Asymptote Fault Tolerance (AFT): Maximal Deterministic Byzantine Safety — All-But-One ($n-1$ of n), Relay-Free, Fully Attributable

IOI

March 17, 2026 (revised September 7, 2026)

Conditionality and exact-figures note. Every result in this paper is proven within the AFT model delta M_{AFT} — an assumed common publication boundary whose non-circular construction is the subject of, and is tracked by, the AFT-CB program. The public claim surface is governed by the whitepaper's §5.3 assumption ledger (A1–A9) and prints only the interim conditional claim until that program's gates pass. Tolerance figures appear in this paper only in exact all-but-one form: "all-but-one ($n-1$ of n) Byzantine safety." A rounded percentage is never printed as a tolerance figure, because with $n = 16$–64 validators the true all-but-one maximum is 93.75%–98.4375%; the all-but-one statement is both stronger and exact.

Abstract

Asymptote Fault Tolerance (AFT) is a family of consensus and settlement constructions for high-throughput ordering, guardian-backed base finality, proof-carrying canonical ordering, and deterministic release of irreversible effects. Its central move is to stop treating dense positive voting as the only possible source of ordering truth. Instead, AFT separates throughput-critical publication and chain progression from authority-critical proof verification, omission detection, and effect collapse.

The protocol family has four interacting surfaces. *GuardianMajority* supplies the production base-finality path: a validator quorum certificate is necessary but not sufficient, because each proposal must also carry a registered guardian committee certificate anchored into shared evidence. *CanonicalOrdering* supplies the proof-carrying ordering path: a slot order is accepted because a canonical certificate is uniquely valid and recoverable from public inputs, not because a dense validator quorum voted the order into existence. *Asymptote* supplies the sealing path for irreversible effects: `BaseFinal` chain progress is upgraded to `SealedFinal` through deterministic witness or observer-backed collapse, with canonical observer challenges dominating unsafe releases. *NestedGuardian* supplies a witness-augmented mode for layered threshold constructions.

This paper is self-contained and normative for the protocol family. Its ordering subtheorem remains all-but-one ($n-1$ of n) equal-authority ordering consensus, conditional on the AFT model delta's assumed common publication boundary and on explicit bulletin-publication, cutoff, omission, protocol-native retrievability, and proof-soundness assumptions. The runtime also internalizes compact signed publication-frontier metadata on the hot path: positive committed blocks carry a predecessor-linked frontier summary and compact availability-receipt binding, while same-slot conflicts or stale parent links admit short objective contradiction objects. Those frontier objects bind the claimed bulletin / ordering surface, while the protocol-native retrievability plane internalizes deterministic extraction-or-abort on the cold path. For each closed slot boundary there exists at most one admissible public execution object, every conflicting fully specified

candidate admits a short public contradiction witness, durable state advances only through canonical collapse, and sealed-effect release is bound to the same collapse surface.

The lower-bound claim is explicit about model membership. The classical $n > 3f$ / $f < n/3$ lower bound remains true in the classical partially synchronous message-passing model whose adversary may keep honest groups mutually ignorant of each other's published views for the duration needed by the indistinguishability argument. AFT does not refute that theorem from inside its quantified model. It operates in the model where an admissible published artifact cannot be indefinitely suppressed from correct parties after stabilization, and it realizes that delta without a trusted relay: honest publishers send artifacts or erasure-coded shards directly over authenticated point-to-point links to validators or deterministic custodians, while validators publish only compact commitments, receipts, frontiers, and challenges as the echo layer. Payload delivery is relay-free; equivocation and omission detection still require compressed honest-party information flow.

In that model delta, and conditional on it, AFT is *relay-free, coordinator-free, pure-software deterministic all-but-one ($n-1$ of n) Byzantine safety*: the classical threshold stops being load-bearing because the DLS adversary's indefinite-suppression move is no longer legal, not because an impossibility theorem is violated inside its own model. Compact bindings stay on the hot path while cold-path recovery / reveal / reconstruction re-materialize the same close, abort, collapse, replay-prefix, execution-restart, and restart-ancestry surfaces through the AFT-native recovery-family contract. Accountable publication and validator removal remain operational reinforcement, not the source of correctness.

Stated at full strength: within the model delta, AFT occupies the two ceilings of the safety–accountability plane simultaneously, and both ceilings are theorems, not comparisons. All-but-one is not merely a large threshold but the *terminal* one — with zero honest signers, equivocation is unpreventable by any protocol in any model (the zero-honest bound), so no consensus method, in any model, can ever claim a stronger safety threshold than $n-1$ of n; AFT claims exactly that threshold, weight-independently, with a mechanized proof. Symmetrically, no protocol can attribute a violation beyond the signers of the conflicting certificates (the attribution cap), and AFT attributes exactly that set — every member of the violating configuration, individually, from the two certificates alone. There is no stronger safety claim and no stronger accountability claim available to any consensus protocol; AFT prints both, paired with the impossibility results that make them ceilings. The price is stated beside the claim: unanimity means one withholder freezes seal cadence (the forced trade of the unanimity bound — never chain progress, which continues on the live tier), live-tier liveness remains synchrony-conditional, and the classical lower bound is refused rather than refuted: its carrier — dense positive voting relayed through the same parties the adversary may control — is replaced by direct publication with compressed public echo, and in the replaced carrier the safety threshold ascends to its information-theoretic maximum. The live tier uses randomized asynchronous termination below one-third Byzantine membership after three certified failed optimistic views; eventual synchrony is required only for the optimistic path's latency, not for that fallback's declared termination model.

1 Scope and Status

This paper contains the full architectural and theorem-level description of the instantiated AFT family. It does not require any companion prose specification to be understood.

The instantiated protocol surfaces are:

- `classic_bft` as the sole admitted AFT PQ v1 production mode, using ML-DSA live votes and the mandatory hash-only asynchronous path.
- `aft_quv_v0`, the interactive-visibility (query-unanimity) authorization profile for irreversible effects at the all-but-one ($n-1$ of n) threshold (Section 12); frozen as the immutable M17Q R2 review candidate after complete M16Q qualification.

- unanimous, configuration-enrolled SLH-DSA boundary seals as the terminal irreversible-effects profile.
- `CanonicalOrdering` through the live `CommittedSurfaceV1` certificate path plus compact `PublicationFrontier` summaries attached to committed blocks.
- `GuardianMajority`, `Asymptote`, and `NestedGuardian` as non-admitted historical research/reference constructions.

Implementation correspondence is included below as exact symbol references. That section is non-normative. The protocol and theorem content of the paper stands on its own.

The strongest theorem surface of the runtime is singular, and it is conditional on the AFT model delta. AFT claims not only all-but-one ($n-1$ of n) equal-authority ordering for the ordering layer, but relay-free, coordinator-free, pure-software deterministic all-but-one ($n-1$ of n) Byzantine safety for durable ordering and sealed-effect safety in the AFT model delta, with public-state continuity as its protocol-internal safety carrier rather than dense positive quorum intersection. Compact hot-path bindings plus cold-path recovery / reveal / reconstruction materialize the same recovered close-or-abort, collapse, replay-prefix, execution-restart, and streamed restart-ancestry surfaces through the AFT-native recovery-family contract. Compact publication/frontier metadata is internalized on the live path throughout, and deeper recovered history is now ordinary endogenous AFT history through the canonical-collapse / replay retrievability anchor and the matching recovered-state historical retrievability surface.

Model discipline: same ambition, explicit delta. No protocol refutes a lower bound from inside the model over which that lower bound universally quantifies. AFT's claim is therefore a membership claim: the classical DLS / PBFT lower bound applies to a model whose adversary can keep honest groups mutually ignorant of published admissible information for the duration needed by the indistinguishability argument. AFT operates in a model delta where that indefinite-suppression move is revoked for content-addressed admissible protocol artifacts after stabilization. The delta is not a hidden helper regime: it is realized by authenticated direct publisher-to-validator or publisher-to-custodian dissemination plus a compact echo layer of commitments, receipts, frontiers, and challenges. The substantive objection would be a hidden relay, privileged coordinator, TEE, bootstrap oracle, external recovery service, or circular realization of the visibility resource by the same Byzantine-majority suppression power. AFT has none in the stated model.

Scope precision is not a concession. A criticism that AFT is "permissioned only" is non-responsive because the promoted claim is a permissioned theorem over fixed authenticated authorities and epoch-scoped identity state. Replacing that domain with open-membership consensus is a target switch, not a rebuttal. The lower-bound question is instead mechanical: execute the classical adversary against AFT and identify the first illegal move. That move is indefinite suppression of a published admissible artifact from correct parties after stabilization. Everything before that point is a standard adversarial schedule; everything after that point is outside the classical model and inside AFT's explicit model delta.

Singular theorem sentence. The theorem surface of this repository is now singular all the way up: AFT claims relay-free, coordinator-free, pure-software deterministic all-but-one ($n-1$ of n) Byzantine safety in the AFT model delta. It breaks the classical threshold in the precise model-relative sense that the theorem is proved in the AFT model delta, where published admissible

artifacts cannot be indefinitely suppressed from correct parties and where that delta is realized without a trusted relay. AFT does not ask credit for defeating DLS / PBFT while remaining inside the bare DLS / PBFT model. It identifies the exact adversary power removed, implements the removal through standard authenticated channels and compressed echo, and then proves the close-or-abort safety surface in that model. Proof-carrying public evidence, endogenous historical retrievability, restart continuity, and collapse-gated durability are the realizing architecture of that claim, not a residual semantic gap.

The interactive edge of the same threshold. One result stands beside the singular ordering theorem rather than inside it. At the irreversible edge, where an executor must act now, no finite portable byte string derived from copyable participant state can certify non-conflict at $f = n - 1$ (Theorem 6); what *can* hold at that threshold is an interactive, known-synchronous, non-portable guarantee obtained by the executor itself from one reachable correct member (Section 12). It is a separate profile with its own assumption ledger, and it raises no coordinate of the ordering or sealing claims.

Claim ladder. The full promoted claim should be read as a constructed ladder:

1. each closed slot boundary admits at most one admissible public execution object and every conflicting fully specified candidate admits a short objective obstruction witness,
2. durable state advances only through canonical collapse on that same close-or-abort surface,
3. irreversible effects release only through that same collapse surface.

Dimension	**AFT realizing surface**	**Promoted singular claim**
Agreement object	canonical close-or-abort / canonical collapse surface	one durable decision surface in M_{AFT}
Safety carrier	compact commitments, short contradiction objects, cold-path recovery, endogenous historical retrievability	dense positive quorum intersection is no longer load-bearing
History / bootstrap	canonical-collapse / replay historical retrievability root plus recovered-state historical retrievability surface	recovered history remains endogenous AFT state
Liveness burden	direct authenticated dissemination, compressed echo, recurrence/reduction/totality/collapse chain, persistent churn simulator	completed theorem surface under the AFT model delta

Target adversary/scheduler model and discharged liveness kernel. The singular theorem is interpreted under one explicit liveness model rather than a vague aspiration. Byzantine authorities may equivocate, omit, reorder, withhold reveals, withhold historical-retrievability objects, rotate maliciously, and crash/restart arbitrarily. Before stabilization the scheduler may delay or interleave publication, registry, recovery, archival, and restart traffic arbitrarily. After stabilization, authenticated channels between correct parties eventually deliver direct publisher sends and compact echo objects; repeatedly published admissible artifacts become visible and content-hash fetchable to correct parties; and governing profile/activation churn quiesces into bounded windows long enough for progress to compose. Under that model, the liveness bridge is the conjunction of five

obligations: frontier-generation progress, canonical-resolution progress, recovery-completion progress, restart/historical-retrievability re-entry progress, and infinite composition of those four obligations across reassignment, outage, profile rotation, and archival page boundaries. That kernel is no longer open; it is the discharged bridge that supports the promoted AFT model-delta theorem.

For the common-publication boundary, the relevant liveness fact is the exact post-stabilization visibility resource used to halt the classical adversary. Separately, the live ordering tier has a hash-only randomized asynchronous fallback under its own static-adversary, (f<n/3), reliable-private-channel profile; that result does not remove the publication boundary's assumptions. The protocol does not require universal timely receipt before the cutoff. It requires eventual delivery, persistence, and hash-fetchability of repeatedly published admissible artifacts after stabilization, realized by direct authenticated dissemination and compressed echo rather than by a trusted publication service.

Current artifact map and completion status. That kernel is now grounded in concrete repository artifacts rather than prose alone. Frontier-generation progress lands on the live `PublicationFrontier` path and its contradiction objects; canonical-resolution progress lands on canonical-order certificates, `CanonicalCollapseObject`, omission dominance, and recursive continuity; recovery-completion progress lands on the coded recovery-family contract and the recovered publication / close-or-abort surface; and restart/historical-retrievability progress lands on the AFT recovered-state surface plus canonical-history retrievability anchors and bounded-memory paging. Those obligations now have both concrete runtime carriers and a completed formal bridge. The formal package contains bounded churn witnesses, recurring cycle cores, a recurrence theorem, a finite reduction, a total classical- agreement history bridge, and a directly discharged semantic-collapse wrapper ending in `NestedGuardianRecoveryClassicalAgreementCollapse.tla`. The exact source of that recurrence / reduction / totality / collapse bridge is embedded in Appendix A, including the final wrapper in Appendix A.6.5. The recurrence / reduction / totality / collapse chain is not narrative glue. Its exact artifact chain is embedded verbatim in Appendix A, ending in the classical-agreement collapse wrapper itself. The runtime mirrors the same story through a persistent historical-retrievability churn/restart simulator over one evolving recovered-history state. The remaining work is therefore no longer theorem completion, but cleanup, stress expansion, and maintenance of the promoted singular claim.

2 Problem Statement

Classical permissioned Byzantine fault tolerance ties deterministic safety to dense positive voting by a large honest majority. Under that model, honest inclusion, total-order agreement, and finality all depend on most validators remaining both correct and available. That is the right model when the protocol is allowed to pay dense coordination cost and accept classical $3f + 1$-style thresholds. It is the wrong model for a system that wants all of the following simultaneously:

- high-throughput publication and chain progression,
- equal-authority participation at the validator layer,
- deterministic rejection of incomplete or conflicting ordered views,
- stronger accountability than "someone must have voted wrong,"
- deterministic gating of irreversible effects.

AFT therefore changes the object of agreement.
Instead of asking:

- "Which order did the validator quorum choose?"

it asks:

- "Which order certificate is uniquely valid for this slot, and which effects are admissible after deterministic collapse?"

The family-level design goals are:

- keep ordering and publication sparse on the hot path,
- keep effect release fail-closed,
- make omissions and conflicts objectively provable,
- keep all validators equally eligible to reveal the canonical order,
- move authority from dense yes-votes to verifiable evidence.

3 Thought Experiment: The Port of Public Manifests and Release Seals

Imagine an international port that handles ordinary cargo and hazardous cargo.

Every ship that wants to unload during tide h must post its cargo manifest to a public harbor board before the closing bell τ_h. The bell is not a private clock in the harbor master's office. It is a certified public event. Once the bell rings, the harbor office takes every admissible manifest, applies the published tide lottery R_h, and derives one canonical docking docket O_h.

The office does not ask every captain to vote on the exact order of every container. It asks a simpler question: has someone produced the one valid docket certificate for the manifests that were publicly posted before the bell? If any dockworker can prove that an admissible manifest was omitted from the announced docket, that docket is rejected immediately.

Ordinary unloading may begin once the berth assignment is base-final. Hazardous cargo is stricter. It may leave the port only after one of two things happens:

- the required independent inspector guilds each sign a release certificate, or
- a deterministically assigned set of external captains publishes a public inspection transcript surface, the public red-flag board remains empty through the close bell, and the port issues the unique release docket implied by that surface.

Any valid red flag aborts release. The cargo does not "probably" leave the port. It does not leave.

Most of the port can be chaotic. Captains can lie, stall, or collude. What they cannot do is create a second valid docket for the same tide or a second valid hazardous-cargo release order, provided the public manifest board remains recoverable and the decisive proof surface stays objectively checkable.

The correspondence is direct:

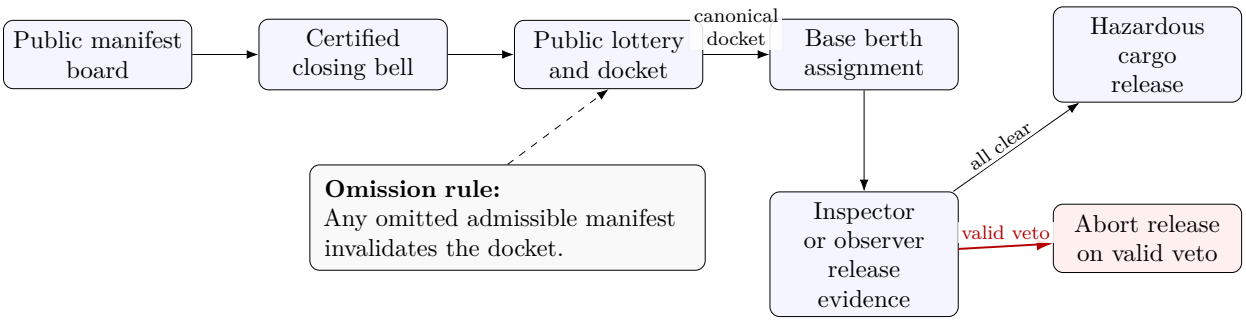

Figure 1: The thought experiment behind AFT: public admission, canonical ordering, base progress, and fail-closed release.

Port object	**AFT object**
Public manifest board	Bulletin / DA surface B_h
Certified closing bell	Canonical cutoff τ_h
Tide lottery	Public randomness beacon R_h
Docking docket	Canonical order O_h
Docket certificate	Canonical-order certificate $OCert_h$
Omitted manifest challenge	Omission proof $\Omega(h, tx)$
Base berth assignment	`BaseFinal`
Independent inspector guilds	Witness committees
Sampled external captains	Equal-authority observers
Red-flag report	Canonical observer challenge
Hazardous cargo release seal	`SealObject` under `SealedFinal`

This thought experiment captures the essential AFT move: sparse operational progress, dense cheap verification, objective omission, and deterministic collapse for irreversible consequences.

4 Protocol Family Overview

AFT is best understood as four planes that compose:

Plane	**Purpose**	**Throughput-critical work**	**Authority object**
Dissemination / bulletin plane	Publish the candidate transaction surface	Data dissemination and bulletin commitments	Public bulletin commitment
Base-finality plane	Advance the chain and commit block identity	Proposal, QC formation, guardianized certification	Validator QC + guardian certificate
Canonical-ordering plane	Prove the unique slot order	Succinct verification, omission dominance	$OCert_h$
Sealed-effect plane	Gate irreversible effects	Asynchronous witness / observer evidence collection	`SealedFinalityProof` and `SealObject`

The protocol modes are:

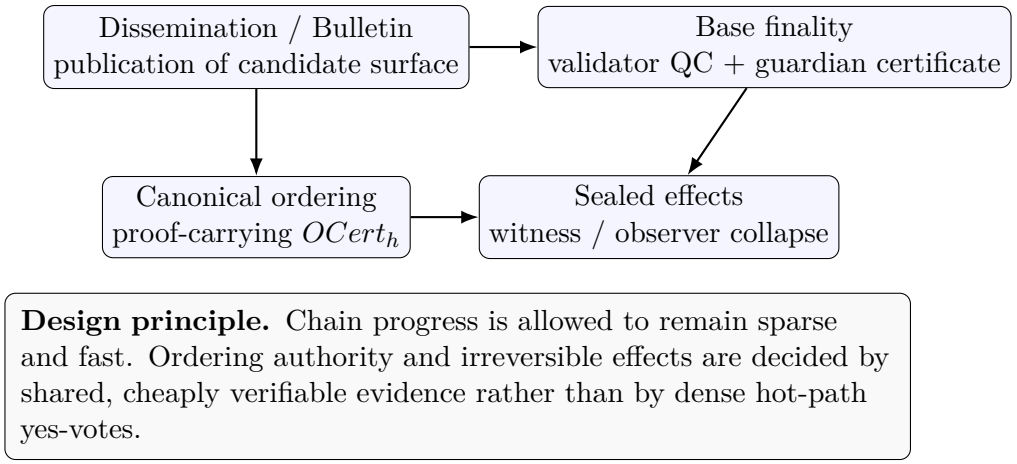

Figure 2: AFT separates publication, base progress, canonical ordering, and sealed effect release, then recomposes them through shared evidence.

Mode	Role	Current status
`GuardianMajority`	Production guardian-backed base finality	Production mode
`CanonicalOrdering`	Proof-carrying equal-authority ordering	Live through `CommittedSurfaceV1`
`Asymptote`	Two-tier sealing and sealed-effect release	Production sealing path
`NestedGuardian`	Witness-augmented layered threshold mode	Experimental operational mode / completed theorem bridge

The key invariant is:

$$\text{throughput-critical work} \neq \text{authority-critical work}$$

5 System Model and Notation

5.1 Actors

- **Validator:** participates in proposal, vote, verification, and chain execution.
- **Guardian committee:** the registered threshold-signing committee protecting one validator's proposal decisions.
- **Guardian member:** an individual signer within a guardian committee.
- **Witness committee:** an external threshold-signing committee used on the sealing path or in `NestedGuardian`.
- **Equal-authority observer:** a validator sampled to audit a slot under `Asymptote`.
- **Registry:** the on-chain source of truth for manifests, policies, epochs, witness sets, and verifier descriptors.

- **Transparency log:** an append-only checkpointed evidence log for guardian and witness statements.
- **External verifier or gateway:** a party that checks `SealObject`s and enforces replay-safe release.

5.2 Bulletin-board time and slot model

Consensus proceeds in heights and views. A slot is identified by $(height, view)$. Epoch-scoped policy and committee state bind the admissible evidence for that slot. The family should be read under a model-delta publication and recoverability discipline rather than a pure point-to-point timely receipt discipline: safety depends on whether the closed boundary fixes a unique public evidence surface, while liveness depends on correct-party delivery and eventual visibility of admissible content-addressed artifacts after stabilization.

Slots are externally paced by a public cutoff object τ_h and a public randomness beacon R_h. Honest validators that observe the same closed public boundary derive the same ordering or sealing close-or-abort result. In the runtime that derivation is no longer keyed off cutoff closure alone: the bulletin plane exports a *canonical bulletin-close object* binding the bulletin commitment, the bulletin-availability certificate, and the canonical cutoff into one public close surface. In this instantiation, positive canonical-order publication is fail-closed at this boundary: a closed-slot order certificate is admitted only if the published bulletin entries deterministically extract the bulletin surface bound by the canonical bulletin-close object and the carried bulletin-availability certificate. Universal timely receipt is not required.

Let

$$h = \text{slot height}, \quad v = \text{slot view},$$

$$B_h = \text{bulletin surface admitted before cutoff } \tau_h,$$

$$R_h = \text{public randomness beacon for slot } h,$$

$$E_h = \{tx \mid \text{Included}(tx, B_h) \wedge \text{Eligible}(tx, root_{h-1})\},$$

$$O_h = \text{CanonicalOrder}(R_h, E_h),$$

$$root_h = \text{Execute}(root_{h-1}, O_h).$$

5.3 Classical lower bound and AFT model delta

Let M_{DLS} denote the standard authenticated partially synchronous message-passing model used by the classical DLS / PBFT lower-bound argument. Its adversary may delay messages so that two honest groups cannot distinguish "the other group is slow" from "the other group is faulty" for the duration required by the indistinguishability construction.

Let M_{AFT} denote the AFT model delta. It keeps fixed authenticated authorities, deterministic local verification, and partial synchrony among correct parties, but it removes one adversary power: after stabilization, an admissible content-addressed protocol artifact repeatedly published by a correct party cannot be indefinitely suppressed from correct parties. Payload delivery is realized by direct authenticated dissemination from the publisher to validators or deterministic custodians; equivocation, omission, and retrievability detection are realized by compact echo objects rather than by payload relay.

Adversary-execution trace. Run the classical indistinguishability adversary against AFT:

1. The adversary may schedule delays, split honest validators into views, and make Byzantine validators equivocate or omit.

2. The adversary may hide a Byzantine sender's conflicting statements until an honest validator publishes a protocol-admissible commitment, receipt, frontier, challenge, or reconstruction / abort artifact.

3. The first illegal step is indefinite suppression of that published admissible artifact from another correct validator after stabilization. In M_{DLS}, that suppression power is available to the adversary long enough to drive the lower-bound execution. In M_{AFT}, the artifact eventually becomes visible and content-hash fetchable.

That stopped step is the model delta. The rest of AFT is protocol design inside M_{AFT}: compact frontiers bind the claimed surface, contradiction objects kill conflicts, retrievability objects reconstruct or abort the full surface, and canonical collapse gates durability.

Lemma 1 (Direct dissemination plus compressed echo realizes the delta)**.** *Assume authenticated point-to-point channels among correct parties that eventually deliver after stabilization. An honest publisher sends its artifact or erasure-coded shards directly to the validators or deterministic custodians named by the slot policy. No non-sender validator must relay the payload for honest-artifact delivery. Validators then publish compact commitments, availability receipts, publication frontiers, observer transcripts, challenges, and reconstruction / abort objects that expose what they received or what the public surface lacks. The payload path discharges honest delivery; the echo path discharges equivocation, omission, and retrievability detection. The classical adversary's indefinite-suppression step is therefore unavailable without introducing a trusted relay, coordinator, TEE, bootstrap oracle, or external recovery service.*

5.4 Core assumptions

The family-level safety claims rely on the following explicit assumptions:

- **Objective publication:** admitted bulletin entries, transcripts, and challenges have stable hashes and objective inclusion paths.

- **Authenticated direct dissemination:** after stabilization, authenticated channels between correct parties eventually deliver the publisher's direct artifact or shard sends to the validators or deterministic custodians named by the slot policy.

- **Compressed echo visibility:** validators publish compact commitments, receipts, frontiers, transcripts, challenges, and reconstruction / abort artifacts that make equivocation, omission, stale lineage, and retrievability failures objectively visible without echoing full payloads.

- **Verifiable bulletin availability:** the protocol exports a bulletin-availability object, compact publication-frontier metadata that binds the same slot surface, and a canonical bulletin-close object for each closed ordering slot; the remaining assumption is that direct dissemination and the compressed echo layer satisfy the retrieval semantics claimed by those objects. The live runtime also requires successful closed-slot extraction before positive canonical-order admission.

- **Compact frontier consistency:** any carried `PublicationFrontier` must bind the same-slot bulletin commitment, ordered surface, and compact availability receipt, and same-slot conflicts or stale predecessor links must admit short objective contradiction objects. These frontier objects summarize the slot surface; they do not reconstruct it.
- **Objective validity only:** theorem-critical rejection or acceptance does not depend on private local facts.
- **Public derivability over universal receipt:** honest validators that observe the same closed public boundary derive the same close-or-abort result; universal timely receipt is not required.
- **Independent echo path:** suppressing negative evidence requires capture of both the payload-delivery path and the compact publication / registry echo surface.
- **Sealed-effect gating:** irreversible effects are released only after surviving the canonical close-or-abort surface.
- **Fixed epoch identities:** the validator set is epoch-scoped and stable while the accountable rules of that epoch are enforced.

Here and throughout, the AFT permissioned model means fixed authenticated authorities, deterministic local verification, authenticated point-to-point channels among correct parties, public hash-addressed protocol objects, epoch-scoped identity state, and eventual visibility / persistence / hash-fetchability of admissible protocol artifacts after stabilization, with no trusted relay, privileged coordinator, TEE, theorem-side bootstrap oracle, or hidden external recovery service. Fixed authenticated membership is part of the theorem statement, not a missing crutch. The hidden-helper exclusions are relay, coordinator, TEE, bootstrap oracle, and external recovery service — not epoch-scoped authenticated identity itself.

5.5 Evidence objects

The family-level evidence objects are:

- validator quorum certificates over block proposals,
- guardian quorum certificates over slot decisions,
- witness certificates over guardianized slots,
- canonical-order certificates over bulletin surfaces,
- omission proofs over ordered-set incompleteness,
- observer transcripts, observer challenges, and canonical close-or-abort objects over sealed finality,
- sealed-effect proof objects with replay-safe nullifiers.

All honest nodes are required to derive the same acceptance result from the same admissible evidence set.

5.6 Accountable publication rules

Objective AFT faults are not merely audit artifacts. In the runtime they are first-class accountable protocol objects:

- `OmissionProof` names the validator accountable for the dominated positive order object.
- `AsymptoteObserverChallenge` determines accountability by kind: `MissingTranscript` and `ConflictingTranscript` blame the assigned observer; `TranscriptMismatch`, `VetoTranscriptPresent`, and `InvalidCanonicalClose` blame the producer / positive-close path.
- Valid objective fault evidence is replay-deduplicated on chain.
- Policy-controlled membership updates are aftermath only: the same evidence may trigger best-effort immediate quarantine and stage next-epoch eviction, but the decisive negative object remains valid even when those membership updates are disabled, delayed, or skipped.

These rules do not change the hot path. They strengthen the adversary model: arbitrary behavior is tolerated only insofar as it remains publicly revealable and penalty-bearing.

6 Public State Continuity with Extractable Obstructions

AFT's deeper theorem target is not a new round gadget. It is a new source of uniqueness. Classical Byzantine agreement derives uniqueness from overlap of dense positive quorums. AFT instead seeks a rigidity theorem over public slot objects: once the slot boundary is closed, the space of admissible public continuations should collapse to one canonical object or one canonical abort.

6.1 Boundary-conditioned execution language

For a closed slot boundary define

$$\partial_h := (root_{h-1}, \beta_h, \tau_h, R_h, p_h),$$

where β_h is the canonical bulletin-close object binding the bulletin commitment, bulletin-availability certificate, and canonical cutoff whose recoverable surface is B_h; τ_h is the canonical cutoff object; R_h is the slot randomness beacon; and p_h is the policy descriptor governing admissible checks for slot h.

Let a *public execution object* for slot h be a normalized public codeword

$$X_h := (B_h, E_h, O_h, root_h, \sigma_h),$$

where E_h is the eligible subset, O_h is the canonical ordered object, $root_h$ is the resulting state commitment, and σ_h is the slot-local sealing surface needed to determine whether the slot closes or aborts. The important point is that X_h is not a distributed transcript. It is the public quotient of the slot after irrelevant schedule variation is factored out.

Define the boundary-conditioned language

$$\mathcal{L}(\partial_h) := \{X \mid \exists \pi \ \mathrm{Accept}(\partial_h, X, \pi) = 1\}.$$

Here π is a succinct admissibility witness. For canonical ordering, π is the proof-carrying order certificate witness. For deterministic observer sealing, π is the witness that the canonical transcript surface, challenge surface, and close-or-abort object satisfy the slot policy.

6.2 Obstruction extractability

The companion rejection relation is

$$\mathrm{Reject}(\partial_h, Y, \omega) = 1,$$

where Y is a fully specified candidate public execution object and ω is a short public objective contradiction witness.

AFT seeks *obstruction-complete local testability*: for a fixed closed boundary, every fully specified candidate lies in exactly one of two states,

$$\Big(\exists \pi\ \mathrm{Accept}(\partial_h, Y, \pi) = 1\Big)\ \oplus\ \Big(\exists \omega\ \mathrm{Reject}(\partial_h, Y, \omega) = 1\Big).$$

This xor formulation is the software-only analogue of state continuity. It is stronger than merely having a valid proof system. It says every non-admissible candidate exposes a short public contradiction witness.

In the live repository, the obstruction basis is not abstract:

- ordering uses a first-class canonical abort taxonomy over the canonical bulletin surface: missing certificates, bulletin-surface reconstruction failures, bulletin-surface mismatches, invalid bulletin-close formation, omission-dominated candidates, certificate-height mismatches, randomness mismatches, ordered-transactions-root mismatches, resulting-state-root mismatches, invalid public-input bindings, invalid bulletin-availability bindings, and invalid proof bindings,
- deterministic observer sealing uses objective challenge kinds over the canonical transcript and challenge surface,
- the accountable-adversary layer turns those same objects into penalty-bearing public fault evidence.

6.3 Unique collapse object

The slot should not merely admit at most one positive object. It should admit at most one *collapse object*:

$$\mathrm{Collapse}(\partial_h) \in \{\mathrm{Close}(X_h), \mathrm{Abort}(\Omega_h)\}.$$

Here $\mathrm{Close}(X_h)$ denotes the unique admissible positive object when the obstruction surface is empty, while $\mathrm{Abort}(\Omega_h)$ denotes the unique negative object induced by a non-empty valid obstruction surface. Negative evidence does not create a second truth. It forces the slot to land on its canonical abort object instead of a close object.

In the runtime this collapse object is not merely an abstract proof target. Validator finalization publishes a first-class `CanonicalCollapseObject` through `guardian_registry`, and later consensus verification cross-checks any published copy against the same proof-carried ordering and sealing surface when parent-state data is available. In the current recursive-continuity slice, `CanonicalCollapseObject` also carries a rolling `previous_canonical_collapse_commitment_hash`, and in the current clean-break runtime slice it also carries `continuity_accumulator_hash` together with `continuity_recursive_proof`, so the current slot's collapse object is linked to the previously persisted or published collapse object rather than standing as an isolated per-slot fact. The newest proposal-surface slice pushes that same predecessor link into the signed header itself: `BlockHeader` now carries `previous_canonical_collapse_commitment_hash` in its signed preimage and also carries a typed `CanonicalCollapseExtensionCertificate`. That certificate now carries the

predecessor commitment plus the predecessor recursive-proof hash, while the public predecessor `CanonicalCollapseObject` now carries only a single recursive proof step rather than a carried predecessor-proof tree. Each recursive proof step still binds a predecessor collapse commitment, predecessor commitment hash, predecessor payload hash, and previous proof hash; the rolling `continuity_accumulator_hash` remains in place as the companion accumulator over the same chain. The implementation surface distinguishes the reference `HashPcdV1` carrier from a backend slot `SuccinctSp1V1`, and the runtime exposes a dedicated continuity-verifier seam for those recursive public inputs via `ioi-api` and `zk-driver-succinct`. That seam is now wired into the live protocol: `GuardianMajority` invokes it while validating proposal and QC continuity for carried or anchored `SuccinctSp1V1` proof steps, and validator durable-state checks invoke the same backend before a persisted `CanonicalCollapseObject` is treated as authoritative. Proposal verification checks that the carried predecessor commitment hash matches the signed predecessor link, checks that the predecessor commitment's resulting state root matches the signed parent-state root, and checks that the carried predecessor proof hash matches the anchored predecessor collapse object already persisted or published in protocol state. Leaders in `Asymptote` stall rather than propose when the required extension certificate or anchored predecessor collapse object is unavailable, proposal verification rechecks that implied predecessor commitment against both the signed predecessor commitment link and the parent-state root, checks the carried predecessor proof hash against the anchored predecessor proof on the public collapse object, and QC progression only treats continuity-linked headers as progress-authoritative.

Theorem 1 (Public State Continuity with Extractable Obstructions)**.** *Assume a closed boundary ∂_h, objective publication, a protocol-native retrievability plane, compact publication-frontier consistency, objective validity only, and proof soundness. Then the instantiated and normative theorem shape for AFT is:*

1. *$\mathcal{L}(\partial_h)$ contains at most one admissible public execution object,*
2. *every fully specified candidate $Y \notin \mathcal{L}(\partial_h)$ admits a short public objective obstruction witness ω such that* $\mathrm{Reject}(\partial_h, Y, \omega) = 1$,
3. *therefore the slot admits at most one admissible collapse object, either the unique close object or the unique abort object.*

This theorem is the precise form of the software-only continuity primitive that the instantiated AFT runtime realizes. The later ordering, collapse, recovery, restart, and sealed-effect sections are concrete theorem instances and composition layers of this same continuity statement, not separate aspirations. It does not claim that the classical DLS / PBFT frontier disappeared inside the standard dense-vote model. The point is different: the uniqueness source has moved from quorum overlap into the public-evidence language itself. In the instantiated runtime, compact publication frontiers internalize same-slot and predecessor consistency inside the live protocol, while the protocol-native retrievability plane discharges closed-bulletin extraction-or-abort on the cold path.

6.4 Lemma program

The clean proof path is a filtration rather than a single opaque argument:

$$\mathcal{L}_0(\partial_h) \supseteq \mathcal{L}_1(\partial_h) \supseteq \cdots \supseteq \mathcal{L}_k(\partial_h),$$

where each refinement adds one structural constraint and each failed refinement admits a short witness.

The repository's theorem program is:

1. **Boundary Fixing Lemma.** The closed boundary fixes the admissible publication domain.
2. **Bulletin Close Lemma.** The canonical bulletin-close object fixes the unique recoverable bulletin surface for the slot.
3. **Eligibility Determinism Lemma.** The predecessor commitment and boundary fix the eligible subset.
4. **Order Determinism Lemma.** The boundary and eligible subset fix the canonical ordered object.
5. **Transition Determinism Lemma.** The ordered object fixes the next state commitment.
6. **Collapse Exclusivity Lemma.** A slot cannot admit both a canonical close object and a canonical abort object.
7. **Obstruction Soundness Lemma.** Every valid omission proof or observer challenge objectively rejects its target candidate.
8. **Obstruction Completeness Lemma.** Every invalid fully specified candidate contains at least one minimal public obstruction witness.
9. **Extractor Sufficiency Lemma.** Given the canonical bulletin-close object, any honest validator can run the protocol-defined bulletin extractor to reconstruct the same bulletin surface; in this instantiation, admission of the positive ordering object already requires this extraction to succeed over the published closed-slot surface. One honest validator remains sufficient to surface the resulting positive or negative object under the instantiated runtime model.

One honest validator is surfacing-sufficient but never trust-bearing: correctness does not attach to the revealer's identity, because any honest validator observing the same closed boundary derives the same decisive object.

The theorem surfaces of this paper are instances of that program:

- `CanonicalOrdering` instantiates unique admissibility and extractable obstruction through proof-carrying order certificates plus omission dominance.
- `Asymptote` instantiates unique collapse through canonical transcript/challenge surfaces plus close-or-abort exclusivity.

7 The AFT Deterministic Base Engine

AFT modes share a deterministic core engine for proposal flow, QC formation, locking, and view progression.

7.1 Deterministic leadership and view progression

For a given height and view, the implementation selects the proposer by deterministic round-robin over the active validator set:

$$\text{leader}(h, v) = V_h[(h - 1 + v) \bmod |V_h|].$$

Here V_h is the active validator set for height h after operational quarantine filters are applied.

View 0 proposals do not carry a timeout certificate. Any non-zero view proposal must carry a valid timeout certificate for the previous unsuccessful view. The pacemaker advances views monotonically and resets timers on observed progress.

7.2 Validator quorum certificates

In `GuardianMajority`, `Asymptote`, and `NestedGuardian`, validator quorum is a strict simple majority of active validator weight, not a classical $2/3 + 1$ quorum:

$$\text{QCQuorum}(V_h) > \frac{1}{2}\text{Weight}(V_h).$$

This is safe only because the validator quorum is *not* the sole authority object. A proposal must also satisfy guardian-layer admissibility.

Timeout certificates use the same simple-majority rule over view-change votes.

7.3 Locking, 2-chain commit, and divergence guard

Validators maintain a lock on the highest safe parent QC they have observed. A validator votes only when the proposal extends the lock or is in a strictly higher view than the lock.

Base commitment uses a 2-chain rule:

- let QC_{parent} certify parent block P,
- let QC_{child} certify child block C,
- if C directly extends P and the views are consecutive, then P becomes a pending commit.

The implementation delays final durability by a guard window Δ_{guard} before a pending commit is released. The purpose of the guard window is operational, not purely algebraic: it gives divergence evidence or panic signals time to propagate before a conflicting branch is treated as durable.

This produces a fail-closed operational behavior:

- sparse chain progress happens on the hot path,
- conflicting evidence is surfaced before irreversible consequences,
- divergence proofs quarantine the affected node rather than silently allowing fork continuation.

8 GuardianMajority: Historical Reference Model

`GuardianMajority` is not a production fault model for AFT PQ v1. ADR 0048 closes its configuration and assurance admission; this section is retained to state the historical construction and its assumptions while shared theorem material is extracted. The production profile is `classic_bft` plus the hash-only randomized asynchronous path. Validator votes alone never make a proposal admissible. Every valid proposal must also carry a guardian committee certificate that verifies against current registered committee state.

8.1 Guardian certificate object

The guardian certificate binds:

- the guardian committee manifest hash,
- the active epoch,
- the canonical decision hash for the slot payload,
- a monotonic guardian counter,
- a guardian trace root,
- a measurement root,
- the signer bitfield,
- the aggregated BLS signature,
- a transparency-log checkpoint,
- and, in `NestedGuardian`, an optional experimental witness certificate.

The guardian decision payload is slot-scoped. Honest guardians are assumed not to sign two different decisions for the same slot.

8.2 Verification rule

A guardianized proposal is admissible only if all of the following hold:

1. the referenced guardian committee manifest is registered on-chain for the active epoch,
2. the certificate manifest hash matches the registered manifest,
3. the certificate epoch matches the manifest epoch and current epoch,
4. the decision hash matches the slot's canonical guardian decision payload,
5. the measurement root matches the decision payload,
6. the signer set is valid, unique, and meets the committee threshold,
7. the aggregated signature verifies against the manifest public keys,
8. the attached transparency-log checkpoint verifies against the anchored log descriptor and append-only proof.

The transparency log is an accountability layer. It does not replace threshold safety. It makes issued decisions, checkpoints, and later slashing evidence publicly checkable.

8.3 Committee threshold model

Let a guardian committee have threshold t and size n. Conflicting slot certificates require enough equivocators to cover the minimum quorum intersection:

$$f < 2t - n.$$

Production committees therefore use strict majority thresholds:

$$t = \left\lfloor \frac{n}{2} \right\rfloor + 1.$$

This guarantees pairwise quorum intersection. It also exposes an important operational nuance:

- odd-sized simple-majority committees tolerate zero equivocating guardian members at the committee layer,
- even-sized majority committees tolerate one equivocator before threshold intersection is exhausted.

Production safety therefore relies on both threshold intersection and stronger guardian non-equivocation guarantees.

8.4 Safety statement

Proposition 1 (GuardianMajority Safety)**.** *No two conflicting blocks can both finalize for the same* $(height, view)$ *if guardian committee manifests are current, committee thresholds are majority thresholds, honest guardians do not equivocate for the same slot, transparency-log checkpoints and registry state are current enough to verify admissibility, and validators finalize only guardian-admissible blocks.*

This is not a classical $3f + 1$ theorem. It is a composed threshold theorem over validators, guardians, registry state, and shared evidence. That statement is local to the guardian-backed base-finality layer. It does not delimit the paper's promoted theorem surface. The promoted durable-agreement claim is carried by the later proof-carrying close-or-abort and canonical- collapse architecture, which is precisely why the classical $3f + 1$ threshold is not load-bearing there.

8.5 Liveness model

Liveness is secondary to safety. Progress requires:

- enough active validators to form the simple-majority validator QC,
- enough guardian members to satisfy the committee threshold,
- current registry state for the epoch,
- current enough checkpoint and log availability.

If these degrade, the protocol prefers stalling over accepting unsafe evidence.

9 Canonical Ordering

`CanonicalOrdering` is the proof-carrying ordering path that gives AFT its all-but-one ($n-1$ of n) equal-authority ordering consensus claim in the AFT model delta.

9.1 Objective

For each slot h, define a unique canonical ordered set O_h and a certificate $OCert_h$ such that:

- every honest verifier can check $OCert_h$ cheaply,
- any omission is objectively provable,
- the ordered set is recoverable from public data,
- the live hot path carries a compact publication frontier that binds the same-slot bulletin / ordering / availability surface and predecessor continuity,
- conflicting valid certificates for the same slot are impossible,
- arbitrary behavior by almost all validators cannot create a conflicting valid ordering outcome because the closed bulletin boundary admits one deterministically derivable close-or-abort result.

9.2 Canonical objects

For slot h,

$$B_h = \text{bulletin surface admitted before } \tau_h,$$
$$R_h = \text{public randomness beacon for slot } h,$$
$$E_h = \{tx \mid \text{Included}(tx, B_h) \wedge \text{Eligible}(tx, root_{h-1})\},$$
$$O_h = \text{CanonicalOrder}(R_h, E_h),$$
$$root_h = \text{Execute}(root_{h-1}, O_h).$$

The abstract certificate is

$$OCert_h = (h, root_{h-1}, cutoff_h, bulletin\_commitment_h, ordered\_commitment_h, root_h, witness_h)\,.$$

9.3 Bulletin, cutoff, and admissibility assumptions

The ordering theorem is conditional on the following named assumptions.

A1. Objective publication. Each admitted transaction or batch in B_h has a stable content hash, a verifiable inclusion path into the bulletin commitment, and an objective publication time or publication position relative to the slot cutoff.

A2. Endogenous retrievability. The bulletin contents committed by $bulletin\_commitment_h$ are reconstructed or deterministically aborted from protocol-native retrievability objects rooted in the canonical bulletin close for slot h. Compact publication-frontier summaries and availability certificates bind that claim on the hot path, while retrievability profiles, shard manifests, custody assignments, custody receipts, custody responses, and objective challenge objects close the extraction-or-abort loop on the cold path. Without that plane, uniqueness may still hold while timely materialization of the same close-or-abort surface can fail.

A3. Append-only or equivocation-evident commitment. The bulletin commitment and any carried publication frontier for slot h must be append-only or otherwise make same-slot equivocation or stale predecessor linkage objectively detectable at the slot boundary.

A4. Eligibility determinism. Eligible$(tx, root_{h-1})$ is deterministic and locally checkable from the committed predecessor root and the transaction object. Honest validators therefore derive the same eligible set E_h from the same public inputs.

A5. Proof soundness. VerifyProof$(witness_h)$ = true implies that the witness obligations for cutoff binding, bulletin binding, ordering, state transition, retrievability commitments, and omission commitments actually hold.

The cutoff τ_h is not a private local clock read. It is a protocol-bound object. In the instantiated runtime, the cutoff is bound directly into the published bulletin commitment and into the canonical public-input set. Eligibility is deterministic with respect to the predecessor state root and the transaction object. This prevents local reinterpretation of the eligible set.

9.4 Current runtime instantiation

The runtime surfaces the abstract objects above as the following concrete objects:

Runtime object	Current fields
BulletinCommitment	(height, cutoff_timestamp_ms, bulletin_root, entry_count)
BulletinSurfaceEntry	(height, tx_hash)
BulletinAvailability Certificate	(height, bulletin_commitment_hash, recoverability_root)
BulletinRetrievabilityProfile	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, recoverability_root, shard_count, recovery_threshold, custody_threshold)
BulletinShardManifest	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, recoverability_root, entry_count, shard_count, recovery_threshold, shard_commitment_root)
BulletinCustodyAssignment	(height, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, validator_set_commitment_hash, assignments)
BulletinCustodyReceipt	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, custodian_count, custody_threshold, custody_root)
BulletinCustodyResponse	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, served_shards)
BulletinRetrievabilityChallenge	(height, kind, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, bulletin_custody_response_hash, details)
BulletinReconstructionCertificate	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, bulletin_custody_response_hash, canonical_bulletin_close_hash, canonical_order_certificate_hash, reconstructed_entry_count, reconstructed_bulletin_root)
BulletinReconstructionAbort	(height, kind, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_receipt_hash, bulletin_retrievability_challenge_hash, canonical_order_abort_hash, details)
PublicationAvailabilityReceipt	(height, bulletin_commitment_hash, ordered_transactions_root_hash, resulting_state_root_hash, receipt_root)
PublicationFrontier	(height, view, counter, parent_frontier_hash, bulletin_commitment_hash, ordered_transactions_root_hash, availability_receipt_hash)
PublicationFrontierContradiction	(height, kind, candidate_frontier, reference_frontier)
CanonicalBulletin Close	(height, cutoff_timestamp_ms, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_receipt_hash, entry_count)
CanonicalOrder PublicInputs	(height, parent_state_root_hash, bulletin_commitment_hash, randomness_beacon, ordered_transactions_root_hash, resulting_state_root_hash, cutoff_timestamp_ms)
CanonicalOrderProofSystem	concrete variants HashBindingV1 and CommittedSurfaceV1
CommittedSurface	(bulletin_availability_certificate_hash, omission_commitment_root,
CanonicalOrderProof	boundary_omission_justification_root)
CanonicalOrder Certificate	(height, bulletin_commitment, bulletin_availability_certificate, randomness_beacon, ordered_transactions_root_hash, resulting_state_root_hash, proof, omission_proofs, boundary_tx_hashes, boundary_omission_justifications)
CanonicalOrder PublicationBundle	(bulletin_commitment, bulletin_entries, bulletin_availability_certificate, bulletin_retrievability_profile, bulletin_shard_manifest, bulletin_custody_receipt, canonical_order_certificate)
CanonicalOrder Abort	(height, reason, details, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_close_hash, canonical_order_certificate_hash)

`HashBindingV1` is the reference verifier family: its proof bytes are a canonical hash binding over the public-input hash. `CommittedSurfaceV1` is the live proof family: its proof payload contains the recoverability root and the omission-commitment root for the committed surface. In the current runtime, positive ordering artifacts are published through the atomic `CanonicalOrderPublicationBundle`, carrying the bound retrievability profile / manifest / custody-receipt objects, while the registry deterministically materializes the coupled custody-assignment and custody-response objects over the same slot surface before admitting the positive lane. Positive committed blocks also carry a compact signed `PublicationFrontier`: it binds the same-slot bulletin commitment, ordered-transactions root, and compact publication-availability receipt, and it hash-links that summary to the previous slot's frontier. Short `PublicationFrontierContradiction` objects make same-slot conflict or stale predecessor linkage objectively rejectable on the hot path. Those frontier objects internalize compact publication consistency, but they do not replace the protocol-native retrievability plane that reconstructs or aborts the full bulletin surface. Validators also run a block-local derivation pass over the committed block boundary: they derive either a `CanonicalOrderExecutionObject` or a `CanonicalOrderAbort`, and publish the abort artifact when the proof-carried positive object cannot be derived. When the endogenous retrievability lane succeeds, the registry materializes a first-class `BulletinReconstructionCertificate` bound to the canonical close, canonical-order certificate, and the same retrievability objects. When endogenous retrievability evidence dominates that same surface, the registry also materializes a specialized `BulletinReconstructionAbort` object bound to the challenge and the paired `CanonicalOrderAbort`, so reconstruction-impossible failure is named as a first-class protocol object rather than only as a generic ordering abort.

9.5 Acceptance rule

Every validator applies the same deterministic rule:

$$\begin{aligned}\mathrm{Accept}(OCert_h) \iff\ & \mathrm{VerifyProof}(witness_h)\\ & \wedge \mathrm{VerifyCutoff}(cutoff_h)\\ & \wedge \mathrm{PredecessorAccepted}(root_{h-1})\\ & \wedge \neg\exists tx : \mathrm{ValidOmission}(h, tx).\end{aligned}$$

In the implementation this specializes to:

- the certificate height and bulletin height must match the block height,
- the randomness beacon must match the slot schedule,
- the bulletin commitment must match the published bulletin when that bulletin is available from anchored state,
- any carried `PublicationFrontier` must match the same-slot bulletin commitment, ordered-transactions root, and publication-availability receipt hash,
- when a predecessor frontier exists, the carried `PublicationFrontier` must extend it by counter and parent hash,
- no published `PublicationFrontierContradiction` may already dominate the slot,

- positive canonical-order admission in the runtime additionally requires successful closed-slot extraction over the published bulletin entries, the carried bulletin-availability certificate, and the canonical bulletin-close object,
- the ordered-transactions root and resulting state root must match the block header,
- the proof must match the canonical public inputs,
- the recoverability root must match the certificate commitments,
- the omission-commitment root must match the omission-proof set.

If the omission-proof set is non-empty, the candidate certificate is rejected immediately.

No dense positive vote on the order is required once a valid $OCert_h$ is surfaced. The current verifier entry point for this rule is `verify_canonical_order_certificate`.

9.6 Omission dominance

An omission is objective when a transaction:

- was included in B_h before τ_h,
- was eligible under $root_{h-1}$,
- and is absent from the committed ordered set O_h.

Define the omission proof

$$\Omega(h, tx) = \begin{pmatrix} inclusion\_proof(tx, bulletin_h), \\ cutoff\_admissibility\_proof(tx, cutoff_h), \\ eligibility\_proof(tx, root_{h-1}), \\ nonmembership\_proof(tx, O_h) \end{pmatrix}.$$

Its semantics are:

$$\text{Valid}(\Omega(h, tx)) \Rightarrow \text{Reject}(OCert_h).$$

This is the central AFT ordering move. A candidate does not need to win a dense round of positive endorsements if any incomplete view can be killed objectively.

Omission dominance is a correctness rule, not a punishment rule. A valid omission proof kills the candidate immediately whether or not slashing, quarantine, or next-epoch removal ever fires.

9.7 Uniqueness, publication frontiers, and recoverability

Theorem 2 (Uniqueness)**.** *If* ValidOrderCert(h, C_1) *and* ValidOrderCert(h, C_2)*, then* C_1 *and* C_2 *commit the same canonical ordered set* O_h*.*

The reason is structural:

- the predecessor root is fixed,
- the cutoff object is canonical,
- the bulletin commitment is canonical,

- eligibility is deterministic,
- the ordering function is deterministic,
- proof soundness forbids a witness for a different ordered set.

The live publication-frontier slice sits between uniqueness and recoverability. A frontier gives a compact signed summary of the claimed bulletin / ordering surface and its predecessor link, so same-slot conflict or stale lineage is objectively rejectable without reconstructing the full bulletin on the hot path. But the frontier only binds hashes and receipt roots; it is not itself the recoverable bulletin surface.

Theorem 3 (Endogenous Retrievability)**.** *If* ValidOrderCert(h, C) *and the canonical bulletin close plus protocol-native retrievability plane for slot h are present and unchallenged, then every honest verifier can reconstruct the same ordered set O_h from protocol objects alone: the canonical close, bound frontier / availability commitments, retrievability profile, shard manifest, custody receipt, positive reconstruction certificate, and published bulletin entries.*

Endogenous retrievability matters because the accepted order is not merely unique. The compact frontier tells validators which surface is being claimed; the retrievability plane tells them that the full surface can actually be reconstructed or objectively aborted from protocol evidence.

The hot path and the cold path do not carry different truths. The frontier binds the claimed surface compactly; the retrievability plane reconstructs or objectively aborts that same surface; both terminate on the same canonical close-or-abort object.

9.8 All-but-one ($n-1$ of n) equal-authority ordering consensus

AFT's ordering claim is a proof-carrying consensus theorem whose decisive object is revealed rather than manufactured by dense positive yes-votes. Revelation is not a weaker post hoc reporting layer here; it is the equal- authority surfacing mechanism for the uniquely valid public execution object already fixed by the closed public boundary.

Theorem 4 (All-But-One ($n-1$ of n) Equal-Authority Ordering Consensus in the AFT Model Delta)**.** *Assume, in the AFT model delta (the assumed common publication boundary):*

1. *the closed bulletin surface B_h bound by the carried publication frontier and bulletin commitment is governed by a canonical bulletin close and protocol-native retrievability plane that deterministically yields extraction or abort,*
2. *the cutoff object for slot h is canonical,*
3. *bulletin commitments, compact publication-frontier bindings, and omission proofs are objectively verifiable,*
4. *same-slot frontier conflicts and stale predecessor links admit short objective contradiction objects,*
5. *the proof system is sound.*

Then even if all but one validator ($n-1$ of n) behaves arbitrarily, they cannot create a conflicting valid canonical-order outcome for slot h. More strongly, every honest validator that observes the same closed ordering boundary derives the same positive order object or omission-dominated abort.

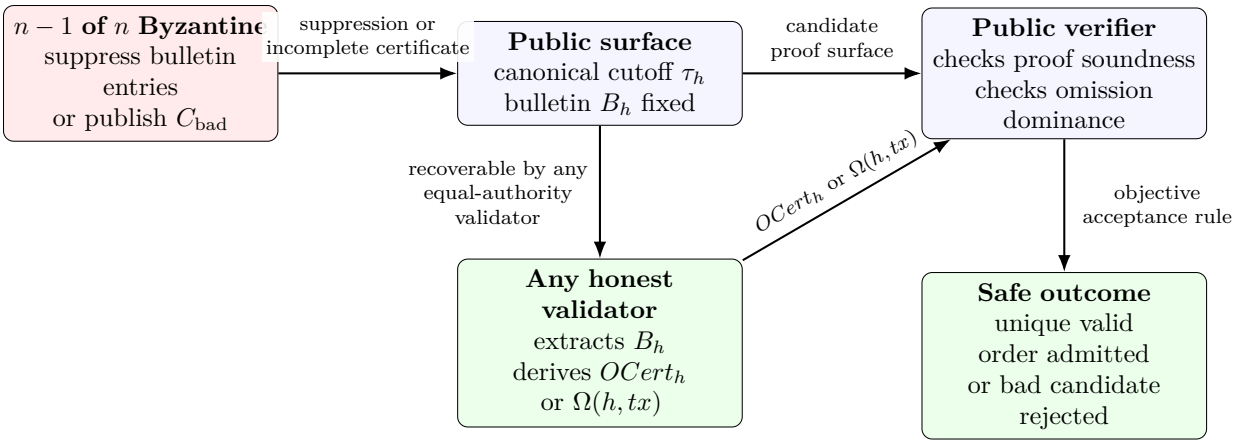

Figure 3: Omission-dominance attack and deterministic extraction. Even if a suppressing supermajority tries to hide admissible bulletin entries or promote an incomplete candidate certificate, any honest validator that reconstructs the same public surface derives the same valid $OCert_h$ or publishes a valid omission proof $\Omega(h, tx)$ that kills the bad candidate.

Compact publication frontiers therefore internalize cheap same-slot and predecessor consistency checks on the live path, while recoverability remains the stronger assumption needed to turn that compact claim into a publicly reconstructable ordered set.

The strongest shorthand, in the AFT model delta, is:

$$n-1 \text{ of } n \text{ validators may behave arbitrarily,}$$

$$\text{because the closed ordering boundary admits one deterministically}$$

$$\text{derivable close-or-abort outcome.}$$

This is equal-authority because any validator may reveal the winning certificate. It tolerates all but one Byzantine validator ($n-1$ of n) because correctness does not depend on a dense majority of positive votes. This section isolates the ordering lane, not the theorem ceiling. Once composed with endogenous retrievability, canonical collapse, recovery and restart continuity, and collapse-bound sealed-effect gating, the promoted AFT theorem surface is exactly the paper's relay-free deterministic all-but-one ($n-1$ of n) Byzantine safety claim in the AFT model delta.

With the accountable publication rules of Section 5, invalid positive ordering attempts are not only rejectable; they are also penalty-bearing and next-epoch removing.

10 Asymptote: Sealed Finality and Irreversible Effects

`Asymptote` is the two-tier settlement mode in the AFT family. It preserves fast `BaseFinal` chain progression while requiring stronger evidence before irreversible effects are released.

10.1 Finality tiers and collapse states

The finality tiers are:

- `BaseFinal`: validator QC plus guardian certificate.
- `SealedFinal`: `BaseFinal` plus sufficient sealing evidence.

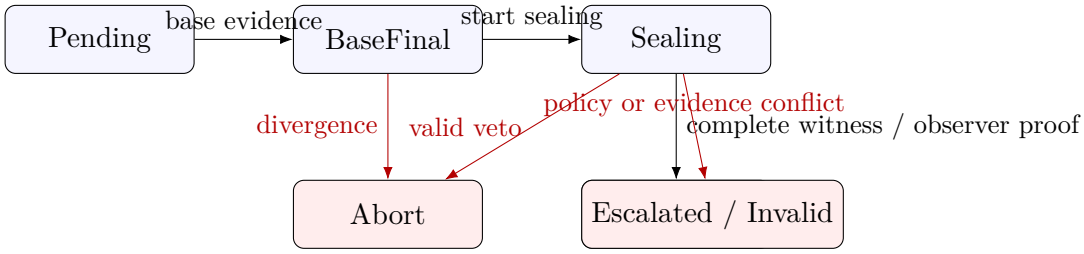

Figure 4: The deterministic collapse surface for `Asymptote`.

The slot collapse states are:

- `Pending`
- `BaseFinal`
- `Sealing`
- `Abort`
- `SealedFinal`
- `Escalated`
- `Invalid`

All honest nodes must derive the same collapse state from the same admissible evidence set.

10.2 SealedFinalityProof

The sealing plane gathers a proof object that binds:

- the epoch,
- the achieved finality tier,
- the collapse state,
- the guardian manifest hash,
- the guardian decision hash,
- the guardian counter,
- the guardian trace root,
- the guardian measurement root,
- the policy hash,
- witness certificates, when the witness path is used,
- observer transcripts, transcript commitment, observer challenges, challenge commitment, and exactly one canonical observer outcome object when the observer path is used,

- divergence signals gathered during sealing.

This object is block-scoped and guardian-bound. A `SealedFinalityProof` that is not bound to the slot's guardian evidence is invalid.

10.3 Witness-backed sealing

The witness path assigns one witness committee per required certification stratum.

Witness assignment is deterministic over:

- the epoch seed,
- the producer account id,
- the slot $(height, view)$,
- the reassignment depth,
- the witness stratum id,
- the witness manifest hash.

Sealed finality is reached on the witness path when:

- `BaseFinal` holds,
- each required witness stratum contributes a valid assigned witness certificate,
- each witness certificate verifies against the registered witness manifest,
- required checkpoints and append-only proofs are current enough under policy.

Witness omission, stale registry participation, checkpoint inconsistency, or conflicting witness certificates produce explicit evidence rather than silent best-effort behavior.

10.4 Equal-authority observer sealing

The normative observer path is `CanonicalChallengeV1`. It treats sampled validators as deterministic duty assignees whose outputs must become public, recoverable protocol objects.

Observer assignment is deterministic over:

- the epoch seed,
- the producer account id,
- the slot $(height, view)$,
- the observation round,
- the observer account id.

The producer is excluded from its own observer pool. The runtime may also filter assignments through a correlation budget over provider, region, host class, and key-authority class.

Each assigned observer publishes one of two things:

- a guardian-backed `AsymptoteObserverTranscript`, or
- an objective `AsymptoteObserverChallenge`.

A transcript binds the canonical slot surface:

- the epoch and assignment,
- the candidate block hash,
- the guardian manifest, decision hash, counter, trace hash, and measurement root,
- the guardian checkpoint root,
- the observer verdict and veto kind,
- the observer evidence hash.

The admissible challenge basis is public-evidence-only:

- `MissingTranscript`,
- `TranscriptMismatch`,
- `VetoTranscriptPresent`,
- `ConflictingTranscript`,
- `InvalidCanonicalClose`.

Each challenge carries its own public evidence object: an assignment, an offending observation request, an offending transcript, or an offending canonical close. The challenge `evidence_hash` must bind exactly that carried evidence.

The slot then carries three canonical bindings:

- a transcript commitment over the transcript surface,
- a challenge commitment over the challenge surface,
- exactly one canonical outcome object: `AsymptoteObserverCanonicalClose` or `AsymptoteObserverCanonicalA`

The collapse rule is close-or-abort and challenge-dominant:

- `SealedFinal` if `BaseFinal` holds, the deterministic assignment set is fully covered by valid canonical `Ok` transcripts, the transcript and challenge commitments verify, the challenge surface is empty, and the canonical close object is valid.
- `Abort` if the transcript and challenge commitments verify, the challenge surface is non-empty, and the canonical abort object is valid.
- `Pending` otherwise.

Challenge dominance is a theorem-level veto, not an operational alarm. A valid challenge collapses the slot to abort before any penalty, governance, or membership aftermath is considered.

This makes observer sealing structurally parallel to canonical ordering: public evidence, canonical commitments, a unique positive object, a unique negative object, and deterministic local close-or-abort derivation from the same public surface.

10.5 Divergence signals and escalation

The sealing plane may also emit divergence signals, including:

- conflicting guardian certificates,
- witness omission,
- stale registry use,
- stale checkpoints,
- transparency-log consistency failure.

These signals support `Escalated` and `Invalid` outcomes and give operators a fail-closed evidence trail.

10.6 Sealed effects and replay-safe release

Irreversible effects are gated by finality tier. The effect plane distinguishes between:

- `BaseFinal` effects, which are allowed on the fast path when policy permits,
- `SealedFinal` effects, which require a valid `SealedFinalityProof`.

For proof-enabled irreversible effects, `Asymptote` carries a self-contained `SealObject` whose current instantiation includes:

- an `EffectIntent`,
- an `EffectPublicInputs` object, including `canonical_collapse_hash`,
- an `EffectProofEnvelope`,
- a replay-safe nullifier.

In the runtime, `SealedFinal` secure-egress requests carry both the `SealedFinalityProof` and the slot's `CanonicalCollapseObject`. The guardian-side seal builder and downstream receipt verifier recompute the same `canonical_collapse_hash` from that collapse object together with the proof-carried observer surface, so a challenge-dominated or otherwise mismatched slot cannot authorize irreversible release.

The release rule is

$$\text{Release}(e) \iff \text{BaseFinal}(\text{slot}(e)) \wedge \text{SealedFinal}(\text{slot}(e)) \wedge \text{VerifySealObject}(e) \wedge \text{CollapseHash}(e) = \text{Hash}(\text{Collapse}($$

This keeps the hot path sparse while making released consequences deterministic and replay-safe.

10.7 Deterministic observer theorem

The deterministic `Asymptote` observer model now proves the following kernel:

- base certificates are unique per slot,
- canonical observer close objects are unique per slot,
- canonical observer close and canonical observer abort cannot coexist for the same slot,
- a challenge-dominated slot cannot produce a valid sealed release.

Theorem 5 (Deterministic Equal-Authority Observer Sealing)**.** *Assume sound deterministic observer assignment, signature-sound guardian-backed observer transcripts and challenges, canonical transcript and challenge commitments, append-only registry and log publication once published, and deterministic transcript/challenge derivation over the closed public observer surface. Then arbitrary behavior by the rest of the validator set cannot create a conflicting valid canonical observer close, cannot make canonical close and canonical abort coexist for the same slot, and cannot authorize a valid sealed irreversible effect release for a challenge-dominated slot.*

This theorem is the observer-lane analogue of canonical ordering's public certificate theorem: public evidence, canonical commitments, negative-authority dominance, and deterministic local close-or-abort derivation from the same public observer surface.

Under the accountable publication rules of Section 5, the same observer challenges are also penalty-bearing objective faults. Unsafe positive close attempts therefore become not only rejectable, but accountable and epoch-removing.

10.8 Legacy sampled comparison

Before `CanonicalChallengeV1`, the observer lane was typically described through a sampled affirmative bound. Let:

- h be the honest fraction of equal-authority validators,
- A_s^j be the sampled observer committee for slot s and round j,
- $k_j = |A_s^j|$,
- $p_{\text{beacon\_bias}}$ be the probability of sufficient beacon bias,
- $p_{\text{veto\_suppression}}$ be the probability that a valid veto surface is fully suppressed.

Then the old affirmative sampled lane admitted the analytical bound

$$P_{\text{unsafe}}(s) \le p_{\text{beacon\_bias}} \cdot p_{\text{veto\_suppression}} \cdot \prod_j (1-h)^{k_j}.$$

For constant committee size k across r rounds:

$$P_{\text{unsafe}}(s) \le p_{\text{beacon\_bias}} \cdot p_{\text{veto\_suppression}} \cdot (1-h)^{kr}.$$

That formula remains a useful historical comparison, but it is no longer the normative theorem for the observer lane. Under `CanonicalChallengeV1`, sampling determines duties, not safety. Safety comes from the canonical public transcript surface, the canonical public challenge surface, and the close-or-abort split that those surfaces induce.

The runtime publishes the canonical observer transcript, commitment, challenge, challenge commitment, and close-or-abort objects as ordinary `guardian_registry` transactions after the sealed header is formed. The same surface is carried inside the proof for immediate block verification.

11 The Common Boundary and the Seal Ring

The AFT-CB program restated the strong-tier settlement layer as a single construction: the *Common Boundary*. Where the Asymptote surface of the previous section describes the deployed observer-sealing runtime, this section describes the mechanized seal-ring architecture that the theorem corpus of Section 15 proves and that the runtime is converging onto. Status labels are exact and load-bearing: the kernels are mechanized (Appendix A), the membership types, membership simulator, registry wiring, hardened signer, and the runtime inversions below are landed in the repository, and the full multi-member ring runtime is future work behind an honestly labeled reference seam.

11.1 The Unanimous Boundary Close

One signer configuration C_v of n members forms one *Unanimous Boundary Close* per seal slot: an n-of-n certificate of individually verifiable final-ack shares over a domain-separated tuple binding the slot's closed boundary root. Three disciplines carry the entire safety story:

- **Journal-guarded single final-ack (A3).** An honest member journals its chosen root for a slot in the same atomic step that lets the share exist; recovery replays bytes and never re-decides. The journal is the only thing standing between $n-1$ Byzantine equivocators and a double seal — and by T1 it suffices.
- **n-of-n close.** Every current member's share over the exact $(\text{slot}, \text{root})$ pair is required; no smaller quorum exists anywhere in the construction. The forced trade (L2) is accepted openly: one withholder freezes seal cadence, never the chain.
- **Validate-and-hold as the signature's meaning (T3).** The final-ack *is* the custody commitment; availability standing derives from the close itself, and a close verifies with zero audit records.

11.2 Two-tier separation

The live tier (GuardianMajority) produces, orders, and commits blocks under its own weight-and-synchrony model; the seal ring closes boundaries above it. The only protocol edge from ring to live tier is the release gate for effects *typed* irreversible: block production consults no seal, and a ring stall delays irreversible-effect release and nothing else (unsealed-over-unsafe, T4a). The trade runs in both directions — the live tier's fault bound never contaminates the ring's weight-independent uniqueness, and the ring's unanimity requirement never throttles chain progress.

11.3 The membership plane

Ring membership is event-driven and type-enforced (`crates/types/src/app/ring_membership.rs` plus the guardian registry's service wiring):

- **Bonded registration and the public queue.** Registrants post a bond and enter a public activation queue with an event-depth delay (D_{act}), so admission cannot be timed adversarially and the honest set always has public warning of a capture attempt.
- **Event-driven versioned configurations.** A configuration lives until a close record names its successor; no calendar time exists anywhere in the plane.

- **The assurance-preserving handover is the only strong-ring transition.** It is constructible solely from signed old-ring-unanimous approvals plus acceptance from every new member. No type represents non-response, so a transition "from silence" has no expressible input — the proof-of-silence attack is unrepresentable rather than merely rejected (T5c′).
- **Typed re-genesis.** An anchored re-genesis is a lineage root, never continuity; the lineage query cannot answer "continuous across a root."
- **Live-tier-only ejection.** Liveness faults eject a member from live-tier duty on live-tier weight; the ejection types are fenced from every seal and ring-configuration surface, so strong-ring standing exits only via handover or re-genesis.
- **Custody succession gates bond release.** A departing member's bond releases only against a custody handover receipt carrying the successor's acknowledgement — succession without reconstruction is refused.
- **Riders.** Seat assignment is deterministic sortition over a beacon abstraction across constituency diversity floors (the beacon is the reference ordering beacon until a vetted VDF lands — an honest label, not a security claim); watchtower countersignatures are accepted from anyone and gate nothing.

The plane is exercised by a membership simulator at $n \leq 8$ whose gates are the corpus's scenario proofs: a withholder freezes cadence while the live tier produces; three-Byzantine-of-four equivocation yields zero conflicting seals; succession without reconstruction refuses bond release; the handover ceremony completes end to end with lineage preserved while a re-genesis severs it; sortition reproduces from the beacon and the seal outcome is byte-identical with and without watchtower records.

11.4 The hardened signer

The seal path's signer (`crates/validator/.../guardian/seal_signer.rs`) implements the wire discipline T7 is proven against, plus the A8 everlasting-safety mechanism:

- **Attribution-preserving shares.** Every share carries its own per-seal public key and signature over a domain-separated tuple — individually verifiable, never an opaque aggregate — so forensic extraction names every double-signer from the shares alone.
- **Per-seal key evolution with verified immediate erasure.** One keypair per seal index derives from a hash-ratcheting seed; the spent seed is overwritten in the emitting call, the erasure is byte-verified against the persisted state, and a spent index is structurally unreachable. Compromise after seal i yields only future seeds.
- **Emitting is publishing.** Share emission returns the publication record in the same act, so withholding requires deliberately discarding what the signer already handed back — the honest-publication duty as mechanism rather than assumption.

11.5 Runtime inversions landed by the program

Four inversions moved the production ordering path from internally-consistent to boundary-honest, each with its defect class named and killed:

- **Boundary-before-block.** The committed-surface certificate builder *consumes* a previously closed boundary; the bulletin-from-block-transactions path is deleted. Certificates carry their boundary and typed omission justifications, verification is self-contained (the included set — boundary minus justified omissions — must reproduce the header's transaction root), and the program's baseline commit records the pre-inversion code verifying a censoring block as valid. The current closer is an honestly labeled reference-single-member seam that the ring plane replaces.
- **Custody-attested availability.** The self-attested availability-certificate constructor is deleted; what remains is a deterministic binding derivable by anyone, and availability *standing* derives only from a verified close. An additive audit lane exists for slashing evidence and is consulted by no verification path.
- **Immutable closes.** An abort supersedes only pre-seal state; an abort against a sealed close is a typed rejection whose attempt is itself recorded. Post-close evidence lands in an append-only resolution log (slashing evidence, forward remedies, descendant fences) — history is never rewritten, and the sealed object is byte-identical after any attempt.
- **Real-or-refuse succinct verification.** The succinct continuity lane verifies only through the real proving backend against a governed, test-pinned verifying key; a build without the backend refuses rather than simulating, and no mock constructor survives anywhere in the release tree (a CI gate holds the absence).

11.6 The assumption lattice at the type layer

The T6 lattice is implemented in the collapse types: every certificate profile carries a machine-readable assumption label, adding a profile without deciding its label is a compile error, and a collapse object's effective guarantee is computed as the meet of its constituents — minimum rank, union of consumed assumptions, AND of the post-quantum bit. Evidence-only constituents (continuity bindings, typed roots) contribute assumptions and the pq bit but neither grant nor degrade rank, and a finality receipt is issuable only for a class the meet honestly supports.

11.7 Continuous conformance

The trace-conformance lane closes the loop between code and model: a reference driver executes boundary-ring steps under the kernel's exact guards and emits a committed trace; a generator turns that trace into a TLC-checkable module extending `BoundaryRing`; the replay runs on every build with deadlock detection as the divergence signal. A step the code takes that the model refuses is a red build, which makes the proven-spec-beside-divergent-code defect class — the class the program was founded on — structurally visible.

12 Interactive Visibility: Query-Unanimity Verification (QUV)

The common-boundary program (Section 15) proves that a closed boundary admits at most one execution object and that a sealed effect releases only through that boundary. It left one question open at the irreversible edge: whether an executor that must act *now* — an external call that cannot be undone — can decide from bytes alone that no conflicting authorization exists, when as many as $n - 1$ of the n configured participants may be Byzantine. This section closes that question in both directions. First it states the negative result: no finite, portable, byte-only offline

authorization can do this (M12a). Then it states the positive result that replaces the portable target with an interactive one: one reachable correct member out of n suffices for online conflict-qualified accepted-value non-conflict and for no-conflict singleton progress, when every relying executor performs a fresh push/write-before-reply verification against every configured member inside a rooted known-synchronous deadline (M12b, realized as `aft_quv_v0`). Both results keep the all-but-one ($n-1$ of n) threshold of the rest of this paper. Neither amplifies the other, and neither raises any coordinate of the ordering or sealing claims (Section 12.6).

Admission status of this section. The M12a lower bound is proved impossible under its stated constraints, with an upheld independent review of the immutable lower-bound candidate. The M12b construction, its theorems Q-T1–Q-T4, its end-to-end lifts Q-E1–Q-E4, and the production profile `aft_quv_v0` are frozen as one immutable review candidate after the complete sixty-three-gate M16Q R2 qualification passed on a clean checkout of that commit:

lower-bound tag	`aft-maximal-visibility-lower-bound-candidate-r3-2026-09-03`
candidate tag	`aft-quv-v0-m17q-candidate-r2-2026-09-07`
tag object	`f0932c71c630ab6c85676cda3606dc285a755070`
peeled commit	`0d8d50d4f22c4f02b98ed06b4066c7aefd6d5992`

The claim printed here is exactly the claim that qualification and the mechanized kernels support; it is not weakened by the pending independent review and it is not strengthened beyond it. Public admission (M18Q) records the disposition of that review; the sentence in the claims list is the only sentence admitted for publication, together with its assumptions and the costs of Section 12.8.

12.1 The lower bound: portable byte-only authorization (M12a)

The original target (M11) was a portable authorization: a finite byte string, derived from the copyable state of participants, that any later party could check offline and that would simultaneously give a lone honest executor non-`Abort` liveness and give every checker transferable non-conflict at $f = n - 1$.

Theorem 6 (M12a, portable visibility dilemma)**.** *No finite, portable, byte-only offline authorization derived from copyable participant state can simultaneously provide solo non-`Abort` effect liveness and transferable non-conflict at $f = n - 1$.*

Assumes. Participant state is copyable; the identity of the sole correct member is unknown to the checker; the authorization is a finite byte string checked without further interaction; $f = n - 1$.
Why. With one unknown correct member, any byte string that a lone honest executor can complete without the cooperation of the others can also be completed, byte for byte, by $n - 1$ Byzantine members holding copies of that state for a conflicting candidate; a checker that accepts the first cannot reject the second. Mechanized as `MaximalVisibilityDilemma.tla` (Appendix A.8.1). The result is specific to portable bytes derived from copyable state. It does not generalize to every participant-only pure-software protocol, and it does not touch the ordering and sealing theorems, whose objects are public boundary closes rather than portable effect authorizations.

The consequence is architectural, not a defeat: a lone executor can be safe at $f = n - 1$ only by *interacting* with the configuration at the moment it acts, and the guarantee it obtains is a fact about that moment, which no later party can recover from bytes. The rest of this section is the interactive construction and its exact reach.

12.2 The operation

Let P be the rooted configured membership of a domain, $n = |P| \geq 2$, and let $H \subseteq P$ be the unknown set of correct members with $|H| \geq 1$. A *conflict domain* is a rooted namespace of *slots*; each slot may carry at most one accepted candidate. A rooted *policy root* fixes the domain, its authority mode (owned or unowned), the decision interval δ_{rt}, the continuation budget, the preparation policy, the admission quotas, and the typed initial bootstrap rule; every candidate binds that root, the domain, the slot, the exact predecessor candidate hash, and its payload hash (Q-EA1 below).

An executor that intends an irreversible effect for slot s runs one *operation*:

1. it draws a fresh verifier nonce, binds it to the candidate and the exact context, and pushes the complete request (`PUSHQUERY`) directly to every member of P over authenticated ML-DSA channels;

2. each correct member linearizes requests per $(\text{configuration}, \text{domain}, \text{slot})$, records the candidate in its durable grow-only conflict journal *before* signing, and only then signs a reply that discloses every candidate it has retained for that slot (*write-before-reply*);

3. the executor admits at most the first authenticated reply from each member, validates its exact binding and signature, and evaluates a total decision predicate at the end of one complete δ_{rt} interval: `Accept`(c) only when every admitted correct snapshot is consistent with the single candidate c; otherwise a typed `Abort`, a typed conflict disclosure, or an attributable owner equivocation record;

4. an accepted operation is consumed once, immediately, by the same process: the executor re-derives authorization from the currently committed admission, enters T10's durable `Claimed` state on the stable resource key, and only then calls the external resource (Section 12.5).

Silence, timeouts, cached transcripts, relays, notaries, trusted hardware, and hidden honest-majority custody create no authority at any step. A reply that arrives after the interval is discarded. A late release, a missed budget, or a saturated lane is a declared failure of the operation, never a lower-assurance success.

12.3 Assumption ledger (Q-A1–Q-A10)

ID	Assumption	Class
Q-A1	Rooted member and authority signatures are unforgeable and domain separated	cryptographic
Q-A2	Every honest executor sends its complete request directly to every member of the rooted configuration	network
Q-A3	For every honest executor operation and every $c \in H$: request delivery, admission and queueing, validation, atomic durable processing, response delivery, and the maximum clock error complete within the rooted δ_{rt}	timing and availability
Q-A4	Each correct member linearizes requests per (configuration, domain, slot) and durably writes before it signs the corresponding reply	storage
Q-A5	Correct conflict knowledge is grow-only, survives restart, cannot roll back, and is retained and reachable for the authority lifetime	storage
Q-A6	Every honest executor performs QUV itself immediately before irreversible externalization and never trusts a cached assertion that another verifier waited	execution
Q-A7	Candidate validity and authority are fixed by the independently provisioned root; local arrival, timeout, and silence create no candidate authority	authorization
Q-A8	During one live operation the verifier admits at most the first authenticated response from each rooted member and then validates its exact binding and signature; a correct member emits exactly one valid response, and later Byzantine duplicates are irrelevant to non-conflict	verifier
Q-A9	Admission control reserves enough authenticated capacity for every correct member to satisfy Q-A3 despite Byzantine query traffic	resource and timing
Q-A10	The fault assignment is static for the rooted configuration and authority lifetime; the result does not tolerate a mobile adversary that later corrupts the last correct state holder	adversary

Q-A3 is a *safety* assumption, not a liveness convenience: if a correct reply may arrive after the deadline, two conflicting accepts become possible. The result is therefore known-synchronous safety. It is not asynchronous safety and not eventual-synchronous safety before an independently established bound is active. Every production domain also declares an operator envelope `qualified_delta_rt_envelope_millis` strictly inside δ_{rt} and a membership envelope `qualified_max_configured_members`; both are deployment-local assertions backed by the retained measurements of Section 12.8, and startup fails closed when either is absent or out of range.

12.4 Theorems (Q-T1–Q-T4)

Theorem 7 (Q-T1, accepted-value non-conflict)**.** *For arbitrary membership size, arbitrary nonempty correct subset, and any number of operations, two owned-mode accepts or two unowned-mode accepts for one rooted configuration, domain, slot, and predecessor name the same candidate.*

Assumes. Q-A1–Q-A10, a fixed rooted configuration, domain, slot, and predecessor, $H \neq \emptyset$, and every honest executor follows the complete-deadline algorithm. **Proof sketch.** Fix any correct $c \in H$. In owned mode, pairwise serialization at c (Q-A4) places one operation's candidate in the other operation's snapshot at c, so an accepting singleton union forces equality. In unowned mode each accept must equal every correct member's immutable first winner (Q-A5), so choosing any c forces equality. Mechanized for arbitrary sets as `QOwnedNonConflict`, `QUnownedNonConflict`, `QOwnedOutcomeNonConflict`, and `QUnownedOutcomeNonConflict` in `QueryUnanimityProof.tla` (Appendix A.8.2).

Theorem 8 (Q-T2, external validity)**.** *Every non-`Abort` outcome belongs to the independently valid candidate set; invalid and forged replies never enter an acceptance predicate.*

Assumes. Q-A1, Q-A7, Q-A8, and rooted candidate validation.
Mechanized as `QOwnedExternalValidity`, `QUnownedExternalValidity`, `QOwnedOutcomeTyped`, and `QUnownedOutcomeTyped`.

Theorem 9 (Q-T3, bounded typed termination)**.** *An honest operation returns a typed candidate or `Abort` after one complete δ_{rt} decision interval. Byzantine silence cannot extend that interval; missing the bound invalidates the model rather than producing a lower-assurance certificate.*

Assumes. Q-A2, Q-A3, Q-A6, Q-A8, Q-A9, a live honest executor, and a fixed rooted operation start. The proof kernel mechanizes total outcome typing once the complete snapshots exist; the timing bridge is definitional from Q-A3 and the algorithm, which evaluates a total predicate at the deadline. This is known-synchronous termination, not asynchronous or eventual-synchronous progress, and it is *decision* termination: a typed `Abort`, refusal, freeze, veto, or equivocation record is not inclusion and not effect liveness.

Theorem 10 (Q-T4, no-conflict progress)**.** *If exactly one independently valid candidate s is introduced for the slot and no separately valid conflict is disclosed, every honest operation on s accepts by its deadline, including with every Byzantine member permanently silent. If all members are correct and introduce the same s, the same argument gives all-correct same-input validity.*

Assumes. Q-A1–Q-A10 and the singleton premise. Mechanized for arbitrary sets as `QOwnedSingleCandidateProgress` and `QUnownedSingleCandidateProgress`; the explicit-time R4 model separately enumerates fresh and pre-populated member state under Byzantine silence and non-conflicting replies for both authority modes. No fairness among competing valid submissions is claimed.

Necessity witnesses. Each theorem is paired with a mutation that removes one assumption and recovers a conflict in the model: a different timely correct witness per operation (split witness) yields conflicts in 2 of 16 enumerated cases; a one-way request bound without the observed reply recovers conflicts (Q-T3); reply-before-durable plus crash loses the operation intersection (Q-A4); unbound cross-slot or cross-configuration replies replay (Q-A1/Q-A8 context binding); and a prior acceptance does not guarantee fresh acceptance after a later valid owner conflict, so the singleton premise of Q-T4 is exact (`OwnedParentReverification.tla`).

12.5 End-to-end composition (Q-EA1–Q-EA8, Q-E1–Q-E4)

QUV is an authorization operation; the effect path around it is what makes an authorization safe to act on. The composition premises are:

ID	Premise
Q-EA1	Every ordered candidate binds its rooted configuration, the independently provisioned policy root (authority, timing, preparation budget, and the typed initial bootstrap rule), domain, slot number, exact predecessor candidate hash, authority mode, and payload/manifest hash.
Q-EA2	A correct runtime appends only the unique accepted candidate for its next slot and durably commits candidate, predecessor, and new head before exposing it.
Q-EA3	Correct recovery begins from that durable head, never truncates or rewrites it, and replays no effect mutation from chain state alone.
Q-EA4	Every irreversible executor performs its own QUV operation against the active rooted membership immediately before entering T10's durable `Claimed` state; after the online wait it re-derives authorization from currently committed admission, keeps that admission and execution height stable through claim and call, and validates the process-local continuation immediately before the claim and any resumed call.
Q-EA5	The effect manifest derives one stable idempotency key from the rooted conflict domain and slot, not from the candidate payload, and commits the exact atomic-resource profile.
Q-EA6	The external resource and executor satisfy T10's atomic idempotency-register and claim-before-call assumptions.
Q-EA7	Reconfiguration is a typed handoff candidate under the old root that binds the exact old-root boundary block, state root, and authenticated ordering QC; before old-root expiry every correct new member performs QUV against every old correct member within the old rooted bound, independently verifies that QC under the old root, and durably installs the accepted predecessor and state root before activating. Successor-root history reaching a process as synced or gossiped bytes never activates it; a process without a successor identity refuses such history and stops.
Q-EA8	A client joining after the old authority is no longer reachable receives the current root through an independently provisioned trust channel; historical bytes alone do not establish currentness.

Theorem 11 (Q-E1, prefix-compatible accepted ordering)**.** *Any two correct durable histories of one conflict domain are prefix compatible.*

Assumes. Q-T1, Q-EA1–Q-EA3, correct runtime admission.
Mechanized as `QHistoryPrefixCompatibility` in `QueryUnanimityCompositionProof.tla` (Appendix A.8.3).

Theorem 12 (Q-E2, accepted-consequence non-conflict)**.** *Two mutation candidates admitted for the same rooted domain and slot are equal; an executor's `Abort`, ambiguity, or attributable owner equivocation record authorizes no other mutation candidate.*

Assumes. Q-T1, Q-T2, Q-EA1, Q-EA4, exact manifest validation. Mechanized as `QConsequenceCandidateNonConflict`.

Theorem 13 (Q-E3, at-most-once physical externalization)**.** *A rooted conflict-domain slot causes at most one modeled external-resource mutation.*

Assumes. Q-E2 and Q-EA4–Q-EA6. Q-E2 gives one candidate class; Q-EA5 gives every duplicate delivery the same stable key; T10's atomic put-if-absent register admits one record and its claim-before-call recovery machine never blindly reinvokes after ambiguity, reconciling by lookup only. The result composes QUV with T10; it infers no endpoint semantics from consensus.

Theorem 14 (Q-E4, live reconfiguration continuity)**.** *No two correct new members activate conflicting handoff roots; with one valid handoff and no conflict, every new-correct operation completes within the old bound. A conflict may fail closed and prevent reconfiguration; it cannot create a second lineage.*

Assumes. Q-T1–Q-T4, Q-EA1–Q-EA3, Q-EA7, and at least one correct member in both rooted configurations. This is live-overlap handoff, not portable long-range verification; Q-EA8 is mandatory for later bootstrap. The composed transition model `QuvEndToEndRefinement.tla` (Appendix A.8.4) and its registered countermodels are part of the mandatory formal corpus.

12.6 Separation of guarantees and no laundering

The interactive result is deliberately printed as five separate properties, because a reader who merges them would read a stronger claim than the one proved.

Guarantee	Statement	Where
Accepted-value non-conflict	No two honest executor operations accept different candidates for one rooted conflict-domain slot	Q-T1, Q-S1; `QueryUnanimityProof.tla`; R4 timed model; composed transition model; process campaigns
Decision termination	Every honest operation ends with a typed outcome after at most its rooted interval plus service budget	Q-T3; runtime service budgets; late replies discarded
Inclusion	Not a QUV guarantee	—
Effect authorization	An irreversible effect executes only as the continuation of the executor's own live operation against the currently committed admission	Q-E2, Q-EA4; executor entries, claim-time fences, claim index, T10
Externalization at most once	Claim before call on the stable key; ambiguous calls reconcile by lookup only	Q-E3, T10; `AtMostOnceExternalization.tla`
Fairness	Not claimed; bounded queued waiters per domain and per principal only	admission tests; measured costs

No coordinate of the guarantee vector is raised by composition. PQ v1 ordering and the QUV effect path stay separate profiles with an isolated dependency graph; a QUV audit transcript is a non-authorizing observation and never raises a portable-finality coordinate; every consequence receipt carries `portable_final_receipt = false`; a rooted policy never raises an assurance beyond its verified constituents (the M4 no-laundering theorem and the `GuaranteeMeet` model). The Hypervisor graph does not import QUV and QUV supplies it no fallback authority.

12.7 Production profile

The frozen candidate instantiates one profile, and the profile identifiers are the single version statement shared by the specifications, the code, the receipts, and this paper: policy root `ioi/aft/quv-policy/v8-push-admission`; member store schema 9 (journal records `AFTQJ001`, anchors `AFTQJA01`); handoff store schema 3; PQ outbox logical schema v2 inside `AFTPQI04` reserved envelopes with `AFTPQI05` indices and `AFTPQA01` payload arenas; consequence receipt envelope `AFTCR001`. Members and executors sign with ML-DSA-44 over strict PQ channels only; the v0 protocol caps configured membership so that its isolated request/reply lane remains bounded; every storage charge (record, anchor, receipt envelope, endpoint record, claim index, outbox arena) is committed by the policy root and reserved before the live operation, so that a member that admits a request already holds the space its write-before-reply needs.

12.8 Explicit costs, host-measured

Every number below was measured on the qualification host (one 24-core laptop-class machine, four validator processes per campaign) by the retained clean M16Q R2 run and the retained standalone campaigns around it. Following Section 18.3, these are host measurements with their reproduction path, not bounds, and the deployment obligation is to measure them again on the target host before rooting a policy.

- **Synchrony.** Rooted $\delta_{\mathrm{rt}} = 5000$ ms; operator-declared reply envelope 4500 ms in both the flood and readiness profiles. Measured valid-reply maxima: under live Byzantine flood 1249, 1368, 2921, 4010 ms (earlier campaigns) and 2389 / 1572 / 424 ms (retained run); in readiness campaigns 2075, 2217, 2524, 2731, 4210 ms, the last with replies waiting about 3 s on the shared single-in-flight PQ peer lanes beside concurrent preparation pushes and an ordering commit; handoff campaigns 786 ms (disjoint) and 689 ms (overlapping). Cold start: the first full-membership operation after a process start delivered its pushes 4.7–4.9 s late; operators warm lanes or treat the first typed abort as expected.
- **Reachability.** Every correct member must be reachable by every relying executor for every operation; a member that misses the interval is not correct for that operation and yields a typed abort, never authority.
- **Service.** The active service budget is charged from exclusive admission and includes reserved-storage initialization and the executor's runtime-finality critical section. Flood profile: 16 s (5 s interval, 11 s continuation) after measured release maxima of 10.25, 10.20, 13.01 (under unrelated host load, an 11 s finality stall inside the operation), 10.78, and 10.80 s. Readiness profile: 10 s, with maxima below 9.6 s. A late release is a declared failure.
- **End-to-end singleton cost.** 5.6–6.6 s unflooded; 8.4–11.0 s under terminal replay pressure; 7.3–10.8 s under live Byzantine flood; non-starvation fence three intervals; at most one waiting foreground operation per principal.
- **Durability and storage.** Schema-9 authenticated journal with reserved record and anchor allocations per domain under rooted lifetime budgets; reserved receipt envelopes of tens of MiB per effect, initialized before the live operation; reserved endpoint records, claim-index files, and outbox arenas; a persistence error quarantines the member until authenticated reopen.
- **Post-quantum.** ML-DSA-44 identities, pushes, and replies; provisional carrier claims bounded to 4 per account and 4096 in total for 30 s; a single durable in-flight record per peer lane, so ordering traffic, pushes, and replies serialize per peer.
- **Reconfiguration.** Live old-root handoff with a QC-certified boundary; retained disjoint and overlapping successor campaigns (328 s and 289 s in the retained run) including restart from the durable local install gate, refusal of a rollback image, refusal of a missing gate, and the retirement of an old-only process that is offered successor-root history.
- **Qualification.** The retained clean run executes sixty-three mandatory gates in 7217 s: the complete formal corpus (2623 s), unit and mutation kernels, reservation and syscall-ancestry gates, and eight process campaigns with evidence checkers that reject any missing member, any missed envelope, any late release, and any scheduling or service failure.

- **Audit and consequence.** Transcripts are non-authorizing observations. The consequence path is T10 claim-before-call on reserved storage with the reconciliation budget taken from the manifest.

12.9 What the interactive result is not

Not admitted, and not printed anywhere in this corpus: portable or offline finality; asynchronous or eventually-synchronous safety before an independently established bound is active; exact-decision Byzantine agreement; classical Byzantine consensus among the members; a transferable finality certificate; inclusion or effect liveness for any input that is refused, aborted, vetoed, frozen, or equivocated; fairness among competing valid submissions; tolerance of a mobile adversary; and any guarantee for external resources that do not satisfy T10's atomic register contract. The original portable milestones M13–M18 remain blocked by Theorem 6; their interactive counterparts M13Q–M18Q are what this section reports.

13 NestedGuardian: Witness-Augmented Research Mode

`NestedGuardian` explores a layered threshold construction:

- validators run the AFT deterministic message flow,
- each proposal carries its guardian committee certificate,
- external witness committees cross-check the slot,
- chain state anchors witness assignments, witness manifests, and slashing evidence.

The instantiated runtime scope includes:

- deterministic witness assignment from the active witness set,
- registered on-chain witness manifests,
- witness certificates bound into guardianized slot certificates,
- safety proofs for the witness-augmented kernel,
- bounded model checking for reassignment, outage, and checkpoint transitions.

The instantiated proof package consists of:

- safety formalization,
- bounded operational model checking for reassignment, outage, and checkpoint transitions,
- a discharged composed-liveness bridge under the target eventual-fair scheduler through the recurrence/reduction/totality/collapse chain together with a matching persistent churn/restart simulator.

`NestedGuardian` is therefore a witness-augmented mode with a finished theorem bridge. It is experimental as an operational mode, not as a theorem crutch. Its witness-coded family supplies an explicit constructive recovery carrier and completed bridge artifacts; it does not reduce the promoted AFT theorem surface to an experimental-only claim. That constructive recovery carrier is a witness-coded publication-bundle recovery contract with compact hot-path bindings and cold-path registry-driven close-or-abort resolution. The abstract coded recovery-family contract is realized by the parametric `SystematicXorKOfKPlus1V1` parity family plus the bounded true non-parity `SystematicGf256KOfNV1` family with at least two parity shares, implemented with systematic Cauchy-style coefficients and exercised in bounded `2-of-4`, `3-of-5`, `3-of-7`, `4-of-6`, and `4-of-7` lanes. These lanes resolve from threshold-many public reveals, objective recovered-support conflict, or deterministic missingness. The bounded `3-of-5`, `4-of-6`, and `4-of-7` lanes restore or exceed threshold-support overlap while the bounded `3-of-7` lane shows that recovered-support conflict remains fail-closed even when overlap is absent. A bounded conformance harness checks that every threshold-sized reveal subset reconstructs and every below-threshold subset fails across the shipped XOR and GF256 realizations of that contract. Those recovered reveals lift deterministically into an explicit positive close-extraction payload and then into an explicit extractable bulletin-surface payload that binds the verifying publication bundle, verifying canonical bulletin close, canonical bulletin-availability bytes, and sorted bulletin entries.

The completed bounded family also carries the ordinary durable and restart surfaces. A bounded recovered-only two-slot harness shows that consecutive recovered surfaces derive the ordinary publication-frontier predecessor chain and the same authoritative `CanonicalCollapseObject` predecessor chain without the ordinary publication-bundle lane. A second bounded recovered-only harness shows a close/abort/close bulletin window with a recovered omission-dominated middle slot: recovered data alone materializes the ordinary positive closes and extracted bulletin surfaces on the outer slots, materializes the ordinary omission-dominated abort in the middle while keeping omission evidence and recovered bulletin entries public, and preserves the same authoritative collapse continuity chain. A bounded read-side harness shows that the same recovered-only durable chain yields the same compact replay-prefix / state-root continuity view that the ordinary lane exposes plus a bounded recovered canonical-header prefix. The execution module verifies that replay tip as the same restart anchor the ordinary lane uses, retains both recovered prefixes in execution state, and uses the recovered canonical-header ancestry for bounded anchored reads plus parent-block / parent-state-root continuity after restart when ordinary recent blocks are absent.

Validator consensus orchestration derives the same bounded recovered restart parent anchor from the recovered canonical-header / collapse surface when the ordinary committed block is absent, and the `GuardianMajority` engine consumes that bounded recovered canonical-header prefix as restart-time parent/QC context for synthetic parent-height QC continuity when the full committed block is absent. The same recovered surface yields a bounded recovered certified-header window plus a restart-only recovered `BlockHeader` / QC cache window for restart-time header / QC lookup plus bounded QC-certified recovered branch reconciliation over the configured local five-entry window. Those windows fold by exact overlap with a configurable budget, exercised at five windows into a longer seventeen-step recovered certified branch without ordinary committed headers. Those same cold-path loaders consume a configurable exact-overlap segment budget, exercised at four exact-overlap segments into a longer fifty-three-step recovered certified branch with direct registry-extraction parity and explicit interior-overlap conflict rejection. Those same restart loaders also compose two overlapping four-segment exact-overlap folds into a longer eighty-nine-step recovered certified branch with direct registry-extraction parity and explicit inter-fold overlap conflict rejection, and tests exercise the same recursive carrier at three overlapping four-segment folds into a one-hundred-twenty-five-step recovered certified branch. The runtime has a

bounded conformance harness over one, two, and three stitched segment folds, matching the same production-loader / registry-extraction carrier at the corresponding fifty-three-step, eighty-nine-step, and one-hundred-twenty-five-step branches.

The restart path also pages older exact-overlap segment folds on demand beneath that bounded suffix via an overlap-checked cursor while keeping only the bounded recent suffix plus the streamed page live in engine caches, and tests exercise that paged carrier at a longer two-hundred-thirty-three-step recovered certified branch with direct registry-extraction parity plus explicit duplicate-page, missing-gap, and late-page overlap rejection. Restart therefore streams recovered certified ancestry until target height or recovered-history exhaustion without a theorem-relevant fixed depth bound.

The archival internalization layer uses an active archived-recovered-history profile naming retention horizon, exact-overlap paging geometry, segment / fold geometry, checkpoint update rule, and profile evolution. Archived segments, archived restart pages, archival checkpoints, and archival retention receipts bind to historically referenced profile hashes and governing activation hashes under a published profile-activation chain before archived continuation is trusted, and ordinary canonical collapse history names the archived checkpoint / activation pair that archived restart is authorized to follow. Archived replay and archived publication correctness are historical and index-free: they follow the canonical-collapse-anchored or object-carried activation hash plus predecessor/checkpoint history rather than whichever archived activation is merely latest in local state.

When the recovered certificate carries objective omission proofs, the same recovered lane routes into the ordinary `OmissionDominated` abort path while keeping omission evidence and recovered bulletin entries public. The recovery-family contract, canonical-collapse / replay retrievability anchor, and recovered-state historical retrievability surface are therefore part of ordinary endogenous AFT history rather than a narrower theorem-side qualifier. The directly discharged recurrence/reduction/totality/collapse chain closes the former distance to the relay-free deterministic all-but-one ($n-1$ of n) Byzantine safety theorem in M_{AFT}, so no recoverability or architecture-specific semantic gap remains at the AFT theorem surface.

14 Security and Assumption Surface

The AFT family depends on explicit assumptions. They are not optional prose.

Layer	Required assumptions	Failure consequence
Base finality	current manifests, current epoch state, majority thresholds, guardian non-equivocation, valid checkpoints, enough validators and guardians to reach quorum	conflicting base-finality evidence or loss of progress
Canonical ordering	objective publication, compact publication-frontier binding and contradiction rules, canonical cutoff, deterministic eligibility, append-only bulletin commitment, proof soundness, protocol-native retrievability profile / manifest / custody / challenge surface, canonical bulletin-close extraction-or-abort, cheap omission verification, accountable omission publication	loss of compact frontier consistency, deterministic extraction-or-abort, accountability, or the ordering theorem
Sealed finality	valid asymptote policy, current witness seed, valid witness or observer assignments, checkpoint freshness, valid log consistency proofs, deterministic transcript/challenge derivation, accountable challenge publication	incorrect sealed release, loss of accountability, or fail-closed stalling
Sealed effects	correct finality-tier policy, sound verifier metadata, nullifier uniqueness, intent/public-input binding	replay or unsafe irreversible effect release
Interactive visibility (QUV)	Q-A1–Q-A10 of Section 12.3: unforgeable rooted signatures, direct push to every member, known end-to-end synchrony within the rooted interval, write-before-reply, grow-only non-rollback retained conflict state, fresh per-executor operation before every irreversible effect, root-fixed authority, first-reply admission, reserved correct-member capacity, static faults	two conflicting accepts (if Q-A3 or Q-A5 fails) or typed abort and fail-closed stalling (every other failure)
NestedGuardian	sound witness assignment and registration, correct reassignment handling, witness committee non-equivocation	degraded safety or open liveness behavior

The family-level discipline is simple: if an assumption fails, AFT should degrade to stalling, aborting, or withholding release before it degrades to unsafe irreversible behavior.

15 The Common-Boundary Theorem Corpus (AFT-CB)

The AFT Common-Boundary research program (AFT-CB) restated the strong-tier settlement story as a theorem corpus with a single machine-checked discipline: every theorem carries an explicit *Assumes* line drawn from a ten-entry assumption ledger, every positive theorem is paired with the lower bound that makes its trade forced, and every claim any AFT surface prints is gated by checkers that fail the build on an unconditional or over-rounded assertion. The full paper proofs live in the corpus document (`specs/common_boundary_theorems.md`); their mechanized successors are the TLAPS/TLC kernels of Appendix A; the ledger's single owner is the public whitepaper's research-candidate section, mirrored here; and the claim surface is bound by the program's claim adjudication (`specs/p4_claim_adjudication.md`). This section is the citation spine the rest of the paper hangs from.

15.1 The assumption ledger (A1–A10)

Every deterministic statement in the corpus consumes only entries from this ledger, named on its own *Assumes* line. Machine discipline (enforced in CI): no safety theorem may cite A5 (synchrony is confined to liveness); A9 appears only where the physical clock is genuinely consumed; A10 belongs to the design-open succession extension alone.

Id	Assumption
A1	Signature unforgeability and hash collision resistance.
A2	Minimal Honesty Axiom (MHA): at least one honest, non-equivocating signer per publicly committed signer configuration. Economically bought (bonds, diversity floors, activation queues), never proven, and stated so.
A3	Honest single-final-ack discipline over crash-consistent storage: the journal entry is written in the same atomic step that lets the share exist, and recovery replays bytes without re-deciding.
A4	Reachability to at least one honest signer — for submitters (completeness) and for retrievers (availability's serving path).
A5	The selected liveness environment and bounded dishonest weight. The optimistic live path and seal cadence use post-GST synchrony as a full honest mesh. The hash-only fallback instead uses a static adversary with (f<n/3), reliable private authenticated channels with eventual delivery, and private random inputs. The profiles remain separately labelled; liveness only, and no strong-lineage safety claim cites A5.
A6	A historical-freshness anchor, exactly one deployed, each separately priced: a recent authenticated checkpoint; forward-secure or puncturable signatures with secure key erasure; or an external objective checkpoint. Verifiable-delay elapsed history is a hardening layer over an anchor, never a standalone anchor.
A7	Proof-system soundness (succinct verification only; full-replay verification needs A1 alone).
A8	Per-seal key evolution with verified immediate erasure: no signing-capable material ever persists to a medium that outlives its slot, so later compromise of former members — including full-media imaging — forges nothing (everlasting safety).
A9	Verifiable-delay-function sequentiality with a bounded, published adversary hardware advantage (a physical assumption). The protocol's objective clock wherever ticks are read; safety-critical consumption is confined to the elapsed-history hardening and the design-open succession extension.
A10	Succession-path observation: a bound on the acting successor's view of pre-consented-succession evidence. PROVISIONAL — consumed by the succession extension alone, whose claims are withdrawn to design-open; the uniqueness and lineage ladder never cites it.

15.2 Theorems T1–T12

Statements are condensed; the corpus document carries each full proof, and each entry names its mechanization artifact and its lower-bound pairing. Throughout, a *Unanimous Boundary Close* (UBC) for a configuration C_v and seal slot s is an n-of-n certificate of individually verifiable final-ack shares over a domain-separated tuple.

T1 — Uniqueness within one configuration. *Assumes A1, A2, A3.* At most one UBC exists per (v, s): two closes over different boundary roots imply every member of C_v final-acked both, so zero members are honest ($\neg$A2). The claim is weight-independent and holds under adversarial delivery and scheduling; retries never weaken it because pre-acks are attempt-tagged and can never be reinterpreted as final-ack shares (a wire-format premise, stated as such). *Mechanization:* `BoundaryRing.tla` + TLAPS inductive invariant; TLC at the one-honest-member corner. *Pairing:* L1.

T2 — Completeness (censorship at the seal layer). *Assumes A1, A2, A4.* An artifact received by at least one honest signer before that signer's local cutoff is contained in every boundary that signer final-acks, hence in the unique close. Censoring an artifact out of a sealed boundary requires corrupting all n signers (¬A2) or preventing it from reaching any honest signer (¬A4). Post-close omission from a sealed batch is either carried by a typed, verifiable omission justification or invalidates the seal. *Mechanization:* completeness invariant over declared sets (`BoundaryRing.tla`). *Pairing:* L-E (eclipse).

T3 — Availability (custody-attested). *Assumes A1, A2, A4.* Validate-and-hold is a signing obligation: the final-ack signature *means* the signer obtained the bytes, recomputed the root, and committed to retain and serve. A UBC therefore implies every compliant signer holds; under A2 one signer actually does, so a sealed boundary's bytes are held and served through the horizon with *zero* audit records — the audit lane produces slashing evidence and is proven non-load-bearing. Deletion by the other $n-1$ members is harmless; retrieval additionally needs A4's reach edge. *Mechanization:* `CustodyObligation.tla` (close-time replication at compliant signers; zero-audit witness). *Pairing:* L-H (holder necessity).

T4a — Live-tier liveness. *Assumes A1, A5.* Block production advances under the selected live profile, independently of the ring. The optimistic profile is responsive post-GST at exact (n=3f+1,q=2f+1). After the durable D2 trigger, the normative hash-only fallback has randomized asynchronous progress against a static Byzantine adversary with (f<n/3), reliable private authenticated channels, eventual delivery, and private random inputs. Randomness schedules and selects; rooted PQ votes authorize. No Boundary Ring rule appears in either live path. A stalled ring delays only effects *typed* irreversible (unsealed-over-unsafe), never unrelated ordering. *Mechanization:* the two-tier separation property of `BoundaryLiveness.tla` plus `OptimisticFallbackComposition.tla`; the adverse production/cold-restart gate closes RES-R10. *Pairing:* L-A (FLP randomness necessity and the optimal (f<n/3) resilience boundary) [6, 7].

T4b — Seal cadence. *Assumes A2, A3, A4, A5.* With all n members responsive and honest-behaving post-GST, seals advance at bounded cadence (each pipeline phase completes within a bounded number of message delays, per-slot volume bounded by the deployment parameter). One non-responsive member freezes seal cadence — never the chain (T4a) — until a handover or a labeled re-genesis; no rule substitutes a smaller quorum. *Mechanization:* `BoundaryLiveness.tla`, model-checked at $n \leq 4$ in both the advancement and freeze directions. *Pairing:* L2 (unanimity forces the one-withholder stall).

T5a — Membership canonicity. *Assumes A1, A2, A3.* For lineage segments whose transitions are UBC-closed, two lineages diverging at version v imply full-configuration self-incrimination at the transition slot (T1's argument verbatim), so under per-configuration MHA the configuration lineage from a root is unique. *Mechanization:* `MembershipTransition.tla`. *Pairing:* L1, applied per configuration.

T5b — Bootstrap (validity and freshness, separately). *Assumes A1, A7, A6 (anchor as deployed; A9 iff the elapsed-history hardening is enabled).* A newcomer that verifies the lineage's recursive proof chain from genesis and runs exactly one deployed A6 freshness mechanism obtains validity under A7 (or A1 alone via full replay) and freshness under the deployed mechanism's own assumption, cited by name. A newcomer with no live A6 mechanism is out of model —

stated, not smoothed. The VDF chain is a hardening layer only: tick length evidences work and elapsed time, never *which* history is live. *Mechanization:* bootstrap-freshness obligation (`MembershipTransition.tla`), A6-conditional in the statement itself. *Pairing:* L-LR (long-range indistinguishability, a necessity bound).

T5c′ — Assurance-preserving reconfiguration. *Assumes A1, A2, A3.* Every strong-ring transition carries old-ring unanimity *and* new-ring acceptance, so configuration lineage is MHA-inductive end to end with no weight anywhere in the strong chain. No proof-of-silence path exists: the transition action set contains no silence-derived action — asserted over the formal action set and made unrepresentable in the runtime's types. An anchored re-genesis is a typed lineage root, never continuity. *Mechanization:* `MembershipTransition.tla` (transition obligation + the no-silence action-set assertion); the membership plane's type layer. *Pairing:* L2 and L1.

T5d — Succession adjudication. *Assumes for scheduled safety: A1, A2, A3, A9, and a formation-time clock-fenced lease.* Responsive succession is impossible in the declared asynchronous model: a dead ring and an alive-but-partitioned ring with a hidden conflicting seal have the same successor-visible prefix, so silence or elapsed time cannot safely authorize failover. Scheduled, slot-disjoint succession is safe under the explicit fence and is mechanized in `SuccessionSchedule.tla`; it is not a death-detection, gapless-continuation, or cadence theorem. A10 is needed only for the stronger gapless variant. *Pairing:* L-S (silence cannot prove inaction; fence necessity).

T6 — Composition (the lattice meet). *Assumes A1.* Requirements, verified evidence, and transformations are separate objects. For each coordinate the verifier derives the meet of its load-bearing constituents; PQ uses conjunction and exact scopes survive only on equality. A wrapper may strengthen one coordinate only when a versioned, coordinate-specific verifier establishes new evidence, names the permitting theorem, and commits to its inputs and output. Unknown or unverified rules refuse. The L-M indistinguishability bound shows why: identical constituent certificate bytes force identical certificate-only output, so a wrapper cannot soundly distinguish an execution where the stronger property is false. *Mechanization:* `GuaranteeMeet.tla`, the opaque runtime `VerifiedGuaranteeV1` algebra, exhaustive profile census, and laundering negative corpus. *Pairing:* L-M.

T7 — Forensic accountability (accountable degradation). *Assumes A1.* Proven against the wire format, not an abstraction: above the fault threshold, either no seal exists for a slot, or any party holding both conflicting seals extracts a cryptographically attributable set containing every participant necessary for the violation — for n-of-n UBCs, the entire configuration. The extraction runs on the two certificates alone; the wire-format smallnesses (individually verifiable shares, domain separation, broadcast duty) are premises, and the theorem is false without them. Holder-relativity is stated: an all-Byzantine set can complete a second certificate privately; surfacing is guaranteed by use, honest broadcast, and watchtowers, never by wire-format magic. *Mechanization:* `ForensicAccountability.tla` over the wire-level certificate structure, including the staged all-Byzantine double-seal trace extracting the full set. *Pairing:* L9.

T8 — Selection supply (probabilistic, forever separate). *Assumes: none — an economic model, not a ledger-consuming theorem.* Under a stated model (adversary budget, bond schedule, constituency-correlation structure over the diversity floors, an adaptive-corruption window from sortition unpredictability, and a standby-capture term), the probability that open selection yields at

least one honest signer in a formed configuration is published *as a probability* with all model inputs. The composition rule is absolute: deterministic safety is only ever "conditioned on the selection event," and no surface may convert this probability into a deterministic Byzantine-tolerance figure — the corpus's checkers enforce the ban. *Mechanization:* none (the economics memo owns the published computation). *Pairing:* L-OPEN — the cheapest-capture supply bound is analysis work, recorded open.

T9 — Maximal accountable safety. *Assumes A1.* Any seal-uniqueness violation cryptographically convicts the entire violating coalition — all n signers. The convicted-to-necessary ratio is 1.0, which is maximal: L9 states no protocol can attribute beyond the signers of the conflicting certificates, and T9 meets that ceiling. Attribution alone supplies no monetary floor; T11 separately verifies distinct, live and enforceable collateral. The lineage-escape maneuver (laundering a double-seal through a self-serving re-genesis) is closed upstream by root-admissibility rules, so the maneuver changes the offense's name, not its price. *Mechanization:* `ForensicAccountability.tla` (attribution completeness); share-level verification in the hardened signer. *Pairing:* L9 (the attribution cap).

T10 — Consequence at-most-once externalization. *Assumes A1.* For an accepted `EffectManifestV1` bound to an atomic put-if-absent, compare-and-set, or equivalent external idempotency register, the durable consequence executor causes at most one modeled resource mutation. It flushes `Claimed` and `InFlight` before its only mutation call; every clear or ambiguous result, including restart from a post-call `InFlight` state, advances only through same-key observation to `Reconciled`. The receipt commits intent, request, predecessor, expected outcome, observed outcome, and reconciliation evidence. A contradictory resource record is attributable only when its evidence verifies under the committed resource profile; network ambiguity alone carries no blame. *Mechanization:* `AtMostOnceExternalization.tla` (66 generated, 42 distinct states, depth 8), Rust trace conformance, all-boundary crash injection, duplicate-delivery and forged-evidence tests. *Pairing:* L-X, the ambiguous-response retry dilemma: without endpoint-visible atomic deduplication, retry may duplicate the mutation while no retry may omit it.

T11 — Evidence-qualified distinct collateral floor. *Assumes A1.* Given transferable signed-fault evidence naming a non-empty implicated set, the offline verifier reports exactly the sum in one native asset of distinct collateral lots that are exclusively bound to those members and the implicated configuration, accept the exact evidence predicate under one slashing contract, were locked at the snapshot, remain locked through the challenge horizon, and are neither withdrawing nor encumbered. Duplicate bond or lot identifiers, shared assignments, mixed assets/contracts, expired locks and mismatched claims refuse. Withholding and silence yield no floor. Oracle assumptions remain visible and cannot inflate native units. *Mechanization:* `DistinctCollateralFloor.tla` (33 generated, 8 distinct states, depth 4), arbitrary-precision offline verification, negative attack tests and guard-deletion mutation. *Pairing:* L-C, the indistinguishability bound that no unproved eligibility condition can raise a sound floor. The acquisition/supply bound remains separate and open.

T12 — Portable PQ-channel coverage for one authorized payload. *Assumes A1.* The receipt verifier reconstructs the exact static member set and canonical finality-bundle hash from a PQ-issued hash-asynchronous certificate. It reports `channel_pq=true` only when it verifies exactly one dual-endpoint `PqChannelCompletionEvidenceV1` for every unordered member pair.

Each edge binds the same network, configuration, epoch, channel profile, ML-KEM-768 transcript, derived-key completion, protected-payload hash, and both rooted ML-DSA-44 identities. Missing, duplicate, foreign, singly attested, classically authenticated, or mutated edges refuse. This is a payload-scoped demonstrated-path result, not a claim about historical traffic, delivery, adaptive security, or unobserved sessions. *Mechanization:* the executable complete-graph verifier and its edge/transcript/scope/payload mutation corpus. *Pairing:* L-PQCH: identical consensus bytes can be carried over PQ or classical channels, so independently authenticated channel evidence is necessary.

Portability does not allow a receipt to select its own authority. A production offline decision additionally consumes a separately provisioned `PortableAssuranceTrustV1` that pins the network, configuration, epoch, terminal-key root, allowed receipt signer, anchors, and relying-party guarantee floor. A self-consistent parallel configuration is valid evidence only relative to its own roots and is refused as authority under different pinned roots.

15.3 Lower-bound pairing

Each positive theorem is paired with the necessity bound that makes its trade forced; one row is honestly open. Per the program's claim ladder, no flagship-completeness rung prints while any row is open — the corpus's own checkers enforce the gate (Section 20).

L-S is the responsive-succession indistinguishability bound. A dead original configuration and an alive-but-partitioned original configuration can expose the same finite successor-visible prefix; therefore silence or elapsed time cannot both trigger live failover and prove that no conflicting original seal exists. A formation-time slot fence makes scheduled ranges disjoint rather than detecting death. Removing that fence reaches the conflicting-seal state in `SuccessionSchedule.tla`.

Positive theorem	**Lower bound**	**Status**
T1 uniqueness	L1 (zero-honest equivocation)	cited
T2 completeness	L-E (eclipse)	cited
T3 availability	L-H (holder necessity)	cited
T4a live-tier liveness	L-A (FLP + optimal (f<n/3) asynchronous resilience)	cited; RES-R10 closed
T4b seal cadence	L2 (unanimity forces the one-withholder stall)	cited
T5a membership canonicity	L1 (per configuration)	cited
T5b bootstrap	L-LR (long-range indistinguishability; a necessity bound that forces *some* anchor)	cited
T5c′ reconfiguration	L2 + L1	cited
T5d succession adjudication	L-S (silence cannot prove inaction; fence necessity)	responsive positive claim refuted; scheduled safety mechanized
T6 composition	L-M (above-meet unsoundness)	cited
T7 forensic accountability	L9 (attribution cap)	cited
T8 selection supply	—	L-OPEN (cheapest-capture supply bound)
T9 maximal accountable safety	L9 (ratio 1.0 is the cap)	cited
T10 consequence externalization	L-X (ambiguous-response retry dilemma)	cited; proved and model-checked
T11 distinct collateral floor	L-C (unproved eligibility cannot raise a floor)	cited; proved and model-checked; T8 separate
T12 portable PQ-channel coverage	L-PQCH (transport bytes do not reveal channel properties)	cited; executable complete-graph proof

16 Formal Results

The repository's machine-checked surfaces align with the protocol family as follows:

Formal module	Core proved results
`GuardianMajorityProof`	`QuorumCertificatesIntersect`, `InvariantImpliesSafety`
`GuardianMajority`	executable bounded checks for finalization witness soundness and slot safety `InvariantImpliesCertifiedUniqueness`, `InvariantImpliesRecoverability`,
`CanonicalOrderingProof`	`InvariantImpliesOmissionDominates`
`CanonicalOrdering`	executable bounded checks for publication, cutoff, certification, omission, and recoverability `InvariantImpliesBaseSafety`, `InvariantImpliesCanonicalCloseSafety`, `InvariantImpliesCloseAbortExclusive`,
`AsymptoteProof`	`InvariantImpliesAbortDominatesSealed`
`Asymptote`	executable bounded checks for transcript publication, challenge publication, canonical close, and canonical abort witness-assignment soundness, witness-certificates-stay-assigned,
`NestedGuardianProof`	`InvariantImpliesSafety`
`NestedGuardian`	executable bounded checks for reassignment, outage, and checkpoint admissibility
`BoundaryRingProof`	T1 uniqueness via a TLAPS inductive invariant over an unconstrained Byzantine adversary; no admitted-equals-canonical definition anywhere in the module
`BoundaryRing`	TLC exploration at the one-honest-member corner ($n = 3$) and at $n = 4$
`BoundaryLiveness`	two-tier separation (live progress under a ring stall) and seal-cadence advancement plus the freeze direction, model-checked at $n \leq 4$
`MembershipTransitionProof`	T5a lineage canonicity, the T5c′ transition obligation, the no-silence action-set assertion, and typed lineage roots
`CustodyObligationProof`	T3 validate-and-hold: close-time replication at compliant signers, the zero-audit witness, and reconstruct-before-release
`ForensicAccountabilityProof`	T7 against the wire-level certificate structure, including the staged all-Byzantine double-seal extraction
`SuccessionClockProof`	the honestly narrowed succession kernel (T5d itself withdrawn; the kernel header states the narrowing)
`BoundaryRingTraceReference`	the trace-conformance lane: a committed reference trace replays against `BoundaryRing` with deadlock detection as the divergence signal, on every build
`AtMostOnceExternalization`	T10 durable claim-before-call, at-most-one invocation/mutation, crash-to-unknown, and lookup-only reconciliation

Every formal artifact named in this section, and every additional proof artifact cited by file name below, is embedded verbatim in Appendix A so that the served yellow paper remains self-contained outside the repository.

These formal surfaces prove the deterministic safety kernels that the broader classical-agreement bridge builds on. The full promoted all-but-one ($n - 1$ of n) Byzantine agreement sentence — conditional on the AFT model delta — is then completed by the recurring, reduction, totality, and collapse layers discussed above, rather than by these module-local safety kernels in isolation. Compact publication-frontier summaries internalize same-slot and predecessor consistency on the live path, while the protocol-native retrievability plane internalizes closed-bulletin extraction-or-abort as an endogenous part of the realizing architecture rather than a purely algebraic consequence of the authenticated message transcript alone.

The accountable-adversary variant therefore composes these formal kernels with runtime accountability rules rather than replacing them: replay-deduplicated objective fault publication and optional policy-controlled membership updates live in the implementation and policy layer above the proved deterministic core.

16.1 Complete omission-dominance witness artifact

The formal package includes an executable TLC witness module, `internal-docs/architecture/protocols/aft/f`
paired with `internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingC`
The exact module and configuration are embedded in Appendix A.5.1 and Appendix A.5.2; the summary below highlights the executed public-evidence shape that the witness reaches.

Executed witness trace. The module intentionally asks TLC to violate the state predicate `NoOmissionDominanceWitness` so that the checker emits a concrete public evidence trace. The executed trace is:

1. `PublishStep`: `tx1` enters slot 1's bulletin surface.
2. `PublishStep`: `tx2` enters the same bulletin surface.
3. `CloseCutoffStep`: the canonical cutoff closes and the recoverable set becomes $\{tx1, tx2\}$.
4. `CandidateCertifyStep`: a candidate certificate for $\{tx1\}$ appears.
5. `ProveOmissionStep`: an omission proof for $tx2$ against that candidate certificate appears.

The same formal package carries a bounded recursive-continuity model, `internal-docs/architecture/protoco` paired with `internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseR` The exact source of both artifacts is embedded in Appendix A.5.3 and Appendix A.5.4. That model captures the reference `HashPcdV1` continuity carrier directly: deterministic recursive proof steps, predecessor-proof hashing, `CanonicalCollapseExtensionCertificate` carriage, and the rule that non-genesis header admission depends on the anchored predecessor proof relation. The runtime exposes a `SuccinctSp1V1` backend seam for those same recursive public inputs, and that seam is exercised by consensus verification and durable-state gating. The formal model still stops at the reference carrier rather than mechanizing a cryptographic recursive accumulator or ZK backend.

At the final state, the incomplete candidate certificate remains *unadmitted* while the omission proof is present. In the executable model this happens because the admitted object must already equal the canonical set; the trace therefore does not replace the theorem. Its purpose is narrower and useful: it gives a bounded, executed witness of the exact public-evidence shape from which omission dominance follows. Readers can run the trace directly and inspect the six-state witness without reconstructing the paper's argument by hand.

17 Implementation Correspondence

This section is non-normative. It names the exact runtime symbols that carry the protocol objects defined above.

17.1 Core ordering symbols

The core ordering data structures and free functions are public Rust symbols re-exported through `ioi_types::app`. They are the runtime carriers for the abstract ordering objects and verifier rules defined above.

Abstract object	Current runtime symbols
Bulletin surface commitment	`BulletinCommitment`, `BulletinSurfaceEntry`
Compact publication availability binding	`PublicationAvailabilityReceipt`
Compact publication frontier	`PublicationFrontier`
Publication-frontier contradiction	`PublicationFrontierContradiction`
Canonical ordering proof families	`CanonicalOrderProofSystem`, with concrete variants `HashBindingV1` and `CommittedSurfaceV1`
Canonical public-input tuple	`CanonicalOrderPublicInputs`
Live succinct witness payload	`CommittedSurfaceCanonicalOrderProof`
Omission evidence	`OmissionProof`
Order certificate	`CanonicalOrderCertificate`
Canonical ordering function	`canonical_order_tx_hashes`
Publication-frontier hashing	`canonical_publication_availability_receipt_hash`, `canonical_publication_frontier_hash`, `canonical_publication_frontier_contradiction_hash`
Publication-frontier construction / verification	`build_publication_availability_receipt`, `build_publication_frontier`, `verify_publication_frontier`, `verify_publication_frontier_contradiction`
Public-input hashing	`canonical_order_public_inputs`, `canonical_order_public_inputs_hash`
Reference proof construction	`build_reference_canonical_order_proof_bytes`
Live certificate construction	`build_committed_surface_canonical_order_certificate`
Certificate verification	`verify_canonical_order_certificate`

17.2 Guardian and sealing symbols

The guardian, witness, observer, and sealed-effect objects are likewise public Rust symbols re-exported through `ioi_types::app`. They encode the exact evidence surface checked by validators and downstream gateways.

Abstract object	Current runtime symbols
Guardian slot certificate	`GuardianQuorumCertificate`
Witness certificate	`GuardianWitnessCertificate`
Equal-authority observer assignment	`AsymptoteObserverAssignment`
Correlation budget	`AsymptoteObserverCorrelationBudget`
Observer observation request	`AsymptoteObserverObservationRequest`
Observer transcript	`AsymptoteObserverTranscript`
Observer transcript commitment	`AsymptoteObserverTranscriptCommitment`
Observer challenge	`AsymptoteObserverChallenge`, with kinds `MissingTranscript`, `TranscriptMismatch`, `VetoTranscriptPresent`, `ConflictingTranscript`, and `InvalidCanonicalClose`
Observer challenge commitment	`AsymptoteObserverChallengeCommitment`
Canonical observer close	`AsymptoteObserverCanonicalClose`
Canonical observer abort	`AsymptoteObserverCanonicalAbort`
Finality tier	`FinalityTier`, with variants `BaseFinal` and `SealedFinal`
Collapse state	`CollapseState`, with variants `Pending`, `BaseFinal`, `Sealing`, `Abort`, `SealedFinal`, `Escalated`, and `Invalid`
Sealing policy	`AsymptotePolicy`
Observer statement	`AsymptoteObserverStatement`
Observer certificate	`AsymptoteObserverCertificate`
Divergence signal	`DivergenceSignal`, `DivergenceSignalKind`
Sealed finality proof	`SealedFinalityProof`
Observer binding for sealed effects	`sealed_finality_proof_observer_binding`
Proof-carrying effect object	`SealObject`

17.3 Common-boundary and membership symbols

The AFT-CB program's runtime surfaces are public Rust symbols on the same re-export path. They realize the objects of Section 11.

Abstract object	Current runtime symbols
Closed boundary (R1)	`ClosedBoundary`, `BoundaryOmissionJustification`, `close_reference_single_member_boundary`
Availability binding / standing (R2)	`derive_bulletin_availability_binding`, `availability_standing_from_close`, `AvailabilityAuditRecord`
Resolution log (R3)	`AftResolutionRecord`, `AftResolutionKind`, `aft_resolution_log_entry_key`
Assumption lattice (T6)	`CertificateProfile`, `AssumptionLabel`, `assumption_meet`, `FinalityReceipt`
Ring membership plane (R5)	`RingMemberRegistration`, `RingActivationQueue`, `BoundaryRingConfig`, `AssurancePreservingHandover`, `AnchoredRegenesisRoot`, `LiveTierEjection`, `CustodyHandoverReceipt`, `assign_seats`, `WatchtowerCountersignRecord`
Membership registry wiring (R5)	`register_ring_member@v1`, `publish_ring_genesis_config@v1`, `publish_ring_handover@v1`
Hardened seal signer (R9)	`EvolvingSealSigner`, `SealShare`, `verify_seal_share`, `extract_double_signers`
Trace conformance (R13)	`BoundaryRingDriver`, the committed reference trace, and the replay generator in the formal-floor CI job
Succinct continuity (R4c)	`SuccinctDriverConfig::pinned`, the governed verifying-key pin, and the no-mock CI gate

17.4 Verifier, orchestration, and persistence symbols

Verification and orchestration entry points are split across four public namespaces: `ioi_types::app`, `ioi_crypto::sign::guardian_committee`, `ioi_services::guardian_registry`, and the `GuardianMajorityEngine` runtime. Engine-local checks are named here by their exact method identifiers.

Responsibility	Current runtime symbols
Guardian certificate verification	`verify_quorum_certificate`
Witness certificate verification	`verify_witness_certificate`
Deterministic observer assignment	`derive_asymptote_observer_assignments`, `derive_asymptote_observer_plan_entries`
Observer-side transcript / challenge derivation	`GuardianContainer::observe_asymptote_request`, `GuardianContainer::request_remote_asymptote_observer_observation`, `GuardianContainer::sign_consensus_with_guardian`
Sealed finality verification in the engine	`GuardianMajorityEngine::verify_asymptote_canonical_observer_sealed_finality`, `GuardianMajorityEngine::verify_asymptote_sealed_finality`
Canonical-order enrichment verification in the engine	`GuardianMajorityEngine::verify_canonical_order_enrichment`; published `CanonicalOrderAbort` dominates positive order certificates and inconsistent mixed parent state
Publication-frontier enrichment verification in the engine	`GuardianMajorityEngine::verify_publication_frontier_enrichment`; published `PublicationFrontierContradiction` dominates conflicting or stale frontiers
Block finalization attachment of order certificates	`build_committed_surface_canonical_order_certificate` during finalization
Block finalization attachment of publication frontier	`build_publication_frontier` during finalization
Canonical observer proof publication	`canonicalize_observer_sealed_finality_proof`, `publish_canonical_observer_artifacts`
Sealed-effect construction and checking	`build_http_egress_seal_object`, `verify_seal_object`
Guardianized proposal verification in the engine	`GuardianMajorityEngine::verify_guardianized_certificate`
Base-engine safety and timing	`SafetyGadget`, `Pacemaker`, `TimeoutCertificate`
Bulletin and order-certificate state access	`load_bulletin_commitment`, `load_canonical_order_certificate`
Publication-frontier state access	`GuardianRegistry::load_publication_frontier`, `GuardianRegistry::load_publication_frontier_contradiction`
Ordering close-or-abort derivation	`derive_canonical_order_execution_object`, `derive_canonical_order_public_obstruction`
Ordering abort taxonomy	`CanonicalOrderAbortReason::MissingOrderCertificate`, `BulletinSurfaceReconstructionFailure`, `BulletinSurfaceMismatch`, `InvalidBulletinClose`, `OmissionDominated`, `CertificateHeightMismatch`, `RandomnessMismatch`, `OrderedTransactionsRootMismatch`, `ResultingStateRootMismatch`, `InvalidPublicInputsHash`, `InvalidBulletinAvailabilityCertificate`, `InvalidProofBinding`
Protocol-wide collapse derivation and verification	`derive_canonical_collapse_object`, `GuardianMajorityEngine::verify_published_canonical_collapse_object`, `BlockHeader::previous_canonical_collapse_commitment_hash`, `BlockHeader::canonical_collapse_extension_certificate`, `CanonicalCollapseExtensionCertificate`, `CanonicalCollapseCommitment`, `verify_block_header_canonical_collapse_evidence`,
Proposal-surface recursive continuity	`ConsensusDecision::ProduceBlock`
Bulletin, order, and collapse state access	`load_bulletin_commitment`, `load_canonical_order_certificate`, `load_canonical_order_abort`, `load_canonical_collapse_object`
	`publish_aft_publication_frontier@v1`, `publish_aft_canonical_order_artifact_bundle@v1`, `publish_aft_canonical_order_abort@v1`, `publish_aft_canonical_collapse_object@v1`, `report_aft_omission@v1`, `publish_asymptote_observer_transcript@v1`, `publish_asymptote_observer_transcript_commitment@v1`, `report_asymptote_observer_challenge@v1`, `publish_asymptote_observer_challenge_commitment@v1`, `publish_asymptote_observer_canonical_close@v1`,

These symbols are named here so that the paper maps directly onto the current runtime without making the reader chase separate prose specifications. In the implementation, validator finalization publishes a protocol-visible `CanonicalCollapseObject` through `publish_aft_canonical_collapse_object@v1` after local post-commit header enrichment, `guardian_registry` exposes `load_canonical_collapse_object`, and `GuardianMajorityEngine::verify_publi` cross-checks any published copy against the proof-carried header surface when it is available. The externally finalized AFT commit path persists the same object keyed by slot height, rather than treating ordering and sealing close-or-abort evidence as loose enrichments detached from state persistence. The same post-commit path also builds `PublicationFrontier` via `build_publication_frontier` and publishes it through `publish_aft_publication_frontier@v1`; `GuardianMajorityEngine::verify_publication_frontier_enrichment` rechecks same-slot binding, predecessor linkage, and any dominating contradiction object. In the current proposal-surface continuity slice, validator orchestration now signs both `previous_canonical_collapse_commitment_hash` and a typed `CanonicalCollapseExtensionCertificate` into each produced AFT block header. In the current clean-break runtime slice, `CanonicalCollapseObject` itself carries `continuity_accumulator_hash` and `continuity_recursive_proof`, so the certificate now carries the predecessor commitment plus predecessor proof hash rather than an explicit carried ancestor vector, full predecessor collapse object, or predecessor recursive proof object. `Asymptote` leaders stall rather than produce when that required extension certificate is unavailable, proposal verification recursively checks that the carried predecessor commitment hash matches the signed predecessor link, checks that the predecessor commitment's resulting state root matches the signed parent-state root, and rechecks those carried public inputs against the anchored predecessor collapse object when one is available. QC promotion therefore relies on a reference recursive proof-carrying continuity relation rather than on a recomputed predecessor digest alone, and only treats continuity-linked headers as progress-bearing.

17.5 Interactive visibility (QUV) symbols

Responsibility	Current runtime symbols
Policy root and profile identifiers	`POLICY_ROOT_DOMAIN_V0` = `ioi/aft/quv-policy/v8-push-admission`, `quv_policy_root`, `AftQuvDomainPolicyV0`, `QuvPushAdmissionPolicyV0`, `QuvPreparationPolicyV0`
Canonical conflict identity and expected head	`QuvConflictSlotV0`, `quv_candidate_hash`, `check_expected_slot`, `process_push_with_byte_limit`
Member write-before-reply and durable conflict state	`DurableQuvMemberV0` (schema 9, `AFTQJ001` / `AFTQJA01`), `MemberDelta::{InsertCandidate, ReservePreparation, AcceptHead}`
Executor operation and deadline finalization	`execute_aft_quv_effect`, `handle_execute_aft_quv_effect`, `reserve_online_authorization`, `finalize_operation_at_deadline`, `finish_active_service`, `QuvOperationAdmissionV0`
Accepted audit and non-portable receipt	`QuvAcceptedAuditEvidenceV0`, `QuvOnlineAuthorizationV0`, `ConsequenceReceiptV1` with `portable_final_receipt = false`, envelope `AFTCR001`
Claim-before-call consequence path	`agentgres::consequence` claim index and receipt reservation, `ImmediateOnlineEffectAuthorizationV1::consume`
Live handoff and successor activation	`DurableQuvHandoffV0` (schema 3), `validate_quv_handoff_candidate`, `authorize_and_install_handoff`, `activate_installed_handoff`, `select_aft_pq_startup_root`, `select_aft_pq_local_role`, `quv_successor_root_gate`, `QUV_RETIRED_SIGNER_REFUSAL`
Strict PQ carrier and reserved outbox	`PqChannelManager`, `QueueQuvPushQuery`, `QueueQuvReply`, `CompleteQuvPush`, outbox envelopes `AFTPQI04` / `AFTPQI05` / `AFTPQA01`
Qualification runner and evidence checkers	`run_aft_m16q_qualification.sh`, `run_aft_formal_checks.sh`, `check_aft_m16q_process_evidence.py`, `check_aft_quv_readiness_evidence.py`, `check_aft_quv_handoff_evidence.py`, `freeze_aft_quv_candidate.sh`

18 Benchmark Context

The performance evidence in this paper has two sources:

- literature-reported baselines for `HotStuff`, `Narwhal/Tusk`, and `Bullshark`,
- the repository's native `benchmark_throughput` target for matched `GuardianMajority` and `Asymptote` scenarios.

The purpose of this section is architectural positioning, not false equivalence across unmatched environments.

18.1 Paper-reported baselines

System	Reported throughput	Reported latency	Source type
HotStuff baseline	1,800 tx/s	1 s	Narwhal/Tusk comparison setting
Narwhal-HotStuff	130,000 tx/s	under 2 s	Narwhal/Tusk paper
Narwhal-HotStuff with additional workers	up to 600,000 tx/s	n/a	Narwhal/Tusk paper
Tusk	160,000 tx/s	about 3 s	Narwhal/Tusk paper
Bullshark	125,000 tx/s	about 2 s	Bullshark paper

The baseline interpretation is:

- `HotStuff` demonstrates the leader-based partially synchronous baseline,
- `Narwhal-HotStuff` demonstrates the gain from separating dissemination from ordering,
- `Bullshark` demonstrates high-throughput DAG-oriented ordering,
- AFT adds a new separation: order surfacing and effect release are no longer identified with dense positive voting.

18.2 Native repository measurements

No performance measurement in this section is load-bearing for any theorem in this paper; the promoted theorem surface stands independently of benchmark tuning, harness state, or optimization maturity.

The corrected native measurements below were produced on March 21, 2026 after bringing the repository's native AFT benchmark path back into sync with the current `GuardianMajority` / `base_final` harness semantics:

- shared-tip readiness no longer treats a transient `get_status.height = 0` as authoritative when a resilient tip probe already sees committed blocks,
- submission alignment and leader-targeted ingress now use a resilient tip block rather than a potentially stale status height,
- authoritative chain-commit scans start from the resilient pre-submission tip instead of rescanning stale empty-block prefixes,
- sustained throughput for fast probes now stops on the authoritative full-chain commit scan for the highest committed height; sampled committed-status visibility is retained only as an auxiliary lag diagnostic,
- the benchmark surface now exposes alignment, spill, and replay diagnostics explicitly so missed due blocks or replay churn are not hidden behind a single TPS number.

The table below reports the current matched bulk-throughput proof point for the native `GuardianMajority` 4-validator `base_final` scenario under that corrected harness. This is a repository-native AFT proof point on the Jellyfish state path with a ‘1 s‘ block interval, not an apples-to-apples claim against published Narwhal/Tusk/Bullshark environments.

scenario	validators	attempted	accepted	committed	blocks	inj. tps	sust. tps	p50	p95	p99	sealed p50	sealed p95
guardian_majority_4v_base_final_bulk	4	16384	16384	16384	1	21370.89	9647.66	1293.45	1603.73	1695.24	–	–

These measurements imply four things:

- the earlier single-digit and low-hundreds TPS reports were artifacts of benchmark-path bugs rather than of the protocol surface itself,
- the current repository now clears the lower edge of the intended `8k-16k TPS` range in a matched native 4-validator `GuardianMajority` bulk probe,
- on this corrected probe the batch lands in a single block, so the main remaining AFT work is ceiling-raising and aggregate-domain multiplication rather than recovering from a broken harness,
- these values should be read as corrected repository baselines, not as optimized ceilings and not as apples-to-apples superiority claims over published `Narwhal/Tusk/Bullshark` environments.

18.3 Measured costs, reproduction-classed

The AFT-CB program's measured-cost discipline: every number carries its reproduction class, and a number without a reproduction path is not printed.

Class A — exact, zero variance (gate-pinned). The SCALE-encoded byte sizes of the canonical objects, pinned by a repository test so any wire-format change surfaces in review:

Quantity	Bytes	Reading
Validate-and-hold binding	72	per sealed slot, ring-size independent
Canonical bulletin close	180	the sealed-slot commitment
Optional audit record	106	strictly marginal: a close verifies with zero audit records, so the at-rest audit cost is zero and 106 B is paid only per recorded probe

Class B — reproduces within CPU variance. The real succinct continuity backend proves one recursion step in roughly 27 seconds of CPU (about 84 seconds wall for the three-step end-to-end chain, CPU backend), reproduced by the repository's nightly proof lane. GPU or prover-network backends would shift these by an order of magnitude and are out of scope for an in-repository measurement.

Class C — measured-container, variance-caveated. Two separately authorized campaigns ran the complete `res-p4.3.v2` matrix on the same exact Akash provider and immutable image. Each discarded one full-matrix warmup and retained five measured passes with ten scenario/lane rows. Their environment manifests match: AMD EPYC 7352, 48 online CPUs, 131753256 KiB memory, Linux 6.8.0-124-generic, `x86_64`, and `schedutil`. The runtime supplied no provider host attestation, so the honesty class is a measured container on one exact audited provider allocation; physical-host placement is unproven.

The table reports campaign-N / campaign-O medians. TPS columns are transactions per second and latency columns are milliseconds.

scenario / lane	inj. TPS	sust. TPS	p50	p95	p99	max	verdict
asymptote_4v / base_final	23112.38 / 20072.61	321.83 / 322.81	1583.16 / 1556.07	1589.41 / 1583.42	1589.94 / 1585.05	1590.11 / 1585.07	caveated
asymptote_4v / canonical_ordering	22156.16 / 17828.87	324.38 / 326.17	1555.21 / 1548.14	1576.13 / 1567.84	1577.47 / 1568.48	1577.70 / 1568.82	caveated
asymptote_4v / durable_collapse	18596.36 / 21552.59	322.01 / 322.63	1540.99 / 1581.05	1583.51 / 1585.45	1589.04 / 1586.13	1589.50 / 1586.26	caveated
asymptote_4v / sealed_final	20248.78 / 19319.42	325.75 / 315.97	1556.70 / 1561.24	1570.01 / 1618.40	1570.42 / 1619.53	1570.92 / 1619.82	within
asymptote_7v / base_final	11799.95 / 26761.25	338.23 / 344.18	2241.47 / 2185.40	2266.60 / 2214.95	2268.63 / 2229.23	2269.23 / 2229.58	caveated
asymptote_7v / canonical_ordering	11387.74 / 31215.72	342.15 / 341.02	2224.57 / 2238.23	2240.87 / 2248.16	2242.20 / 2249.77	2243.71 / 2250.00	caveated
asymptote_7v / durable_collapse	25359.49 / 32596.58	343.97 / 342.35	2221.34 / 2211.22	2229.33 / 2239.72	2231.37 / 2241.34	2231.53 / 2241.73	caveated
asymptote_7v / sealed_final	28148.01 / 32207.55	337.85 / 342.19	2212.87 / 2232.53	2269.57 / 2241.25	2271.57 / 2242.89	2272.29 / 2243.35	caveated
guardian_majority_4v / base_final	9119.25 / 9421.75	332.00 / 329.50	1536.53 / 1549.81	1540.12 / 1552.24	1541.05 / 1552.85	1541.35 / 1553.08	within
guardian_majority_7v / base_final	29314.23 / 30175.52	342.84 / 339.08	2194.89 / 2213.13	2201.76 / 2262.40	2238.58 / 2263.98	2239.15 / 2264.00	within

The predeclared thresholds were 10% for injection TPS, sustained TPS, p50, and p95, and 15% for p99 and max. Every sustained-TPS and commit-latency comparison passed. Injection TPS exceeded 10% in seven rows, so the aggregate is variance-caveated; no row was discarded and no threshold was changed after measurement. Both leases closed and financially reconciled with zero campaign-scoped open or unknown exposure. The full per-pass values and variance envelopes remain bound in the campaign certificates. This closes the live RES-P4.3 measurement loop with an honest caveat, not a promoted reproduced-within-threshold claim and not a bare-metal claim.

These numbers did not enter this table by editorial insertion. Both campaigns were executed under the estate's governed authority contract — model-proposed, wallet-authorized, VM-isolated, with a durable intent commitment before the provider effect and an outcome commitment after

provider-native evidence. The retained campaign bundle was then evaluated by a separate Rust relying-party verifier with its own typed evidence model, no shared certificate code generation, and filesystem-only portable input; it re-derives the summary statistics and variance verdicts from the raw five-pass samples, rejects forty fully resealed semantic mutations of the bundle, and admits a conforming bundle only through a named acceptance policy. The measured-results registry transitioned from revision 0 to revision 1 through that policy gate, with the accepted row bound to its certificate, environment, immutable image, exact provider, honesty class, and `variance_caveated` verdict. The verifier is operationally separate, not organizationally independent — IOI maintains both producer and verifier — so this is a first-party relying-party admission, not third-party validation. Promotion of a reproduced-within-threshold row remains a separate future registry transition, never a table edit.

19 Related Work

Geeq's Proof of Honesty (PoH) is important prior work in the "99%" (their figure, not an AFT claim) direction: it argues for globally honest and canonical chain recovery in pure software under assumptions including at least one honest node and an honest relay, and it uses public evidence plus user-side verification to push beyond ordinary dense-vote intuition [11]. AFT differs in a key respect: it is relay-free and coordinator-free. Correctness does not depend on any designated honest relay, broadcaster, or privileged recovery role. Instead, durable ordering and sealed effects collapse through protocol-native public-evidence objects whose negative forms are immediately decisive.

Narwhal/Tusk and Bullshark separate dissemination from ordering, but they still ground ordering safety in dense positive certificates and quorum-intersection arguments over the ordered object itself [2, 3]. HoneyBadgerBFT and Dumbo move to asynchronous common-subset or MVBA-style agreement, but they still rely on classical Byzantine-majority thresholds rather than a public-state-continuity theorem over a canonical public surface [4, 5]. Chainlink CCIP and optimistic cross-chain mechanisms add secondary monitoring layers, risk-management committees, or challenge windows to detect and halt bad transfers, but the decisive recovery rule is operational or economic rather than the immediate acceptance of a uniquely valid proof-carrying canonical object recoverable by any equal-authority validator [10]. Accountable subgroup multisigs and finality gadgets add equivocation evidence and slashing, yet still keep dense subgroup votes as the positive safety object [8, 9]. AFT differs by making ordering and sealed release relay-free public-evidence close-or-abort objects with dominant negative proofs: honest validators that observe the same closed public boundary derive the same decisive object, and publication is downstream of that deterministic derivation rather than its source.

20 Claims and Non-Claims

20.1 The headline claim

The claim this paper prints, adopted verbatim from the AFT-CB program's claim adjudication and machine-gated in CI (no stronger sentence can enter this corpus without failing the build):

> **Deterministic all-but-one safety for certified boundaries; live consensus under separately bounded adversarial weight and eventual synchrony.**

Both halves state their own conditions. The first half is T1 (Section 15.2): weight-independent, mechanized, and unconditional under the Minimal Honesty Axiom — which is economically bought,

never proven, and priced rather than assumed away. The second half is T4a, and it *names* its two conditions (bounded adversarial weight; eventual synchrony), which is exactly why the named asynchronous residual does not undercut it: the claim never asserted asynchronous liveness. Stronger "completeness"-class rungs exist in the program's claim ladder and are BLOCKED — they do not print here or anywhere, and the enumerated open gates live in the adjudication document.

20.2 The maximality claims

Two claims in this paper are superlatives, and both are theorems rather than comparisons — each is a cited positive-theorem/lower-bound pair in the corpus (Section 15.3), so the ceiling and the achievement print together:

- **Safety at the terminal threshold (T1 paired with L1).** With zero honest signers, two conflicting n-of-n certificates are constructible by any adversary in any model — equivocation is unpreventable, so *no consensus protocol can claim a stronger safety threshold than all-but-one.* Within the model delta, AFT's certified-boundary safety meets that terminal threshold exactly, weight-independently, under A1–A3 alone, with the mechanized kernel of Appendix A.7.2 as the machine-checked proof. On the safety axis there is nothing above this point to claim; the axis ends here.
- **Accountability at the attribution ceiling (T9 paired with L9).** No protocol can attribute a violation beyond the signers of the conflicting certificates — ratio 1.0 of convicted to necessary is the cap. AFT meets the cap: a seal-uniqueness violation convicts the entire configuration, member by member, from the two certificates alone, with slashable floor $n \times$ bond. Safety violations are not merely detected; they are maximally expensive and maximally attributed, by construction.

The classical comparison, stated precisely: no protocol in the classical carrier exceeds $f < n/3$ for agreement, because the indefinite-suppression move is legal there. AFT does not out-tune that family inside its own model — it replaces the carrier, which makes the suppression move illegal, and in the replaced carrier the safety threshold ascends from a fraction to the information-theoretic maximum. Prior work has gestured at this territory with rounded prose figures; this paper's form of the claim is exact ($n - 1$ of n, never a rounded percentage), mechanized (TLAPS, with the trace lane holding the runtime to the kernel on every build), and attributed (every violator convicted individually). The liveness price and the residual list (Section 20.5) print beside these claims and are part of them.

The same refusal discipline governs measurement. Every performance number in this paper carries a named reproduction class, and entry into the measured-results registry is a policy-gated relying-party state transition — a producer cannot obtain an accepted row by editing a table, recomputing an outer hash, or weakening a verdict (Section 18.3). A paper that claims the two provable ceilings can afford to be exact about everything beneath them; the claim ladder, the machine gate, and the registry contract are that exactness, mechanized.

20.3 Claims

This paper additionally claims:

- a production guardian-backed base-finality model in which validator quorums are composed with guardian committee admissibility,

- proof-carrying canonical ordering with omission dominance and uniqueness, plus protocol-native endogenous retrievability,
- boundary-honest ordering: the committed-surface certificate consumes a previously closed boundary, carries typed omission justifications, and its verification is self-contained — an unjustified omission is an invalid block, so censorship is observable by construction,
- compact signed publication-frontier summaries and short contradiction objects that internalize same-slot and predecessor consistency on the live path,
- a public-state-continuity formulation in which closed slot boundaries admit at most one positive public execution object and conflicting candidates admit short objective obstruction witnesses,
- all-but-one ($n-1$ of n) equal-authority ordering consensus in the AFT model delta, under explicit assumptions,
- deterministic sealed-effect collapse with challenge dominance, with sealed closes immutable and post-close evidence confined to an append-only resolution log,
- a relay-free, coordinator-free, pure-software model-delta theorem yielding deterministic all-but-one ($n-1$ of n) Byzantine safety for durable ordering and sealed-effect safety over the explicit bulletin / retrievability surface, conditional on the AFT model delta,
- that the whole AFT stack breaks the classical indefinite-suppression lower bound, in the AFT model delta, by identifying the first illegal adversary step and realizing the visibility delta without a trusted relay — the classical threshold is not out-tuned but made moot by replacing its carrier,
- safety at the terminal threshold: no protocol in any model can exceed all-but-one for safety (L1), and AFT meets that ceiling weight-independently with a mechanized proof (Section 20.2),
- accountability at the attribution ceiling: convicted-to-necessary ratio 1.0 (T9/L9), every violator individually, from the two certificates alone,
- the common-boundary theorem corpus of Section 15: T1–T12 with explicit Assumes lines over the A1–A10 ledger, each theorem paired with its necessity bound, with the mechanized kernels of Appendix A as their machine-checked successors and a trace-conformance lane holding code to the uniqueness kernel on every build,
- forensic accountability at the attribution ceiling: a seal-uniqueness violation convicts the entire configuration individually from the two certificates alone (T7/T9), with the wire discipline implemented in the hardened signer,
- a witness-augmented mode for layered threshold constructions,
- machine-checked deterministic safety kernels for the major protocol surfaces.
- at the irreversible edge, the interactive-visibility result of Section 12: one reachable correct configured member out of n suffices for online conflict-qualified accepted-value non-conflict and no-conflict singleton progress when every relying executor performs fresh push/write-before-reply verification against every configured member within a rooted known-synchronous end-to-end deadline, and correct-member conflict state is atomic, durable, monotone, correctly scoped, and non-rollback (Q-T1–Q-T4, Q-E1–Q-E4), paired with the portable-visibility lower bound (Theorem 6) that makes the interactive form necessary at $f = n - 1$,

20.4 Non-claims

This paper does **not** claim:

- that public state continuity arises for free inside the classical DLS / PBFT partially synchronous message-passing model,
- that the classical $n > 3f$ / $f < n/3$ lower bound has been refuted inside that bare model; the claim is instead that AFT operates in the explicit M_{AFT} model delta where the adversary cannot indefinitely suppress published admissible artifacts from correct parties after stabilization. Remove that visibility resource, or realize it circularly through the same Byzantine-majority relay set, and the paper is no longer claiming the same result. This is not a retreat; it is the exact location of the break. AFT does not merely vary a parameter inside the dense-vote carrier. It replaces the carrier with direct dissemination plus compressed public echo and states the new proof obligation at theorem surface,
- freedom from bulletin-publication, protocol-native retrievability obligations, or proof-soundness assumptions,
- that compact publication-frontier summaries make the bulletin surface self-extracting inside the classical authenticated-message model,
- a full asynchronous liveness theorem under arbitrary registry, log, witness, and outage schedules,
- for the interactive-visibility result: portable or offline finality; asynchronous or eventually-synchronous safety before an independently established bound is active; exact-decision Byzantine agreement or classical Byzantine consensus among the members; a transferable finality certificate; inclusion or effect liveness for any input that is refused, aborted, vetoed, frozen, or equivocated; fairness among competing valid submissions; or tolerance of a mobile adversary (Section 12.9),
- an apples-to-apples measured throughput superiority over `HotStuff`, `Narwhal`, `Tusk`, or `Bullshark` from the current repository alone.

20.5 Named residuals (the limitations list)

The AFT-CB program closed complete-with-residuals, and this paper carries the residual list verbatim rather than smoothing it. None of these is a silent gap; each names its closing condition in the program record.

- **The MHA is bought, not proven.** T1's safety is conditional on at least one honest signer per configuration; the economics memo prices how open selection *supplies* that condition (T8, as a probability only), and no surface converts that price into a deterministic tolerance figure.
- **RES-R10 — closed asynchronous-live-tier residual.** The normative hash-only fallback now supplies randomized asynchronous progress for a static adversary with (f<n/3) over reliable private authenticated channels, without private threshold setup or DKG. D1–D4, the load-bearing mutation checks, exact (n=130) benchmark, same-height production race, cold restart, and native PQ re-entry are complete. No adaptive-security or bounded latency claim follows. T8 remains the open lower-bound row; responsive T5d is separately refuted and paired with L-S.

- **RES-R11 — the VDF clock plane.** Every candidate construction requires externally vetted cryptography (class-group, isogeny, or checkpointed-hash approaches each fail a stated requirement today); selection and review are an owner action. Sortition's beacon is the reference ordering beacon until this closes — an honest label, not a security bound.

- **RES-R12 and the T5d resolution — succession.** Three responsive succession formulations were refuted, and L-S closes the question: silence cannot prove future inaction in the declared asynchronous model. Scheduled slot-disjoint succession is safe under a formation-time clock-fenced lease and is mechanized, but no production path activates it and it supplies no responsive cadence theorem. The operative exits from a dead ring remain the unanimous handover and labeled re-genesis.

- **RES-P4.3-throughput.** The live measurement loop is closed with an honest caveat, not a promoted claim: two separately authorized campaigns on the same exact audited provider allocation and immutable image reproduced every sustained-TPS and commit-latency comparison within the predeclared thresholds, while injection TPS exceeded its threshold in seven of ten rows, so the printed Class C figures are `variance_caveated` (Section 18.3) and were admitted into the measured-results registry under exactly that verdict. What remains open is strictly stronger: a reproduced-within-threshold promotion for injection TPS, and any measurement whose honesty class exceeds a measured container — the runtime supplied no provider host attestation, so physical-host placement is unproven.

- **R14 — the verified verifier.** A spec-extracted or refinement-proven UBC verifier remains a design-permitted residual; its outward complement (an independent twin implementation against exported conformance vectors) is filed as an owner packet.

- **RES-QUV — the interactive-visibility candidate under review.** The construction, proofs, and production profile of Section 12 are frozen as one immutable candidate whose sixty-three-gate qualification passed; the fresh independent M17Q review of that exact commit is the admission gate, and the packet admits only the sentence printed in the claims list with its assumptions and costs. Two measured transport costs are carried as deployment obligations rather than guarantees: replies share the single in-flight PQ lane per peer with ordering traffic and preparation pushes (a 4210 ms valid reply was observed inside the 5000 ms interval), and the first operation after a process start delivers late. Neither is attributed to a specific record at the retained logging level, and neither is a theorem premise.

20.6 Model-delta public-state-continuity theorem

The runtime supports a stronger theorem than the older accountable summary because the mechanism and system model are now stated directly at theorem surface.

Theorem 15 (Relay-Free Deterministic All-But-One ($n-1$ of n) Byzantine Safety in M_{AFT})**.** *Assume the system model of Section 5, including the direct-dissemination / compressed-echo delta of Lemma 1, guardian non-equivocation, proof soundness, compact signed publication-frontier summaries whose same-slot and predecessor-link contradictions are objectively checkable, protocol-native retrievability objects plus canonical bulletin-close extraction-or-abort for the bulletin surface bound by those frontiers, deterministic transcript/challenge derivation for sealing, durable state advancement only through canonical collapse, and irreversible-effect release only through collapse-bound sealed proofs. Then all but one validator ($n-1$ of n) may behave arbitrarily without creating a conflicting valid durable canonical-order outcome or a conflicting valid sealed-effect release outcome. Honest validators*

that observe the same closed public boundary derive the same close-or-abort result; publication, penalties, and validator-set updates only accelerate convergence and future hygiene. The classical $n > 3f$ / $f < n/3$ threshold is not load-bearing in M_{AFT}, because the classical adversary's indefinite-suppression step is no longer legal.

The theorem does not claim that DLS / PBFT has been refuted inside M_{DLS}. It claims that AFT is not a member of that model: the first illegal lower-bound adversary step is indefinite suppression of a published admissible artifact from correct parties after stabilization. Bulletin close, publication frontier, retrievability, recovery, restart continuity, canonical collapse, and collapse-bound sealing are not theorem-side caveats; they are the protocol-internal realizing mechanism of M_{AFT}. A criticism that AFT "changes the model" is therefore descriptive rather than fatal unless it identifies circular realization of the visibility resource or a hidden relay, coordinator, TEE, bootstrap oracle, or external recovery service.

The theorem is stronger than the older accountable summary because invalid positive objects are not merely rejectable or penalty-bearing. They are already dominated by deterministic negative objects and collapse-gated durable state before any penalty logic runs. Readers trained on dense-vote BFT will be tempted to treat the absence of quorum-intersection safety as disqualifying. In AFT, that is precisely the point: authority is carried by uniquely valid public execution objects and omission-dominant contradiction witnesses, with durable advancement gated by canonical collapse. Compact publication frontiers keep the live hot path cheap by internalizing consistency and lineage checks, while the protocol-native retrievability plane discharges deterministic extraction-or-abort on the full bulletin surface.

The recurrence / reduction / totality / collapse chain is not narrative glue. Its exact artifact chain is embedded verbatim in Appendix A, ending in the classical-agreement collapse wrapper itself.

Theorem 16 (Realizing-Mechanism Statement)**.** *The new publication-frontier slice is not the theorem by itself; the theorem is the promoted AFT model-delta sentence, carried by the full AFT public-evidence continuity architecture. Compact signed publication-frontier summaries, compact availability receipts, short frontier contradiction objects, endogenous retrievability profiles, manifests, custody assignments, custody receipts, custody responses, retrievability challenges, canonical extraction-or-abort, canonical collapse, and historical retrievability together realize the direct-dissemination / compressed-echo visibility resource stated above. Removing that protocol-native retrievability plane removes part of the realizing mechanism rather than merely shaving off an optional refinement.*

Proposition 2 (Scoped Witness-Coded Constructive Corollary)**.** *Assume the hypotheses of the relay-free deterministic all-but-one ($n - 1$ of n) Byzantine safety theorem in M_{AFT}, plus sound witness assignment and witness certificate binding, a completed witness-coded recovery family with compact hot-path bindings and cold-path share delivery / reveal / reconstruction, and availability of the retained recovered publication surface together with any older recovered-history pages needed for restart-time extraction. Then the* `NestedGuardian` *witness-coded carrier deterministically materializes the same positive close or canonical abort surface, the same authoritative collapse chain, the same replay-prefix / restart-anchor surface, and the same restart-time canonical-header / certified-header / block-header ancestry surface as the ordinary lane for the completed recovery families. Restart can therefore stream recovered certified ancestry until target height or recovered-history exhaustion without a theorem-relevant fixed depth bound, while keeping only compact bindings on the hot path and bounded-memory paging on the restart path.*

This runtime result is stronger than the separation closeout because it no longer stops at "compact frontiers bind a recoverability claim." It shows that the live AFT stack can re-materialize

the same durable and restart-critical surfaces without violating the throughput guardrails, while keeping deeper recovered history inside ordinary endogenous AFT state.

This is one constructive route to Byzantine agreement in a realizable permissioned model delta rather than a weaker contrast class. In the classical dense-vote presentation, worst-case progress is usually phrased in terms of $> 2/3$ honest participation and quorum intersection as the main safety mechanism. In AFT's realization, authority is obtained instead through uniquely valid public-evidence objects plus dominant negative proofs, without trusted relays, privileged coordinators, TEEs, or dense positive quorum intersection. The live publication-frontier slice internalizes compact same-slot and predecessor consistency on the hot path, while explicit recoverability of the bound bulletin surface, canonical collapse, and historical retrievability complete the same AFT model-delta theorem. Publication and accountability serve as reinforcement rather than the source of correctness. Liveness then follows from direct authenticated dissemination, compressed echo visibility, and eventual publication of the protocol-defined artifacts, and the recurrence/reduction/totality/collapse chain closes that route formally. That is the precise sense in which the classical $n > 3f$ / $f < n/3$ barrier stops being load-bearing here: the promoted theorem no longer grants the adversary indefinite suppression of published admissible information.

The correct one-line summary is:

> AFT is a proof-carrying, guardian-backed, omission-dominant consensus and settlement family in which base finality, canonical ordering, and irreversible-effect release are separated and then recomposed through shared evidence.

A Embedded Formal Artifacts

This appendix embeds verbatim every proof artifact referenced by name in the yellow paper, together with the repository path from which it is sourced. The goal is to keep the served paper self-contained even when it is read outside the repository. The artifacts are grouped by protocol surface and rendered with line numbers so the appendix reads as a standalone proof bundle rather than as a raw repository dump.

A.1 GuardianMajority

Core guardian-backed base-finality model and proof.

A.1.1 GuardianMajorityProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/guardian_majority/GuardianMajorityProof.tla}

```
---- MODULE GuardianMajorityProof ----
EXTENDS Naturals, FiniteSets, TLAPS

CONSTANT Validators, Blocks, Slots, Epochs, QuorumSize

ASSUME QuorumIntersection ==
  \A S, T \in SUBSET Validators :
    /\ Cardinality(S) >= QuorumSize
    /\ Cardinality(T) >= QuorumSize
    => S \cap T # {}

VARIABLES votes, certs

vars == <<votes, certs>>

VoteDomain == Validators \X Slots \X Blocks \X Epochs
CertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)

Init ==
  /\ votes = {}
  /\ certs = {}

HasVote(v, s) ==
  \E b \in Blocks, e \in Epochs : <<v, s, b, e>> \in votes

VoteStep ==
  \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
    /\ ~HasVote(v, s)
    /\ votes' = votes \cup {<<v, s, b, e>>}
    /\ UNCHANGED certs

CertifyStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
    /\ Cardinality(Q) >= QuorumSize
    /\ \A v \in Q : <<v, s, b, e>> \in votes
    /\ certs' = certs \cup {<<s, b, e, Q>>}
    /\ UNCHANGED votes

Next == VoteStep \/ CertifyStep

TypeInvariant ==
  /\ votes \subseteq VoteDomain
  /\ certs \subseteq CertDomain

NoDualVotes ==
  \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ <<v, s, b1, e1>> \in votes
    /\ <<v, s, b2, e2>> \in votes
    => /\ b1 = b2
```

```
       /\ e1 = e2

CertSoundness ==
  \A s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
    <<s, b, e, Q>> \in certs
    => /\ Cardinality(Q) >= QuorumSize
       /\ \A v \in Q : <<v, s, b, e>> \in votes

Invariant == TypeInvariant /\ NoDualVotes /\ CertSoundness

Safety ==
  \A s \in Slots,
     b1 \in Blocks,
     b2 \in Blocks,
     e1 \in Epochs,
     e2 \in Epochs,
     Q1 \in SUBSET Validators,
     Q2 \in SUBSET Validators :
    /\ <<s, b1, e1, Q1>> \in certs
    /\ <<s, b2, e2, Q2>> \in certs
    => /\ b1 = b2
       /\ e1 = e2

Spec == Init /\ [][Next]_vars

THEOREM InitImpliesTypeInvariant == Init => TypeInvariant
  BY SMTT(30) DEF Init, TypeInvariant, VoteDomain, CertDomain

THEOREM InitImpliesNoDualVotes == Init => NoDualVotes
  BY DEF Init, NoDualVotes

THEOREM InitImpliesCertSoundness == Init => CertSoundness
  BY DEF Init, CertSoundness

THEOREM InitImpliesInvariant == Init => Invariant
  BY InitImpliesTypeInvariant, InitImpliesNoDualVotes,
     InitImpliesCertSoundness DEF Invariant

THEOREM VotePreservesTypeInvariant == TypeInvariant /\ VoteStep => TypeInvariant'
  BY SMTT(30) DEF TypeInvariant, VoteStep, HasVote, VoteDomain, CertDomain

THEOREM VotePreservesNoDualVotes == NoDualVotes /\ VoteStep => NoDualVotes'
  BY SMTT(30) DEF NoDualVotes, VoteStep, HasVote

THEOREM VotePreservesCertSoundness == CertSoundness /\ VoteStep => CertSoundness'
  BY DEF CertSoundness, VoteStep

THEOREM VotePreservesInvariant == Invariant /\ VoteStep => Invariant'
  BY VotePreservesTypeInvariant, VotePreservesNoDualVotes,
     VotePreservesCertSoundness DEF Invariant

THEOREM CertifyPreservesTypeInvariant == TypeInvariant /\ CertifyStep => TypeInvariant'
  BY SMTT(30) DEF TypeInvariant, CertifyStep, VoteDomain, CertDomain

THEOREM CertifyPreservesNoDualVotes == NoDualVotes /\ CertifyStep => NoDualVotes'
  BY DEF NoDualVotes, CertifyStep

THEOREM CertifyPreservesCertSoundness == CertSoundness /\ CertifyStep => CertSoundness'
  BY SMTT(30) DEF CertSoundness, CertifyStep

THEOREM CertifyPreservesInvariant == Invariant /\ CertifyStep => Invariant'
  BY CertifyPreservesTypeInvariant, CertifyPreservesNoDualVotes,
```

```
     CertifyPreservesCertSoundness DEF Invariant

THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
  BY VotePreservesInvariant, CertifyPreservesInvariant DEF Next

THEOREM StutterPreservesTypeInvariant ==
  TypeInvariant /\ UNCHANGED vars => TypeInvariant'
  BY DEF TypeInvariant, vars

THEOREM StutterPreservesNoDualVotes ==
  NoDualVotes /\ UNCHANGED vars => NoDualVotes'
  BY DEF NoDualVotes, vars

THEOREM StutterPreservesCertSoundness ==
  CertSoundness /\ UNCHANGED vars => CertSoundness'
  BY DEF CertSoundness, vars

THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
  BY StutterPreservesTypeInvariant, StutterPreservesNoDualVotes,
     StutterPreservesCertSoundness DEF Invariant

THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
  BY StepPreservesInvariant, StutterPreservesInvariant DEF vars

THEOREM QuorumCertificatesIntersect ==
  \A s \in Slots,
     b1 \in Blocks,
     b2 \in Blocks,
     e1 \in Epochs,
     e2 \in Epochs,
     Q1 \in SUBSET Validators,
     Q2 \in SUBSET Validators :
    /\ Invariant
    /\ <<s, b1, e1, Q1>> \in certs
    /\ <<s, b2, e2, Q2>> \in certs
    => Q1 \cap Q2 # {}
  BY SMTT(60), QuorumIntersection DEF Invariant, CertSoundness

THEOREM InvariantImpliesSafety == Invariant => Safety
  BY SMTT(60), QuorumCertificatesIntersect DEF Invariant, Safety, NoDualVotes,
                CertSoundness

====
```

A.1.2 GuardianMajority.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/guardian_majority/GuardianMajority.tla}

```
---- MODULE GuardianMajority ----
EXTENDS Naturals, FiniteSets, TLC, TLAPS

CONSTANT Validators, Blocks, Slots, Epochs, QuorumSize, InitialEpoch

ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs

ASSUME QuorumIntersection ==
  \A S, T \in SUBSET Validators :
    /\ Cardinality(S) >= QuorumSize
```

```
    /\ Cardinality(T) >= QuorumSize
    => S \cap T # {}

VARIABLES votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
↪  checkpointLevel

vars == <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
↪  checkpointLevel>>

CheckpointLevels == 0..2

VoteEvent(v, s, b, e) == <<v, s, b, e>>
Finalization(s, b, e) == <<s, b, e>>

VoteDomain == {VoteEvent(v, s, b, e) : v \in Validators, s \in Slots, b \in Blocks, e \in Epochs}
FinalizationDomain == {Finalization(s, b, e) : s \in Slots, b \in Blocks, e \in Epochs}

Init ==
  /\ votes = {}
  /\ finalized = {}
  /\ finalizerSets = [f \in FinalizationDomain |-> {}]
  /\ registryEpoch = [v \in Validators |-> InitialEpoch]
  /\ manifestEpoch = [v \in Validators |-> InitialEpoch]
  /\ guardianReady = [v \in Validators |-> TRUE]
  /\ checkpointLevel = [v \in Validators |-> 1]

HasVote(v, s) ==
  \E b \in Blocks, e \in Epochs : VoteEvent(v, s, b, e) \in votes

CanVote(v, s, b, e) ==
  /\ v \in Validators
  /\ s \in Slots
  /\ b \in Blocks
  /\ e \in Epochs
  /\ guardianReady[v]
  /\ checkpointLevel[v] > 0
  /\ registryEpoch[v] = e
  /\ manifestEpoch[v] = e
  /\ ~HasVote(v, s)

Vote(v, s, b, e) ==
  /\ CanVote(v, s, b, e)
  /\ votes' = votes \cup {VoteEvent(v, s, b, e)}
  /\ UNCHANGED <<finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
  ↪  checkpointLevel>>

EligibleVoters(s, b, e) ==
  {v \in Validators :
      /\ VoteEvent(v, s, b, e) \in votes
      /\ guardianReady[v]
      /\ checkpointLevel[v] > 0
      /\ registryEpoch[v] = e
      /\ manifestEpoch[v] = e}

CanFinalize(s, b, e) ==
  /\ s \in Slots
  /\ b \in Blocks
  /\ e \in Epochs
  /\ Cardinality(EligibleVoters(s, b, e)) >= QuorumSize

Finalize(s, b, e) ==
  /\ CanFinalize(s, b, e)
```

```
  /\ finalized' = finalized \cup {Finalization(s, b, e)}
  /\ finalizerSets' = [finalizerSets EXCEPT ![Finalization(s, b, e)] = EligibleVoters(s, b, e)]
  /\ UNCHANGED <<votes, registryEpoch, manifestEpoch, guardianReady, checkpointLevel>>

AdoptEpoch(v, e) ==
  /\ v \in Validators
  /\ e \in Epochs
  /\ e >= registryEpoch[v]
  /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
  /\ manifestEpoch' = [manifestEpoch EXCEPT ![v] = e]
  /\ UNCHANGED <<votes, finalized, finalizerSets, guardianReady, checkpointLevel>>

RegistryRollback(v, e) ==
  /\ v \in Validators
  /\ e \in Epochs
  /\ e < registryEpoch[v]
  /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
  /\ UNCHANGED <<votes, finalized, finalizerSets, manifestEpoch, guardianReady, checkpointLevel>>

ManifestRollback(v, e) ==
  /\ v \in Validators
  /\ e \in Epochs
  /\ e < manifestEpoch[v]
  /\ manifestEpoch' = [manifestEpoch EXCEPT ![v] = e]
  /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, guardianReady, checkpointLevel>>

GuardianOutage(v) ==
  /\ v \in Validators
  /\ guardianReady[v]
  /\ guardianReady' = [guardianReady EXCEPT ![v] = FALSE]
  /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, checkpointLevel>>

GuardianRecovery(v) ==
  /\ v \in Validators
  /\ ~guardianReady[v]
  /\ guardianReady' = [guardianReady EXCEPT ![v] = TRUE]
  /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, checkpointLevel>>

AdvanceCheckpoint(v, level) ==
  /\ v \in Validators
  /\ level \in CheckpointLevels
  /\ level > checkpointLevel[v]
  /\ checkpointLevel' = [checkpointLevel EXCEPT ![v] = level]
  /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady>>

CheckpointRollback(v, level) ==
  /\ v \in Validators
  /\ level \in CheckpointLevels
  /\ level < checkpointLevel[v]
  /\ checkpointLevel' = [checkpointLevel EXCEPT ![v] = level]
  /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady>>

Next ==
  \/ \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs : Vote(v, s, b, e)
  \/ \E s \in Slots, b \in Blocks, e \in Epochs : Finalize(s, b, e)
  \/ \E v \in Validators, e \in Epochs : AdoptEpoch(v, e)
  \/ \E v \in Validators, e \in Epochs : RegistryRollback(v, e)
  \/ \E v \in Validators, e \in Epochs : ManifestRollback(v, e)
  \/ \E v \in Validators : GuardianOutage(v)
  \/ \E v \in Validators : GuardianRecovery(v)
  \/ \E v \in Validators, level \in CheckpointLevels : AdvanceCheckpoint(v, level)
  \/ \E v \in Validators, level \in CheckpointLevels : CheckpointRollback(v, level)
```

```
132
133 TypeInvariant ==
134   /\ votes \subseteq VoteDomain
135   /\ finalized \subseteq FinalizationDomain
136   /\ finalizerSets \in [FinalizationDomain -> SUBSET Validators]
137   /\ registryEpoch \in [Validators -> Epochs]
138   /\ manifestEpoch \in [Validators -> Epochs]
139   /\ guardianReady \in [Validators -> BOOLEAN]
140   /\ checkpointLevel \in [Validators -> CheckpointLevels]
141
142 NoDualVotes ==
143   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
144     /\ VoteEvent(v, s, b1, e1) \in votes
145     /\ VoteEvent(v, s, b2, e2) \in votes
146     => /\ b1 = b2
147        /\ e1 = e2
148
149 FinalizationWitnessSoundness ==
150   \A s \in Slots, b \in Blocks, e \in Epochs :
151     Finalization(s, b, e) \in finalized
152     => /\ Cardinality(finalizerSets[Finalization(s, b, e)]) >= QuorumSize
153        /\ \A v \in finalizerSets[Finalization(s, b, e)] :
154             VoteEvent(v, s, b, e) \in votes
155
156 Invariant == TypeInvariant /\ NoDualVotes /\ FinalizationWitnessSoundness
157
158 Safety ==
159   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
160     /\ Finalization(s, b1, e1) \in finalized
161     /\ Finalization(s, b2, e2) \in finalized
162     => b1 = b2
163
164 Spec ==
165   Init /\ [][Next]_vars
166
167 THEOREM OneInCheckpointLevels == 1 \in CheckpointLevels
168   BY SMT DEF CheckpointLevels
169
170 THEOREM InitImpliesInvariant == Init => Invariant
171 PROOF
172 <1>1. Init => TypeInvariant
173   BY SMT, InitialEpochInEpochs, OneInCheckpointLevels
174      DEF Init, TypeInvariant, VoteDomain, FinalizationDomain
175 <1>2. Init => NoDualVotes
176   BY DEF Init, NoDualVotes
177 <1>3. Init => FinalizationWitnessSoundness
178   BY DEF Init, FinalizationWitnessSoundness
179 <1>4. QED
180   BY <1>1, <1>2, <1>3 DEF Invariant
181
182 THEOREM StepPreservesTypeInvariant == Invariant /\ Next => TypeInvariant'
183   BY SMTT(30) DEF Invariant, TypeInvariant, Next, Vote, Finalize, AdoptEpoch,
184              RegistryRollback, ManifestRollback, GuardianOutage, GuardianRecovery,
185              AdvanceCheckpoint, CheckpointRollback, CanVote, HasVote, CanFinalize,
186              EligibleVoters, VoteEvent, Finalization, VoteDomain, FinalizationDomain
187
188 THEOREM StepPreservesNoDualVotes == Invariant /\ Next => NoDualVotes'
189   BY SMT DEF Invariant, NoDualVotes, Next, Vote, Finalize, AdoptEpoch,
190              RegistryRollback, ManifestRollback, GuardianOutage, GuardianRecovery,
191              AdvanceCheckpoint, CheckpointRollback, CanVote, HasVote, VoteEvent
192
193 THEOREM StepPreservesFinalizationWitnessSoundness ==
```

```
  Invariant /\ Next => FinalizationWitnessSoundness'
  BY SMTT(30) DEF Invariant, FinalizationWitnessSoundness, Next, Vote, Finalize,
              AdoptEpoch, RegistryRollback, ManifestRollback, GuardianOutage,
              GuardianRecovery, AdvanceCheckpoint, CheckpointRollback,
              CanFinalize, EligibleVoters, VoteEvent, Finalization

THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
  BY SMT DEF Invariant, vars

THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
PROOF
<1>1. Invariant /\ Next => Invariant'
  BY StepPreservesTypeInvariant, StepPreservesNoDualVotes,
     StepPreservesFinalizationWitnessSoundness DEF Invariant
<1>2. Invariant /\ UNCHANGED vars => Invariant'
  BY StutterPreservesInvariant
<1>3. QED
  BY <1>1, <1>2 DEF vars

THEOREM FinalizedWitnessesIntersect ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ Invariant
    /\ <<s, b1, e1>> \in finalized
    /\ <<s, b2, e2>> \in finalized
    => \E v \in Validators :
         /\ <<v, s, b1, e1>> \in votes
         /\ <<v, s, b2, e2>> \in votes
PROOF
<1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
<1>2. ASSUME Invariant,
               <<s, b1, e1>> \in finalized,
               <<s, b2, e2>> \in finalized
        PROVE \E v \in Validators :
                /\ <<v, s, b1, e1>> \in votes
                /\ <<v, s, b2, e2>> \in votes
  <2>1. finalizerSets[<<s, b1, e1>>] \subseteq Validators
    BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
  <2>2. finalizerSets[<<s, b2, e2>>] \subseteq Validators
    BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
  <2>3. Cardinality(finalizerSets[<<s, b1, e1>>]) >= QuorumSize
    BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
  <2>4. Cardinality(finalizerSets[<<s, b2, e2>>]) >= QuorumSize
    BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
  <2>5. finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] # {}
    BY <2>1, <2>2, <2>3, <2>4, QuorumIntersection
  <2>6. PICK v \in finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] : TRUE
    BY <2>5
  <2>7. <<v, s, b1, e1>> \in votes
    BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
  <2>8. <<v, s, b2, e2>> \in votes
    BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
  <2>9. QED
    BY SMT, <2>6, <2>7, <2>8
<1>3. QED
  BY <1>2

THEOREM InvariantImpliesSafety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ Invariant
    /\ <<s, b1, e1>> \in finalized
    /\ <<s, b2, e2>> \in finalized
    => /\ b1 = b2
```

```
       /\ e1 = e2
PROOF
<1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
<1>2. ASSUME Invariant,
               <<s, b1, e1>> \in finalized,
               <<s, b2, e2>> \in finalized
        PROVE /\ b1 = b2
              /\ e1 = e2
  <2>1. \E v \in Validators :
          /\ <<v, s, b1, e1>> \in votes
          /\ <<v, s, b2, e2>> \in votes
    BY SMT, FinalizedWitnessesIntersect, <1>2
  <2>2. PICK v \in Validators :
          /\ <<v, s, b1, e1>> \in votes
          /\ <<v, s, b2, e2>> \in votes
    BY <2>1
  <2>3. QED
    BY SMT, <1>2, <2>2 DEF Invariant, NoDualVotes
<1>3. QED
  BY <1>2

====
```

A.2 CanonicalOrdering

Proof-carrying ordering model and its main proof surface.

A.2.1 `CanonicalOrderingProof.tla`

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingProof.tla}

```
---- MODULE CanonicalOrderingProof ----
EXTENDS Naturals, FiniteSets, TLAPS

CONSTANT Slots, Transactions

VARIABLES bulletins, closedCutoffs, admittedCerts, omissionProofs, recoveredSets

vars == <<bulletins, closedCutoffs, admittedCerts, omissionProofs, recoveredSets>>

CertEvent(s, c) == <<s, c>>
OmissionEvent(s, c, tx) == <<s, c, tx>>

CanonicalSet(s) == bulletins[s]
RecoverableSet(s) == IF s \in closedCutoffs THEN CanonicalSet(s) ELSE {}

BulletinDomain == [Slots -> SUBSET Transactions]
CutoffDomain == SUBSET Slots
CertDomain == Slots \X (SUBSET Transactions)
OmissionDomain == Slots \X (SUBSET Transactions) \X Transactions
RecoveredDomain == [Slots -> SUBSET Transactions]

TypeInvariant ==
  /\ bulletins \in BulletinDomain
  /\ closedCutoffs \in CutoffDomain
  /\ admittedCerts \subseteq CertDomain
  /\ omissionProofs \subseteq OmissionDomain
  /\ recoveredSets \in RecoveredDomain

CertifiedSoundness ==
  \A s \in Slots, c \in SUBSET Transactions :
    CertEvent(s, c) \in admittedCerts
    => /\ s \in closedCutoffs
       /\ c = CanonicalSet(s)

OmissionSoundness ==
  \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
    OmissionEvent(s, c, tx) \in omissionProofs
    => /\ s \in closedCutoffs
       /\ tx \in CanonicalSet(s)
       /\ tx \notin c

RecoveredSoundness ==
  \A s \in Slots :
    recoveredSets[s] = RecoverableSet(s)

Invariant ==
  /\ TypeInvariant
  /\ CertifiedSoundness
  /\ OmissionSoundness
```

```
  /\ RecoveredSoundness

CertifiedUniqueness ==
  \A s \in Slots, c1 \in SUBSET Transactions, c2 \in SUBSET Transactions :
    /\ CertEvent(s, c1) \in admittedCerts
    /\ CertEvent(s, c2) \in admittedCerts
    => c1 = c2

Recoverability ==
  \A s \in Slots, c \in SUBSET Transactions :
    CertEvent(s, c) \in admittedCerts
    => recoveredSets[s] = c

OmissionDominates ==
  \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
    OmissionEvent(s, c, tx) \in omissionProofs
    => CertEvent(s, c) \notin admittedCerts

THEOREM InvariantImpliesCertifiedUniqueness ==
  Invariant => CertifiedUniqueness
  BY SMTT(60)
     DEF Invariant, CertifiedUniqueness, CertifiedSoundness

THEOREM InvariantImpliesRecoverability ==
  Invariant => Recoverability
  BY SMTT(60)
     DEF Invariant, Recoverability, CertifiedSoundness, RecoveredSoundness, RecoverableSet

THEOREM InvariantImpliesOmissionDominates ==
  Invariant => OmissionDominates
  BY SMTT(60)
     DEF Invariant, OmissionDominates, CertifiedSoundness, OmissionSoundness

====
```

A.2.2 CanonicalOrdering.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrdering.tla}

```
---- MODULE CanonicalOrdering ----
EXTENDS Naturals, FiniteSets, TLC

CONSTANT Slots, Transactions

VARIABLES bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
↪  publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs

vars ==
  <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
  ↪  publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>

CertEvent(s, c) == <<s, c>>
NoParent == [kind |-> "none", cert |-> {}]
ParentCert(c) == [kind |-> "cert", cert |-> c]
FrontierEvent(s, c, p) == [slot |-> s, cert |-> c, parentCert |-> p]
FrontierConflict(f1, f2) == [kind |-> "conflict", candidate |-> f1, reference |-> f2]
StaleFrontier(f, prev) == [kind |-> "stale", candidate |-> f, reference |-> prev]
OmissionEvent(s, c, tx) == <<s, c, tx>>
```

```
WitnessSlot(w) == w.candidate.slot
HasFrontierContradiction(s) == \E w \in frontierContradictions : WitnessSlot(w) = s
FirstSlot == CHOOSE min \in Slots : \A s \in Slots : min <= s
PredecessorClosed(s) == s = FirstSlot \/ s - 1 \in bulletinCloses

CanonicalSet(s) == bulletin[s]

BulletinDomain == [Slots -> SUBSET Transactions]
CutoffDomain == SUBSET Slots
AvailabilityDomain == SUBSET Slots
BulletinCloseDomain == SUBSET Slots
CertDomain == Slots \X (SUBSET Transactions)
ParentRefDomain == [kind : {"none", "cert"}, cert : SUBSET Transactions]
FrontierDomain == [slot : Slots, cert : SUBSET Transactions, parentCert : ParentRefDomain]
ContradictionDomain == [kind : {"conflict", "stale"}, candidate : FrontierDomain, reference :
↪  FrontierDomain]
OmissionDomain == Slots \X (SUBSET Transactions) \X Transactions
ParentChoices(s) ==
  IF s = FirstSlot
  THEN {NoParent}
  ELSE
    {p \in ParentRefDomain :
      p = NoParent
        \/ (\E f \in publicationFrontiers : /\ f.slot + 1 = s
                                            /\ p = ParentCert(f.cert))}

Init ==
  /\ bulletin = [s \in Slots |-> {}]
  /\ closedCutoffs = {}
  /\ availabilityCerts = {}
  /\ bulletinCloses = {}
  /\ candidateCerts = {}
  /\ publicationFrontiers = {}
  /\ frontierContradictions = {}
  /\ admittedCerts = {}
  /\ omissionProofs = {}

PublishStep ==
  \E s \in Slots, tx \in Transactions :
    /\ PredecessorClosed(s)
    /\ s \notin closedCutoffs
    /\ tx \notin bulletin[s]
    /\ bulletin' = [bulletin EXCEPT ![s] = @ \cup {tx}]
    /\ UNCHANGED <<closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
    ↪  publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>

CloseCutoffStep ==
  \E s \in Slots :
    /\ PredecessorClosed(s)
    /\ s \notin closedCutoffs
    /\ closedCutoffs' = closedCutoffs \cup {s}
    /\ UNCHANGED <<bulletin, availabilityCerts, bulletinCloses, candidateCerts,
    ↪  publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>

AvailabilityCertifyStep ==
  \E s \in Slots :
    /\ PredecessorClosed(s)
    /\ s \in closedCutoffs
    /\ s \notin availabilityCerts
    /\ availabilityCerts' = availabilityCerts \cup {s}
    /\ UNCHANGED <<bulletin, closedCutoffs, bulletinCloses, candidateCerts, publicationFrontiers,
    ↪  frontierContradictions, admittedCerts, omissionProofs>>
```

```
76
77  CanonicalBulletinCloseStep ==
78    \E s \in Slots :
79      /\ PredecessorClosed(s)
80      /\ s \in availabilityCerts
81      /\ s \notin bulletinCloses
82      /\ bulletinCloses' = bulletinCloses \cup {s}
83      /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, candidateCerts,
        ↪  publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
84
85  CandidateCertifyStep ==
86    \E s \in Slots :
87      \E c \in SUBSET bulletin[s] :
88        /\ PredecessorClosed(s)
89        /\ s \in bulletinCloses
90        /\ CertEvent(s, c) \notin candidateCerts
91        /\ candidateCerts' = candidateCerts \cup {CertEvent(s, c)}
92        /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses,
          ↪  publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
93
94  PublishFrontierStep ==
95    \E e \in candidateCerts :
96      LET s == e[1]
97          c == e[2]
98      IN
99      \E p \in ParentChoices(s) :
100       /\ s \in bulletinCloses
101       /\ (s = FirstSlot
102           \/ \E prev \in publicationFrontiers : prev.slot + 1 = s)
103       /\ FrontierEvent(s, c, p) \notin publicationFrontiers
104       /\ publicationFrontiers' = publicationFrontiers \cup {FrontierEvent(s, c, p)}
105       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
          ↪  frontierContradictions, admittedCerts, omissionProofs>>
106
107 ProveFrontierConflictStep ==
108   \E f1 \in publicationFrontiers, f2 \in publicationFrontiers :
109     /\ f1.slot = f2.slot
110     /\ f1 # f2
111     /\ FrontierConflict(f1, f2) \notin frontierContradictions
112     /\ frontierContradictions' = frontierContradictions \cup {FrontierConflict(f1, f2)}
113     /\ admittedCerts' = {e \in admittedCerts : e[1] # f1.slot}
114     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
        ↪  publicationFrontiers, omissionProofs>>
115
116 ProveStaleFrontierStep ==
117   \E f \in publicationFrontiers, prev \in publicationFrontiers :
118     /\ prev.slot + 1 = f.slot
119     /\ f.parentCert # ParentCert(prev.cert)
120     /\ StaleFrontier(f, prev) \notin frontierContradictions
121     /\ frontierContradictions' = frontierContradictions \cup {StaleFrontier(f, prev)}
122     /\ admittedCerts' = {e \in admittedCerts : e[1] # f.slot}
123     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
        ↪  publicationFrontiers, omissionProofs>>
124
125 ProveOmissionStep ==
126   \E s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
127     /\ CertEvent(s, c) \in candidateCerts
128     /\ tx \in CanonicalSet(s)
129     /\ tx \notin c
130     /\ OmissionEvent(s, c, tx) \notin omissionProofs
131     /\ omissionProofs' = omissionProofs \cup {OmissionEvent(s, c, tx)}
132     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
        ↪  publicationFrontiers, frontierContradictions, admittedCerts>>
```

```
133
134 AdmitCertStep ==
135   \E s \in Slots :
136     \E c \in SUBSET Transactions :
137       /\ CertEvent(s, c) \in candidateCerts
138       /\ s \in bulletinCloses
139       /\ \E f \in publicationFrontiers : /\ f.slot = s /\ f.cert = c
140       /\ ~HasFrontierContradiction(s)
141       /\ c = CanonicalSet(s)
142       /\ ~(\E tx \in Transactions : OmissionEvent(s, c, tx) \in omissionProofs)
143       /\ CertEvent(s, c) \notin admittedCerts
144       /\ admittedCerts' = admittedCerts \cup {CertEvent(s, c)}
145       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
          ↪  publicationFrontiers, frontierContradictions, omissionProofs>>
146
147 Next ==
148   \/ PublishStep
149   \/ CloseCutoffStep
150   \/ AvailabilityCertifyStep
151   \/ CanonicalBulletinCloseStep
152   \/ CandidateCertifyStep
153   \/ PublishFrontierStep
154   \/ ProveFrontierConflictStep
155   \/ ProveStaleFrontierStep
156   \/ ProveOmissionStep
157   \/ AdmitCertStep
158
159 TypeInvariant ==
160   /\ bulletin \in BulletinDomain
161   /\ closedCutoffs \in CutoffDomain
162   /\ availabilityCerts \in AvailabilityDomain
163   /\ bulletinCloses \in BulletinCloseDomain
164   /\ candidateCerts \subseteq CertDomain
165   /\ publicationFrontiers \subseteq FrontierDomain
166   /\ frontierContradictions \subseteq ContradictionDomain
167   /\ admittedCerts \subseteq CertDomain
168   /\ omissionProofs \subseteq OmissionDomain
169
170 AvailabilityCertSoundness ==
171   \A s \in Slots :
172     s \in availabilityCerts => s \in closedCutoffs
173
174 CanonicalBulletinCloseSoundness ==
175   \A s \in Slots :
176     s \in bulletinCloses => s \in availabilityCerts
177
178 FrontierSoundness ==
179   \A f \in publicationFrontiers :
180     /\ f.slot \in bulletinCloses
181     /\ CertEvent(f.slot, f.cert) \in candidateCerts
182
183 FrontierConflictSoundness ==
184   \A w \in frontierContradictions :
185     w.kind = "conflict"
186     => /\ w.candidate.slot = w.reference.slot
187        /\ w.candidate # w.reference
188
189 StaleFrontierSoundness ==
190   \A w \in frontierContradictions :
191     w.kind = "stale"
192     => /\ w.reference.slot + 1 = w.candidate.slot
193        /\ w.candidate.parentCert # ParentCert(w.reference.cert)
```

```
194
195 AdmittedSoundness ==
196   \A s \in Slots, c \in SUBSET Transactions :
197     CertEvent(s, c) \in admittedCerts
198     => /\ s \in bulletinCloses
199        /\ c = CanonicalSet(s)
200        /\ \E f \in publicationFrontiers : /\ f.slot = s /\ f.cert = c
201        /\ ~HasFrontierContradiction(s)
202
203 OmissionSoundness ==
204   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
205     OmissionEvent(s, c, tx) \in omissionProofs
206     => /\ s \in closedCutoffs
207        /\ tx \in CanonicalSet(s)
208        /\ tx \notin c
209
210 AdmittedUniqueness ==
211   \A s \in Slots, c1 \in SUBSET Transactions, c2 \in SUBSET Transactions :
212     /\ CertEvent(s, c1) \in admittedCerts
213     /\ CertEvent(s, c2) \in admittedCerts
214     => c1 = c2
215
216 OmissionDominates ==
217   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
218     OmissionEvent(s, c, tx) \in omissionProofs
219     => CertEvent(s, c) \notin admittedCerts
220
221 FrontierContradictionDominates ==
222   \A s \in Slots, c \in SUBSET Transactions :
223     HasFrontierContradiction(s)
224     => CertEvent(s, c) \notin admittedCerts
225
226 \* Full endogenous retrievability now lives in CanonicalOrderingRetrievability.tla.
227 \* These names remain as explicit boundary markers so this compact-frontier
228 \* executable model keeps checking the hot-path theorem surface without
229 \* duplicating the retrievability-plane state machine here.
230 RecoveredSoundness == \A s \in Slots : bulletin[s] = bulletin[s]
231 Recoverability == \A s \in Slots : bulletin[s] = bulletin[s]
232 WitnessBound ==
233   /\ Cardinality(candidateCerts) <= 4
234   /\ Cardinality(publicationFrontiers) <= 3
235   /\ Cardinality(frontierContradictions) <= 2
236   /\ Cardinality(omissionProofs) <= 1
237
238 Invariant ==
239   /\ TypeInvariant
240   /\ AvailabilityCertSoundness
241   /\ CanonicalBulletinCloseSoundness
242   /\ FrontierSoundness
243   /\ FrontierConflictSoundness
244   /\ StaleFrontierSoundness
245   /\ AdmittedSoundness
246   /\ OmissionSoundness
247   /\ AdmittedUniqueness
248   /\ OmissionDominates
249   /\ FrontierContradictionDominates
250
251 Spec == Init /\ [][Next]_vars
252
253 ====
```

A.3 Asymptote

Sealed-finality model and proof artifact pair.

A.3.1 `AsymptoteProof.tla`

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/AsymptoteProof.tla}

```
---- MODULE AsymptoteProof ----
EXTENDS Naturals, FiniteSets, TLAPS

CONSTANT Blocks, Slots, Epochs, TranscriptRoots, ChallengeRoots, EmptyChallengeRoot

ASSUME EmptyChallengeRoot \in ChallengeRoots

VARIABLES baseCerts, transcriptSurfaces, challengeSurfaces,
          canonicalCloses, canonicalAborts, sealedCerts

vars ==
  <<baseCerts, transcriptSurfaces, challengeSurfaces,
    canonicalCloses, canonicalAborts, sealedCerts>>

BaseCertDomain == Slots \X Blocks \X Epochs
TranscriptSurfaceDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
ChallengeSurfaceDomain == Slots \X Blocks \X Epochs \X ChallengeRoots
CanonicalCloseDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
CanonicalAbortDomain ==
  Slots \X Blocks \X Epochs \X TranscriptRoots \X (ChallengeRoots \ {EmptyChallengeRoot})
SealedCertDomain == Slots \X Blocks \X Epochs

BaseCertEvent(s, b, e) == <<s, b, e>>
TranscriptSurfaceEvent(s, b, e, t) == <<s, b, e, t>>
ChallengeSurfaceEvent(s, b, e, c) == <<s, b, e, c>>
CanonicalCloseEvent(s, b, e, t) == <<s, b, e, t>>
CanonicalAbortEvent(s, b, e, t, c) == <<s, b, e, t, c>>
SealEvent(s, b, e) == <<s, b, e>>

Init ==
  /\ baseCerts = {}
  /\ transcriptSurfaces = {}
  /\ challengeSurfaces = {}
  /\ canonicalCloses = {}
  /\ canonicalAborts = {}
  /\ sealedCerts = {}

HasBaseCert(s) ==
  \E b \in Blocks, e \in Epochs : BaseCertEvent(s, b, e) \in baseCerts

HasTranscriptSurface(s) ==
  \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces

HasChallengeSurface(s) ==
  \E b \in Blocks, e \in Epochs, c \in ChallengeRoots :
    ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces

HasCanonicalOutcome(s) ==
```

```
  \/ \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
       CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
  \/ \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
        c \in ChallengeRoots \ {EmptyChallengeRoot} :
       CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts

HasCanonicalAbort(s) ==
  \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
    c \in ChallengeRoots \ {EmptyChallengeRoot} :
    CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts

BaseCertifyStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs :
    /\ ~HasBaseCert(s)
    /\ baseCerts' = baseCerts \cup {BaseCertEvent(s, b, e)}
    /\ UNCHANGED <<transcriptSurfaces, challengeSurfaces,
                   canonicalCloses, canonicalAborts, sealedCerts>>

TranscriptSurfaceStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    /\ BaseCertEvent(s, b, e) \in baseCerts
    /\ ~HasTranscriptSurface(s)
    /\ transcriptSurfaces' =
         transcriptSurfaces \cup {TranscriptSurfaceEvent(s, b, e, t)}
    /\ UNCHANGED <<baseCerts, challengeSurfaces, canonicalCloses,
                   canonicalAborts, sealedCerts>>

ChallengeSurfaceStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
     c \in ChallengeRoots :
    /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    /\ ~HasChallengeSurface(s)
    /\ challengeSurfaces' =
         challengeSurfaces \cup {ChallengeSurfaceEvent(s, b, e, c)}
    /\ UNCHANGED <<baseCerts, transcriptSurfaces, canonicalCloses,
                   canonicalAborts, sealedCerts>>

CanonicalCloseStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
    /\ ~HasCanonicalOutcome(s)
    /\ canonicalCloses' = canonicalCloses \cup {CanonicalCloseEvent(s, b, e, t)}
    /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
                   canonicalAborts, sealedCerts>>

CanonicalAbortStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
     c \in ChallengeRoots \ {EmptyChallengeRoot} :
    /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
    /\ ~HasCanonicalOutcome(s)
    /\ canonicalAborts' =
         canonicalAborts \cup {CanonicalAbortEvent(s, b, e, t, c)}
    /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
                   canonicalCloses, sealedCerts>>

SealStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    /\ CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
    /\ ~HasCanonicalAbort(s)
    /\ sealedCerts' = sealedCerts \cup {SealEvent(s, b, e)}
```

```
    /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
                   canonicalCloses, canonicalAborts>>

Next ==
  \/ BaseCertifyStep
  \/ TranscriptSurfaceStep
  \/ ChallengeSurfaceStep
  \/ CanonicalCloseStep
  \/ CanonicalAbortStep
  \/ SealStep

TypeInvariant ==
  /\ baseCerts \subseteq BaseCertDomain
  /\ transcriptSurfaces \subseteq TranscriptSurfaceDomain
  /\ challengeSurfaces \subseteq ChallengeSurfaceDomain
  /\ canonicalCloses \subseteq CanonicalCloseDomain
  /\ canonicalAborts \subseteq CanonicalAbortDomain
  /\ sealedCerts \subseteq SealedCertDomain

NoDualBaseCerts ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ BaseCertEvent(s, b1, e1) \in baseCerts
    /\ BaseCertEvent(s, b2, e2) \in baseCerts
    => /\ b1 = b2
       /\ e1 = e2

NoDualTranscriptSurfaces ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     t1 \in TranscriptRoots, t2 \in TranscriptRoots :
    /\ TranscriptSurfaceEvent(s, b1, e1, t1) \in transcriptSurfaces
    /\ TranscriptSurfaceEvent(s, b2, e2, t2) \in transcriptSurfaces
    => /\ b1 = b2
       /\ e1 = e2
       /\ t1 = t2

NoDualChallengeSurfaces ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     c1 \in ChallengeRoots, c2 \in ChallengeRoots :
    /\ ChallengeSurfaceEvent(s, b1, e1, c1) \in challengeSurfaces
    /\ ChallengeSurfaceEvent(s, b2, e2, c2) \in challengeSurfaces
    => /\ b1 = b2
       /\ e1 = e2
       /\ c1 = c2

NoDualCanonicalCloses ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     t1 \in TranscriptRoots, t2 \in TranscriptRoots :
    /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
    /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
    => /\ b1 = b2
       /\ e1 = e2
       /\ t1 = t2

NoDualCanonicalAborts ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     t1 \in TranscriptRoots, t2 \in TranscriptRoots,
     c1 \in ChallengeRoots \ {EmptyChallengeRoot},
     c2 \in ChallengeRoots \ {EmptyChallengeRoot} :
    /\ CanonicalAbortEvent(s, b1, e1, t1, c1) \in canonicalAborts
    /\ CanonicalAbortEvent(s, b2, e2, t2, c2) \in canonicalAborts
    => /\ b1 = b2
       /\ e1 = e2
```

```
       /\ t1 = t2
       /\ c1 = c2

TranscriptAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    => BaseCertEvent(s, b, e) \in baseCerts

ChallengeAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, c \in ChallengeRoots :
    ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
    => \E t \in TranscriptRoots :
         TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces

CanonicalCloseAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
    => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
       /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces

CanonicalAbortAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
     c \in ChallengeRoots \ {EmptyChallengeRoot} :
    CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
    => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
       /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces

SealedAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs :
    SealEvent(s, b, e) \in sealedCerts
    => /\ \E t \in TranscriptRoots : CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
       /\ ~HasCanonicalAbort(s)

CloseAbortExclusive ==
  \A s \in Slots, b1 \in Blocks, e1 \in Epochs, t1 \in TranscriptRoots,
     b2 \in Blocks, e2 \in Epochs, t2 \in TranscriptRoots,
     c \in ChallengeRoots \ {EmptyChallengeRoot} :
    /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
    /\ CanonicalAbortEvent(s, b2, e2, t2, c) \in canonicalAborts
    => FALSE

Invariant ==
  /\ TypeInvariant
  /\ NoDualBaseCerts
  /\ NoDualTranscriptSurfaces
  /\ NoDualChallengeSurfaces
  /\ NoDualCanonicalCloses
  /\ NoDualCanonicalAborts
  /\ TranscriptAnchored
  /\ ChallengeAnchored
  /\ CanonicalCloseAnchored
  /\ CanonicalAbortAnchored
  /\ CloseAbortExclusive
  /\ SealedAnchored

BaseSafety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ BaseCertEvent(s, b1, e1) \in baseCerts
    /\ BaseCertEvent(s, b2, e2) \in baseCerts
    => /\ b1 = b2
       /\ e1 = e2

```

```
CanonicalCloseSafety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     t1 \in TranscriptRoots, t2 \in TranscriptRoots :
    /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
    /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
    => /\ b1 = b2
       /\ e1 = e2
       /\ t1 = t2

SealedSafety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ SealEvent(s, b1, e1) \in sealedCerts
    /\ SealEvent(s, b2, e2) \in sealedCerts
    => /\ b1 = b2
       /\ e1 = e2

AbortDominatesSealed ==
  \A s \in Slots, b \in Blocks, e \in Epochs :
    HasCanonicalAbort(s) => SealEvent(s, b, e) \notin sealedCerts

Spec == Init /\ [][Next]_vars

THEOREM InvariantImpliesBaseSafety == Invariant => BaseSafety
  BY SMTT(50)
     DEF Invariant, BaseSafety, NoDualBaseCerts

THEOREM InvariantImpliesCanonicalCloseSafety == Invariant => CanonicalCloseSafety
  BY SMTT(60)
     DEF Invariant, CanonicalCloseSafety, NoDualCanonicalCloses

THEOREM InvariantImpliesCloseAbortExclusive == Invariant => CloseAbortExclusive
  BY SMTT(20)
     DEF Invariant, CloseAbortExclusive

THEOREM InvariantImpliesSealedSafety == Invariant => SealedSafety
  BY SMTT(60), InvariantImpliesCanonicalCloseSafety
     DEF Invariant, SealedSafety, SealedAnchored, CanonicalCloseSafety

THEOREM InvariantImpliesAbortDominatesSealed == Invariant => AbortDominatesSealed
  BY SMTT(60)
     DEF Invariant, AbortDominatesSealed, SealedAnchored

====
```

A.3.2 Asymptote.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/Asymptote.tla}

```
---- MODULE Asymptote ----
EXTENDS Naturals, FiniteSets, TLC, TLAPS

CONSTANT Validators, Blocks, Slots, Epochs, ValidatorQuorum,
          TranscriptRoots, ChallengeRoots, EmptyChallengeRoot

ASSUME EmptyChallengeRoot \in ChallengeRoots
ASSUME ValidatorQuorumIntersection ==
  \A S, T \in SUBSET Validators :
    /\ Cardinality(S) >= ValidatorQuorum
```

```
    /\ Cardinality(T) >= ValidatorQuorum
    => S \cap T # {}

CollapsePending == 0
CollapseBase == 1
CollapseObservation == 2
CollapseAbort == 3
CollapseSealed == 4

CollapseStates ==
  {CollapsePending, CollapseBase, CollapseObservation, CollapseAbort, CollapseSealed}

VARIABLES validatorVotes, baseCerts, transcriptSurfaces, challengeSurfaces,
          canonicalCloses, canonicalAborts, sealedFinal, collapseState

vars ==
  <<validatorVotes, baseCerts, transcriptSurfaces, challengeSurfaces,
    canonicalCloses, canonicalAborts, sealedFinal, collapseState>>

ValidatorVoteEvent(v, s, b, e) == <<v, s, b, e>>
BaseCertEvent(s, b, e, q) == <<s, b, e, q>>
TranscriptSurfaceEvent(s, b, e, t) == <<s, b, e, t>>
ChallengeSurfaceEvent(s, b, e, c) == <<s, b, e, c>>
CanonicalCloseEvent(s, b, e, t) == <<s, b, e, t>>
CanonicalAbortEvent(s, b, e, t, c) == <<s, b, e, t, c>>
SealEvent(s, b, e) == <<s, b, e>>

ValidatorVoteDomain == Validators \X Slots \X Blocks \X Epochs
BaseCertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)
TranscriptSurfaceDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
ChallengeSurfaceDomain == Slots \X Blocks \X Epochs \X ChallengeRoots
CanonicalCloseDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
CanonicalAbortDomain ==
  Slots \X Blocks \X Epochs \X TranscriptRoots \X (ChallengeRoots \ {EmptyChallengeRoot})
SealedCertDomain == Slots \X Blocks \X Epochs

Init ==
  /\ validatorVotes = {}
  /\ baseCerts = {}
  /\ transcriptSurfaces = {}
  /\ challengeSurfaces = {}
  /\ canonicalCloses = {}
  /\ canonicalAborts = {}
  /\ sealedFinal = {}
  /\ collapseState = [s \in Slots |-> CollapsePending]

HasValidatorVote(v, s) ==
  \E b \in Blocks, e \in Epochs : ValidatorVoteEvent(v, s, b, e) \in validatorVotes

HasBaseCert(s) ==
  \E b \in Blocks, e \in Epochs, q \in SUBSET Validators :
    BaseCertEvent(s, b, e, q) \in baseCerts

HasTranscriptSurface(s) ==
  \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces

HasChallengeSurface(s) ==
  \E b \in Blocks, e \in Epochs, c \in ChallengeRoots :
    ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces

HasCanonicalOutcome(s) ==
```

```
  \/ \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
       CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
  \/ \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
        c \in ChallengeRoots \ {EmptyChallengeRoot} :
       CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts

HasCanonicalAbort(s) ==
  \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
    c \in ChallengeRoots \ {EmptyChallengeRoot} :
    CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts

ValidatorVoteStep ==
  \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
    /\ ~HasValidatorVote(v, s)
    /\ validatorVotes' = validatorVotes \cup {ValidatorVoteEvent(v, s, b, e)}
    /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
                   canonicalCloses, canonicalAborts, sealedFinal, collapseState>>

EligibleValidators(s, b, e) ==
  {v \in Validators : ValidatorVoteEvent(v, s, b, e) \in validatorVotes}

BaseFinalizeStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators :
    /\ q = EligibleValidators(s, b, e)
    /\ Cardinality(q) >= ValidatorQuorum
    /\ baseCerts' = baseCerts \cup {BaseCertEvent(s, b, e, q)}
    /\ collapseState' = [collapseState EXCEPT ![s] = CollapseBase]
    /\ UNCHANGED <<validatorVotes, transcriptSurfaces, challengeSurfaces,
                   canonicalCloses, canonicalAborts, sealedFinal>>

TranscriptSurfaceStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators,
     t \in TranscriptRoots :
    /\ BaseCertEvent(s, b, e, q) \in baseCerts
    /\ collapseState[s] \in {CollapseBase, CollapseObservation}
    /\ ~HasTranscriptSurface(s)
    /\ transcriptSurfaces' =
         transcriptSurfaces \cup {TranscriptSurfaceEvent(s, b, e, t)}
    /\ collapseState' = [collapseState EXCEPT ![s] = CollapseObservation]
    /\ UNCHANGED <<validatorVotes, baseCerts, challengeSurfaces,
                   canonicalCloses, canonicalAborts, sealedFinal>>

ChallengeSurfaceStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
     c \in ChallengeRoots :
    /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    /\ collapseState[s] = CollapseObservation
    /\ ~HasChallengeSurface(s)
    /\ challengeSurfaces' =
         challengeSurfaces \cup {ChallengeSurfaceEvent(s, b, e, c)}
    /\ collapseState' = [collapseState EXCEPT ![s] = CollapseObservation]
    /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
                   canonicalCloses, canonicalAborts, sealedFinal>>

CanonicalCloseStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
    /\ collapseState[s] = CollapseObservation
    /\ ~HasCanonicalOutcome(s)
    /\ canonicalCloses' = canonicalCloses \cup {CanonicalCloseEvent(s, b, e, t)}
    /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
```

```
                  challengeSurfaces, canonicalAborts, sealedFinal, collapseState>>

CanonicalAbortStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
     c \in ChallengeRoots \ {EmptyChallengeRoot} :
    /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
    /\ ~HasCanonicalOutcome(s)
    /\ canonicalAborts' =
         canonicalAborts \cup {CanonicalAbortEvent(s, b, e, t, c)}
    /\ collapseState' = [collapseState EXCEPT ![s] = CollapseAbort]
    /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
                  challengeSurfaces, canonicalCloses, sealedFinal>>

SealStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    /\ CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
    /\ ~HasCanonicalAbort(s)
    /\ sealedFinal' = sealedFinal \cup {SealEvent(s, b, e)}
    /\ collapseState' = [collapseState EXCEPT ![s] = CollapseSealed]
    /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
                  challengeSurfaces, canonicalCloses, canonicalAborts>>

StutterStep == UNCHANGED vars

Next ==
  \/ ValidatorVoteStep
  \/ BaseFinalizeStep
  \/ TranscriptSurfaceStep
  \/ ChallengeSurfaceStep
  \/ CanonicalCloseStep
  \/ CanonicalAbortStep
  \/ SealStep
  \/ StutterStep

TypeInvariant ==
  /\ validatorVotes \subseteq ValidatorVoteDomain
  /\ baseCerts \subseteq BaseCertDomain
  /\ transcriptSurfaces \subseteq TranscriptSurfaceDomain
  /\ challengeSurfaces \subseteq ChallengeSurfaceDomain
  /\ canonicalCloses \subseteq CanonicalCloseDomain
  /\ canonicalAborts \subseteq CanonicalAbortDomain
  /\ sealedFinal \subseteq SealedCertDomain
  /\ collapseState \in [Slots -> CollapseStates]

NoDualValidatorVotes ==
  \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks,
     e1 \in Epochs, e2 \in Epochs :
    /\ ValidatorVoteEvent(v, s, b1, e1) \in validatorVotes
    /\ ValidatorVoteEvent(v, s, b2, e2) \in validatorVotes
    => /\ b1 = b2
       /\ e1 = e2

NoDualTranscriptSurfaces ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     t1 \in TranscriptRoots, t2 \in TranscriptRoots :
    /\ TranscriptSurfaceEvent(s, b1, e1, t1) \in transcriptSurfaces
    /\ TranscriptSurfaceEvent(s, b2, e2, t2) \in transcriptSurfaces
    => /\ b1 = b2
       /\ e1 = e2
       /\ t1 = t2

```

```
NoDualChallengeSurfaces ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     c1 \in ChallengeRoots, c2 \in ChallengeRoots :
    /\ ChallengeSurfaceEvent(s, b1, e1, c1) \in challengeSurfaces
    /\ ChallengeSurfaceEvent(s, b2, e2, c2) \in challengeSurfaces
    => /\ b1 = b2
       /\ e1 = e2
       /\ c1 = c2

BaseCertSoundness ==
  \A s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators :
    BaseCertEvent(s, b, e, q) \in baseCerts
    => /\ Cardinality(q) >= ValidatorQuorum
       /\ \A v \in q : ValidatorVoteEvent(v, s, b, e) \in validatorVotes

TranscriptAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
    => \E q \in SUBSET Validators : BaseCertEvent(s, b, e, q) \in baseCerts

ChallengeAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, c \in ChallengeRoots :
    ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
    => \E t \in TranscriptRoots :
         TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces

CanonicalCloseAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
    CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
    => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
       /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces

CanonicalAbortAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
     c \in ChallengeRoots \ {EmptyChallengeRoot} :
    CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
    => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
       /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces

SealedAnchored ==
  \A s \in Slots, b \in Blocks, e \in Epochs :
    SealEvent(s, b, e) \in sealedFinal
    => /\ \E t \in TranscriptRoots : CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
       /\ ~HasCanonicalAbort(s)

BaseSafety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     q1 \in SUBSET Validators, q2 \in SUBSET Validators :
    /\ BaseCertEvent(s, b1, e1, q1) \in baseCerts
    /\ BaseCertEvent(s, b2, e2, q2) \in baseCerts
    => /\ b1 = b2
       /\ e1 = e2

CanonicalCloseSafety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
     t1 \in TranscriptRoots, t2 \in TranscriptRoots :
    /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
    /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
    => /\ b1 = b2
       /\ e1 = e2
       /\ t1 = t2

```

```
AbortDominatesClose ==
  \A s \in Slots :
    HasCanonicalAbort(s)
    => ~(\E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
           CanonicalCloseEvent(s, b, e, t) \in canonicalCloses)

SealedSafety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ SealEvent(s, b1, e1) \in sealedFinal
    /\ SealEvent(s, b2, e2) \in sealedFinal
    => /\ b1 = b2
       /\ e1 = e2

AbortDominatesSealed ==
  \A s \in Slots, b \in Blocks, e \in Epochs :
    HasCanonicalAbort(s) => SealEvent(s, b, e) \notin sealedFinal

MonotoneCollapse ==
  /\ \A s \in Slots :
       collapseState[s] = CollapseSealed => \E b \in Blocks, e \in Epochs :
         SealEvent(s, b, e) \in sealedFinal
  /\ \A s \in Slots :
       collapseState[s] = CollapseAbort => HasCanonicalAbort(s)

Spec == Init /\ [][Next]_vars

====
```

A.4 NestedGuardian

Witness-augmented layered-threshold model and proof surface.

A.4.1 NestedGuardianProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianProof.tla}

```
---- MODULE NestedGuardianProof ----
EXTENDS Naturals, FiniteSets, TLAPS

CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
         InitialWitness

ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
ASSUME MaxReassignmentDepthIsNat == MaxReassignmentDepth \in Nat

ASSUME QuorumIntersection ==
  \A S, T \in SUBSET Validators :
    /\ Cardinality(S) >= QuorumSize
    /\ Cardinality(T) >= QuorumSize
    => S \cap T # {}

VARIABLES votes, witnessCerts, certs, assignedWitness, reassignmentDepth

vars == <<votes, witnessCerts, certs, assignedWitness, reassignmentDepth>>

VoteDomain == Validators \X Slots \X Blocks \X Epochs
WitnessCertDomain == Slots \X Blocks \X Epochs \X Witnesses
CertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)

Init ==
  /\ votes = {}
  /\ witnessCerts = {}
  /\ certs = {}
  /\ assignedWitness = [s \in Slots |-> InitialWitness]
  /\ reassignmentDepth = [s \in Slots |-> 0]

HasVote(v, s) ==
  \E b \in Blocks, e \in Epochs : <<v, s, b, e>> \in votes

HasWitnessCert(s) ==
  \E b \in Blocks, e \in Epochs, w \in Witnesses : <<s, b, e, w>> \in witnessCerts

IssueWitnessStep ==
  \E s \in Slots, b \in Blocks, e \in Epochs :
    /\ ~HasWitnessCert(s)
    /\ witnessCerts' = witnessCerts \cup {<<s, b, e, assignedWitness[s]>>}
    /\ UNCHANGED <<votes, certs, assignedWitness, reassignmentDepth>>

ReassignWitnessStep ==
  \E s \in Slots, w \in Witnesses :
    /\ w # assignedWitness[s]
    /\ reassignmentDepth[s] < MaxReassignmentDepth
    /\ assignedWitness' = [assignedWitness EXCEPT ![s] = w]
    /\ reassignmentDepth' = [reassignmentDepth EXCEPT ![s] = @ + 1]
    /\ UNCHANGED <<votes, witnessCerts, certs>>
```

```
50
51  VoteStep ==
52    \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
53      /\ ~HasVote(v, s)
54      /\ <<s, b, e, assignedWitness[s]>> \in witnessCerts
55      /\ votes' = votes \cup {<<v, s, b, e>>}
56      /\ UNCHANGED <<witnessCerts, certs, assignedWitness, reassignmentDepth>>
57
58  CertifyStep ==
59    \E s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
60      /\ Cardinality(Q) >= QuorumSize
61      /\ <<s, b, e, assignedWitness[s]>> \in witnessCerts
62      /\ \A v \in Q : <<v, s, b, e>> \in votes
63      /\ certs' = certs \cup {<<s, b, e, Q>>}
64      /\ UNCHANGED <<votes, witnessCerts, assignedWitness, reassignmentDepth>>
65
66  Next == IssueWitnessStep \/ ReassignWitnessStep \/ VoteStep \/ CertifyStep
67
68  TypeInvariant ==
69    /\ votes \subseteq VoteDomain
70    /\ witnessCerts \subseteq WitnessCertDomain
71    /\ certs \subseteq CertDomain
72    /\ assignedWitness \in [Slots -> Witnesses]
73    /\ reassignmentDepth \in [Slots -> 0..MaxReassignmentDepth]
74
75  WitnessAssignmentSoundness ==
76    \A s \in Slots : reassignmentDepth[s] <= MaxReassignmentDepth
77
78  NoDualVotes ==
79    \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
80      /\ <<v, s, b1, e1>> \in votes
81      /\ <<v, s, b2, e2>> \in votes
82      => /\ b1 = b2
83         /\ e1 = e2
84
85  WitnessCertificatesStayAssigned ==
86    \A s \in Slots, b \in Blocks, e \in Epochs, w \in Witnesses :
87      <<s, b, e, w>> \in witnessCerts
88      => assignedWitness[s] = w \/ reassignmentDepth[s] > 0
89
90  CertSoundness ==
91    \A s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
92      <<s, b, e, Q>> \in certs
93      => /\ Cardinality(Q) >= QuorumSize
94         /\ \A v \in Q : <<v, s, b, e>> \in votes
95
96  Invariant ==
97    /\ TypeInvariant
98    /\ WitnessAssignmentSoundness
99    /\ NoDualVotes
100   /\ WitnessCertificatesStayAssigned
101   /\ CertSoundness
102
103 Safety ==
104   \A s \in Slots,
105      b1 \in Blocks,
106      b2 \in Blocks,
107      e1 \in Epochs,
108      e2 \in Epochs,
109      Q1 \in SUBSET Validators,
110      Q2 \in SUBSET Validators :
111     /\ <<s, b1, e1, Q1>> \in certs
```

```
    /\ <<s, b2, e2, Q2>> \in certs
    => /\ b1 = b2
       /\ e1 = e2

Spec == Init /\ [][Next]_vars

THEOREM InitImpliesTypeInvariant == Init => TypeInvariant
  BY SMTT(30), InitialWitnessInWitnesses, MaxReassignmentDepthIsNat DEF Init, TypeInvariant,
  ↪  VoteDomain,
               WitnessCertDomain, CertDomain

THEOREM InitImpliesWitnessAssignmentSoundness == Init => WitnessAssignmentSoundness
  BY SMTT(30), MaxReassignmentDepthIsNat DEF Init, WitnessAssignmentSoundness

THEOREM InitImpliesNoDualVotes == Init => NoDualVotes
  BY DEF Init, NoDualVotes

THEOREM InitImpliesWitnessCertificatesStayAssigned == Init => WitnessCertificatesStayAssigned
  BY DEF Init, WitnessCertificatesStayAssigned

THEOREM InitImpliesCertSoundness == Init => CertSoundness
  BY DEF Init, CertSoundness

THEOREM InitImpliesInvariant == Init => Invariant
  BY InitImpliesTypeInvariant, InitImpliesWitnessAssignmentSoundness,
     InitImpliesNoDualVotes, InitImpliesWitnessCertificatesStayAssigned,
     InitImpliesCertSoundness DEF Invariant

THEOREM IssueWitnessPreservesInvariant == Invariant /\ IssueWitnessStep => Invariant'
  BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
               WitnessCertificatesStayAssigned, CertSoundness, IssueWitnessStep,
               HasWitnessCert, WitnessCertDomain

THEOREM ReassignWitnessPreservesInvariant == Invariant /\ ReassignWitnessStep => Invariant'
  BY SMTT(30), MaxReassignmentDepthIsNat DEF Invariant, TypeInvariant,
  ↪  WitnessAssignmentSoundness, NoDualVotes,
               WitnessCertificatesStayAssigned, CertSoundness, ReassignWitnessStep

THEOREM VotePreservesInvariant == Invariant /\ VoteStep => Invariant'
  BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
               WitnessCertificatesStayAssigned, CertSoundness, VoteStep, HasVote,
               VoteDomain

THEOREM CertifyPreservesInvariant == Invariant /\ CertifyStep => Invariant'
  BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
               WitnessCertificatesStayAssigned, CertSoundness, CertifyStep, CertDomain

THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
  BY IssueWitnessPreservesInvariant, ReassignWitnessPreservesInvariant,
     VotePreservesInvariant, CertifyPreservesInvariant DEF Next

THEOREM StutterPreservesTypeInvariant ==
  TypeInvariant /\ UNCHANGED vars => TypeInvariant'
  BY DEF TypeInvariant, vars

THEOREM StutterPreservesWitnessAssignmentSoundness ==
  WitnessAssignmentSoundness /\ UNCHANGED vars => WitnessAssignmentSoundness'
  BY DEF WitnessAssignmentSoundness, vars

THEOREM StutterPreservesNoDualVotes ==
  NoDualVotes /\ UNCHANGED vars => NoDualVotes'
  BY DEF NoDualVotes, vars
```

```
172
173 THEOREM StutterPreservesWitnessCertificatesStayAssigned ==
174   WitnessCertificatesStayAssigned /\ UNCHANGED vars => WitnessCertificatesStayAssigned'
175   BY DEF WitnessCertificatesStayAssigned, vars
176
177 THEOREM StutterPreservesCertSoundness ==
178   CertSoundness /\ UNCHANGED vars => CertSoundness'
179   BY DEF CertSoundness, vars
180
181 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
182   BY StutterPreservesTypeInvariant, StutterPreservesWitnessAssignmentSoundness,
183      StutterPreservesNoDualVotes, StutterPreservesWitnessCertificatesStayAssigned,
184      StutterPreservesCertSoundness DEF Invariant
185
186 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
187   BY StepPreservesInvariant, StutterPreservesInvariant DEF vars
188
189 THEOREM QuorumCertificatesIntersect ==
190   \A s \in Slots,
191      b1 \in Blocks,
192      b2 \in Blocks,
193      e1 \in Epochs,
194      e2 \in Epochs,
195      Q1 \in SUBSET Validators,
196      Q2 \in SUBSET Validators :
197     /\ Invariant
198     /\ <<s, b1, e1, Q1>> \in certs
199     /\ <<s, b2, e2, Q2>> \in certs
200     => Q1 \cap Q2 # {}
201   BY SMTT(60), QuorumIntersection DEF Invariant, CertSoundness
202
203 THEOREM InvariantImpliesSafety == Invariant => Safety
204   BY SMTT(60), QuorumCertificatesIntersect DEF Invariant, Safety, NoDualVotes,
205                 CertSoundness
206
207 ====
```

A.4.2 NestedGuardian.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardian.tla}

```
1  ---- MODULE NestedGuardian ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5           InitialEpoch, InitialWitness
6
7  ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
8  ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
9
10 ASSUME QuorumIntersection ==
11   \A S, T \in SUBSET Validators :
12     /\ Cardinality(S) >= QuorumSize
13     /\ Cardinality(T) >= QuorumSize
14     => S \cap T # {}
15
16 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
↪  witnessOnline,
```

```
          witnessCheckpoint, assignedWitness, reassignmentDepth

vars == <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
          witnessOnline, witnessCheckpoint, assignedWitness, reassignmentDepth>>

CheckpointLevels == 0..2

VoteEvent(v, s, b, e) == <<v, s, b, e>>
WitnessEvent(w, s, b, e) == <<w, s, b, e>>
Finalization(s, b, e) == <<s, b, e>>

VoteDomain == {VoteEvent(v, s, b, e) : v \in Validators, s \in Slots, b \in Blocks, e \in Epochs}
WitnessDomain == {WitnessEvent(w, s, b, e) : w \in Witnesses, s \in Slots, b \in Blocks, e \in
↪  Epochs}
FinalizationDomain == {Finalization(s, b, e) : s \in Slots, b \in Blocks, e \in Epochs}

Init ==
  /\ votes = {}
  /\ witnessCerts = {}
  /\ finalized = {}
  /\ finalizerSets = [f \in FinalizationDomain |-> {}]
  /\ registryEpoch = [v \in Validators |-> InitialEpoch]
  /\ guardianReady = [v \in Validators |-> TRUE]
  /\ witnessOnline = [w \in Witnesses |-> TRUE]
  /\ witnessCheckpoint = [w \in Witnesses |-> 1]
  /\ assignedWitness = [s \in Slots |-> InitialWitness]
  /\ reassignmentDepth = [s \in Slots |-> 0]

HasVote(v, s) ==
  \E b \in Blocks, e \in Epochs : VoteEvent(v, s, b, e) \in votes

HasWitnessCert(w, s) ==
  \E b \in Blocks, e \in Epochs : WitnessEvent(w, s, b, e) \in witnessCerts

CanIssueWitness(w, s, b, e) ==
  /\ w \in Witnesses
  /\ s \in Slots
  /\ b \in Blocks
  /\ e \in Epochs
  /\ witnessOnline[w]
  /\ witnessCheckpoint[w] > 0
  /\ assignedWitness[s] = w
  /\ ~HasWitnessCert(w, s)

IssueWitness(w, s, b, e) ==
  /\ CanIssueWitness(w, s, b, e)
  /\ witnessCerts' = witnessCerts \cup {WitnessEvent(w, s, b, e)}
  /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, guardianReady, witnessOnline,
                 witnessCheckpoint, assignedWitness, reassignmentDepth>>

CanReassign(s, w) ==
  /\ s \in Slots
  /\ w \in Witnesses
  /\ w # assignedWitness[s]
  /\ ~witnessOnline[assignedWitness[s]]
  /\ reassignmentDepth[s] < MaxReassignmentDepth

ReassignWitness(s, w) ==
  /\ CanReassign(s, w)
  /\ assignedWitness' = [assignedWitness EXCEPT ![s] = w]
  /\ reassignmentDepth' = [reassignmentDepth EXCEPT ![s] = @ + 1]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
```

```
                witnessOnline, witnessCheckpoint>>

CanVote(v, s, b, e) ==
  /\ v \in Validators
  /\ s \in Slots
  /\ b \in Blocks
  /\ e \in Epochs
  /\ guardianReady[v]
  /\ registryEpoch[v] = e
  /\ ~HasVote(v, s)
  /\ WitnessEvent(assignedWitness[s], s, b, e) \in witnessCerts

Vote(v, s, b, e) ==
  /\ CanVote(v, s, b, e)
  /\ votes' = votes \cup {VoteEvent(v, s, b, e)}
  /\ UNCHANGED <<witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
  ↪  witnessOnline,
                witnessCheckpoint, assignedWitness, reassignmentDepth>>

EligibleVoters(s, b, e) ==
  {v \in Validators :
      /\ VoteEvent(v, s, b, e) \in votes
      /\ guardianReady[v]
      /\ registryEpoch[v] = e
      /\ WitnessEvent(assignedWitness[s], s, b, e) \in witnessCerts}

CanFinalize(s, b, e) ==
  /\ s \in Slots
  /\ b \in Blocks
  /\ e \in Epochs
  /\ Cardinality(EligibleVoters(s, b, e)) >= QuorumSize

Finalize(s, b, e) ==
  /\ CanFinalize(s, b, e)
  /\ finalized' = finalized \cup {Finalization(s, b, e)}
  /\ finalizerSets' = [finalizerSets EXCEPT ![Finalization(s, b, e)] = EligibleVoters(s, b, e)]
  /\ UNCHANGED <<votes, witnessCerts, registryEpoch, guardianReady, witnessOnline,
                witnessCheckpoint, assignedWitness, reassignmentDepth>>

AdoptEpoch(v, e) ==
  /\ v \in Validators
  /\ e \in Epochs
  /\ e >= registryEpoch[v]
  /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, guardianReady, witnessOnline,
                witnessCheckpoint, assignedWitness, reassignmentDepth>>

RegistryRollback(v, e) ==
  /\ v \in Validators
  /\ e \in Epochs
  /\ e < registryEpoch[v]
  /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, guardianReady, witnessOnline,
                witnessCheckpoint, assignedWitness, reassignmentDepth>>

GuardianOutage(v) ==
  /\ v \in Validators
  /\ guardianReady[v]
  /\ guardianReady' = [guardianReady EXCEPT ![v] = FALSE]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, witnessOnline,
                witnessCheckpoint, assignedWitness, reassignmentDepth>>

```

```
WitnessOutage(w) ==
  /\ w \in Witnesses
  /\ witnessOnline[w]
  /\ witnessOnline' = [witnessOnline EXCEPT ![w] = FALSE]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
                 witnessCheckpoint, assignedWitness, reassignmentDepth>>

WitnessRecovery(w) ==
  /\ w \in Witnesses
  /\ ~witnessOnline[w]
  /\ witnessOnline' = [witnessOnline EXCEPT ![w] = TRUE]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
                 witnessCheckpoint, assignedWitness, reassignmentDepth>>

AdvanceWitnessCheckpoint(w, level) ==
  /\ w \in Witnesses
  /\ level \in CheckpointLevels
  /\ level > witnessCheckpoint[w]
  /\ witnessCheckpoint' = [witnessCheckpoint EXCEPT ![w] = level]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
                 witnessOnline, assignedWitness, reassignmentDepth>>

WitnessCheckpointRollback(w, level) ==
  /\ w \in Witnesses
  /\ level \in CheckpointLevels
  /\ level < witnessCheckpoint[w]
  /\ witnessCheckpoint' = [witnessCheckpoint EXCEPT ![w] = level]
  /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
                 witnessOnline, assignedWitness, reassignmentDepth>>

Next ==
  \/ \E w \in Witnesses, s \in Slots, b \in Blocks, e \in Epochs : IssueWitness(w, s, b, e)
  \/ \E s \in Slots, w \in Witnesses : ReassignWitness(s, w)
  \/ \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs : Vote(v, s, b, e)
  \/ \E s \in Slots, b \in Blocks, e \in Epochs : Finalize(s, b, e)
  \/ \E v \in Validators, e \in Epochs : AdoptEpoch(v, e)
  \/ \E v \in Validators, e \in Epochs : RegistryRollback(v, e)
  \/ \E v \in Validators : GuardianOutage(v)
  \/ \E w \in Witnesses : WitnessOutage(w)
  \/ \E w \in Witnesses : WitnessRecovery(w)
  \/ \E w \in Witnesses, level \in CheckpointLevels : AdvanceWitnessCheckpoint(w, level)
  \/ \E w \in Witnesses, level \in CheckpointLevels : WitnessCheckpointRollback(w, level)

TypeInvariant ==
  /\ votes \subseteq VoteDomain
  /\ witnessCerts \subseteq WitnessDomain
  /\ finalized \subseteq FinalizationDomain
  /\ finalizerSets \in [FinalizationDomain -> SUBSET Validators]
  /\ registryEpoch \in [Validators -> Epochs]
  /\ guardianReady \in [Validators -> BOOLEAN]
  /\ witnessOnline \in [Witnesses -> BOOLEAN]
  /\ witnessCheckpoint \in [Witnesses -> CheckpointLevels]
  /\ assignedWitness \in [Slots -> Witnesses]
  /\ reassignmentDepth \in [Slots -> 0..MaxReassignmentDepth]

WitnessAssignmentSoundness ==
  \A s \in Slots :
    reassignmentDepth[s] <= MaxReassignmentDepth

NoDualVotes ==
  \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ VoteEvent(v, s, b1, e1) \in votes
```

```
    /\ VoteEvent(v, s, b2, e2) \in votes
    => /\ b1 = b2
       /\ e1 = e2

WitnessCertificatesStayAssigned ==
  \A w \in Witnesses, s \in Slots, b \in Blocks, e \in Epochs :
    WitnessEvent(w, s, b, e) \in witnessCerts
    => assignedWitness[s] = w \/ reassignmentDepth[s] > 0

FinalizationWitnessSoundness ==
  \A s \in Slots, b \in Blocks, e \in Epochs :
    Finalization(s, b, e) \in finalized
    => /\ Cardinality(finalizerSets[Finalization(s, b, e)]) >= QuorumSize
       /\ \A v \in finalizerSets[Finalization(s, b, e)] :
            VoteEvent(v, s, b, e) \in votes

Invariant ==
  /\ TypeInvariant
  /\ WitnessAssignmentSoundness
  /\ NoDualVotes
  /\ WitnessCertificatesStayAssigned
  /\ FinalizationWitnessSoundness

Safety ==
  \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
    /\ Finalization(s, b1, e1) \in finalized
    /\ Finalization(s, b2, e2) \in finalized
    => b1 = b2

Spec ==
  Init /\ [][Next]_vars

THEOREM OneInCheckpointLevels == 1 \in CheckpointLevels
  BY SMT DEF CheckpointLevels

THEOREM InitImpliesInvariant == Init => Invariant
PROOF
<1>1. Init => TypeInvariant
  BY SMT, InitialEpochInEpochs, InitialWitnessInWitnesses, OneInCheckpointLevels
     DEF Init, TypeInvariant, VoteDomain, WitnessDomain, FinalizationDomain
<1>2. Init => WitnessAssignmentSoundness
  BY DEF Init, WitnessAssignmentSoundness
<1>3. Init => NoDualVotes
  BY DEF Init, NoDualVotes
<1>4. Init => WitnessCertificatesStayAssigned
  BY DEF Init, WitnessCertificatesStayAssigned
<1>5. Init => FinalizationWitnessSoundness
  BY DEF Init, FinalizationWitnessSoundness
<1>6. QED
  BY <1>1, <1>2, <1>3, <1>4, <1>5 DEF Invariant

THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
  BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
              WitnessCertificatesStayAssigned, FinalizationWitnessSoundness, Next,
              IssueWitness, ReassignWitness, Vote, Finalize, AdoptEpoch,
              RegistryRollback, GuardianOutage, WitnessOutage, WitnessRecovery,
              AdvanceWitnessCheckpoint, WitnessCheckpointRollback, CanIssueWitness,
              CanVote, HasVote, CanFinalize, EligibleVoters, VoteEvent,
              WitnessEvent, Finalization, VoteDomain, WitnessDomain, FinalizationDomain

THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
  BY SMT DEF Invariant, vars
```

```
263
264 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
265   BY SMT, StepPreservesInvariant, StutterPreservesInvariant DEF vars
266
267 THEOREM FinalizedWitnessesIntersect ==
268   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
269     /\ Invariant
270     /\ <<s, b1, e1>> \in finalized
271     /\ <<s, b2, e2>> \in finalized
272     => \E v \in Validators :
273          /\ <<v, s, b1, e1>> \in votes
274          /\ <<v, s, b2, e2>> \in votes
275 PROOF
276 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
277 <1>2. ASSUME Invariant,
278                <<s, b1, e1>> \in finalized,
279                <<s, b2, e2>> \in finalized
280         PROVE \E v \in Validators :
281                 /\ <<v, s, b1, e1>> \in votes
282                 /\ <<v, s, b2, e2>> \in votes
283   <2>1. finalizerSets[<<s, b1, e1>>] \subseteq Validators
284     BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
285   <2>2. finalizerSets[<<s, b2, e2>>] \subseteq Validators
286     BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
287   <2>3. Cardinality(finalizerSets[<<s, b1, e1>>]) >= QuorumSize
288     BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
289   <2>4. Cardinality(finalizerSets[<<s, b2, e2>>]) >= QuorumSize
290     BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
291   <2>5. finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] # {}
292     BY <2>1, <2>2, <2>3, <2>4, QuorumIntersection
293   <2>6. PICK v \in finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] : TRUE
294     BY <2>5
295   <2>7. <<v, s, b1, e1>> \in votes
296     BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
297   <2>8. <<v, s, b2, e2>> \in votes
298     BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
299   <2>9. QED
300     BY SMT, <2>6, <2>7, <2>8
301 <1>3. QED
302   BY <1>2
303
304 THEOREM InvariantImpliesSafety == Invariant => Safety
305 PROOF
306 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
307 <1>2. ASSUME Invariant,
308                <<s, b1, e1>> \in finalized,
309                <<s, b2, e2>> \in finalized
310         PROVE b1 = b2
311   <2>1. \E v \in Validators :
312           /\ <<v, s, b1, e1>> \in votes
313           /\ <<v, s, b2, e2>> \in votes
314     BY SMT, FinalizedWitnessesIntersect, <1>2
315   <2>2. PICK v \in Validators :
316           /\ <<v, s, b1, e1>> \in votes
317           /\ <<v, s, b2, e2>> \in votes
318     BY <2>1
319   <2>3. QED
320     BY SMT, <1>2, <2>2 DEF Invariant, NoDualVotes
321 <1>3. QED
322   BY <1>2 DEF Safety
323
324 ====
```

A.5 CanonicalOrdering Companion Witnesses

Executable witness artifacts for omission dominance and recursive continuity.

A.5.1 CanonicalOrderingOmissionTrace.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingOmissionTrace.tla}

```
---- MODULE CanonicalOrderingOmissionTrace ----
EXTENDS CanonicalOrdering

TargetSlot == 1
TargetCert == {"tx1"}
TargetTx == "tx2"

WitnessState ==
  /\ TargetSlot \in closedCutoffs
  /\ bulletin[TargetSlot] = {"tx1", "tx2"}
  /\ CertEvent(TargetSlot, TargetCert) \in candidateCerts
  /\ OmissionEvent(TargetSlot, TargetCert, TargetTx) \in omissionProofs
  /\ CertEvent(TargetSlot, TargetCert) \notin admittedCerts

NoOmissionDominanceWitness == ~WitnessState

====
```

A.5.2 CanonicalOrderingOmissionTrace.cfg

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingOmissionTrace.cfg}

```
SPECIFICATION Spec

CONSTANTS
  Slots = {1}
  Transactions = {"tx1", "tx2"}

INVARIANTS
  TypeInvariant
  AdmittedSoundness
  OmissionSoundness
  RecoveredSoundness
  AdmittedUniqueness
  Recoverability
  OmissionDominates
  NoOmissionDominanceWitness
```

A.5.3 CanonicalCollapseRecursiveContinuity.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseRecursiveContinuity.tla}

```
---- MODULE CanonicalCollapseRecursiveContinuity ----
EXTENDS Naturals, Sequences, TLC

CONSTANT MaxHeight

Slots == 1..MaxHeight
Genesis == 1
NonGenesisSlots == Slots \ {Genesis}
NullHash == 0

VARIABLES publishedCollapses, publishedProofs, publishedExtensionCertificates, admittedHeaders

vars ==
  <<publishedCollapses, publishedProofs, publishedExtensionCertificates, admittedHeaders>>

RECURSIVE AccumulatorHash(_), ProofHash(_), ProofChain(_)

PayloadHash(s) == <<"payload", s>>

AccumulatorHash(s) ==
  IF s = Genesis
    THEN <<"accum", s, NullHash, PayloadHash(s)>>
    ELSE <<"accum", s, AccumulatorHash(s - 1), PayloadHash(s)>>

Commitment(s) ==
  [height |-> s,
   continuityAccumulatorHash |-> AccumulatorHash(s),
   resultingStateRootHash |-> <<"state", s>>]

CommitmentHash(s) == <<"commitment-hash", Commitment(s)>>

PreviousCommitmentHash(s) ==
  IF s = Genesis THEN NullHash ELSE CommitmentHash(s - 1)

StatementHash(s) ==
  <<"statement-hash", Commitment(s), PreviousCommitmentHash(s), PayloadHash(s)>>

PreviousProofHash(s) ==
  IF s = Genesis THEN NullHash ELSE ProofHash(s - 1)

ProofBytes(s) ==
  <<"hash-pcd-v1", StatementHash(s), PreviousProofHash(s)>>

ProofStep(s) ==
  [commitment |-> Commitment(s),
   previousCommitmentHash |-> PreviousCommitmentHash(s),
   payloadHash |-> PayloadHash(s),
   previousProofHash |-> PreviousProofHash(s),
   proofBytes |-> ProofBytes(s)]

ProofHash(s) == <<"proof-hash", ProofStep(s)>>

ProofChain(s) ==
  IF s = Genesis
    THEN <<ProofStep(s)>>
    ELSE Append(ProofChain(s - 1), ProofStep(s))

ExtensionCertificate(s) ==
  [coveredHeight |-> s,
   predecessorCommitment |-> Commitment(s - 1),
   predecessorProofHash |-> ProofHash(s - 1)]
```

```
62
63  Last(seq) == seq[Len(seq)]
64
65  Init ==
66    /\ publishedCollapses = {}
67    /\ publishedProofs = {}
68    /\ publishedExtensionCertificates = {}
69    /\ admittedHeaders = {}
70
71  PublishCollapseStep ==
72    \E s \in Slots :
73      /\ s \notin publishedCollapses
74      /\ publishedCollapses' = publishedCollapses \cup {s}
75      /\ UNCHANGED <<publishedProofs, publishedExtensionCertificates, admittedHeaders>>
76
77  PublishRecursiveProofStep ==
78    \E s \in Slots :
79      /\ s \in publishedCollapses
80      /\ s \notin publishedProofs
81      /\ IF s = Genesis THEN TRUE ELSE s - 1 \in publishedProofs
82      /\ publishedProofs' = publishedProofs \cup {s}
83      /\ UNCHANGED <<publishedCollapses, publishedExtensionCertificates, admittedHeaders>>
84
85  PublishExtensionCertificateStep ==
86    \E s \in NonGenesisSlots :
87      /\ s \in publishedCollapses
88      /\ s - 1 \in publishedProofs
89      /\ s \notin publishedExtensionCertificates
90      /\ publishedExtensionCertificates' = publishedExtensionCertificates \cup {s}
91      /\ UNCHANGED <<publishedCollapses, publishedProofs, admittedHeaders>>
92
93  AdmitHeaderStep ==
94    \E s \in Slots :
95      /\ s \in publishedCollapses
96      /\ s \notin admittedHeaders
97      /\ IF s = Genesis THEN TRUE ELSE s \in publishedExtensionCertificates
98      /\ admittedHeaders' = admittedHeaders \cup {s}
99      /\ UNCHANGED <<publishedCollapses, publishedProofs, publishedExtensionCertificates>>
100
101 Next ==
102   \/ PublishCollapseStep
103   \/ PublishRecursiveProofStep
104   \/ PublishExtensionCertificateStep
105   \/ AdmitHeaderStep
106
107 TypeInvariant ==
108   /\ publishedCollapses \subseteq Slots
109   /\ publishedProofs \subseteq Slots
110   /\ publishedExtensionCertificates \subseteq NonGenesisSlots
111   /\ admittedHeaders \subseteq Slots
112
113 RecursiveProofSoundness ==
114   \A s \in publishedProofs :
115     /\ s \in publishedCollapses
116     /\ IF s = Genesis THEN TRUE ELSE s - 1 \in publishedProofs
117     /\ LET step == ProofStep(s) IN
118          /\ step.commitment = Commitment(s)
119          /\ step.previousCommitmentHash = PreviousCommitmentHash(s)
120          /\ step.payloadHash = PayloadHash(s)
121          /\ step.previousProofHash = PreviousProofHash(s)
122          /\ step.proofBytes = ProofBytes(s)
123          /\ step.commitment.continuityAccumulatorHash = AccumulatorHash(s)
```

```
124
125 RecursiveProofChainSoundness ==
126   \A s \in publishedProofs :
127     /\ Len(ProofChain(s)) = s
128     /\ Last(ProofChain(s)) = ProofStep(s)
129     /\ IF s = Genesis
130          THEN TRUE
131          ELSE /\ ProofChain(s)[Len(ProofChain(s)) - 1] = ProofStep(s - 1)
132
133 ExtensionCertificateSoundness ==
134   \A s \in publishedExtensionCertificates :
135     /\ s \in NonGenesisSlots
136     /\ s - 1 \in publishedProofs
137     /\ ExtensionCertificate(s).coveredHeight = s
138     /\ ExtensionCertificate(s).predecessorCommitment = Commitment(s - 1)
139     /\ ExtensionCertificate(s).predecessorProofHash = ProofHash(s - 1)
140
141 HeaderAdmissionSoundness ==
142   \A s \in admittedHeaders :
143     /\ s \in publishedCollapses
144     /\ IF s = Genesis
145          THEN TRUE
146          ELSE /\ s \in publishedExtensionCertificates
147               /\ s - 1 \in publishedProofs
148               /\ ExtensionCertificate(s).predecessorCommitment = Commitment(s - 1)
149               /\ ExtensionCertificate(s).predecessorProofHash = ProofHash(s - 1)
150
151 Invariant ==
152   /\ TypeInvariant
153   /\ RecursiveProofSoundness
154   /\ RecursiveProofChainSoundness
155   /\ ExtensionCertificateSoundness
156   /\ HeaderAdmissionSoundness
157
158 Spec == Init /\ [][Next]_vars
159
160 ====
```

A.5.4 CanonicalCollapseRecursiveContinuity.cfg

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseRecursiveContinuity.cfg}

```
1  SPECIFICATION Spec
2
3  CONSTANTS
4    MaxHeight = 3
5
6  INVARIANTS
7    TypeInvariant
8    RecursiveProofSoundness
9    RecursiveProofChainSoundness
10   ExtensionCertificateSoundness
11   HeaderAdmissionSoundness
```

A.6 Recovery and Classical-Agreement Bridge

Recurring recovery kernel, finite reduction, totality bridge, and final classical-agreement collapse wrapper.

A.6.1 `NestedGuardianRecoveryRecurringInductionCore.tla`

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryRecurringInductionCore.tla}

```
---- MODULE NestedGuardianRecoveryRecurringInductionCore ----
(*************************************************************************)
(* PREMISE-CONTRACT KERNEL (requalified 2026-08-17, AFT-CB P2.2, standing  *)
(* rule 2 -- labels claim only what is checked).  `RecoveryInductionBase == *)
(* <>RecoveryClosedPrefix(1)` and the step contracts below are PREMISES:   *)
(* assumed contracts whose sufficiency to close the prefix is what this    *)
(* module checks -- the premises themselves are NOT derived here from a     *)
(* Byzantine network plus a dishonest-weight bound (the Q3 finding).  The  *)
(* boundary lane's successor does not inherit these premises: the          *)
(* common-boundary cadence obligations are checked directly, under         *)
(* explicit fairness, in common_boundary/BoundaryLiveness.tla              *)
(* (model-checked at n≤4, stated as such -- never "proven").               *)
(*************************************************************************)
EXTENDS Naturals, FiniteSets, TLC

CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
         InitialEpoch, InitialWitness, TotalCycles,
         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
         SmallRecoveryCommittee, LargeRecoveryCommittee

ASSUME InitialEpoch \in Epochs
ASSUME InitialWitness \in Witnesses
ASSUME QuorumSize \in 1..Cardinality(Validators)
ASSUME TotalCycles \in Nat
ASSUME TotalCycles >= 1
ASSUME \A c \in 1..TotalCycles :
  /\ TargetSlotOf(c) \in Slots
  /\ TargetBlockOf(c) \in Blocks
  /\ StableEpochOf(c) \in Epochs

VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
↪  witnessOnline,
          witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
          continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
          continuationPagePublished, continuationPageBoundary, continuationFetched,
          shareReceipts, shareConflicts, windowClosed, missingShareClaims,
          missingThresholdCertificates, recovered, recoveredSurfaces,
          recoveryConflicts, aborted

Core ==
  INSTANCE NestedGuardianRecoveryRecurringLivenessCore
  WITH Validators <- Validators,
       Witnesses <- Witnesses,
       Blocks <- Blocks,
       Slots <- Slots,
       Epochs <- Epochs,
       QuorumSize <- QuorumSize,
```

```
        MaxReassignmentDepth <- MaxReassignmentDepth,
        InitialEpoch <- InitialEpoch,
        InitialWitness <- InitialWitness,
        TotalCycles <- TotalCycles,
        TargetSlotOf <- TargetSlotOf,
        TargetBlockOf <- TargetBlockOf,
        StableEpochOf <- StableEpochOf,
        SmallCommitteeSlot <- SmallCommitteeSlot,
        SmallRecoveryThreshold <- SmallRecoveryThreshold,
        LargeRecoveryThreshold <- LargeRecoveryThreshold,
        SmallRecoveryCommittee <- SmallRecoveryCommittee,
        LargeRecoveryCommittee <- LargeRecoveryCommittee,
        votes <- votes,
        witnessCerts <- witnessCerts,
        finalized <- finalized,
        finalizerSets <- finalizerSets,
        registryEpoch <- registryEpoch,
        guardianReady <- guardianReady,
        witnessOnline <- witnessOnline,
        witnessCheckpoint <- witnessCheckpoint,
        assignedWitness <- assignedWitness,
        reassignmentDepth <- reassignmentDepth,
        currentCycle <- currentCycle,
        phase <- phase,
        churnStage <- churnStage,
        continuationBoundary <- continuationBoundary,
        continuationAnchorPublished <- continuationAnchorPublished,
        continuationAnchorBoundary <- continuationAnchorBoundary,
        continuationPagePublished <- continuationPagePublished,
        continuationPageBoundary <- continuationPageBoundary,
        continuationFetched <- continuationFetched,
        shareReceipts <- shareReceipts,
        shareConflicts <- shareConflicts,
        windowClosed <- windowClosed,
        missingShareClaims <- missingShareClaims,
        missingThresholdCertificates <- missingThresholdCertificates,
        recovered <- recovered,
        recoveredSurfaces <- recoveredSurfaces,
        recoveryConflicts <- recoveryConflicts,
        aborted <- aborted

vars == Core!vars
CycleRange == Core!CycleRange
PrefixAdvanceRange == Core!PrefixAdvanceRange
RecoveryClosedPrefix(c) == Core!RecoveryClosedPrefix(c)

PriorCycleRange(c) ==
  IF c = 1 THEN {}
  ELSE 1..(c - 1)

RecoveryInductionBase ==
  <>RecoveryClosedPrefix(1)

RecoveryInductionStep(c) ==
  /\ c \in PrefixAdvanceRange
  /\ Core!RecoveryPrefixAdvanceContract(c)

RecoveryInductionPremisesUpTo(c) ==
  /\ c \in CycleRange
  /\ RecoveryInductionBase
  /\ \A i \in PriorCycleRange(c) :
       RecoveryInductionStep(i)
```

```
110
111 RecoveryRecurringInductionPremises ==
112   /\ RecoveryInductionBase
113   /\ \A i \in PrefixAdvanceRange :
114        RecoveryInductionStep(i)
115
116 RecoveryClosedPrefixInductionKernel(c) ==
117   /\ c \in CycleRange
118   /\ (RecoveryInductionPremisesUpTo(c)
119       => <>RecoveryClosedPrefix(c))
120
121 RecoveryRecurringInductionKernel ==
122   \A c \in CycleRange :
123     RecoveryClosedPrefixInductionKernel(c)
124
125 ====
```

A.6.2 NestedGuardianRecoveryRecurringProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryRecurringProof.tla}

```
1  ---- MODULE NestedGuardianRecoveryRecurringProof ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5           InitialEpoch, InitialWitness, TotalCycles,
6           TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7           SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8           SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
11 ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
12 ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCyclesIsNat == TotalCycles \in Nat
14 ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
15 ASSUME TargetCycleDomains ==
16   \A c \in 1..TotalCycles :
17   /\ TargetSlotOf(c) \in Slots
18   /\ TargetBlockOf(c) \in Blocks
19   /\ StableEpochOf(c) \in Epochs
20
21 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
   ↪  witnessOnline,
22           witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
23           continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
24           continuationPagePublished, continuationPageBoundary, continuationFetched,
25           shareReceipts, shareConflicts, windowClosed, missingShareClaims,
26           missingThresholdCertificates, recovered, recoveredSurfaces,
27           recoveryConflicts, aborted
28
29 Core ==
30   INSTANCE NestedGuardianRecoveryRecurringInductionCore
31   WITH Validators <- Validators,
32        Witnesses <- Witnesses,
33        Blocks <- Blocks,
34        Slots <- Slots,
35        Epochs <- Epochs,
36        QuorumSize <- QuorumSize,
```

```
        MaxReassignmentDepth <- MaxReassignmentDepth,
        InitialEpoch <- InitialEpoch,
        InitialWitness <- InitialWitness,
        TotalCycles <- TotalCycles,
        TargetSlotOf <- TargetSlotOf,
        TargetBlockOf <- TargetBlockOf,
        StableEpochOf <- StableEpochOf,
        SmallCommitteeSlot <- SmallCommitteeSlot,
        SmallRecoveryThreshold <- SmallRecoveryThreshold,
        LargeRecoveryThreshold <- LargeRecoveryThreshold,
        SmallRecoveryCommittee <- SmallRecoveryCommittee,
        LargeRecoveryCommittee <- LargeRecoveryCommittee,
        votes <- votes,
        witnessCerts <- witnessCerts,
        finalized <- finalized,
        finalizerSets <- finalizerSets,
        registryEpoch <- registryEpoch,
        guardianReady <- guardianReady,
        witnessOnline <- witnessOnline,
        witnessCheckpoint <- witnessCheckpoint,
        assignedWitness <- assignedWitness,
        reassignmentDepth <- reassignmentDepth,
        currentCycle <- currentCycle,
        phase <- phase,
        churnStage <- churnStage,
        continuationBoundary <- continuationBoundary,
        continuationAnchorPublished <- continuationAnchorPublished,
        continuationAnchorBoundary <- continuationAnchorBoundary,
        continuationPagePublished <- continuationPagePublished,
        continuationPageBoundary <- continuationPageBoundary,
        continuationFetched <- continuationFetched,
        shareReceipts <- shareReceipts,
        shareConflicts <- shareConflicts,
        windowClosed <- windowClosed,
        missingShareClaims <- missingShareClaims,
        missingThresholdCertificates <- missingThresholdCertificates,
        recovered <- recovered,
        recoveredSurfaces <- recoveredSurfaces,
        recoveryConflicts <- recoveryConflicts,
        aborted <- aborted

RecoveryRecurringClosedPrefixes ==
  \A c \in Core!CycleRange :
    <>Core!RecoveryClosedPrefix(c)

RecoveryParametricRecurrenceArgument ==
  /\ Core!RecoveryRecurringInductionPremises
  /\ Core!RecoveryRecurringInductionKernel
  => RecoveryRecurringClosedPrefixes

THEOREM PriorCycleRangeImpliesTotalCyclesNotOne ==
  \A c \in Core!CycleRange :
    \A i \in Core!PriorCycleRange(c) :
      TotalCycles # 1
  BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
     DEF Core!CycleRange, Core!Core!CycleRange, Core!PriorCycleRange

THEOREM PriorCycleRangeImpliesPrefixAdvanceInterval ==
  \A c \in Core!CycleRange :
    \A i \in Core!PriorCycleRange(c) :
      i \in 1..(TotalCycles - 1)
  BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
```

```
     DEF Core!CycleRange, Core!Core!CycleRange, Core!PriorCycleRange

THEOREM RecoveryInductionPremisesRestrict ==
  \A c \in Core!CycleRange :
    Core!RecoveryRecurringInductionPremises
      => Core!RecoveryInductionPremisesUpTo(c)
PROOF
  <1>1. TAKE c \in Core!CycleRange
  <1>2. ASSUME Core!RecoveryRecurringInductionPremises
        PROVE Core!RecoveryInductionPremisesUpTo(c)
    <2>1. Core!RecoveryInductionBase
          BY <1>2 DEF Core!RecoveryRecurringInductionPremises
    <2>2. \A i \in Core!PriorCycleRange(c) :
             Core!RecoveryInductionStep(i)
      PROOF
        <3>1. TAKE i \in Core!PriorCycleRange(c)
        <3>2. i \in Core!PrefixAdvanceRange
          PROOF
            <4>1. TotalCycles # 1
                  BY PriorCycleRangeImpliesTotalCyclesNotOne, <1>1, <3>1
            <4>2. i \in 1..(TotalCycles - 1)
                  BY PriorCycleRangeImpliesPrefixAdvanceInterval, <1>1, <3>1
            <4>3. TotalCycles # 1
                  BY <4>1
            <4>6. QED
                  BY <4>3, <4>2
                     DEF Core!PrefixAdvanceRange, Core!Core!PrefixAdvanceRange
        <3>3. Core!RecoveryInductionStep(i)
              BY <1>2, <3>2 DEF Core!RecoveryRecurringInductionPremises
        <3>4. QED
              BY <3>3
      <2>3. QED
          BY <1>1, <2>1, <2>2
             DEF Core!RecoveryInductionPremisesUpTo
  <1>3. QED
        BY <1>2

THEOREM RecoveryInductionKernelRestrict ==
  \A c \in Core!CycleRange :
    Core!RecoveryRecurringInductionKernel
      => Core!RecoveryClosedPrefixInductionKernel(c)
  BY DEF Core!RecoveryRecurringInductionKernel,
         Core!RecoveryClosedPrefixInductionKernel,
         Core!CycleRange

THEOREM RecoveryRecurringClosedPrefixFromKernel ==
  \A c \in Core!CycleRange :
    /\ Core!RecoveryRecurringInductionPremises
    /\ Core!RecoveryRecurringInductionKernel
    => <>Core!RecoveryClosedPrefix(c)
PROOF
  <1>1. TAKE c \in Core!CycleRange
  <1>2. ASSUME Core!RecoveryRecurringInductionPremises,
               Core!RecoveryRecurringInductionKernel
        PROVE <>Core!RecoveryClosedPrefix(c)
    <2>1. Core!RecoveryInductionPremisesUpTo(c)
          BY RecoveryInductionPremisesRestrict, <1>1, <1>2
    <2>2. Core!RecoveryClosedPrefixInductionKernel(c)
          BY RecoveryInductionKernelRestrict, <1>1, <1>2
    <2>3. QED
          BY <2>1, <2>2
             DEF Core!RecoveryClosedPrefixInductionKernel
```

```
  <1>3. QED
        BY <1>2

THEOREM RecoveryParametricRecurrenceArgumentTheorem ==
  RecoveryParametricRecurrenceArgument
  BY RecoveryRecurringClosedPrefixFromKernel
     DEF RecoveryParametricRecurrenceArgument,
         RecoveryRecurringClosedPrefixes

====
```

A.6.3 NestedGuardianRecoveryClassicalAgreementReduction.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementReduction.tla}

```
---- MODULE NestedGuardianRecoveryClassicalAgreementReduction ----
EXTENDS Naturals, FiniteSets, TLC, TLAPS

CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
         InitialEpoch, InitialWitness, TotalCycles,
         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
         SmallRecoveryCommittee, LargeRecoveryCommittee

ASSUME InitialEpoch \in Epochs
ASSUME InitialWitness \in Witnesses
ASSUME QuorumSize \in 1..Cardinality(Validators)
ASSUME TotalCycles \in Nat
ASSUME TotalCycles >= 1
ASSUME \A c \in 1..TotalCycles :
  /\ TargetSlotOf(c) \in Slots
  /\ TargetBlockOf(c) \in Blocks
  /\ StableEpochOf(c) \in Epochs

VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
↪  witnessOnline,
          witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
          continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
          continuationPagePublished, continuationPageBoundary, continuationFetched,
          shareReceipts, shareConflicts, windowClosed, missingShareClaims,
          missingThresholdCertificates, recovered, recoveredSurfaces,
          recoveryConflicts, aborted

Proof ==
  INSTANCE NestedGuardianRecoveryRecurringProof
  WITH Validators <- Validators,
       Witnesses <- Witnesses,
       Blocks <- Blocks,
       Slots <- Slots,
       Epochs <- Epochs,
       QuorumSize <- QuorumSize,
       MaxReassignmentDepth <- MaxReassignmentDepth,
       InitialEpoch <- InitialEpoch,
       InitialWitness <- InitialWitness,
       TotalCycles <- TotalCycles,
       TargetSlotOf <- TargetSlotOf,
       TargetBlockOf <- TargetBlockOf,
       StableEpochOf <- StableEpochOf,
```

```
        SmallCommitteeSlot <- SmallCommitteeSlot,
        SmallRecoveryThreshold <- SmallRecoveryThreshold,
        LargeRecoveryThreshold <- LargeRecoveryThreshold,
        SmallRecoveryCommittee <- SmallRecoveryCommittee,
        LargeRecoveryCommittee <- LargeRecoveryCommittee,
        votes <- votes,
        witnessCerts <- witnessCerts,
        finalized <- finalized,
        finalizerSets <- finalizerSets,
        registryEpoch <- registryEpoch,
        guardianReady <- guardianReady,
        witnessOnline <- witnessOnline,
        witnessCheckpoint <- witnessCheckpoint,
        assignedWitness <- assignedWitness,
        reassignmentDepth <- reassignmentDepth,
        currentCycle <- currentCycle,
        phase <- phase,
        churnStage <- churnStage,
        continuationBoundary <- continuationBoundary,
        continuationAnchorPublished <- continuationAnchorPublished,
        continuationAnchorBoundary <- continuationAnchorBoundary,
        continuationPagePublished <- continuationPagePublished,
        continuationPageBoundary <- continuationPageBoundary,
        continuationFetched <- continuationFetched,
        shareReceipts <- shareReceipts,
        shareConflicts <- shareConflicts,
        windowClosed <- windowClosed,
        missingShareClaims <- missingShareClaims,
        missingThresholdCertificates <- missingThresholdCertificates,
        recovered <- recovered,
        recoveredSurfaces <- recoveredSurfaces,
        recoveryConflicts <- recoveryConflicts,
        aborted <- aborted

CycleRange == 1..TotalCycles

ClassicalAgreementDecisionObject(c) ==
  [cycle |-> c,
   slot |-> TargetSlotOf(c),
   block |-> TargetBlockOf(c),
   epoch |-> StableEpochOf(c)]

ClassicalAgreementPrefixObject(c) ==
  [i \in 1..c |-> ClassicalAgreementDecisionObject(i)]

FiniteClassicalAgreementPrefixRealized(c) ==
  /\ c \in CycleRange
  /\ <>Proof!Core!RecoveryClosedPrefix(c)

FiniteClassicalAgreementLiveness ==
  \A c \in CycleRange :
    FiniteClassicalAgreementPrefixRealized(c)

ClassicalAgreementFiniteReduction ==
  Proof!RecoveryRecurringClosedPrefixes
    => FiniteClassicalAgreementLiveness

THEOREM ProofCycleRangeMatchesCycleRange ==
  Proof!Core!CycleRange = CycleRange
  BY DEF Proof!Core!CycleRange, Proof!Core!Core!CycleRange, CycleRange

THEOREM FiniteClassicalAgreementPrefixFromRecurringPrefixes ==
```

```
  \A c \in CycleRange :
    Proof!RecoveryRecurringClosedPrefixes
      => FiniteClassicalAgreementPrefixRealized(c)
PROOF
  <1>1. TAKE c \in CycleRange
  <1>2. ASSUME Proof!RecoveryRecurringClosedPrefixes
        PROVE FiniteClassicalAgreementPrefixRealized(c)
    <2>1. c \in Proof!Core!CycleRange
          BY ProofCycleRangeMatchesCycleRange, <1>1
    <2>2. <>Proof!Core!RecoveryClosedPrefix(c)
          BY <1>2, <2>1 DEF Proof!RecoveryRecurringClosedPrefixes
    <2>3. QED
          BY <1>1, <2>2 DEF FiniteClassicalAgreementPrefixRealized
  <1>3. QED
        BY <1>2

THEOREM ClassicalAgreementFiniteReductionTheorem ==
  ClassicalAgreementFiniteReduction
  BY FiniteClassicalAgreementPrefixFromRecurringPrefixes
     DEF ClassicalAgreementFiniteReduction,
         FiniteClassicalAgreementLiveness

====
```

A.6.4 NestedGuardianRecoveryClassicalAgreementTotality.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementTotality.tla}

```
---- MODULE NestedGuardianRecoveryClassicalAgreementTotality ----
EXTENDS Naturals, FiniteSets, TLC, TLAPS

CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
         InitialEpoch, InitialWitness, TotalCycles,
         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
         SmallRecoveryCommittee, LargeRecoveryCommittee

ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
ASSUME TotalCyclesIsNat == TotalCycles \in Nat
ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
ASSUME TargetCycleDomains ==
  \A c \in 1..TotalCycles :
  /\ TargetSlotOf(c) \in Slots
  /\ TargetBlockOf(c) \in Blocks
  /\ StableEpochOf(c) \in Epochs

VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
↪  witnessOnline,
          witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
          continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
          continuationPagePublished, continuationPageBoundary, continuationFetched,
          shareReceipts, shareConflicts, windowClosed, missingShareClaims,
          missingThresholdCertificates, recovered, recoveredSurfaces,
          recoveryConflicts, aborted

Reduction ==
```

```
  INSTANCE NestedGuardianRecoveryClassicalAgreementReduction
  WITH Validators <- Validators,
       Witnesses <- Witnesses,
       Blocks <- Blocks,
       Slots <- Slots,
       Epochs <- Epochs,
       QuorumSize <- QuorumSize,
       MaxReassignmentDepth <- MaxReassignmentDepth,
       InitialEpoch <- InitialEpoch,
       InitialWitness <- InitialWitness,
       TotalCycles <- TotalCycles,
       TargetSlotOf <- TargetSlotOf,
       TargetBlockOf <- TargetBlockOf,
       StableEpochOf <- StableEpochOf,
       SmallCommitteeSlot <- SmallCommitteeSlot,
       SmallRecoveryThreshold <- SmallRecoveryThreshold,
       LargeRecoveryThreshold <- LargeRecoveryThreshold,
       SmallRecoveryCommittee <- SmallRecoveryCommittee,
       LargeRecoveryCommittee <- LargeRecoveryCommittee,
       votes <- votes,
       witnessCerts <- witnessCerts,
       finalized <- finalized,
       finalizerSets <- finalizerSets,
       registryEpoch <- registryEpoch,
       guardianReady <- guardianReady,
       witnessOnline <- witnessOnline,
       witnessCheckpoint <- witnessCheckpoint,
       assignedWitness <- assignedWitness,
       reassignmentDepth <- reassignmentDepth,
       currentCycle <- currentCycle,
       phase <- phase,
       churnStage <- churnStage,
       continuationBoundary <- continuationBoundary,
       continuationAnchorPublished <- continuationAnchorPublished,
       continuationAnchorBoundary <- continuationAnchorBoundary,
       continuationPagePublished <- continuationPagePublished,
       continuationPageBoundary <- continuationPageBoundary,
       continuationFetched <- continuationFetched,
       shareReceipts <- shareReceipts,
       shareConflicts <- shareConflicts,
       windowClosed <- windowClosed,
       missingShareClaims <- missingShareClaims,
       missingThresholdCertificates <- missingThresholdCertificates,
       recovered <- recovered,
       recoveredSurfaces <- recoveredSurfaces,
       recoveryConflicts <- recoveryConflicts,
       aborted <- aborted

CycleRange == 1..TotalCycles

RecoveryRecurringClosedPrefixes == Reduction!Proof!RecoveryRecurringClosedPrefixes
RecoveryRecurringInductionPremises == Reduction!Proof!Core!RecoveryRecurringInductionPremises
RecoveryRecurringInductionKernel == Reduction!Proof!Core!RecoveryRecurringInductionKernel
RecoveryClosedPrefix(c) == Reduction!Proof!Core!RecoveryClosedPrefix(c)

ClassicalAgreementTotalHistoryObject ==
  [i \in CycleRange |-> Reduction!ClassicalAgreementDecisionObject(i)]

ClassicalAgreementTotalHistoryExtendsFinitePrefixes ==
  \A c \in CycleRange :
    [i \in 1..c |-> ClassicalAgreementTotalHistoryObject[i]]
      = Reduction!ClassicalAgreementPrefixObject(c)
```

```
92
93  TotalClassicalAgreementHistoryRealized ==
94    /\ <>RecoveryClosedPrefix(TotalCycles)
95    /\ ClassicalAgreementTotalHistoryExtendsFinitePrefixes
96
97  TotalClassicalAgreementReduction ==
98    Reduction!FiniteClassicalAgreementLiveness
99      => TotalClassicalAgreementHistoryRealized
100
101 TotalClassicalAgreementFromRecurringPrefixes ==
102   RecoveryRecurringClosedPrefixes
103     => TotalClassicalAgreementHistoryRealized
104
105 TotalClassicalAgreementFromRecurrenceKernel ==
106   /\ RecoveryRecurringInductionPremises
107   /\ RecoveryRecurringInductionKernel
108   => TotalClassicalAgreementHistoryRealized
109
110 THEOREM TotalHistoryEntryMatchesDecisionObject ==
111   \A c \in CycleRange :
112     \A i \in 1..c :
113       ClassicalAgreementTotalHistoryObject[i]
114         = Reduction!ClassicalAgreementDecisionObject(i)
115   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
116      DEF CycleRange, ClassicalAgreementTotalHistoryObject
117
118 THEOREM ClassicalAgreementTotalHistoryExtendsFinitePrefixesTheorem ==
119   ClassicalAgreementTotalHistoryExtendsFinitePrefixes
120 PROOF
121   <1>1. \A c \in CycleRange :
122             [i \in 1..c |-> ClassicalAgreementTotalHistoryObject[i]]
123               = Reduction!ClassicalAgreementPrefixObject(c)
124     PROOF
125       <2>1. TAKE c \in CycleRange
126       <2>2. \A i \in 1..c :
127                ClassicalAgreementTotalHistoryObject[i]
128                  = Reduction!ClassicalAgreementDecisionObject(i)
129             BY TotalHistoryEntryMatchesDecisionObject, <2>1
130       <2>3. QED
131             BY IsaWithSetExtensionality, <2>1, <2>2
132                DEF Reduction!ClassicalAgreementPrefixObject
133   <1>2. QED
134         BY <1>1 DEF ClassicalAgreementTotalHistoryExtendsFinitePrefixes
135
136 THEOREM TotalCyclesInCycleRange ==
137   TotalCycles \in CycleRange
138   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne DEF CycleRange
139
140 THEOREM TotalFinitePrefixFromFiniteLiveness ==
141   Reduction!FiniteClassicalAgreementLiveness
142     => Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
143 PROOF
144   <1>1. ASSUME Reduction!FiniteClassicalAgreementLiveness
145         PROVE Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
146     <2>1. TotalCycles \in Reduction!CycleRange
147           BY TotalCyclesInCycleRange DEF CycleRange, Reduction!CycleRange
148     <2>2. QED
149           BY <1>1, <2>1 DEF Reduction!FiniteClassicalAgreementLiveness
150   <1>2. QED
151         BY <1>1
152
153 THEOREM TotalRecoveryClosedPrefixFromFiniteLiveness ==
```

```
  Reduction!FiniteClassicalAgreementLiveness
    => <>RecoveryClosedPrefix(TotalCycles)
  BY TotalFinitePrefixFromFiniteLiveness
     DEF Reduction!FiniteClassicalAgreementPrefixRealized,
         RecoveryClosedPrefix

THEOREM TotalClassicalAgreementReductionTheorem ==
  TotalClassicalAgreementReduction
  BY TotalRecoveryClosedPrefixFromFiniteLiveness,
     ClassicalAgreementTotalHistoryExtendsFinitePrefixesTheorem
     DEF TotalClassicalAgreementReduction,
         TotalClassicalAgreementHistoryRealized

THEOREM ReductionLivenessFromRecurringPrefixes ==
  RecoveryRecurringClosedPrefixes
    => Reduction!FiniteClassicalAgreementLiveness
PROOF
  <1>1. ASSUME RecoveryRecurringClosedPrefixes
        PROVE Reduction!FiniteClassicalAgreementLiveness
    <2>1. \A c \in Reduction!CycleRange :
             Reduction!FiniteClassicalAgreementPrefixRealized(c)
      PROOF
        <3>1. TAKE c \in Reduction!CycleRange
        <3>2. c \in Reduction!Proof!Core!CycleRange
              BY <3>1
                 DEF Reduction!CycleRange,
                     Reduction!Proof!Core!CycleRange,
                     Reduction!Proof!Core!Core!CycleRange
        <3>3. Reduction!Proof!RecoveryRecurringClosedPrefixes
              BY <1>1 DEF RecoveryRecurringClosedPrefixes
        <3>4. <>Reduction!Proof!Core!RecoveryClosedPrefix(c)
              BY <3>2, <3>3 DEF Reduction!Proof!RecoveryRecurringClosedPrefixes
        <3>5. QED
              BY <3>1, <3>4 DEF Reduction!FiniteClassicalAgreementPrefixRealized
    <2>2. QED
          BY <2>1 DEF Reduction!FiniteClassicalAgreementLiveness
  <1>2. QED
        BY <1>1

THEOREM TotalClassicalAgreementFromRecurringPrefixesTheorem ==
  TotalClassicalAgreementFromRecurringPrefixes
PROOF
  <1>1. ASSUME RecoveryRecurringClosedPrefixes
        PROVE TotalClassicalAgreementHistoryRealized
    <2>1. Reduction!FiniteClassicalAgreementLiveness
          BY ReductionLivenessFromRecurringPrefixes, <1>1
    <2>2. Reduction!FiniteClassicalAgreementLiveness
             => TotalClassicalAgreementHistoryRealized
          BY TotalClassicalAgreementReductionTheorem
             DEF TotalClassicalAgreementReduction
    <2>3. QED
          BY <2>1, <2>2
  <1>2. QED
        BY <1>1 DEF TotalClassicalAgreementFromRecurringPrefixes

THEOREM RecoveryRecurringClosedPrefixesFromRecurrenceKernel ==
  /\ RecoveryRecurringInductionPremises
  /\ RecoveryRecurringInductionKernel
  => RecoveryRecurringClosedPrefixes
PROOF
  <1>1. ASSUME RecoveryRecurringInductionPremises,
               RecoveryRecurringInductionKernel
```

```
        PROVE RecoveryRecurringClosedPrefixes
    <2>1. \A c \in Reduction!Proof!Core!CycleRange :
             <>Reduction!Proof!Core!RecoveryClosedPrefix(c)
      PROOF
        <3>1. TAKE c \in Reduction!Proof!Core!CycleRange
        <3>2. Reduction!Proof!Core!RecoveryInductionPremisesUpTo(c)
          PROOF
            <4>1. Reduction!Proof!Core!RecoveryInductionBase
                  BY <1>1
                     DEF RecoveryRecurringInductionPremises,
                         Reduction!Proof!Core!RecoveryRecurringInductionPremises
            <4>2. \A i \in Reduction!Proof!Core!PriorCycleRange(c) :
                     Reduction!Proof!Core!RecoveryInductionStep(i)
              PROOF
                <5>1. TAKE i \in Reduction!Proof!Core!PriorCycleRange(c)
                <5>2. TotalCycles # 1
                      BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne, <3>1, <5>1
                         DEF Reduction!Proof!Core!CycleRange,
                             Reduction!Proof!Core!Core!CycleRange,
                             Reduction!Proof!Core!PriorCycleRange
                <5>3. i \in 1..(TotalCycles - 1)
                      BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne, <3>1, <5>1
                         DEF Reduction!Proof!Core!CycleRange,
                             Reduction!Proof!Core!Core!CycleRange,
                             Reduction!Proof!Core!PriorCycleRange
                <5>4. i \in Reduction!Proof!Core!PrefixAdvanceRange
                      BY <5>2, <5>3
                         DEF Reduction!Proof!Core!PrefixAdvanceRange,
                             Reduction!Proof!Core!Core!PrefixAdvanceRange
                <5>5. QED
                      BY <1>1, <5>4
                         DEF RecoveryRecurringInductionPremises,
                             Reduction!Proof!Core!RecoveryRecurringInductionPremises
              <4>3. QED
                    BY <3>1, <4>1, <4>2
                       DEF Reduction!Proof!Core!RecoveryInductionPremisesUpTo
        <3>3. Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel(c)
              BY <1>1, <3>1
                 DEF RecoveryRecurringInductionKernel,
                     Reduction!Proof!Core!RecoveryRecurringInductionKernel
            <3>4. QED
                  BY <3>2, <3>3
                     DEF Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel
    <2>2. QED
          BY <2>1 DEF RecoveryRecurringClosedPrefixes,
                     Reduction!Proof!RecoveryRecurringClosedPrefixes
  <1>2. QED
        BY <1>1
           DEF RecoveryRecurringClosedPrefixes,
               RecoveryRecurringInductionPremises,
               RecoveryRecurringInductionKernel,
               Reduction!Proof!RecoveryRecurringClosedPrefixes

THEOREM TotalClassicalAgreementFromRecurrenceKernelTheorem ==
  TotalClassicalAgreementFromRecurrenceKernel
PROOF
  <1>1. ASSUME RecoveryRecurringInductionPremises,
               RecoveryRecurringInductionKernel
        PROVE TotalClassicalAgreementHistoryRealized
    <2>1. RecoveryRecurringClosedPrefixes
          BY RecoveryRecurringClosedPrefixesFromRecurrenceKernel, <1>1
    <2>2. TotalClassicalAgreementFromRecurringPrefixes
```

```
          BY TotalClassicalAgreementFromRecurringPrefixesTheorem
    <2>3. QED
          BY <2>1, <2>2 DEF TotalClassicalAgreementFromRecurringPrefixes
  <1>2. QED
        BY <1>1 DEF TotalClassicalAgreementFromRecurrenceKernel

====
```

A.6.5 NestedGuardianRecoveryClassicalAgreementCollapse.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementCollapse.tla}

```
---- MODULE NestedGuardianRecoveryClassicalAgreementCollapse ----
EXTENDS Naturals, FiniteSets, TLC, TLAPS

CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
         InitialEpoch, InitialWitness, TotalCycles,
         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
         SmallRecoveryCommittee, LargeRecoveryCommittee

ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
ASSUME TotalCyclesIsNat == TotalCycles \in Nat
ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
ASSUME TargetCycleDomains ==
  \A c \in 1..TotalCycles :
  /\ TargetSlotOf(c) \in Slots
  /\ TargetBlockOf(c) \in Blocks
  /\ StableEpochOf(c) \in Epochs

VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
↪  witnessOnline,
          witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
          continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
          continuationPagePublished, continuationPageBoundary, continuationFetched,
          shareReceipts, shareConflicts, windowClosed, missingShareClaims,
          missingThresholdCertificates, recovered, recoveredSurfaces,
          recoveryConflicts, aborted

Totality ==
  INSTANCE NestedGuardianRecoveryClassicalAgreementTotality
  WITH Validators <- Validators,
       Witnesses <- Witnesses,
       Blocks <- Blocks,
       Slots <- Slots,
       Epochs <- Epochs,
       QuorumSize <- QuorumSize,
       MaxReassignmentDepth <- MaxReassignmentDepth,
       InitialEpoch <- InitialEpoch,
       InitialWitness <- InitialWitness,
       TotalCycles <- TotalCycles,
       TargetSlotOf <- TargetSlotOf,
       TargetBlockOf <- TargetBlockOf,
       StableEpochOf <- StableEpochOf,
       SmallCommitteeSlot <- SmallCommitteeSlot,
       SmallRecoveryThreshold <- SmallRecoveryThreshold,
```

```
        LargeRecoveryThreshold <- LargeRecoveryThreshold,
        SmallRecoveryCommittee <- SmallRecoveryCommittee,
        LargeRecoveryCommittee <- LargeRecoveryCommittee,
        votes <- votes,
        witnessCerts <- witnessCerts,
        finalized <- finalized,
        finalizerSets <- finalizerSets,
        registryEpoch <- registryEpoch,
        guardianReady <- guardianReady,
        witnessOnline <- witnessOnline,
        witnessCheckpoint <- witnessCheckpoint,
        assignedWitness <- assignedWitness,
        reassignmentDepth <- reassignmentDepth,
        currentCycle <- currentCycle,
        phase <- phase,
        churnStage <- churnStage,
        continuationBoundary <- continuationBoundary,
        continuationAnchorPublished <- continuationAnchorPublished,
        continuationAnchorBoundary <- continuationAnchorBoundary,
        continuationPagePublished <- continuationPagePublished,
        continuationPageBoundary <- continuationPageBoundary,
        continuationFetched <- continuationFetched,
        shareReceipts <- shareReceipts,
        shareConflicts <- shareConflicts,
        windowClosed <- windowClosed,
        missingShareClaims <- missingShareClaims,
        missingThresholdCertificates <- missingThresholdCertificates,
        recovered <- recovered,
        recoveredSurfaces <- recoveredSurfaces,
        recoveryConflicts <- recoveryConflicts,
        aborted <- aborted

CycleRange == Totality!CycleRange

ClassicalAgreementDecision(c) ==
  [cycle |-> c,
   slot |-> TargetSlotOf(c),
   block |-> TargetBlockOf(c),
   epoch |-> StableEpochOf(c)]

ClassicalAgreementHistory(history) ==
  \A i \in CycleRange :
    history[i] = ClassicalAgreementDecision(i)

OrdinaryClassicalAgreementHistory(history) ==
  /\ ClassicalAgreementHistory(history)
  /\ <>Totality!RecoveryClosedPrefix(TotalCycles)

UnconditionalClassicalAgreementSentence ==
  OrdinaryClassicalAgreementHistory(
    Totality!ClassicalAgreementTotalHistoryObject
  )

THEOREM TotalHistoryMatchesCollapsedClassicalHistory ==
  ClassicalAgreementHistory(Totality!ClassicalAgreementTotalHistoryObject)
PROOF
  <1>1. \A i \in CycleRange :
            Totality!ClassicalAgreementTotalHistoryObject[i]
              = ClassicalAgreementDecision(i)
    PROOF
      <2>1. TAKE i \in CycleRange
      <2>2. QED
```

```
            BY <2>1
               DEF ClassicalAgreementDecision,
                   CycleRange,
                   Totality!ClassicalAgreementTotalHistoryObject,
                   Totality!Reduction!ClassicalAgreementDecisionObject
  <1>2. QED
        BY <1>1 DEF ClassicalAgreementHistory

THEOREM TotalityCyclePrefixEntriesStayInCycleRange ==
  \A c \in Totality!CycleRange :
    \A i \in 1..c :
      i \in Totality!CycleRange
  BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
     DEF Totality!CycleRange

THEOREM TotalityReductionCorePriorCycleRangeImpliesPrefixAdvanceRange ==
  \A c \in Totality!Reduction!Proof!Core!CycleRange :
    \A i \in Totality!Reduction!Proof!Core!PriorCycleRange(c) :
      i \in Totality!Reduction!Proof!Core!PrefixAdvanceRange
  BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
     DEF Totality!Reduction!Proof!Core!CycleRange,
         Totality!Reduction!Proof!Core!Core!CycleRange,
         Totality!Reduction!Proof!Core!PriorCycleRange,
         Totality!Reduction!Proof!Core!PrefixAdvanceRange,
         Totality!Reduction!Proof!Core!Core!PrefixAdvanceRange

THEOREM LocalReductionLivenessFromRecurringPrefixesTheorem ==
  Totality!RecoveryRecurringClosedPrefixes
    => Totality!Reduction!FiniteClassicalAgreementLiveness
PROOF
  <1>1. ASSUME Totality!RecoveryRecurringClosedPrefixes
        PROVE Totality!Reduction!FiniteClassicalAgreementLiveness
    <2>1. \A c \in Totality!Reduction!CycleRange :
             Totality!Reduction!FiniteClassicalAgreementPrefixRealized(c)
      PROOF
        <3>1. TAKE c \in Totality!Reduction!CycleRange
        <3>2. c \in Totality!Reduction!Proof!Core!CycleRange
              BY <3>1
                 DEF Totality!Reduction!CycleRange,
                     Totality!Reduction!Proof!Core!CycleRange,
                     Totality!Reduction!Proof!Core!Core!CycleRange
        <3>3. <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
              BY <1>1, <3>2
                 DEF Totality!RecoveryRecurringClosedPrefixes,
                     Totality!Reduction!Proof!RecoveryRecurringClosedPrefixes
        <3>4. QED
              BY <3>1, <3>3
                 DEF Totality!Reduction!FiniteClassicalAgreementPrefixRealized
      <2>2. QED
            BY <2>1 DEF Totality!Reduction!FiniteClassicalAgreementLiveness
  <1>2. QED
        BY <1>1

THEOREM LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem ==
  Totality!Reduction!FiniteClassicalAgreementLiveness
    => Totality!TotalClassicalAgreementHistoryRealized
PROOF
  <1>1. ASSUME Totality!Reduction!FiniteClassicalAgreementLiveness
        PROVE Totality!TotalClassicalAgreementHistoryRealized
    <2>1. TotalCycles \in Totality!Reduction!CycleRange
          BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
             DEF Totality!Reduction!CycleRange
```

```
    <2>2. Totality!Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
          BY <1>1, <2>1 DEF Totality!Reduction!FiniteClassicalAgreementLiveness
    <2>3. <>Totality!RecoveryClosedPrefix(TotalCycles)
          BY <2>2
             DEF Totality!Reduction!FiniteClassicalAgreementPrefixRealized,
                 Totality!RecoveryClosedPrefix
    <2>4. Totality!ClassicalAgreementTotalHistoryExtendsFinitePrefixes
      PROOF
        <3>1. \A c \in Totality!CycleRange :
                  [i \in 1..c |-> Totality!ClassicalAgreementTotalHistoryObject[i]]
                    = Totality!Reduction!ClassicalAgreementPrefixObject(c)
          PROOF
            <4>1. TAKE c \in Totality!CycleRange
            <4>2. \A i \in 1..c :
                     Totality!ClassicalAgreementTotalHistoryObject[i]
                       = Totality!Reduction!ClassicalAgreementDecisionObject(i)
              PROOF
                <5>1. TAKE i \in 1..c
                <5>2. c \in 1..TotalCycles
                      BY <4>1 DEF Totality!CycleRange
                <5>3. i \in Totality!CycleRange
                      BY TotalityCyclePrefixEntriesStayInCycleRange, <4>1, <5>1
                <5>4. QED
                      BY <5>3
                         DEF Totality!ClassicalAgreementTotalHistoryObject
              <4>3. QED
                    BY IsaWithSetExtensionality, <4>1, <4>2
                       DEF Totality!Reduction!ClassicalAgreementPrefixObject
        <3>2. QED
              BY <3>1 DEF Totality!ClassicalAgreementTotalHistoryExtendsFinitePrefixes
    <2>5. QED
          BY <2>3, <2>4 DEF Totality!TotalClassicalAgreementHistoryRealized
  <1>2. QED
        BY <1>1

THEOREM LocalRecurringPrefixesFromRecurrenceKernelTheorem ==
  /\ Totality!RecoveryRecurringInductionPremises
  /\ Totality!RecoveryRecurringInductionKernel
  => Totality!RecoveryRecurringClosedPrefixes
PROOF
  <1>1. ASSUME Totality!RecoveryRecurringInductionPremises,
               Totality!RecoveryRecurringInductionKernel
        PROVE Totality!RecoveryRecurringClosedPrefixes
    <2>1. \A c \in Totality!Reduction!Proof!Core!CycleRange :
             <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
      PROOF
        <3>1. TAKE c \in Totality!Reduction!Proof!Core!CycleRange
        <3>2. /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
              /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel
              BY <1>1
                 DEF Totality!RecoveryRecurringInductionPremises,
                     Totality!RecoveryRecurringInductionKernel
        <3>3. Totality!Reduction!Proof!Core!RecoveryInductionPremisesUpTo(c)
          PROOF
            <4>1. Totality!Reduction!Proof!Core!RecoveryInductionBase
                  BY <3>2
                     DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
            <4>2. \A i \in Totality!Reduction!Proof!Core!PriorCycleRange(c) :
                     Totality!Reduction!Proof!Core!RecoveryInductionStep(i)
              PROOF
                <5>1. TAKE i \in Totality!Reduction!Proof!Core!PriorCycleRange(c)
                <5>2. i \in Totality!Reduction!Proof!Core!PrefixAdvanceRange
```

```
                      BY TotalityReductionCorePriorCycleRangeImpliesPrefixAdvanceRange,
                         <3>1, <5>1
                <5>3. QED
                      BY <3>2, <5>2
                         DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
            <4>3. QED
                  BY <3>1, <4>1, <4>2
                     DEF Totality!Reduction!Proof!Core!RecoveryInductionPremisesUpTo
        <3>4. Totality!Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel(c)
              BY <3>1, <3>2
                 DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel
        <3>5. <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
              BY <3>3, <3>4
                 DEF Totality!Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel
        <3>6. (/\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
                /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel)
                => <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
              BY <3>5
        <3>7. QED
              BY <3>5
    <2>2. QED
            BY <2>1
               DEF Totality!RecoveryRecurringClosedPrefixes,
                   Totality!Reduction!Proof!RecoveryRecurringClosedPrefixes
  <1>2. QED
        BY <1>1

THEOREM UnconditionalClassicalAgreementFromTotalHistoryTheorem ==
  Totality!TotalClassicalAgreementHistoryRealized
    => UnconditionalClassicalAgreementSentence
PROOF
  <1>1. ASSUME Totality!TotalClassicalAgreementHistoryRealized
        PROVE UnconditionalClassicalAgreementSentence
    <2>1. ClassicalAgreementHistory(Totality!ClassicalAgreementTotalHistoryObject)
          BY TotalHistoryMatchesCollapsedClassicalHistory
    <2>2. <>Totality!RecoveryClosedPrefix(TotalCycles)
          BY <1>1 DEF Totality!TotalClassicalAgreementHistoryRealized
    <2>3. QED
          BY <2>1, <2>2
             DEF UnconditionalClassicalAgreementSentence,
                 OrdinaryClassicalAgreementHistory
  <1>2. QED
        BY <1>1

THEOREM UnconditionalClassicalAgreementFromRecurringPrefixesTheorem ==
  Totality!RecoveryRecurringClosedPrefixes
    => UnconditionalClassicalAgreementSentence
PROOF
  <1>1. ASSUME Totality!RecoveryRecurringClosedPrefixes
        PROVE UnconditionalClassicalAgreementSentence
    <2>1. Totality!Reduction!FiniteClassicalAgreementLiveness
          BY LocalReductionLivenessFromRecurringPrefixesTheorem, <1>1
    <2>2. Totality!TotalClassicalAgreementHistoryRealized
          BY LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem, <2>1
    <2>3. QED
          BY UnconditionalClassicalAgreementFromTotalHistoryTheorem, <2>2
  <1>2. QED
        BY <1>1

THEOREM UnconditionalClassicalAgreementFromRecurrenceKernelTheorem ==
  /\ Totality!RecoveryRecurringInductionPremises
  /\ Totality!RecoveryRecurringInductionKernel
```

```
  => UnconditionalClassicalAgreementSentence
PROOF
  <1>1. ASSUME Totality!RecoveryRecurringInductionPremises,
               Totality!RecoveryRecurringInductionKernel
        PROVE UnconditionalClassicalAgreementSentence
    <2>1. Totality!RecoveryRecurringClosedPrefixes
          BY LocalRecurringPrefixesFromRecurrenceKernelTheorem, <1>1
    <2>2. Totality!Reduction!FiniteClassicalAgreementLiveness
          BY LocalReductionLivenessFromRecurringPrefixesTheorem, <2>1
    <2>3. Totality!TotalClassicalAgreementHistoryRealized
          BY LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem, <2>2
    <2>4. QED
          BY UnconditionalClassicalAgreementFromTotalHistoryTheorem, <2>3
  <1>3. QED
        BY <1>1

====
```

A.7 Common Boundary (AFT-CB)

The mechanized kernels of the common-boundary theorem corpus (Section 15): the Boundary Ring uniqueness kernel and its TLAPS discharge, the liveness and two-tier separation model, the membership-transition and custody-obligation kernels with their proofs, the wire-level forensic-accountability kernel, and the honestly narrowed succession clock (its full safety claim withdrawn; the kernel header states the narrowing).

A.7.1 `BoundaryRing.tla`

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/BoundaryRing.tla}

```
---- MODULE BoundaryRing ----
(***************************************************************************)
(* AFT-CB P2.1 -- the Boundary Ring action-level uniqueness kernel.         *)
(*                                                                         *)
(* Models one signer configuration (Ring) forming Unanimous Boundary      *)
(* Closes per slot.  The adversary schedules all delivery (no synchrony    *)
(* appears anywhere) and emits arbitrary well-signed messages for corrupt  *)
(* members -- the ONLY exclusion is forging an honest member's signature   *)
(* (A1), modeled by HonestFinalAck being the sole producer of honest ack   *)
(* records.  Honest final-acks are guarded by a durable single-entry       *)
(* journal (A3).  Uniqueness is DERIVED from the inductive invariant under *)
(* the Minimal Honesty Axiom (A2) -- no admitted-equals-canonical           *)
(* definition exists in this module (the Q2 defect class this kernel       *)
(* exists to kill).                                                        *)
(*                                                                         *)
(* Spec source: specs/common_boundary.md §§0-2, §11-§12; theorem T1 in    *)
(* specs/common_boundary_theorems.md.  The TLAPS discharge lives in        *)
(* BoundaryRingProof.tla; the TLC configuration explores n = 3 with        *)
(* exactly one honest member (the MHA corner).                             *)
(***************************************************************************)
EXTENDS Naturals, FiniteSets

CONSTANTS
  Ring,           \* the signer configuration C_v
  HonestMembers,  \* the honest subset (MHA: nonempty -- assumed in the proof)
  Slots,          \* seal slots s
  Roots,          \* candidate boundary roots
  Artifacts,      \* submittable artifacts (COLLECT/RECONCILE structure)
  NoRoot          \* journal sentinel: "no final-ack recorded for this slot"

ASSUME ConstantAssumptions ==
  /\ HonestMembers \subseteq Ring
  /\ NoRoot \notin Roots

VARIABLES
  delivered,  \* subset of Artifacts \X Ring: what the adversary delivered where
  declared,   \* [Ring -> [Slots -> SUBSET Artifacts]]: declared sets
  journal,    \* [HonestMembers -> [Slots -> Roots \cup {NoRoot}]]: A3 journal
  acks,       \* subset of Ring \X Slots \X Roots: final-ack shares in existence
  closes      \* subset of Slots \X Roots: assembled UBCs

vars == <<delivered, declared, journal, acks, closes>>

```

```
TypeOK ==
  /\ delivered \subseteq Artifacts \X Ring
  /\ declared \in [Ring -> [Slots -> SUBSET Artifacts]]
  /\ journal \in [HonestMembers -> [Slots -> Roots \cup {NoRoot}]]
  /\ acks \subseteq Ring \X Slots \X Roots
  /\ closes \subseteq Slots \X Roots

Init ==
  /\ delivered = {}
  /\ declared = [m \in Ring |-> [s \in Slots |-> {}]]
  /\ journal = [m \in HonestMembers |-> [s \in Slots |-> NoRoot]]
  /\ acks = {}
  /\ closes = {}

(*************************************************************************)
(* COLLECT: the adversary schedules delivery of any artifact to any      *)
(* member, in any order, at any time.  No fairness, no bound -- this is  *)
(* where asynchrony lives.                                               *)
(*************************************************************************)
Deliver(a, m) ==
  /\ delivered' = delivered \cup {<<a, m>>}
  /\ UNCHANGED <<declared, journal, acks, closes>>

(*************************************************************************)
(* RECONCILE/DECLARE: an honest member's declared set for a slot is drawn *)
(* from what it actually received (spec §3.1: cutoff is local -- the     *)
(* declared set is whatever was delivered when the member declares).     *)
(* Corrupt members' declared sets are set arbitrarily by the adversary.  *)
(*************************************************************************)
HonestDeclare(m, s, A) ==
  /\ m \in HonestMembers
  /\ A \subseteq {a \in Artifacts : <<a, m>> \in delivered}
  /\ declared' = [declared EXCEPT ![m] = [declared[m] EXCEPT ![s] = A]]
  /\ UNCHANGED <<delivered, journal, acks, closes>>

ByzantineDeclare(m, s, A) ==
  /\ m \in Ring \ HonestMembers
  /\ A \subseteq Artifacts
  /\ declared' = [declared EXCEPT ![m] = [declared[m] EXCEPT ![s] = A]]
  /\ UNCHANGED <<delivered, journal, acks, closes>>

(*************************************************************************)
(* FINAL-ACK, honest: journal-guarded (A3).  The journal entry is written *)
(* in the same atomic step that lets the share exist -- spec §2.1's      *)
(* "journal before the share leaves the process".  The guard             *)
(* journal[m][s] = NoRoot is the single-final-ack discipline: once a root *)
(* is journaled for (m, s), no second honest share for that slot can ever *)
(* be produced (recovery replays bytes; it never re-decides -- §2.2).    *)
(*************************************************************************)
HonestFinalAck(m, s, r) ==
  /\ m \in HonestMembers
  /\ r \in Roots
  /\ s \in Slots
  /\ journal[m][s] = NoRoot
  /\ journal' = [journal EXCEPT ![m] = [journal[m] EXCEPT ![s] = r]]
  /\ acks' = acks \cup {<<m, s, r>>}
  /\ UNCHANGED <<delivered, declared, closes>>

(*************************************************************************)
(* The unconstrained Byzantine action: a corrupt member emits ANY        *)
(* well-formed signed message, for any slot, any root, any number of     *)
(* times, on any schedule.  Forging an honest signature is the only      *)
```

```
(* exclusion (A1): m ranges over corrupt members only.  No journal binds   *)
(* corrupt members.                                                        *)
(*************************************************************************)
ByzantineEmit(m, s, r) ==
  /\ m \in Ring \ HonestMembers
  /\ s \in Slots
  /\ r \in Roots
  /\ acks' = acks \cup {<<m, s, r>>}
  /\ UNCHANGED <<delivered, declared, journal, closes>>

(*************************************************************************)
(* CLOSE: anyone (the adversary included) may assemble a UBC from shares   *)
(* that exist -- n-of-n: EVERY ring member's share over this exact          *)
(* (slot, root).  There is no smaller quorum anywhere in this module (the  *)
(* (n-1)-close mutation drill targets exactly this conjunct).              *)
(*************************************************************************)
CloseAct(s, r) ==
  /\ s \in Slots
  /\ r \in Roots
  /\ \A m \in Ring : <<m, s, r>> \in acks
  /\ closes' = closes \cup {<<s, r>>}
  /\ UNCHANGED <<delivered, declared, journal, acks>>

Next ==
  \/ \E a \in Artifacts, m \in Ring : Deliver(a, m)
  \/ \E m \in Ring, s \in Slots, A \in SUBSET Artifacts :
       HonestDeclare(m, s, A) \/ ByzantineDeclare(m, s, A)
  \/ \E m \in Ring, s \in Slots, r \in Roots :
       HonestFinalAck(m, s, r) \/ ByzantineEmit(m, s, r)
  \/ \E s \in Slots, r \in Roots : CloseAct(s, r)

Spec == Init /\ [][Next]_vars

(*************************************************************************)
(* The derived property.  Uniqueness is NOT assumed anywhere above: no     *)
(* definition equates "admitted" with "canonical"; closes is an ordinary   *)
(* set any adversary schedule can try to populate twice.                   *)
(*************************************************************************)
Uniqueness ==
  \A s \in Slots, r1 \in Roots, r2 \in Roots :
    (<<s, r1>> \in closes /\ <<s, r2>> \in closes) => r1 = r2

(*************************************************************************)
(* The inductive invariant (discharged in BoundaryRingProof.tla):          *)
(* an honest share always matches the journal entry (so an honest member   *)
(* has at most one share per slot), and every close carries every          *)
(* member's share.  Uniqueness follows under MHA.                          *)
(*************************************************************************)
Inv ==
  /\ TypeOK
  /\ \A m \in HonestMembers, s \in Slots, r \in Roots :
       (<<m, s, r>> \in acks) => journal[m][s] = r
  /\ \A s \in Slots, r \in Roots :
       (<<s, r>> \in closes) => \A m \in Ring : <<m, s, r>> \in acks

====
```

A.7.2 BoundaryRingProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/BoundaryRingProof.tla}

```
---- MODULE BoundaryRingProof ----
(***************************************************************************)
(* AFT-CB P2.1 -- TLAPS discharge of Boundary Ring uniqueness (T1).         *)
(*                                                                         *)
(* Structure: Init => Inv; Inv /\ [Next]_vars => Inv'; Inv /\ MHA =>       *)
(* Uniqueness; composed to THEOREM Spec => []Uniqueness under the MHA      *)
(* constant assumption.  Uniqueness is derived, never defined into         *)
(* existence.                                                              *)
(***************************************************************************)
EXTENDS BoundaryRing, TLAPS

ASSUME MHA == HonestMembers # {}

LEMMA InvInit == Init => Inv
  BY ConstantAssumptions DEF Init, Inv, TypeOK

LEMMA InvNext == Inv /\ [Next]_vars => Inv'
<1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
  OBVIOUS
<1>2. CASE UNCHANGED vars
  BY <1>1, <1>2 DEF Inv, TypeOK, vars
<1>3. ASSUME NEW a \in Artifacts, NEW m \in Ring, Deliver(a, m)
      PROVE Inv'
  BY <1>1, <1>3 DEF Inv, TypeOK, Deliver
<1>4. ASSUME NEW m \in Ring, NEW s \in Slots, NEW A \in SUBSET Artifacts,
             HonestDeclare(m, s, A)
      PROVE Inv'
  BY <1>1, <1>4 DEF Inv, TypeOK, HonestDeclare
<1>5. ASSUME NEW m \in Ring, NEW s \in Slots, NEW A \in SUBSET Artifacts,
             ByzantineDeclare(m, s, A)
      PROVE Inv'
  BY <1>1, <1>5 DEF Inv, TypeOK, ByzantineDeclare
<1>6. ASSUME NEW m \in Ring, NEW s \in Slots, NEW r \in Roots,
             HonestFinalAck(m, s, r)
      PROVE Inv'
  <2>1. m \in HonestMembers /\ journal[m][s] = NoRoot
    BY <1>6 DEF HonestFinalAck
  <2>2. TypeOK'
    BY <1>1, <1>6, <2>1 DEF Inv, TypeOK, HonestFinalAck
  <2>3. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
          (<<mm, ss, rr>> \in acks') => journal'[mm][ss] = rr
    <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
                          NEW rr \in Roots, <<mm, ss, rr>> \in acks'
          PROVE journal'[mm][ss] = rr
      OBVIOUS
    <3>2. acks' = acks \cup {<<m, s, r>>}
      BY <1>6 DEF HonestFinalAck
    <3>3. CASE <<mm, ss, rr>> = <<m, s, r>>
      <4>1. mm = m /\ ss = s /\ rr = r
        BY <3>3
      <4>2. journal'[m][s] = r
        BY <1>1, <1>6, <2>1 DEF Inv, TypeOK, HonestFinalAck
      <4> QED BY <4>1, <4>2
    <3>4. CASE <<mm, ss, rr>> \in acks
      <4>1. journal[mm][ss] = rr
```

```
        BY <1>1, <3>4 DEF Inv
      <4>2. CASE mm = m /\ ss = s
        <5>1. journal[m][s] = rr
          BY <4>1, <4>2
        <5>2. rr \in Roots /\ NoRoot \notin Roots
          BY <3>1, ConstantAssumptions
        <5>3. FALSE
          BY <2>1, <5>1, <5>2
        <5> QED BY <5>3
      <4>3. CASE ~(mm = m /\ ss = s)
        <5>1. journal'[mm][ss] = journal[mm][ss]
          BY <1>1, <1>6, <2>1, <4>3 DEF Inv, TypeOK, HonestFinalAck
        <5> QED BY <4>1, <5>1
      <4> QED BY <4>2, <4>3
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2>4. \A ss \in Slots, rr \in Roots :
          (<<ss, rr>> \in closes') => \A mm \in Ring : <<mm, ss, rr>> \in acks'
    BY <1>1, <1>6 DEF Inv, HonestFinalAck
  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
<1>7. ASSUME NEW m \in Ring, NEW s \in Slots, NEW r \in Roots,
             ByzantineEmit(m, s, r)
      PROVE Inv'
  <2>1. m \notin HonestMembers
    BY <1>7 DEF ByzantineEmit
  <2>2. TypeOK'
    BY <1>1, <1>7 DEF Inv, TypeOK, ByzantineEmit
  <2>3. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
          (<<mm, ss, rr>> \in acks') => journal'[mm][ss] = rr
    BY <1>1, <1>7, <2>1 DEF Inv, ByzantineEmit
  <2>4. \A ss \in Slots, rr \in Roots :
          (<<ss, rr>> \in closes') => \A mm \in Ring : <<mm, ss, rr>> \in acks'
    BY <1>1, <1>7 DEF Inv, ByzantineEmit
  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
<1>8. ASSUME NEW s \in Slots, NEW r \in Roots, CloseAct(s, r)
      PROVE Inv'
  <2>1. TypeOK'
    BY <1>1, <1>8 DEF Inv, TypeOK, CloseAct
  <2>2. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
          (<<mm, ss, rr>> \in acks') => journal'[mm][ss] = rr
    BY <1>1, <1>8 DEF Inv, CloseAct
  <2>3. \A ss \in Slots, rr \in Roots :
          (<<ss, rr>> \in closes') => \A mm \in Ring : <<mm, ss, rr>> \in acks'
    <3>1. SUFFICES ASSUME NEW ss \in Slots, NEW rr \in Roots,
                          <<ss, rr>> \in closes'
          PROVE \A mm \in Ring : <<mm, ss, rr>> \in acks'
      OBVIOUS
    <3>2. closes' = closes \cup {<<s, r>>} /\ acks' = acks
      BY <1>8 DEF CloseAct
    <3>3. CASE <<ss, rr>> \in closes
      BY <1>1, <3>2, <3>3 DEF Inv
    <3>4. CASE <<ss, rr>> = <<s, r>>
      BY <1>8, <3>2, <3>4 DEF CloseAct
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2> QED BY <2>1, <2>2, <2>3 DEF Inv
<1>9. QED
  BY <1>1, <1>2, <1>3, <1>4, <1>5, <1>6, <1>7, <1>8 DEF Next, vars

LEMMA InvUniqueness == Inv => Uniqueness
<1>1. SUFFICES ASSUME Inv, NEW s \in Slots, NEW r1 \in Roots, NEW r2 \in Roots,
                      <<s, r1>> \in closes, <<s, r2>> \in closes
      PROVE r1 = r2
  BY DEF Uniqueness
```

```
<1>2. PICK h \in HonestMembers : TRUE
  BY MHA
<1>3. h \in Ring
  BY ConstantAssumptions
<1>4. <<h, s, r1>> \in acks /\ <<h, s, r2>> \in acks
  BY <1>1, <1>3 DEF Inv
<1>5. journal[h][s] = r1 /\ journal[h][s] = r2
  BY <1>1, <1>2, <1>4 DEF Inv
<1>6. QED
  BY <1>5

THEOREM BoundaryUniqueness == Spec => []Uniqueness
<1>1. Init => Inv
  BY InvInit
<1>2. Inv /\ [Next]_vars => Inv'
  BY InvNext
<1>3. Spec => []Inv
  BY <1>1, <1>2, PTL DEF Spec
<1>4. Inv => Uniqueness
  BY InvUniqueness
<1>5. QED
  BY <1>3, <1>4, PTL

====
```

A.7.3 BoundaryLiveness.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/BoundaryLiveness.tla}

```
---- MODULE BoundaryLiveness ----
(***************************************************************************)
(* AFT-CB P2.2 -- SEAL CADENCE, not chain liveness (T4b), and the two-tier  *)
(* separation property.                                                     *)
(*                                                                          *)
(* Claim scope (standing rule 2, stated exactly): the cadence and           *)
(* separation results below are model-checked at n≤4 -- never "proven".     *)
(* Live-tier liveness (T4a) is the engine's own obligation, tracked at      *)
(* R10 and deliberately NOT claimed here: LiveBlock below is a progress     *)
(* ABSTRACTION that reads no ring variable, so the separation property is   *)
(* visible by construction and checked by TLC.                              *)
(*                                                                          *)
(* A5 is modeled as FAIRNESS: weak fairness on responsive members' acks     *)
(* and on seal assembly is exactly the post-GST eventual-delivery           *)
(* abstraction (T4b's Assumes line carries A5; this module says where it    *)
(* lives).  A2/A3's safety content is BoundaryRing's business (P2.1) --     *)
(* this module checks progress, with honest-shaped members only.            *)
(*                                                                          *)
(* The action set includes the assurance-preserving handover (§9): an      *)
(* unresponsive member can be replaced by a responsive standby when         *)
(* HandoverEnabled -- modeled as an available transition; its safety terms  *)
(* (old-ring unanimity + new-ring acceptance) are P2.3's obligation, not    *)
(* re-proved here.  NO EJECTION ACTION EXISTS AT THIS LAYER: weighted       *)
(* ejection is live-tier-only (spec §5), and the strong ring changes only   *)
(* by handover here.                                                        *)
(*                                                                          *)
(* Mutation drill (leg-mandated): one member permanently silent with        *)
(* handover disabled ⇒ SealCadenceReached goes RED while                     *)
(* LiveTierProgressReached stays GREEN -- both directions of the tier       *)
```

```
(* split, demonstrated by the same counterexample run.                    *)
(*************************************************************************)
EXTENDS Naturals, FiniteSets

CONSTANTS
  Members,          \* the initial ring configuration
  Standbys,         \* standby pool for handover (disjoint from Members)
  Responsive,       \* the responsive subset of Members \cup Standbys
  MaxSeals,         \* cadence target the property demands be reached
  MaxBlocks,        \* live-tier progress target
  HandoverEnabled   \* BOOLEAN: is the §9 handover transition available?

ASSUME LivenessConstants ==
  /\ Standbys \cap Members = {}
  /\ Responsive \subseteq (Members \cup Standbys)
  /\ MaxSeals \in Nat \ {0}
  /\ MaxBlocks \in Nat \ {0}
  /\ HandoverEnabled \in BOOLEAN

VARIABLES
  ring,        \* current configuration (changes only by handover)
  acked,       \* members of ring that final-acked the CURRENT slot
  sealCount,   \* seals closed so far (the cadence measure)
  liveBlocks,  \* live-tier progress abstraction (reads no ring state)
  version      \* configuration version (increments on handover)

vars == <<ring, acked, sealCount, liveBlocks, version>>

TypeInvariant ==
  /\ ring \subseteq (Members \cup Standbys)
  /\ acked \subseteq ring
  /\ sealCount \in 0..MaxSeals
  /\ liveBlocks \in 0..MaxBlocks
  /\ version \in Nat

Init ==
  /\ ring = Members
  /\ acked = {}
  /\ sealCount = 0
  /\ liveBlocks = 0
  /\ version = 0

(* A responsive ring member final-acks the current slot.  Unresponsive    *)
(* members simply never take this action -- silence is the absence of a    *)
(* step, not a message (spec §10's discipline holds even in the liveness  *)
(* model).                                                                 *)
Ack(m) ==
  /\ m \in ring
  /\ m \in Responsive
  /\ m \notin acked
  /\ sealCount < MaxSeals
  /\ acked' = acked \cup {m}
  /\ UNCHANGED <<ring, sealCount, liveBlocks, version>>

(* The n-of-n close: EVERY current ring member has acked.  Unanimity is    *)
(* the whole point -- one silent member freezes this action forever (L2's  *)
(* forced trade, conceded by T4b).                                         *)
SealClose ==
  /\ acked = ring
  /\ ring # {}
  /\ sealCount < MaxSeals
  /\ sealCount' = sealCount + 1
```

```
  /\ acked' = {}
  /\ UNCHANGED <<ring, liveBlocks, version>>

(* The live tier produces blocks reading NO ring variable: the guard and   *)
(* effect mention only liveBlocks.  A stalled ring cannot disable this     *)
(* action -- that is the separation property, structurally.                 *)
LiveBlock ==
  /\ liveBlocks < MaxBlocks
  /\ liveBlocks' = liveBlocks + 1
  /\ UNCHANGED <<ring, acked, sealCount, version>>

(* Assurance-preserving handover (§9), abstracted to its liveness role:    *)
(* an unresponsive member is replaced by a responsive standby and the      *)
(* configuration version advances.  Enabled only when the deployment       *)
(* provides the ceremony (HandoverEnabled) -- the mutation drill disables   *)
(* it to prove the cadence property fails for the right reason.            *)
Handover(m, sb) ==
  /\ HandoverEnabled
  /\ m \in ring
  /\ m \notin Responsive
  /\ sb \in Standbys
  /\ sb \in Responsive
  /\ sb \notin ring
  /\ ring' = (ring \ {m}) \cup {sb}
  /\ acked' = acked \ {m}
  /\ version' = version + 1
  /\ UNCHANGED <<sealCount, liveBlocks>>

(* Terminal self-loop so a finished run is quiescence, not deadlock.       *)
Terminating ==
  /\ sealCount = MaxSeals
  /\ liveBlocks = MaxBlocks
  /\ UNCHANGED vars

SomeAck == \E m \in Members \cup Standbys : Ack(m)
SomeHandover == \E m \in Members, sb \in Standbys : Handover(m, sb)

Next ==
  \/ SomeAck
  \/ SealClose
  \/ LiveBlock
  \/ SomeHandover
  \/ Terminating

(* Fairness IS the A5 abstraction: post-GST, enabled progress steps        *)
(* eventually happen.  Weak fairness per responsive participant's ack,     *)
(* plus seal assembly, live production, and the handover ceremony when     *)
(* available.                                                              *)
LivenessSpec ==
  /\ Init
  /\ [][Next]_vars
  /\ \A m \in Members \cup Standbys : WF_vars(Ack(m))
  /\ WF_vars(SealClose)
  /\ WF_vars(LiveBlock)
  /\ WF_vars(SomeHandover)

(* The cadence property (T4b shape): seals actually reach the target --    *)
(* positive seals, not merely eventual aborts.                             *)
SealCadenceReached == <>(sealCount = MaxSeals)

(* The separation property: the live tier reaches its target regardless    *)
(* of ring state -- checked GREEN in every configuration including the      *)
```

```
(* mutation drill where cadence is RED.                                    *)
LiveTierProgressReached == <>(liveBlocks = MaxBlocks)

====
```

A.7.4 MembershipTransition.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/MembershipTransition.tla}

```
---- MODULE MembershipTransition ----
(***************************************************************************)
(* AFT-CB P2.3 -- membership transitions, the no-silence action set, and   *)
(* typed lineage roots.                                                    *)
(*                                                                         *)
(* Event-driven, versioned configurations: a transition record is closed   *)
(* by the CURRENT configuration's unanimous transition-acks (old-ring      *)
(* unanimity) plus every successor member's acceptance (T5c′) -- the only  *)
(* strong-ring transition action in this module.  There is NO calendar     *)
(* epoch anywhere: nothing advances by time.                               *)
(*                                                                         *)
(* Silence records EXIST in this model -- the adversary mints attested     *)
(* non-response records freely -- and can never justify a transition: the  *)
(* TransitionsJustified invariant states that every closed transition is   *)
(* fully signature-justified (acks + acceptances), which is the spec-level *)
(* no-silence-derived-action assertion the leg demands (spec §10.2).  The  *)
(* mutation drill adds a silence-justified closure action and watches this *)
(* invariant go red.                                                       *)
(*                                                                         *)
(* Re-genesis is a TYPED ROOT: every transition record carries exactly one *)
(* lineage id, roots mint fresh lineage ids, and a well-formed history is  *)
(* single-lineage BY TYPE -- a root-crossing history is distinguishable by *)
(* construction (spec §14.4).  Uniqueness (T5a) quantifies WITHIN one      *)
(* lineage, from a common parent configuration, under per-configuration    *)
(* MHA stated as an explicit theorem condition.                            *)
(*                                                                         *)
(* Bootstrap: the deployed A6 freshness anchor is an EXPLICIT CONSTANT     *)
(* (DeployedAnchor), and the freshness theorem in the proof module is      *)
(* conditional on it IN THE STATEMENT -- never in prose.                   *)
(***************************************************************************)
EXTENDS Naturals, FiniteSets

CONSTANTS
  AllMembers,      \* the member universe
  HonestMembers,   \* honest subset (per-config MHA is a theorem condition)
  Lineages,        \* lineage identifiers (roots mint them)
  GenesisLineage,  \* the anchored genesis lineage
  Versions,        \* transition version numbers (event-driven, no epochs)
  NoSucc,          \* journal sentinel
  Anchors,         \* the A6 anchor menu (explicit constants)
  DeployedAnchor,  \* the deployed freshness anchor, or NoAnchor
  NoAnchor

Configs == (SUBSET AllMembers) \ {{}}

ASSUME MembershipConstants ==
  /\ HonestMembers \subseteq AllMembers
  /\ GenesisLineage \in Lineages
  /\ NoAnchor \notin Anchors
```

```
  /\ DeployedAnchor \in Anchors \cup {NoAnchor}
\* NoSucc \notin Configs is additionally assumed in the PROOF module only:
\* TLC cannot membership-test a model value against this set of sets, and
\* TLC's own model-value semantics already guarantee the disequality.

VARIABLES
  tjournal,        \* [HonestMembers -> [Lineages -> [Versions -> Configs \cup {NoSucc}]]]
  tacks,           \* HONEST old-ring transition final-acks: <<m, l, v, succ>>
  taccepts,        \* HONEST new-ring seat acceptances: <<m, l, v, succ>>
  transitions,     \* closed records: <<l, v, parent, succ>>
  roots,           \* minted lineage roots: <<newLineage, priorLineage>>
  silenceRecords   \* attested non-response records: <<attester, subject>>

vars == <<tjournal, tacks, taccepts, transitions, roots, silenceRecords>>

(***************************************************************************)
(* The unconstrained adversary, modeled STRUCTURALLY: a corrupt member's   *)
(* signature over ANY transition content is always available (A1 excludes  *)
(* only forging honest signatures), so Byzantine acks and acceptances are  *)
(* not state -- they are simply TRUE.  Only honest signatures are state,    *)
(* because only honest members refuse to sign twice (the journal).  This   *)
(* keeps the adversary maximal while keeping the state space enumerable.   *)
(***************************************************************************)
AckedBy(m, l, v, succ) ==
  IF m \in HonestMembers THEN <<m, l, v, succ>> \in tacks ELSE TRUE

AcceptedBy(m, l, v, succ) ==
  IF m \in HonestMembers THEN <<m, l, v, succ>> \in taccepts ELSE TRUE

TypeOK ==
  /\ tjournal \in [HonestMembers -> [Lineages -> [Versions -> Configs \cup {NoSucc}]]]
  /\ tacks \subseteq HonestMembers \X Lineages \X Versions \X Configs
  /\ taccepts \subseteq HonestMembers \X Lineages \X Versions \X Configs
  /\ transitions \subseteq Lineages \X Versions \X Configs \X Configs
  /\ roots \subseteq Lineages \X Lineages
  /\ silenceRecords \subseteq AllMembers \X AllMembers

Init ==
  /\ tjournal = [m \in HonestMembers |-> [l \in Lineages |-> [v \in Versions |-> NoSucc]]]
  /\ tacks = {}
  /\ taccepts = {}
  /\ transitions = {}
  /\ roots = {}
  /\ silenceRecords = {}

(* Old-ring transition final-ack, journal-guarded per (lineage, version):  *)
(* the A3 discipline at the transition layer -- an honest member acks at    *)
(* most one successor per version of a lineage, ever.                      *)
HonestTransitionAck(m, l, v, succ) ==
  /\ m \in HonestMembers
  /\ l \in Lineages /\ v \in Versions /\ succ \in Configs
  /\ tjournal[m][l][v] = NoSucc
  /\ tjournal' = [tjournal EXCEPT ![m] =
       [tjournal[m] EXCEPT ![l] = [tjournal[m][l] EXCEPT ![v] = succ]]]
  /\ tacks' = tacks \cup {<<m, l, v, succ>>}
  /\ UNCHANGED <<taccepts, transitions, roots, silenceRecords>>

(* Honest new-ring acceptance of a seat in a successor configuration the   *)
(* member belongs to.  Acceptance is not the safety bottleneck (the old    *)
(* ring's unanimity is), so no journal binds it; corrupt members'          *)
(* acceptances are always available (AcceptedBy).                          *)
Accept(m, l, v, succ) ==
```

```
  /\ m \in HonestMembers
  /\ l \in Lineages /\ v \in Versions /\ succ \in Configs
  /\ m \in succ
  /\ taccepts' = taccepts \cup {<<m, l, v, succ>>}
  /\ UNCHANGED <<tjournal, tacks, transitions, roots, silenceRecords>>

(* THE ONLY strong-ring transition action: assurance-preserving handover -- *)
(* old-ring unanimity (every parent member's ack, with corrupt signatures  *)
(* always available) AND new-ring acceptance (every successor member's).   *)
(* No other action in this module writes `transitions`; in particular no   *)
(* action consumes silenceRecords.                                         *)
CloseTransition(l, v, parent, succ) ==
  /\ l \in Lineages /\ v \in Versions
  /\ parent \in Configs /\ succ \in Configs
  /\ \A m \in parent : AckedBy(m, l, v, succ)
  /\ \A m \in succ : AcceptedBy(m, l, v, succ)
  /\ transitions' = transitions \cup {<<l, v, parent, succ>>}
  /\ UNCHANGED <<tjournal, tacks, taccepts, roots, silenceRecords>>

(* Anchored re-genesis: a typed lineage ROOT.  It mints a FRESH lineage id *)
(* -- never the genesis lineage, never an id already rooted -- and records   *)
(* the prior lineage it administratively succeeds.  Nothing about it is a  *)
(* transition: it writes `roots`, not `transitions`, and the lineage seam  *)
(* is visible in the type itself.                                          *)
MintRoot(newL, priorL) ==
  /\ newL \in Lineages /\ priorL \in Lineages
  /\ newL # GenesisLineage
  /\ newL # priorL
  /\ \A p \in Lineages : <<newL, p>> \notin roots
  /\ roots' = roots \cup {<<newL, priorL>>}
  /\ UNCHANGED <<tjournal, tacks, taccepts, transitions, silenceRecords>>

(* Attested non-response records: ANYONE may mint one about ANY member, in *)
(* any quantity -- and the state machine gives them no power: no guard of   *)
(* any transition-writing action mentions this variable.                   *)
EmitSilenceRecord(attester, subject) ==
  /\ attester \in AllMembers /\ subject \in AllMembers
  /\ silenceRecords' = silenceRecords \cup {<<attester, subject>>}
  /\ UNCHANGED <<tjournal, tacks, taccepts, transitions, roots>>

Next ==
  \/ \E m \in AllMembers, l \in Lineages, v \in Versions, succ \in Configs :
       HonestTransitionAck(m, l, v, succ) \/ Accept(m, l, v, succ)
  \/ \E l \in Lineages, v \in Versions, parent \in Configs, succ \in Configs :
       CloseTransition(l, v, parent, succ)
  \/ \E newL \in Lineages, priorL \in Lineages : MintRoot(newL, priorL)
  \/ \E a \in AllMembers, s \in AllMembers : EmitSilenceRecord(a, s)

Spec == Init /\ [][Next]_vars

(***************************************************************************)
(* The spec-level no-silence assertion: every closed transition is FULLY   *)
(* justified by signatures -- old-ring unanimity and new-ring acceptance --  *)
(* while silence records float freely in the state with no effect.  The    *)
(* silence-mutation drill adds a closure action justified by               *)
(* silenceRecords instead, and this invariant goes red.                    *)
(***************************************************************************)
TransitionsJustified ==
  \A l \in Lineages, v \in Versions, parent \in Configs, succ \in Configs :
    (<<l, v, parent, succ>> \in transitions) =>
      /\ \A m \in parent : AckedBy(m, l, v, succ)
      /\ \A m \in succ : AcceptedBy(m, l, v, succ)
```

```
(* T5a at the transition layer: within ONE lineage, from a common parent   *)
(* configuration containing at least one honest member, the closed         *)
(* successor at any version is unique.  Divergence convicts the FULL       *)
(* parent configuration (every member double-acked) -- checked by TLC and   *)
(* proven in MembershipTransitionProof.                                    *)
LineageUniqueness ==
  \A l \in Lineages, v \in Versions, parent \in Configs,
     s1 \in Configs, s2 \in Configs :
    (/\ <<l, v, parent, s1>> \in transitions
     /\ <<l, v, parent, s2>> \in transitions
     /\ parent \cap HonestMembers # {}) => s1 = s2

(* Root-crossing distinguishability (spec §14.4): every record carries its *)
(* lineage id in the first component, so a verifier's well-formedness rule *)
(* -- a single history holds records of ONE lineage -- makes a root-crossing *)
(* history distinguishable by construction.  WellFormedHistory is the      *)
(* verifier's rule; CrossLineageUnclaimed states what the model does NOT   *)
(* claim: uniqueness never quantifies across lineages, and the drill that  *)
(* mutates WellFormedHistory to ignore lineage ids watches a cross-lineage *)
(* "uniqueness" invariant collapse immediately (two lineages may close     *)
(* different successors at the same version, legitimately).                *)
WellFormedHistory(H) ==
  \E l \in Lineages : \A r \in H : r[1] = l

CrossLineageUniquenessWouldBeFalse ==
  \A H \in SUBSET transitions :
    WellFormedHistory(H) =>
      \A r1 \in H, r2 \in H, v \in Versions :
        (r1[2] = v /\ r2[2] = v /\ r1[3] = r2[3]
         /\ r1[3] \cap HonestMembers # {}) => r1[4] = r2[4]

Inv ==
  /\ TypeOK
  /\ \A m \in HonestMembers, l \in Lineages, v \in Versions, succ \in Configs :
       (<<m, l, v, succ>> \in tacks) => tjournal[m][l][v] = succ
  /\ TransitionsJustified

====
```

A.7.5 MembershipTransitionProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/MembershipTransitionProof.tla}

```
---- MODULE MembershipTransitionProof ----
(***************************************************************************)
(* AFT-CB P2.3 -- TLAPS discharge: T5a lineage uniqueness under             *)
(* per-configuration MHA (stated as a theorem CONDITION, not a global      *)
(* assumption), the signature-justification invariant (the no-silence      *)
(* assertion), and the A6-conditional bootstrap-freshness statement --      *)
(* conditionality in the theorem, never in prose.                          *)
(***************************************************************************)
EXTENDS MembershipTransition, TLAPS

(* Assumed here rather than in the model module: TLC cannot evaluate this  *)
(* membership test over a model value, while its model-value semantics     *)
(* already make it true; tlapm consumes it for the journal-conflict step.  *)
ASSUME NoSuccOutsideConfigs == NoSucc \notin Configs
```

```
15
16  LEMMA MInvInit == Init => Inv
17    BY MembershipConstants DEF Init, Inv, TypeOK, TransitionsJustified
18
19  LEMMA MInvNext == Inv /\ [Next]_vars => Inv'
20  <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
21    OBVIOUS
22  <1>2. CASE UNCHANGED vars
23    BY <1>1, <1>2 DEF Inv, TypeOK, TransitionsJustified, AckedBy, AcceptedBy,
24       vars
25  <1>3. ASSUME NEW m \in AllMembers, NEW l \in Lineages, NEW v \in Versions,
26               NEW succ \in Configs, HonestTransitionAck(m, l, v, succ)
27        PROVE Inv'
28    <2>1. m \in HonestMembers /\ tjournal[m][l][v] = NoSucc
29      BY <1>3 DEF HonestTransitionAck
30    <2>2. TypeOK'
31      BY <1>1, <1>3, <2>1 DEF Inv, TypeOK, HonestTransitionAck
32    <2>3. \A mm \in HonestMembers, ll \in Lineages, vv \in Versions,
33            ss \in Configs :
34            (<<mm, ll, vv, ss>> \in tacks') => tjournal'[mm][ll][vv] = ss
35      <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ll \in Lineages,
36                            NEW vv \in Versions, NEW ss \in Configs,
37                            <<mm, ll, vv, ss>> \in tacks'
38            PROVE tjournal'[mm][ll][vv] = ss
39        OBVIOUS
40      <3>2. tacks' = tacks \cup {<<m, l, v, succ>>}
41        BY <1>3 DEF HonestTransitionAck
42      <3>3. CASE <<mm, ll, vv, ss>> = <<m, l, v, succ>>
43        <4>1. mm = m /\ ll = l /\ vv = v /\ ss = succ
44          BY <3>3
45        <4>2. tjournal'[m][l][v] = succ
46          BY <1>1, <1>3, <2>1 DEF Inv, TypeOK, HonestTransitionAck
47        <4> QED BY <4>1, <4>2
48      <3>4. CASE <<mm, ll, vv, ss>> \in tacks
49        <4>1. tjournal[mm][ll][vv] = ss
50          BY <1>1, <3>4 DEF Inv
51        <4>2. CASE mm = m /\ ll = l /\ vv = v
52          <5>1. tjournal[m][l][v] = ss
53            BY <4>1, <4>2
54          <5>2. tjournal[m][l][v] = NoSucc
55            BY <2>1
56          <5>3. ss # NoSucc
57            BY <3>1, NoSuccOutsideConfigs
58          <5> QED BY <5>1, <5>2, <5>3
59        <4>3. CASE ~(mm = m /\ ll = l /\ vv = v)
60          <5>1. tjournal'[mm][ll][vv] = tjournal[mm][ll][vv]
61            BY <1>1, <1>3, <2>1, <4>3 DEF Inv, TypeOK, HonestTransitionAck
62          <5> QED BY <4>1, <5>1
63        <4> QED BY <4>2, <4>3
64      <3> QED BY <3>1, <3>2, <3>3, <3>4
65    <2>4. TransitionsJustified'
66      <3>1. SUFFICES ASSUME NEW ll \in Lineages, NEW vv \in Versions,
67                            NEW pp \in Configs, NEW ss \in Configs,
68                            <<ll, vv, pp, ss>> \in transitions'
69            PROVE /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)'
70                  /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)'
71        BY DEF TransitionsJustified
72      <3>2. transitions' = transitions /\ tacks' = tacks \cup {<<m, l, v, succ>>}
73            /\ taccepts' = taccepts
74        BY <1>3 DEF HonestTransitionAck
75      <3>3. /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)
76            /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)
```

```
      BY <1>1, <3>1, <3>2 DEF Inv, TransitionsJustified
    <3> QED BY <3>1, <3>2, <3>3 DEF AckedBy, AcceptedBy
  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
<1>5. ASSUME NEW m \in AllMembers, NEW l \in Lineages, NEW v \in Versions,
             NEW succ \in Configs, Accept(m, l, v, succ)
      PROVE Inv'
  <2>1. TypeOK'
    BY <1>1, <1>5 DEF Inv, TypeOK, Accept
  <2>2. \A mm \in HonestMembers, ll \in Lineages, vv \in Versions,
          ss \in Configs :
          (<<mm, ll, vv, ss>> \in tacks') => tjournal'[mm][ll][vv] = ss
    BY <1>1, <1>5 DEF Inv, Accept
  <2>3. TransitionsJustified'
    <3>1. SUFFICES ASSUME NEW ll \in Lineages, NEW vv \in Versions,
                          NEW pp \in Configs, NEW ss \in Configs,
                          <<ll, vv, pp, ss>> \in transitions'
          PROVE /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)'
                /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)'
      BY DEF TransitionsJustified
    <3>2. transitions' = transitions /\ tacks' = tacks
          /\ taccepts' = taccepts \cup {<<m, l, v, succ>>}
      BY <1>5 DEF Accept
    <3>3. /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)
          /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)
      BY <1>1, <3>1, <3>2 DEF Inv, TransitionsJustified
    <3> QED BY <3>1, <3>2, <3>3 DEF AckedBy, AcceptedBy
  <2> QED BY <2>1, <2>2, <2>3 DEF Inv
<1>6. ASSUME NEW l \in Lineages, NEW v \in Versions, NEW parent \in Configs,
             NEW succ \in Configs, CloseTransition(l, v, parent, succ)
      PROVE Inv'
  <2>1. TypeOK'
    BY <1>1, <1>6 DEF Inv, TypeOK, CloseTransition
  <2>2. \A mm \in HonestMembers, ll \in Lineages, vv \in Versions,
          ss \in Configs :
          (<<mm, ll, vv, ss>> \in tacks') => tjournal'[mm][ll][vv] = ss
    BY <1>1, <1>6 DEF Inv, TransitionsJustified, CloseTransition
  <2>3. TransitionsJustified'
    BY <1>1, <1>6 DEF Inv, TransitionsJustified, CloseTransition,
       AckedBy, AcceptedBy
  <2> QED BY <2>1, <2>2, <2>3 DEF Inv
<1>7. ASSUME NEW newL \in Lineages, NEW priorL \in Lineages,
             MintRoot(newL, priorL)
      PROVE Inv'
  BY <1>1, <1>7 DEF Inv, TypeOK, TransitionsJustified, AckedBy, AcceptedBy,
     MintRoot
<1>8. ASSUME NEW a \in AllMembers, NEW s \in AllMembers,
             EmitSilenceRecord(a, s)
      PROVE Inv'
  BY <1>1, <1>8 DEF Inv, TypeOK, TransitionsJustified, AckedBy, AcceptedBy,
     EmitSilenceRecord
<1>9. QED
  BY <1>1, <1>2, <1>3, <1>5, <1>6, <1>7, <1>8 DEF Next, vars

(***************************************************************************)
(* T5a: within one lineage, from a common parent configuration that        *)
(* contains at least one honest member (per-configuration MHA as an        *)
(* explicit CONDITION of the statement), closed successors are unique.     *)
(***************************************************************************)
LEMMA InvLineageUniqueness == Inv => LineageUniqueness
<1>1. SUFFICES ASSUME Inv, NEW l \in Lineages, NEW v \in Versions,
                      NEW parent \in Configs,
                      NEW s1 \in Configs, NEW s2 \in Configs,
```

```
                      <<l, v, parent, s1>> \in transitions,
                      <<l, v, parent, s2>> \in transitions,
                      parent \cap HonestMembers # {}
      PROVE s1 = s2
  BY DEF LineageUniqueness
<1>2. PICK h \in parent \cap HonestMembers : TRUE
  BY <1>1
<1>3. h \in parent /\ h \in HonestMembers
  BY <1>2
<1>4. <<h, l, v, s1>> \in tacks /\ <<h, l, v, s2>> \in tacks
  BY <1>1, <1>3 DEF Inv, TransitionsJustified, AckedBy
<1>5. tjournal[h][l][v] = s1 /\ tjournal[h][l][v] = s2
  BY <1>1, <1>3, <1>4 DEF Inv
<1>6. QED
  BY <1>5

THEOREM MembershipSafety == Spec => [](LineageUniqueness /\ TransitionsJustified)
<1>1. Init => Inv
  BY MInvInit
<1>2. Inv /\ [Next]_vars => Inv'
  BY MInvNext
<1>3. Spec => []Inv
  BY <1>1, <1>2, PTL DEF Spec
<1>4. Inv => (LineageUniqueness /\ TransitionsJustified)
  BY InvLineageUniqueness DEF Inv
<1>5. QED
  BY <1>3, <1>4, PTL

(*************************************************************************)
(* Bootstrap freshness, A6-conditional IN THE STATEMENT: with a deployed   *)
(* anchor, a verifier that partitions records by lineage id (well-formed   *)
(* histories) obtains per-lineage uniqueness wherever the parent satisfies *)
(* MHA -- the anchor's job (selecting the live lineage among the            *)
(* partitions) is exactly the deployed mechanism's own assumption, which   *)
(* is why the implication's hypothesis names it.  With NO anchor deployed  *)
(* the theorem asserts nothing -- out-of-model, stated structurally.        *)
(*************************************************************************)
THEOREM BootstrapFreshness ==
  (DeployedAnchor \in Anchors) =>
    (Spec => [](\A l \in Lineages, v \in Versions, parent \in Configs,
                  s1 \in Configs, s2 \in Configs :
                 (/\ <<l, v, parent, s1>> \in transitions
                  /\ <<l, v, parent, s2>> \in transitions
                  /\ parent \cap HonestMembers # {}) => s1 = s2))
<1>1. SUFFICES ASSUME DeployedAnchor \in Anchors PROVE
        Spec => []LineageUniqueness
  BY DEF LineageUniqueness
<1>2. Spec => [](LineageUniqueness /\ TransitionsJustified)
  BY MembershipSafety
<1>3. QED
  BY <1>2, PTL

====
```

A.7.6 CustodyObligation.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/CustodyObligation.tla}

```
---- MODULE CustodyObligation ----
(***************************************************************************)
(* AFT-CB P2.4 -- validate-and-hold as a signing obligation (T3), the       *)
(* retention/succession discipline, and the OPTIONAL audit lane.           *)
(*                                                                         *)
(* The obligation (spec §4): an honest member final-acks a boundary only   *)
(* having OBTAINED the full byte set, and the ack commits it to RETAIN and *)
(* SERVE those bytes -- so a close implies close-time replication at every  *)
(* compliant signer, and one honest signer (A2, stated as a theorem        *)
(* condition) yields conditional availability deterministically.           *)
(*                                                                         *)
(* Pairwise audits are OPTIONAL enforcement actions producing slashing     *)
(* evidence that NOTHING consumes as a close precondition: the audit       *)
(* variable is written by the audit action alone and read by no guard of   *)
(* any close-path action -- mirroring P2.3's silence-record discipline.     *)
(* The zero-audit reachability witness (a trace that closes a slot with    *)
(* auditEvidence = {}) is produced by TLC via the NoCloseEver violation    *)
(* run recorded in the leg ledger.                                         *)
(*                                                                         *)
(* Succession (spec §6): custody releases only RECONSTRUCT-BEFORE-RELEASE  *)
(* -- an honest holder may drop a slot's bytes (and its bond may release)   *)
(* only once a successor member has reconstructed full custody of them.    *)
(*                                                                         *)
(* Adversary: structural (the P2.3 pattern) -- corrupt members' acks are    *)
(* always available and their storage is arbitrary (they may sign without  *)
(* holding and delete at will, which is why availability quantifies over   *)
(* honest holders only).  Deletion by the other n−1 is harmless BY         *)
(* THEOREM, not by hope.                                                   *)
(***************************************************************************)
EXTENDS Naturals, FiniteSets

CONSTANTS
  Ring,           \* the signer configuration
  HonestMembers,  \* honest subset (MHA enters theorems as a condition)
  Slots,
  Artifacts,      \* the byte universe; a boundary is a nonempty subset
  NoBoundary      \* journal sentinel

ASSUME CustodyConstants ==
  /\ HonestMembers \subseteq Ring

Boundaries == (SUBSET Artifacts) \ {{}}

VARIABLES
  delivered,      \* Artifacts \X Ring: adversary-scheduled delivery
  holds,          \* [HonestMembers -> SUBSET Artifacts]: bytes actually held
  journal,        \* [HonestMembers -> [Slots -> Boundaries \cup {NoBoundary}]]
  acks,           \* honest final-acks <<m, s, B>>
  retained,       \* [HonestMembers -> [Slots -> SUBSET Artifacts]]: serving custody
  closes,         \* closed slots <<s, B>>
  reconstructed,  \* succession receipts <<successor, s>>: full custody rebuilt
  bondReleased,   \* [HonestMembers -> BOOLEAN]
  auditEvidence   \* audit-lane records <<auditor, subject, s>> -- never load-bearing

vars == <<delivered, holds, journal, acks, retained, closes, reconstructed,
          bondReleased, auditEvidence>>

AckedBy(m, s, B) ==
  IF m \in HonestMembers THEN <<m, s, B>> \in acks ELSE TRUE

TypeOK ==
```

```
  /\ delivered \subseteq Artifacts \X Ring
  /\ holds \in [HonestMembers -> SUBSET Artifacts]
  /\ journal \in [HonestMembers -> [Slots -> Boundaries \cup {NoBoundary}]]
  /\ acks \subseteq HonestMembers \X Slots \X Boundaries
  /\ retained \in [HonestMembers -> [Slots -> SUBSET Artifacts]]
  /\ closes \subseteq Slots \X Boundaries
  /\ reconstructed \subseteq HonestMembers \X Slots
  /\ bondReleased \in [HonestMembers -> BOOLEAN]
  /\ auditEvidence \subseteq Ring \X Ring \X Slots

Init ==
  /\ delivered = {}
  /\ holds = [m \in HonestMembers |-> {}]
  /\ journal = [m \in HonestMembers |-> [s \in Slots |-> NoBoundary]]
  /\ acks = {}
  /\ retained = [m \in HonestMembers |-> [s \in Slots |-> {}]]
  /\ closes = {}
  /\ reconstructed = {}
  /\ bondReleased = [m \in HonestMembers |-> FALSE]
  /\ auditEvidence = {}

Deliver(a, m) ==
  /\ delivered' = delivered \cup {<<a, m>>}
  /\ UNCHANGED <<holds, journal, acks, retained, closes, reconstructed,
                 bondReleased, auditEvidence>>

(* An honest member obtains bytes it was actually delivered.              *)
Fetch(m, a) ==
  /\ m \in HonestMembers
  /\ <<a, m>> \in delivered
  /\ holds' = [holds EXCEPT ![m] = holds[m] \cup {a}]
  /\ UNCHANGED <<delivered, journal, acks, retained, closes, reconstructed,
                 bondReleased, auditEvidence>>

(* VALIDATE-AND-HOLD: the guard B \subseteq holds[m] is the obligation --  *)
(* the member has OBTAINED every byte of the boundary it signs -- and the   *)
(* effect commits it to retain/serve (retained[m][s] = B).  The journal    *)
(* keeps single-final-ack (A3).  The mutation drill removes the holds      *)
(* guard and the T3-shaped invariant goes red.                             *)
HonestFinalAck(m, s, B) ==
  /\ m \in HonestMembers
  /\ ~bondReleased[m]   \* a departed member signs nothing new
  /\ s \in Slots /\ B \in Boundaries
  /\ B \subseteq holds[m]
  /\ journal[m][s] = NoBoundary
  /\ journal' = [journal EXCEPT ![m] = [journal[m] EXCEPT ![s] = B]]
  /\ acks' = acks \cup {<<m, s, B>>}
  /\ retained' = [retained EXCEPT ![m] = [retained[m] EXCEPT ![s] = B]]
  /\ UNCHANGED <<delivered, holds, closes, reconstructed, bondReleased,
                 auditEvidence>>

(* n-of-n close: honest acks are state; corrupt signatures always          *)
(* available (they may sign never having held a byte -- that is the fault   *)
(* model, and why the theorem leans on the honest signer only).            *)
CloseSlot(s, B) ==
  /\ s \in Slots /\ B \in Boundaries
  /\ \A m \in Ring : AckedBy(m, s, B)
  /\ closes' = closes \cup {<<s, B>>}
  /\ UNCHANGED <<delivered, holds, journal, acks, retained, reconstructed,
                 bondReleased, auditEvidence>>

(* Succession: a successor honest member reconstructs FULL custody of a    *)
```

```
(* slot's closed boundary from bytes it actually holds.  Reconstruction    *)
(* ADDS custody (union) -- it never shrinks anyone's serving set; custody   *)
(* drops ride bond release, outside this kernel's mutation surface.        *)
Reconstruct(succ, s, B) ==
  /\ succ \in HonestMembers
  /\ ~bondReleased[succ]   \* a departed member takes on no new custody
  /\ <<s, B>> \in closes
  /\ B \subseteq holds[succ]
  /\ retained' = [retained EXCEPT ![succ] =
       [retained[succ] EXCEPT ![s] = retained[succ][s] \cup B]]
  /\ reconstructed' = reconstructed \cup {<<succ, s>>}
  /\ UNCHANGED <<delivered, holds, journal, acks, closes, bondReleased,
                 auditEvidence>>

(* RECONSTRUCT-BEFORE-RELEASE: an honest member's bond releases only when  *)
(* every slot it retains custody for has a DIFFERENT reconstructed         *)
(* successor.  (Custody drop rides bond release; the invariant below is    *)
(* what the mutation targets.)                                             *)
ReleaseBond(m) ==
  /\ m \in HonestMembers
  /\ \A s \in Slots :
       (retained[m][s] # {}) =>
         \E succ \in HonestMembers : succ # m /\ <<succ, s>> \in reconstructed
  /\ bondReleased' = [bondReleased EXCEPT ![m] = TRUE]
  /\ UNCHANGED <<delivered, holds, journal, acks, retained, closes,
                 reconstructed, auditEvidence>>

(* The OPTIONAL audit lane: an auditor challenges a subject over a slot    *)
(* and a record is minted.  No close-path guard reads auditEvidence --     *)
(* audits produce slashing evidence, never availability.                   *)
Audit(auditor, subject, s) ==
  /\ auditor \in Ring /\ subject \in Ring /\ s \in Slots
  /\ auditEvidence' = auditEvidence \cup {<<auditor, subject, s>>}
  /\ UNCHANGED <<delivered, holds, journal, acks, retained, closes,
                 reconstructed, bondReleased>>

Next ==
  \/ \E a \in Artifacts, m \in Ring : Deliver(a, m)
  \/ \E m \in HonestMembers, a \in Artifacts : Fetch(m, a)
  \/ \E m \in HonestMembers, s \in Slots, B \in Boundaries :
       HonestFinalAck(m, s, B)
  \/ \E s \in Slots, B \in Boundaries : CloseSlot(s, B)
  \/ \E succ \in HonestMembers, s \in Slots, B \in Boundaries :
       Reconstruct(succ, s, B)
  \/ \E m \in HonestMembers : ReleaseBond(m)
  \/ \E auditor \in Ring, subject \in Ring, s \in Slots :
       Audit(auditor, subject, s)

Spec == Init /\ [][Next]_vars

(***************************************************************************)
(* The T3-shaped invariant: every honest ack carries full custody -- the   *)
(* acked boundary is retained by the acker (or a reconstructed successor   *)
(* stands in for it once released; in this kernel retained never shrinks,  *)
(* so the direct form holds).  From it: a closed slot with one honest ring *)
(* member is AVAILABLE -- some honest member retains every byte.            *)
(***************************************************************************)
CustodyInvariant ==
  \A m \in HonestMembers, s \in Slots, B \in Boundaries :
    (<<m, s, B>> \in acks) => B \subseteq retained[m][s]

ConditionalAvailability ==
```

```
  \A s \in Slots, B \in Boundaries :
    (<<s, B>> \in closes /\ Ring \cap HonestMembers # {}) =>
      \E m \in HonestMembers : B \subseteq retained[m][s]

ReleaseDiscipline ==
  \A m \in HonestMembers :
    bondReleased[m] =>
      \A s \in Slots :
        (retained[m][s] # {}) =>
          \E succ \in HonestMembers : succ # m /\ <<succ, s>> \in reconstructed

HonestAckJournal ==
  \A m \in HonestMembers, s \in Slots, B \in Boundaries :
    (<<m, s, B>> \in acks) => journal[m][s] = B

(* The T3 truth the validate-and-hold guard protects: every byte a member  *)
(* CLAIMS custody of (retained) is a byte it actually HOLDS and can serve. *)
(* The leg's mutation drill removes the guard and THIS invariant goes red  *)
(* -- a member would then claim custody of bytes it never obtained.         *)
RetainedBacked ==
  \A m \in HonestMembers, s \in Slots : retained[m][s] \subseteq holds[m]

Inv ==
  /\ TypeOK
  /\ HonestAckJournal
  /\ RetainedBacked
  /\ CustodyInvariant
  /\ \A s \in Slots, B \in Boundaries :
       (<<s, B>> \in closes) => \A m \in Ring : AckedBy(m, s, B)
  /\ ReleaseDiscipline


(* WITNESS-ONLY definition for the zero-audit reachability run: TLC        *)
(* violating this invariant yields a shortest trace that closes a slot --  *)
(* and a shortest trace contains no Audit step, which IS the required      *)
(* witness that audits are not load-bearing for a close.                   *)
NoCloseEver == closes = {}

====
```

A.7.7 CustodyObligationProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/CustodyObligationProof.tla}

```
---- MODULE CustodyObligationProof ----
(***************************************************************************)
(* AFT-CB P2.4 -- TLAPS discharge: the custody invariant is inductive, and  *)
(* conditional availability (T3's shape) follows with MHA as an explicit   *)
(* theorem condition.  No proof step mentions auditEvidence: the audit     *)
(* lane is not load-bearing, by the shape of the proof itself.             *)
(***************************************************************************)
EXTENDS CustodyObligation, TLAPS

ASSUME NoBoundaryOutsideBoundaries == NoBoundary \notin Boundaries

LEMMA CInvInit == Init => Inv
  BY CustodyConstants DEF Init, Inv, TypeOK, HonestAckJournal,
     RetainedBacked, CustodyInvariant, ReleaseDiscipline, AckedBy
```

```
15
16 LEMMA CInvNext == Inv /\ [Next]_vars => Inv'
17 <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
18   OBVIOUS
19 <1>2. CASE UNCHANGED vars
20   BY <1>1, <1>2 DEF Inv, TypeOK, HonestAckJournal, RetainedBacked,
21      CustodyInvariant, ReleaseDiscipline, AckedBy, vars
22 <1>3. ASSUME NEW a \in Artifacts, NEW m \in Ring, Deliver(a, m)
23       PROVE Inv'
24   BY <1>1, <1>3 DEF Inv, TypeOK, HonestAckJournal, RetainedBacked,
25      CustodyInvariant, ReleaseDiscipline, AckedBy, Deliver
26 <1>4. ASSUME NEW m \in HonestMembers, NEW a \in Artifacts, Fetch(m, a)
27       PROVE Inv'
28   <2>1. TypeOK'
29     BY <1>1, <1>4, CustodyConstants DEF Inv, TypeOK, Fetch
30   <2>2. HonestAckJournal'
31     BY <1>1, <1>4 DEF Inv, HonestAckJournal, Fetch
32   <2>3. CustodyInvariant'
33     BY <1>1, <1>4 DEF Inv, CustodyInvariant, Fetch
34   <2>4. (\A s2 \in Slots, B2 \in Boundaries :
35           (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
36     BY <1>1, <1>4 DEF Inv, AckedBy, Fetch
37   <2>5. ReleaseDiscipline'
38     BY <1>1, <1>4 DEF Inv, ReleaseDiscipline, Fetch
39   <2>6. RetainedBacked'
40     <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots
41           PROVE retained'[mm][ss] \subseteq holds'[mm]
42       BY DEF RetainedBacked
43     <3>2. retained' = retained
44       BY <1>4 DEF Fetch
45     <3>3. holds[mm] \subseteq holds'[mm]
46       BY <1>1, <1>4, CustodyConstants DEF Inv, TypeOK, Fetch
47     <3>4. retained[mm][ss] \subseteq holds[mm]
48       BY <1>1, <3>1 DEF Inv, RetainedBacked
49     <3> QED BY <3>1, <3>2, <3>3, <3>4
50   <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
51 <1>5. ASSUME NEW m \in HonestMembers, NEW s \in Slots, NEW B \in Boundaries,
52              HonestFinalAck(m, s, B)
53       PROVE Inv'
54   <2>1. B \subseteq holds[m] /\ journal[m][s] = NoBoundary
55     BY <1>5 DEF HonestFinalAck
56   <2>2. TypeOK'
57     BY <1>1, <1>5, <2>1, CustodyConstants
58        DEF Inv, TypeOK, HonestFinalAck
59   <2>3. HonestAckJournal'
60     <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
61                           NEW BB \in Boundaries, <<mm, ss, BB>> \in acks'
62           PROVE journal'[mm][ss] = BB
63       BY DEF HonestAckJournal
64     <3>2. acks' = acks \cup {<<m, s, B>>}
65       BY <1>5 DEF HonestFinalAck
66     <3>3. CASE <<mm, ss, BB>> = <<m, s, B>>
67       <4>1. journal'[m][s] = B
68         BY <1>1, <1>5, <2>1 DEF Inv, TypeOK, HonestFinalAck
69       <4> QED BY <3>3, <4>1
70     <3>4. CASE <<mm, ss, BB>> \in acks
71       <4>1. journal[mm][ss] = BB
72         BY <1>1, <3>4 DEF Inv, HonestAckJournal
73       <4>2. CASE mm = m /\ ss = s
74         <5>1. journal[m][s] = BB
75           BY <4>1, <4>2
76         <5>2. BB # NoBoundary
```

```
          BY <3>1, NoBoundaryOutsideBoundaries
        <5> QED BY <2>1, <5>1, <5>2
      <4>3. CASE ~(mm = m /\ ss = s)
        <5>1. journal'[mm][ss] = journal[mm][ss]
          BY <1>1, <1>5, <2>1, <4>3 DEF Inv, TypeOK, HonestFinalAck
        <5> QED BY <4>1, <5>1
      <4> QED BY <4>2, <4>3
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2>4. CustodyInvariant'
    <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
                          NEW BB \in Boundaries, <<mm, ss, BB>> \in acks'
          PROVE BB \subseteq retained'[mm][ss]
      BY DEF CustodyInvariant
    <3>2. acks' = acks \cup {<<m, s, B>>}
      BY <1>5 DEF HonestFinalAck
    <3>3. CASE <<mm, ss, BB>> = <<m, s, B>>
      <4>1. retained'[m][s] = B
        BY <1>1, <1>5, CustodyConstants DEF Inv, TypeOK, HonestFinalAck
      <4> QED BY <3>3, <4>1
    <3>4. CASE <<mm, ss, BB>> \in acks
      <4>1. BB \subseteq retained[mm][ss]
        BY <1>1, <3>4 DEF Inv, CustodyInvariant
      <4>2. CASE mm = m /\ ss = s
        <5>1. journal[m][s] = BB
          BY <1>1, <3>4, <4>2 DEF Inv, HonestAckJournal
        <5>2. BB # NoBoundary
          BY <3>1, NoBoundaryOutsideBoundaries
        <5> QED BY <2>1, <5>1, <5>2
      <4>3. CASE ~(mm = m /\ ss = s)
        <5>1. retained'[mm][ss] = retained[mm][ss]
          BY <1>1, <1>5, CustodyConstants, <4>3
             DEF Inv, TypeOK, HonestFinalAck
        <5> QED BY <4>1, <5>1
      <4> QED BY <4>2, <4>3
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2>5. (\A s2 \in Slots, B2 \in Boundaries :
          (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
    BY <1>1, <1>5 DEF Inv, AckedBy, HonestFinalAck
  <2>6. ReleaseDiscipline'
    BY <1>1, <1>5, CustodyConstants
       DEF Inv, TypeOK, ReleaseDiscipline, HonestFinalAck
  <2>7. RetainedBacked'
    BY <1>1, <1>5, <2>1, CustodyConstants
       DEF Inv, TypeOK, RetainedBacked, HonestFinalAck
  <2> QED BY <2>2, <2>3, <2>4, <2>5, <2>6, <2>7 DEF Inv
<1>6. ASSUME NEW s \in Slots, NEW B \in Boundaries, CloseSlot(s, B)
      PROVE Inv'
  <2>1. TypeOK'
    BY <1>1, <1>6 DEF Inv, TypeOK, CloseSlot
  <2>2. HonestAckJournal'
    BY <1>1, <1>6 DEF Inv, HonestAckJournal, CloseSlot
  <2>3. CustodyInvariant'
    BY <1>1, <1>6 DEF Inv, CustodyInvariant, CloseSlot
  <2>4. (\A s2 \in Slots, B2 \in Boundaries :
          (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
    <3>1. SUFFICES ASSUME NEW s2 \in Slots, NEW B2 \in Boundaries,
                          <<s2, B2>> \in closes', NEW mm \in Ring
          PROVE AckedBy(mm, s2, B2)'
      OBVIOUS
    <3>2. closes' = closes \cup {<<s, B>>} /\ acks' = acks
      BY <1>6 DEF CloseSlot
    <3>3. CASE <<s2, B2>> \in closes
```

```
      BY <1>1, <3>2, <3>3 DEF Inv, AckedBy
    <3>4. CASE <<s2, B2>> = <<s, B>>
      BY <1>6, <3>2, <3>4 DEF CloseSlot, AckedBy
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2>5. ReleaseDiscipline'
    BY <1>1, <1>6 DEF Inv, ReleaseDiscipline, CloseSlot
  <2>6. RetainedBacked'
    BY <1>1, <1>6 DEF Inv, RetainedBacked, CloseSlot
  <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
<1>7. ASSUME NEW succ \in HonestMembers, NEW s \in Slots, NEW B \in Boundaries,
             Reconstruct(succ, s, B)
      PROVE Inv'
  <2>1. TypeOK'
    BY <1>1, <1>7, CustodyConstants DEF Inv, TypeOK, Reconstruct
  <2>2. CustodyInvariant'
    <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
                          NEW BB \in Boundaries, <<mm, ss, BB>> \in acks'
          PROVE BB \subseteq retained'[mm][ss]
      BY DEF CustodyInvariant
    <3>2. acks' = acks
      BY <1>7 DEF Reconstruct
    <3>3. BB \subseteq retained[mm][ss]
      BY <1>1, <3>1, <3>2 DEF Inv, CustodyInvariant
    <3>4. retained[mm][ss] \subseteq retained'[mm][ss]
      (* Reconstruction only ever UNIONs custody in, so every member's      *)
      (* serving set is monotone under this action.                         *)
      BY <1>1, <1>7, CustodyConstants DEF Inv, TypeOK, Reconstruct
    <3> QED BY <3>3, <3>4
  <2>3. HonestAckJournal'
    BY <1>1, <1>7 DEF Inv, HonestAckJournal, Reconstruct
  <2>4. (\A s2 \in Slots, B2 \in Boundaries :
           (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
    BY <1>1, <1>7 DEF Inv, AckedBy, Reconstruct
  <2>5. ReleaseDiscipline'
    BY <1>1, <1>7, CustodyConstants
       DEF Inv, TypeOK, ReleaseDiscipline, Reconstruct
  <2>6. RetainedBacked'
    BY <1>1, <1>7, CustodyConstants
       DEF Inv, TypeOK, RetainedBacked, Reconstruct
  <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
<1>8. ASSUME NEW m \in HonestMembers, ReleaseBond(m)
      PROVE Inv'
  <2>1. TypeOK'
    BY <1>1, <1>8, CustodyConstants DEF Inv, TypeOK, ReleaseBond
  <2>2. HonestAckJournal'
    BY <1>1, <1>8 DEF Inv, HonestAckJournal, ReleaseBond
  <2>3. CustodyInvariant'
    BY <1>1, <1>8 DEF Inv, CustodyInvariant, ReleaseBond
  <2>4. (\A s2 \in Slots, B2 \in Boundaries :
           (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
    BY <1>1, <1>8 DEF Inv, AckedBy, ReleaseBond
  <2>5. ReleaseDiscipline'
    <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, bondReleased'[mm],
                          NEW ss \in Slots, retained'[mm][ss] # {}
          PROVE \E succ \in HonestMembers :
                  succ # mm /\ <<succ, ss>> \in reconstructed'
      BY DEF ReleaseDiscipline
    <3>2. /\ bondReleased' = [bondReleased EXCEPT ![m] = TRUE]
          /\ retained' = retained /\ reconstructed' = reconstructed
      BY <1>8 DEF ReleaseBond
    <3>3. CASE mm = m
      BY <1>1, <1>8, <3>1, <3>2, <3>3, CustodyConstants
```

```
         DEF Inv, TypeOK, ReleaseBond
    <3>4. CASE mm # m
      <4>1. bondReleased[mm]
        BY <1>1, <3>1, <3>2, <3>4, CustodyConstants DEF Inv, TypeOK
      <4> QED BY <1>1, <3>1, <3>2, <4>1 DEF Inv, ReleaseDiscipline
    <3> QED BY <3>1, <3>3, <3>4
  <2>6. RetainedBacked'
    BY <1>1, <1>8 DEF Inv, RetainedBacked, ReleaseBond
  <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
<1>9. ASSUME NEW auditor \in Ring, NEW subject \in Ring, NEW s \in Slots,
             Audit(auditor, subject, s)
      PROVE Inv'
  BY <1>1, <1>9 DEF Inv, TypeOK, HonestAckJournal, RetainedBacked,
     CustodyInvariant, ReleaseDiscipline, AckedBy, Audit
<1>10. QED
  BY <1>1, <1>2, <1>3, <1>4, <1>5, <1>6, <1>7, <1>8, <1>9 DEF Next, vars

LEMMA InvAvailability == Inv => ConditionalAvailability
<1>1. SUFFICES ASSUME Inv, NEW s \in Slots, NEW B \in Boundaries,
                      <<s, B>> \in closes, Ring \cap HonestMembers # {}
      PROVE \E m \in HonestMembers : B \subseteq retained[m][s]
  BY DEF ConditionalAvailability
<1>2. PICK h \in Ring \cap HonestMembers : TRUE
  BY <1>1
<1>3. AckedBy(h, s, B)
  BY <1>1, <1>2 DEF Inv
<1>4. <<h, s, B>> \in acks
  BY <1>2, <1>3 DEF AckedBy
<1>5. B \subseteq retained[h][s]
  BY <1>1, <1>2, <1>4 DEF Inv, CustodyInvariant
<1>6. QED
  BY <1>2, <1>5

THEOREM CustodySafety ==
  Spec => [](ConditionalAvailability /\ ReleaseDiscipline)
<1>1. Init => Inv
  BY CInvInit
<1>2. Inv /\ [Next]_vars => Inv'
  BY CInvNext
<1>3. Spec => []Inv
  BY <1>1, <1>2, PTL DEF Spec
<1>4. Inv => (ConditionalAvailability /\ ReleaseDiscipline)
  BY InvAvailability DEF Inv
<1>5. QED
  BY <1>3, <1>4, PTL

====
```

A.7.8 ForensicAccountability.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/ForensicAccountability.tla}

```
---- MODULE ForensicAccountability ----
(***************************************************************************)
(* AFT-CB P2.6 -- forensic accountability (T7), proven against the         *)
(* WIRE-LEVEL certificate format.                                          *)
(*                                                                         *)
(* The wire format is part of the theorem (spec §12): a seal certificate   *)
```

```
(* is a bitmap multisig -- a record carrying, for EVERY ring member, that   *)
(* member's individually verifiable signature share over the exact         *)
(* domain-separated tuple.  Assembly is impossible without all n shares,   *)
(* and decomposition back into per-member shares is total: any holder of   *)
(* two conflicting certificates extracts, for every member, a pair of      *)
(* conflicting shares -- a self-contained conviction (T7), covering every   *)
(* participant necessary for the violation (all n; the L9 maximum).        *)
(*                                                                         *)
(* The mutation drill swaps the encoding for an attribution-destroying     *)
(* OPAQUE AGGREGATE -- a certificate that proves "the ring signed" while    *)
(* decomposing into nobody -- and the attributability invariant goes red:   *)
(* exactly the trade the spec's §12.4 forbids on any seal path.            *)
(*                                                                         *)
(* The staged double-seal gate runs at the ALL-BYZANTINE configuration     *)
(* (HonestMembers = {}): both conflicting certificates assemble (nothing   *)
(* stops an all-corrupt ring from double-sealing -- that is above the       *)
(* fault threshold), and TLC's DoubleSealReached violation trace IS the    *)
(* extraction witness: the full necessary-participant set, recorded in     *)
(* the leg ledger.                                                         *)
(*                                                                         *)
(* Adversary: structural (P2.3's pattern).  Corrupt members' shares over   *)
(* any tuple are always available; honest shares exist only via the        *)
(* journal-guarded honest ack (A3), which is why conflicting certificates  *)
(* under MHA are impossible (P2.1's theorem, restated here over the wire   *)
(* structure as CertUniqueness).                                           *)
(***************************************************************************)
EXTENDS Naturals, FiniteSets

CONSTANTS
  Ring,
  HonestMembers,
  Slots,
  Roots,
  NoRoot

ASSUME ForensicConstants ==
  /\ HonestMembers \subseteq Ring

(* A wire-level share: member m's signature over the domain-separated      *)
(* tuple (slot, root).  A certificate is bitmap-complete: it embeds the    *)
(* share of EVERY member over ITS OWN (slot, root) -- the record structure  *)
(* IS the format; there is nothing else on the wire.                       *)
Share(m, s, r) == <<m, s, r>>

Cert(s, r) == [slot |-> s, root |-> r,
               shares |-> {Share(m, s, r) : m \in Ring}]

VARIABLES
  journal,   \* [HonestMembers -> [Slots -> Roots \cup {NoRoot}]] (A3)
  hshares,   \* honest shares in existence
  certs      \* assembled certificates (wire records)

vars == <<journal, hshares, certs>>

ShareAvailable(m, s, r) ==
  IF m \in HonestMembers THEN Share(m, s, r) \in hshares ELSE TRUE

TypeOK ==
  /\ journal \in [HonestMembers -> [Slots -> Roots \cup {NoRoot}]]
  /\ hshares \subseteq HonestMembers \X Slots \X Roots
  /\ certs \subseteq [slot : Slots, root : Roots,
                      shares : SUBSET (Ring \X Slots \X Roots)]
```

```
69
70  Init ==
71    /\ journal = [m \in HonestMembers |-> [s \in Slots |-> NoRoot]]
72    /\ hshares = {}
73    /\ certs = {}
74
75  HonestSign(m, s, r) ==
76    /\ m \in HonestMembers
77    /\ s \in Slots /\ r \in Roots
78    /\ journal[m][s] = NoRoot
79    /\ journal' = [journal EXCEPT ![m] = [journal[m] EXCEPT ![s] = r]]
80    /\ hshares' = hshares \cup {Share(m, s, r)}
81    /\ UNCHANGED certs
82
83  (* Assembly: anyone may assemble the bitmap-complete certificate once      *)
84  (* every member's share over the exact tuple is available (corrupt         *)
85  (* members' shares always are -- A1 excludes only forging honest ones).     *)
86  Assemble(s, r) ==
87    /\ s \in Slots /\ r \in Roots
88    /\ \A m \in Ring : ShareAvailable(m, s, r)
89    /\ certs' = certs \cup {Cert(s, r)}
90    /\ UNCHANGED <<journal, hshares>>
91
92  Next ==
93    \/ \E m \in Ring, s \in Slots, r \in Roots : HonestSign(m, s, r)
94    \/ \E s \in Slots, r \in Roots : Assemble(s, r)
95
96  Spec == Init /\ [][Next]_vars
97
98  (*************************************************************************)
99  (* THE EXTRACTION PROCEDURE (spec §12.5), specified over the wire records: *)
100 (* given two certificates for one slot with different roots, the           *)
101 (* extracted set is every member exhibiting a share in BOTH -- a pair of    *)
102 (* signatures by one key over conflicting tuples, each pair a              *)
103 (* self-contained conviction.                                              *)
104 (*************************************************************************)
105 ExtractedSet(c1, c2) ==
106   {m \in Ring : /\ Share(m, c1.slot, c1.root) \in c1.shares
107                 /\ Share(m, c2.slot, c2.root) \in c2.shares}
108
109 Conflicting(c1, c2) ==
110   /\ c1.slot = c2.slot
111   /\ c1.root # c2.root
112
113 (* T7 at the wire level: ANY two conflicting certificates decompose into   *)
114 (* conflicting share pairs for EVERY ring member -- the full necessary-     *)
115 (* participant set (n-of-n makes each member necessary; L9 makes all-n     *)
116 (* the maximum any protocol can attribute).                                *)
117 AttributionComplete ==
118   \A c1 \in certs, c2 \in certs :
119     Conflicting(c1, c2) => ExtractedSet(c1, c2) = Ring
120
121 (* P2.1's uniqueness, restated over the wire structure: under MHA (as a    *)
122 (* condition), conflicting certificates cannot both assemble.              *)
123 CertUniqueness ==
124   \A c1 \in certs, c2 \in certs :
125     (Conflicting(c1, c2)) => HonestMembers = {}
126
127 (* Reachability witness target for the all-Byzantine staged double-seal:   *)
128 (* TLC violating this invariant produces the trace in which both           *)
129 (* conflicting certificates assemble and the extraction yields all n.      *)
130 DoubleSealReached ==
```

```
  ~\E c1 \in certs, c2 \in certs : Conflicting(c1, c2)

Inv ==
  /\ TypeOK
  /\ \A m \in HonestMembers, s \in Slots, r \in Roots :
       (Share(m, s, r) \in hshares) => journal[m][s] = r
  /\ \A c \in certs :
       /\ c.shares = {Share(m, c.slot, c.root) : m \in Ring}
       /\ \A m \in HonestMembers : Share(m, c.slot, c.root) \in hshares

====
```

A.7.9 ForensicAccountabilityProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/ForensicAccountabilityProof.tla}

```
---- MODULE ForensicAccountabilityProof ----
(***************************************************************************)
(* AFT-CB P2.6 -- TLAPS discharge of T7 over the wire format: attribution   *)
(* completeness is a consequence of the bitmap-complete record structure,  *)
(* and certificate uniqueness under MHA restates P2.1's theorem over the   *)
(* wire.  The extraction procedure is ExtractedSet, and the theorem says   *)
(* it returns ALL of Ring on any conflicting pair.                         *)
(***************************************************************************)
EXTENDS ForensicAccountability, TLAPS

ASSUME NoRootOutsideRootsF == NoRoot \notin Roots

LEMMA FInvInit == Init => Inv
  BY ForensicConstants DEF Init, Inv, TypeOK

LEMMA FInvNext == Inv /\ [Next]_vars => Inv'
<1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
  OBVIOUS
<1>2. CASE UNCHANGED vars
  BY <1>1, <1>2 DEF Inv, TypeOK, vars
<1>3. ASSUME NEW m \in Ring, NEW s \in Slots, NEW r \in Roots,
             HonestSign(m, s, r)
      PROVE Inv'
  <2>1. m \in HonestMembers /\ journal[m][s] = NoRoot
    BY <1>3 DEF HonestSign
  <2>2. TypeOK'
    BY <1>1, <1>3, <2>1, ForensicConstants DEF Inv, TypeOK, HonestSign, Share
  <2>3. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
          (Share(mm, ss, rr) \in hshares') => journal'[mm][ss] = rr
    <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
                          NEW rr \in Roots, Share(mm, ss, rr) \in hshares'
          PROVE journal'[mm][ss] = rr
      OBVIOUS
    <3>2. hshares' = hshares \cup {Share(m, s, r)}
      BY <1>3 DEF HonestSign
    <3>3. CASE Share(mm, ss, rr) = Share(m, s, r)
      <4>1. mm = m /\ ss = s /\ rr = r
        BY <3>3 DEF Share
      <4>2. journal'[m][s] = r
        BY <1>1, <1>3, <2>1 DEF Inv, TypeOK, HonestSign
      <4> QED BY <4>1, <4>2
    <3>4. CASE Share(mm, ss, rr) \in hshares
```

```
      <4>1. journal[mm][ss] = rr
        BY <1>1, <3>4 DEF Inv
      <4>2. CASE mm = m /\ ss = s
        <5>1. journal[m][s] = rr
          BY <4>1, <4>2
        <5>2. rr # NoRoot
          BY <3>1, NoRootOutsideRootsF
        <5> QED BY <2>1, <5>1, <5>2
      <4>3. CASE ~(mm = m /\ ss = s)
        <5>1. journal'[mm][ss] = journal[mm][ss]
          BY <1>1, <1>3, <2>1, <4>3 DEF Inv, TypeOK, HonestSign
        <5> QED BY <4>1, <5>1
      <4> QED BY <4>2, <4>3
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2>4. \A c \in certs' :
          /\ c.shares = {Share(mm, c.slot, c.root) : mm \in Ring}
          /\ \A mm \in HonestMembers : Share(mm, c.slot, c.root) \in hshares'
    <3>1. certs' = certs
      BY <1>3 DEF HonestSign
    <3>2. hshares \subseteq hshares'
      BY <1>3 DEF HonestSign
    <3> QED BY <1>1, <3>1, <3>2 DEF Inv
  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
<1>4. ASSUME NEW s \in Slots, NEW r \in Roots, Assemble(s, r)
      PROVE Inv'
  <2>1. TypeOK'
    <3>1. Cert(s, r) \in [slot : Slots, root : Roots,
                          shares : SUBSET (Ring \X Slots \X Roots)]
      BY <1>4 DEF Cert, Share
    <3> QED BY <1>1, <1>4, <3>1 DEF Inv, TypeOK, Assemble
  <2>2. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
          (Share(mm, ss, rr) \in hshares') => journal'[mm][ss] = rr
    BY <1>1, <1>4 DEF Inv, Assemble
  <2>3. \A c \in certs' :
          /\ c.shares = {Share(mm, c.slot, c.root) : mm \in Ring}
          /\ \A mm \in HonestMembers : Share(mm, c.slot, c.root) \in hshares'
    <3>1. SUFFICES ASSUME NEW c \in certs'
          PROVE /\ c.shares = {Share(mm, c.slot, c.root) : mm \in Ring}
                /\ \A mm \in HonestMembers :
                     Share(mm, c.slot, c.root) \in hshares'
      OBVIOUS
    <3>2. certs' = certs \cup {Cert(s, r)} /\ hshares' = hshares
      BY <1>4 DEF Assemble
    <3>3. CASE c \in certs
      BY <1>1, <3>2, <3>3 DEF Inv
    <3>4. CASE c = Cert(s, r)
      <4>1. c.shares = {Share(mm, s, r) : mm \in Ring}
            /\ c.slot = s /\ c.root = r
        BY <3>4 DEF Cert
      <4>2. \A mm \in HonestMembers : Share(mm, s, r) \in hshares
        <5>1. \A mm \in Ring : ShareAvailable(mm, s, r)
          BY <1>4 DEF Assemble
        <5> QED BY <5>1, ForensicConstants DEF ShareAvailable
      <4> QED BY <3>2, <4>1, <4>2
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2> QED BY <2>1, <2>2, <2>3 DEF Inv
<1>5. QED
  BY <1>1, <1>2, <1>3, <1>4 DEF Next, vars

(***************************************************************************)
(* T7: any two conflicting wire certificates decompose into conflicting    *)
(* share pairs for EVERY ring member -- the extraction procedure returns    *)
```

```
(* the full necessary-participant set.  Purely structural: the record      *)
(* format guarantees it, which is exactly why the format is part of the    *)
(* theorem and an opaque aggregate destroys it (the mutation drill).       *)
(***************************************************************************)
LEMMA InvAttribution == Inv => AttributionComplete
<1>1. SUFFICES ASSUME Inv, NEW c1 \in certs, NEW c2 \in certs,
                      Conflicting(c1, c2)
      PROVE ExtractedSet(c1, c2) = Ring
  BY DEF AttributionComplete
<1>2. c1.shares = {Share(mm, c1.slot, c1.root) : mm \in Ring}
      /\ c2.shares = {Share(mm, c2.slot, c2.root) : mm \in Ring}
  BY <1>1 DEF Inv
<1>3. \A m \in Ring : /\ Share(m, c1.slot, c1.root) \in c1.shares
                      /\ Share(m, c2.slot, c2.root) \in c2.shares
  BY <1>2
<1>4. QED
  BY <1>3 DEF ExtractedSet

(***************************************************************************)
(* Certificate uniqueness under MHA, restated over the wire: a conflicting *)
(* pair requires an honest member's shares over two roots at one slot,     *)
(* which the journal forbids -- so conflict implies zero honest members.    *)
(***************************************************************************)
LEMMA InvCertUniqueness == Inv => CertUniqueness
<1>1. SUFFICES ASSUME Inv, NEW c1 \in certs, NEW c2 \in certs,
                      Conflicting(c1, c2)
      PROVE HonestMembers = {}
  BY DEF CertUniqueness
<1>2. SUFFICES ASSUME NEW h \in HonestMembers PROVE FALSE
  OBVIOUS
<1>3. Share(h, c1.slot, c1.root) \in hshares
      /\ Share(h, c2.slot, c2.root) \in hshares
  BY <1>1, <1>2 DEF Inv
<1>4. journal[h][c1.slot] = c1.root /\ journal[h][c2.slot] = c2.root
  <2>1. c1.slot \in Slots /\ c1.root \in Roots
        /\ c2.slot \in Slots /\ c2.root \in Roots
    BY <1>1 DEF Inv, TypeOK
  <2> QED BY <1>1, <1>2, <1>3, <2>1 DEF Inv
<1>5. c1.slot = c2.slot /\ c1.root # c2.root
  BY <1>1 DEF Conflicting
<1>6. QED
  BY <1>4, <1>5

THEOREM ForensicSafety ==
  Spec => [](AttributionComplete /\ CertUniqueness)
<1>1. Init => Inv
  BY FInvInit
<1>2. Inv /\ [Next]_vars => Inv'
  BY FInvNext
<1>3. Spec => []Inv
  BY <1>1, <1>2, PTL DEF Spec
<1>4. Inv => (AttributionComplete /\ CertUniqueness)
  BY InvAttribution, InvCertUniqueness
<1>5. QED
  BY <1>3, <1>4, PTL

====
```

A.7.10 SuccessionClock.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/SuccessionClock.tla}

```
---- MODULE SuccessionClock ----
(***************************************************************************)
(* AFT-CB P2.7 -- the §16 v3 succession kernel.  HONEST LABEL AFTER REVIEW *)
(* ROUND 5: this module discharges a STRICTLY NARROWED statement, not §16 *)
(* (R5-F3): the Byzantine standby member has no modeled behavior,         *)
(* obs_commit is computed as the honest union (never adversary-chosen),   *)
(* lastSealTick never advances, and the MHA-corner config cannot hold two *)
(* conflicting old seals.  The proof is internally valid and RETAINED as  *)
(* the record of what v3's refusal mechanism does prove; it is NOT a      *)
(* discharge of §16, whose v3 was refuted (see the spec banner and        *)
(* p2-7-respec-review.md).  A faithful re-mechanization is a v4           *)
(* precondition.                                                          *)
(*                                                                         *)
(* v3's design (spec §16, RESPECIFIED -- pending its own review): the      *)
(* standby S_v adjudicates under its OWN per-configuration MHA.  A rival  *)
(* old-ring seal BINDS iff it is in the fallback seal's UBC-committed     *)
(* observation set; an honest standby member REFUSES to sign a fallback   *)
(* seal that omits an observed object or covers a slot where it observed  *)
(* a rival seal at all -- preemption by refusal, before any certificate   *)
(* exists.  Unobserved rivals are VOID by the old ring's own              *)
(* formation-time signature.  No bulletin, no stamps, no delivery bound   *)
(* on any medium exists in this design.                                   *)
(*                                                                         *)
(* Time: ticks advance monotonically (A9 gives objectivity; the chain     *)
(* condition "≥ T_halt seal-free ticks" is read off the single-valued     *)
(* seal-seeded chain, modeled as lastSealTick).  The adversary schedules  *)
(* everything, including when submissions become observations (A10 -- an  *)
(* object submitted to the standby reaches ≥1 honest member within        *)
(* G_deliv -- is the FAIRNESS the old ring's protection rides; the safety *)
(* theorem below never consumes it, which is the point: safety holds      *)
(* even when A10 fails, at the old ring's liveness cost).                 *)
(*                                                                         *)
(* Single contested slot; roots are the contested content.                *)
(***************************************************************************)
EXTENDS Naturals, FiniteSets

CONSTANTS
  OldRing,
  HonestOld,      \* honest subset of the old ring
  Standby,
  HonestStandby,  \* honest subset of the standby (A2(S_v) enters theorems)
  Roots,
  NoRoot,
  T_halt,         \* activation threshold (ticks since last seal)
  G,              \* release wait after activation
  MaxTick

ASSUME SuccessionConstants ==
  /\ HonestOld \subseteq OldRing
  /\ HonestStandby \subseteq Standby
  /\ T_halt \in Nat \ {0}
  /\ G \in Nat
  /\ MaxTick \in Nat

VARIABLES
  tick,           \* the objective clock (monotone; A9)
```

```
  lastSealTick,   \* tick of the last old-ring seal folded into the chain
  policySigned,   \* BOOLEAN: formation-time unanimous succession policy
  oldJournal,     \* [HonestOld -> Roots \cup {NoRoot}]: single-final-ack (A3)
  oldSeals,       \* assembled old-ring seals for the contested slot: roots
  submitted,      \* old seals submitted to the standby set: roots
  observedBy,     \* [HonestStandby -> SUBSET Roots]: rival seals observed
  activation,     \* 0 = none, else the activation tick T_a
  fallbackSeals,  \* [root : Roots, obs : SUBSET Roots]: obs = obs_commit
  released        \* roots whose fallback irreversible effect released

vars == <<tick, lastSealTick, policySigned, oldJournal, oldSeals, submitted,
          observedBy, activation, fallbackSeals, released>>

OldAckedBy(m, r) ==
  IF m \in HonestOld THEN oldJournal[m] = r ELSE TRUE

TypeOK ==
  /\ tick \in 0..MaxTick
  /\ lastSealTick \in 0..MaxTick
  /\ policySigned \in BOOLEAN
  /\ oldJournal \in [HonestOld -> Roots \cup {NoRoot}]
  /\ oldSeals \subseteq Roots
  /\ submitted \subseteq Roots
  /\ observedBy \in [HonestStandby -> SUBSET Roots]
  /\ activation \in 0..MaxTick
  /\ fallbackSeals \subseteq [root : Roots, obs : SUBSET Roots]
  /\ released \subseteq Roots

Init ==
  /\ tick = 0
  /\ lastSealTick = 0
  /\ policySigned = TRUE   \* formation-time unanimity (the drill flips it)
  /\ oldJournal = [m \in HonestOld |-> NoRoot]
  /\ oldSeals = {}
  /\ submitted = {}
  /\ observedBy = [h \in HonestStandby |-> {}]
  /\ activation = 0
  /\ fallbackSeals = {}
  /\ released = {}

Tick ==
  /\ tick < MaxTick
  /\ tick' = tick + 1
  /\ UNCHANGED <<lastSealTick, policySigned, oldJournal, oldSeals, submitted,
                 observedBy, activation, fallbackSeals, released>>

(* Old-ring final-ack (honest, journal-guarded) and structural Byzantine   *)
(* availability let old seals assemble; the standby race is what this      *)
(* module studies, so seal assembly is one action.                         *)
OldAck(m, r) ==
  /\ m \in HonestOld
  /\ r \in Roots
  /\ oldJournal[m] = NoRoot
  /\ oldJournal' = [oldJournal EXCEPT ![m] = r]
  /\ UNCHANGED <<tick, lastSealTick, policySigned, oldSeals, submitted,
                 observedBy, activation, fallbackSeals, released>>

AssembleOldSeal(r) ==
  /\ r \in Roots
  /\ \A m \in OldRing : OldAckedBy(m, r)
  /\ oldSeals' = oldSeals \cup {r}
  /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, submitted,
```

```
                observedBy, activation, fallbackSeals, released>>

(* Submission of a rival old seal toward the standby set, and its          *)
(* adversary-scheduled arrival at an honest standby member's observation.  *)
(* A10 (reach within G_deliv) is deliberately NOT enforced here: safety    *)
(* must hold on every schedule, including ones where A10 fails.            *)
Submit(r) ==
  /\ r \in oldSeals
  /\ submitted' = submitted \cup {r}
  /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
                 observedBy, activation, fallbackSeals, released>>

Observe(h, r) ==
  /\ h \in HonestStandby
  /\ r \in submitted
  /\ observedBy' = [observedBy EXCEPT ![h] = observedBy[h] \cup {r}]
  /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
                 submitted, activation, fallbackSeals, released>>

(* Activation: valid only through the formation-time policy AND the        *)
(* objective chain condition -- ≥ T_halt seal-free ticks (A9; the chain is  *)
(* single-valued because only seals fold into it).  No silence-derived     *)
(* input exists anywhere in this action.                                   *)
Activate ==
  /\ policySigned
  /\ activation = 0
  /\ tick - lastSealTick >= T_halt
  /\ activation' = tick
  /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
                 submitted, observedBy, fallbackSeals, released>>

(* The observation-committed fallback seal.  The honest-refusal guard is   *)
(* the v3 mechanism: every honest standby member's observations must be     *)
(* inside obs_commit, and NO honest member may have observed ANY rival --   *)
(* otherwise no certificate can exist (A2(S_v) makes the honest share      *)
(* necessary).  obs_commit is exactly the union of honest observations at  *)
(* signing time.                                                           *)
AssembleFallback(r) ==
  /\ activation > 0
  /\ r \in Roots
  /\ \A h \in HonestStandby : \A rv \in observedBy[h] : rv = r
  /\ fallbackSeals' = fallbackSeals \cup
       {[root |-> r, obs |-> UNION {observedBy[h] : h \in HonestStandby}]}
  /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
                 submitted, observedBy, activation, released>>

(* Release: the fallback irreversible effect for r, after the wait.        *)
Release(r) ==
  /\ activation > 0
  /\ tick >= activation + G
  /\ \E fs \in fallbackSeals : fs.root = r
  /\ released' = released \cup {r}
  /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
                 submitted, observedBy, activation, fallbackSeals>>

Next ==
  \/ Tick
  \/ \E m \in OldRing, r \in Roots : OldAck(m, r)
  \/ \E r \in Roots : AssembleOldSeal(r) \/ Submit(r) \/ AssembleFallback(r)
                      \/ Release(r)
  \/ \E h \in Standby, r \in Roots : Observe(h, r)
  \/ Activate
```

```
181
182 Spec == Init /\ [][Next]_vars
183
184 (***************************************************************************)
185 (* Bindingness (§16.5): a rival old seal binds against fallback root r iff *)
186 (* it is in some r-fallback-seal's committed observation set.  VoidByPolicy *)
187 (* is its complement -- definitional, by the old ring's own formation       *)
188 (* signature.                                                              *)
189 (***************************************************************************)
190 Binding(rv, r) ==
191   \E fs \in fallbackSeals : fs.root = r /\ rv \in fs.obs /\ rv # r
192
193 (* T5d-v3: no released fallback effect coexists with a BINDING rival.      *)
194 NoBindingRivalReleased ==
195   \A r \in released, rv \in Roots : ~Binding(rv, r)
196
197 (* Activation is never silence-justified: it exists only with the          *)
198 (* formation policy and the chain condition met at its tick.               *)
199 ActivationJustified ==
200   (activation > 0) => (policySigned /\ activation - lastSealTick >= T_halt)
201
202 Inv ==
203   /\ TypeOK
204   /\ \A fs \in fallbackSeals :
205        \A rv \in fs.obs : rv = fs.root
206   /\ ActivationJustified
207
208 ====
```

A.7.11 SuccessionClockProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/SuccessionClockProof.tla}

```
1  ---- MODULE SuccessionClockProof ----
2  (***************************************************************************)
3  (* AFT-CB P2.7 -- TLAPS discharge of the §16 v3 kernel.                    *)
4  (*                                                                         *)
5  (* T5d-v3's safety is BY REFUSAL: the honest-standby guard means no        *)
6  (* fallback certificate ever commits a rival, so no released effect can    *)
7  (* face a BINDING rival -- the theorem is the guard's inductive            *)
8  (* preservation, under A2(S_v) modeled structurally (the honest share is   *)
9  (* necessary, so the guard binds every assembled certificate).  A          *)
10 (* Byzantine signer padding obs_commit could only ADD rivals -- which      *)
11 (* blocks its own fallback (Binding fires) -- the safe direction; the      *)
12 (* kernel therefore commits the honest union exactly.                      *)
13 (*                                                                         *)
14 (* ActivationJustified: activation exists only with the formation policy   *)
15 (* and the objective chain condition at its own tick -- no silence input.  *)
16 (***************************************************************************)
17 EXTENDS SuccessionClock, TLAPS
18
19 ASSUME NoRootOutsideRootsS == NoRoot \notin Roots
20
21 LEMMA SInvInit == Init => Inv
22   BY SuccessionConstants DEF Init, Inv, TypeOK, ActivationJustified
23
24 LEMMA SInvNext == Inv /\ [Next]_vars => Inv'
25 <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
```

```
  OBVIOUS
<1>2. CASE UNCHANGED vars
  BY <1>1, <1>2 DEF Inv, TypeOK, ActivationJustified, vars
<1>3. CASE Tick
  <2>1. tick < MaxTick /\ tick' = tick + 1
    BY <1>3 DEF Tick
  <2>2. TypeOK'
    BY <1>1, <2>1, <1>3, SuccessionConstants DEF Inv, TypeOK, Tick
  <2>3. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
    BY <1>1, <1>3 DEF Inv, Tick
  <2>4. ActivationJustified'
    BY <1>1, <1>3 DEF Inv, ActivationJustified, Tick
  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
<1>4. ASSUME NEW m \in OldRing, NEW r \in Roots, OldAck(m, r)
      PROVE Inv'
  BY <1>1, <1>4 DEF Inv, TypeOK, ActivationJustified, OldAck
<1>5. ASSUME NEW r \in Roots, AssembleOldSeal(r)
      PROVE Inv'
  BY <1>1, <1>5 DEF Inv, TypeOK, ActivationJustified, AssembleOldSeal
<1>6. ASSUME NEW r \in Roots, Submit(r)
      PROVE Inv'
  BY <1>1, <1>6 DEF Inv, TypeOK, ActivationJustified, Submit
<1>7. ASSUME NEW h \in Standby, NEW r \in Roots, Observe(h, r)
      PROVE Inv'
  <2>1. h \in HonestStandby /\ r \in submitted
    BY <1>7 DEF Observe
  <2>2. TypeOK'
    BY <1>1, <1>7, <2>1, SuccessionConstants DEF Inv, TypeOK, Observe
  <2>3. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
    BY <1>1, <1>7 DEF Inv, Observe
  <2>4. ActivationJustified'
    BY <1>1, <1>7 DEF Inv, ActivationJustified, Observe
  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
<1>8. CASE Activate
  <2>1. TypeOK'
    BY <1>1, <1>8 DEF Inv, TypeOK, Activate
  <2>2. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
    BY <1>1, <1>8 DEF Inv, Activate
  <2>3. ActivationJustified'
    BY <1>1, <1>8 DEF Inv, TypeOK, ActivationJustified, Activate
  <2> QED BY <2>1, <2>2, <2>3 DEF Inv
<1>9. ASSUME NEW r \in Roots, AssembleFallback(r)
      PROVE Inv'
  <2>1. TypeOK'
    <3>1. UNION {observedBy[h] : h \in HonestStandby} \subseteq Roots
      BY <1>1, SuccessionConstants DEF Inv, TypeOK
    <3> QED BY <1>1, <1>9, <3>1 DEF Inv, TypeOK, AssembleFallback
  <2>2. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
    <3>1. SUFFICES ASSUME NEW fs \in fallbackSeals', NEW rv \in fs.obs
          PROVE rv = fs.root
      OBVIOUS
    <3>2. fallbackSeals' = fallbackSeals \cup
            {[root |-> r, obs |-> UNION {observedBy[h] : h \in HonestStandby}]}
      BY <1>9 DEF AssembleFallback
    <3>3. CASE fs \in fallbackSeals
      BY <1>1, <3>3 DEF Inv
    <3>4. CASE fs = [root |-> r, obs |-> UNION {observedBy[h] : h \in HonestStandby}]
      <4>1. \A h \in HonestStandby : \A rv2 \in observedBy[h] : rv2 = r
        BY <1>9 DEF AssembleFallback
      <4>2. rv \in UNION {observedBy[h] : h \in HonestStandby}
        BY <3>1, <3>4
      <4>3. rv = r
```

```
        BY <4>1, <4>2
      <4> QED BY <3>4, <4>3
    <3> QED BY <3>1, <3>2, <3>3, <3>4
  <2>3. ActivationJustified'
    BY <1>1, <1>9 DEF Inv, ActivationJustified, AssembleFallback
  <2> QED BY <2>1, <2>2, <2>3 DEF Inv
<1>10. ASSUME NEW r \in Roots, Release(r)
      PROVE Inv'
  BY <1>1, <1>10 DEF Inv, TypeOK, ActivationJustified, Release
<1>11. QED
  BY <1>1, <1>2, <1>3, <1>4, <1>5, <1>6, <1>7, <1>8, <1>9, <1>10
  DEF Next, vars

LEMMA InvNoBindingRival == Inv => NoBindingRivalReleased
<1>1. SUFFICES ASSUME Inv, NEW r \in released, NEW rv \in Roots,
                      Binding(rv, r)
      PROVE FALSE
  BY DEF NoBindingRivalReleased
<1>2. PICK fs \in fallbackSeals : fs.root = r /\ rv \in fs.obs /\ rv # r
  BY <1>1 DEF Binding
<1>3. rv = fs.root
  BY <1>1, <1>2 DEF Inv
<1>4. QED
  BY <1>2, <1>3

THEOREM SuccessionSafety ==
  Spec => []NoBindingRivalReleased /\ ActivationJustified)
<1>1. Init => Inv
  BY SInvInit
<1>2. Inv /\ [Next]_vars => Inv'
  BY SInvNext
<1>3. Spec => []Inv
  BY <1>1, <1>2, PTL DEF Spec
<1>4. Inv => (NoBindingRivalReleased /\ ActivationJustified)
  BY InvNoBindingRival DEF Inv
<1>5. QED
  BY <1>3, <1>4, PTL

====
```

A.8 Interactive Visibility (QUV)

The mechanized kernels of Section 12: the portable visibility dilemma (Theorem 6), the arbitrary-set query-unanimity proof (Q-T1–Q-T4), the composition proof (Q-E1, Q-E2), and the composed end-to-end transition model checked with its countermodels in the mandatory formal corpus.

A.8.1 `MaximalVisibilityDilemma.tla`

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/maximal_visibility/MaximalVisibilityDilemma.tla}

```
------------------------ MODULE MaximalVisibilityDilemma ------------------------
EXTENDS FiniteSets, Naturals

CONSTANTS Members, Values

ASSUME /\ IsFiniteSet(Members)
       /\ Cardinality(Members) >= 2
       /\ IsFiniteSet(Values)
       /\ Cardinality(Values) >= 2

(*
An accepted proof is abstracted by the configured identities whose
unforgeable acts it needs.  Each value can have an arbitrary acceptance
family, so this is strictly more general than one q-of-n threshold.
*)
AcceptedFamilies == [Values -> SUBSET (SUBSET Members)]

VARIABLE accepted

vars == <<accepted>>

Init == accepted \in AcceptedFamilies

Next == UNCHANGED vars

(* Every possible sole-honest member can produce an accepted proof for every
   valid value while all other members remain silent. *)
SilentNm1EffectLiveness ==
    \A value \in Values :
      \A honest \in Members :
        \E support \in accepted[value] : support \subseteq {honest}

(* Against an unknown n-1 Byzantine set, conflicting accepted proofs must
   share an unforgeable dependency on whichever member is actually honest.
   Since every singleton honest set is admissible, their intersection must be
   the entire configured membership. *)
TransferableNonConflict ==
    \A left_value, right_value \in Values :
      left_value # right_value =>
        \A left_support \in accepted[left_value] :
          \A right_support \in accepted[right_value] :
            left_support \cap right_support = Members

Dilemma == ~(SilentNm1EffectLiveness /\ TransferableNonConflict)

Spec == Init /\ [][Next]_vars
```

```
47
48 ============================================================================
```

A.8.2 QueryUnanimityProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/maximal_visibility/QueryUnanimityProof.tla}

```
1  ---- MODULE QueryUnanimityProof ----
2  (***************************************************************************)
3  (* AFT QUV M13Q -- arbitrary-set proof kernel for online authorization.     *)
4  (*                                                                         *)
5  (* Snapshots abstracts the complete, nonce/context-bound responses that    *)
6  (* Q-A2/Q-A3 require an honest operation to receive from EVERY correct     *)
7  (* member. SerializationDisclosure is the relational consequence of each  *)
8  (* correct member's atomic, grow-only write-before-reply state machine: for *)
9  (* every pair of operations, the later snapshot at a fixed correct member  *)
10 (* contains the earlier candidate. Byzantine replies are omitted here      *)
11 (* because they can only add valid candidates and therefore turn accept    *)
12 (* into reject; they cannot enable either acceptance predicate.            *)
13 (***************************************************************************)
14 EXTENDS FiniteSets, TLAPS
15
16 CONSTANTS
17   Members,
18   Correct,
19   Candidates,
20   ValidCandidates,
21   Ops,
22   CandidateOf,
23   Snapshots,
24   FirstWinner,
25   Abort
26
27 ASSUME QAssumptions ==
28   /\ Correct \subseteq Members
29   /\ Correct # {}
30   /\ ValidCandidates \subseteq Candidates
31   /\ Abort \notin Candidates
32   /\ CandidateOf \in [Ops -> ValidCandidates]
33   /\ Snapshots \in [Ops -> [Correct -> SUBSET ValidCandidates]]
34   /\ FirstWinner \in [Correct -> ValidCandidates]
35   /\ \A o \in Ops, c \in Correct : CandidateOf[o] \in Snapshots[o][c]
36   /\ \A o1 \in Ops, o2 \in Ops, c \in Correct :
37        \/ CandidateOf[o1] \in Snapshots[o2][c]
38        \/ CandidateOf[o2] \in Snapshots[o1][c]
39
40 CorrectSeen(o) == UNION {Snapshots[o][c] : c \in Correct}
41
42 OwnedAccept(o) == CorrectSeen(o) = {CandidateOf[o]}
43
44 UnownedAccept(o) ==
45   \A c \in Correct : FirstWinner[c] = CandidateOf[o]
46
47 OwnedOutcome(o) == IF OwnedAccept(o) THEN CandidateOf[o] ELSE Abort
48
49 UnownedOutcome(o) == IF UnownedAccept(o) THEN CandidateOf[o] ELSE Abort
50
51 THEOREM QOwnedNonConflict ==
```

```
  \A o1 \in Ops, o2 \in Ops :
    (OwnedAccept(o1) /\ OwnedAccept(o2)) =>
      CandidateOf[o1] = CandidateOf[o2]
<1>1. SUFFICES ASSUME NEW o1 \in Ops, NEW o2 \in Ops,
                      OwnedAccept(o1), OwnedAccept(o2)
      PROVE CandidateOf[o1] = CandidateOf[o2]
  OBVIOUS
<1>2. PICK c \in Correct : TRUE
  BY QAssumptions
<1>3. \/ CandidateOf[o1] \in Snapshots[o2][c]
       \/ CandidateOf[o2] \in Snapshots[o1][c]
  BY <1>1, <1>2, QAssumptions
<1>4. CASE CandidateOf[o1] \in Snapshots[o2][c]
  <2>1. CandidateOf[o1] \in CorrectSeen(o2)
    BY <1>1, <1>2, <1>4 DEF CorrectSeen
  <2> QED
    BY <1>1, <2>1 DEF OwnedAccept
<1>5. CASE CandidateOf[o2] \in Snapshots[o1][c]
  <2>1. CandidateOf[o2] \in CorrectSeen(o1)
    BY <1>1, <1>2, <1>5 DEF CorrectSeen
  <2> QED
    BY <1>1, <2>1 DEF OwnedAccept
<1> QED BY <1>3, <1>4, <1>5

THEOREM QUnownedNonConflict ==
  \A o1 \in Ops, o2 \in Ops :
    (UnownedAccept(o1) /\ UnownedAccept(o2)) =>
      CandidateOf[o1] = CandidateOf[o2]
<1>1. SUFFICES ASSUME NEW o1 \in Ops, NEW o2 \in Ops,
                      UnownedAccept(o1), UnownedAccept(o2)
      PROVE CandidateOf[o1] = CandidateOf[o2]
  OBVIOUS
<1>2. PICK c \in Correct : TRUE
  BY QAssumptions
<1>3. FirstWinner[c] = CandidateOf[o1]
  BY <1>1, <1>2 DEF UnownedAccept
<1>4. FirstWinner[c] = CandidateOf[o2]
  BY <1>1, <1>2 DEF UnownedAccept
<1> QED BY <1>3, <1>4

THEOREM QOwnedExternalValidity ==
  \A o \in Ops : OwnedAccept(o) => CandidateOf[o] \in ValidCandidates
  BY QAssumptions

THEOREM QUnownedExternalValidity ==
  \A o \in Ops : UnownedAccept(o) => CandidateOf[o] \in ValidCandidates
  BY QAssumptions

THEOREM QOwnedOutcomeTyped ==
  \A o \in Ops : OwnedOutcome(o) \in ValidCandidates \cup {Abort}
  BY QAssumptions DEF OwnedOutcome

THEOREM QUnownedOutcomeTyped ==
  \A o \in Ops : UnownedOutcome(o) \in ValidCandidates \cup {Abort}
  BY QAssumptions DEF UnownedOutcome

THEOREM QOwnedSingleCandidateProgress ==
  \A s \in ValidCandidates :
    ValidCandidates = {s} => \A o \in Ops : OwnedAccept(o)
<1>1. SUFFICES ASSUME NEW s \in ValidCandidates, ValidCandidates = {s},
                      NEW o \in Ops
      PROVE OwnedAccept(o)
```

```
  OBVIOUS
<1>2. CandidateOf[o] = s
  BY <1>1, QAssumptions
<1>3. \A c \in Correct : Snapshots[o][c] = {s}
  <2>1. SUFFICES ASSUME NEW c \in Correct PROVE Snapshots[o][c] = {s}
    OBVIOUS
  <2>2. Snapshots[o][c] \subseteq {s}
    BY <1>1, <2>1, QAssumptions
  <2>3. s \in Snapshots[o][c]
    BY <1>1, <1>2, <2>1, QAssumptions
  <2> QED BY <2>2, <2>3
<1>4. CorrectSeen(o) = {s}
  BY <1>1, <1>3, QAssumptions DEF CorrectSeen
<1> QED BY <1>2, <1>4 DEF OwnedAccept

THEOREM QUnownedSingleCandidateProgress ==
  \A s \in ValidCandidates :
    ValidCandidates = {s} => \A o \in Ops : UnownedAccept(o)
  BY QAssumptions DEF UnownedAccept

THEOREM QOwnedOutcomeNonConflict ==
  \A o1 \in Ops, o2 \in Ops :
    (OwnedOutcome(o1) # Abort /\ OwnedOutcome(o2) # Abort) =>
      OwnedOutcome(o1) = OwnedOutcome(o2)
  BY QOwnedNonConflict DEF OwnedOutcome

THEOREM QUnownedOutcomeNonConflict ==
  \A o1 \in Ops, o2 \in Ops :
    (UnownedOutcome(o1) # Abort /\ UnownedOutcome(o2) # Abort) =>
      UnownedOutcome(o1) = UnownedOutcome(o2)
  BY QUnownedNonConflict DEF UnownedOutcome

====
```

A.8.3 QueryUnanimityCompositionProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/maximal_visibility/QueryUnanimityCompositionProof.tla}

```
---- MODULE QueryUnanimityCompositionProof ----
(***************************************************************************)
(* AFT QUV M14Q -- lift accepted-value uniqueness into prefix-compatible    *)
(* histories and non-conflicting consequence candidates.                    *)
(*                                                                          *)
(* AcceptedUnique is supplied by the M13Q QueryUnanimityProof theorem.      *)
(* HistoryAdmission is the predecessor/next-slot runtime rule.              *)
(* MutationAuthorization is the executor-side QUV-before-effect rule.       *)
(* Physical duplicate suppression for the one stable slot/effect key is     *)
(* separately proved by T10/AtMostOnceExternalization.                      *)
(***************************************************************************)
EXTENDS Naturals, Sequences, FiniteSets, TLAPS

CONSTANTS Candidates, Accepted, Histories, Mutations

ASSUME QLiftAssumptions ==
  /\ Accepted \subseteq Nat \X Candidates
  /\ Histories \subseteq Seq(Candidates)
  /\ Mutations \subseteq Nat \X Candidates
  /\ \A slot \in Nat, x \in Candidates, y \in Candidates :
```

```
       (<<slot, x>> \in Accepted /\ <<slot, y>> \in Accepted) => x = y
  /\ \A h \in Histories :
       \A i \in 1..Len(h) : <<i, h[i]>> \in Accepted
  /\ \A slot \in Nat, c \in Candidates :
       <<slot, c>> \in Mutations => <<slot, c>> \in Accepted

PrefixCompatible ==
  \A h1 \in Histories, h2 \in Histories :
    \A i \in Nat :
      (i \in 1..Len(h1) /\ i \in 1..Len(h2)) => h1[i] = h2[i]

NoConflictingMutationCandidates ==
  \A slot \in Nat, x \in Candidates, y \in Candidates :
    (<<slot, x>> \in Mutations /\ <<slot, y>> \in Mutations) => x = y

THEOREM QHistoryPrefixCompatibility == PrefixCompatible
<1>1. SUFFICES ASSUME NEW h1 \in Histories, NEW h2 \in Histories,
                      NEW i \in Nat,
                      i \in 1..Len(h1), i \in 1..Len(h2)
      PROVE h1[i] = h2[i]
  BY DEF PrefixCompatible
<1>2. <<i, h1[i]>> \in Accepted /\ <<i, h2[i]>> \in Accepted
  BY <1>1, QLiftAssumptions
<1> QED BY <1>1, <1>2, QLiftAssumptions

THEOREM QConsequenceCandidateNonConflict == NoConflictingMutationCandidates
<1>1. SUFFICES ASSUME NEW slot \in Nat,
                      NEW x \in Candidates,
                      NEW y \in Candidates,
                      <<slot, x>> \in Mutations,
                      <<slot, y>> \in Mutations
      PROVE x = y
  BY DEF NoConflictingMutationCandidates
<1>2. <<slot, x>> \in Accepted /\ <<slot, y>> \in Accepted
  BY <1>1, QLiftAssumptions
<1> QED BY <1>1, <1>2, QLiftAssumptions

====
```

A.8.4 QuvEndToEndRefinement.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/maximal_visibility/QuvEndToEndRefinement.tla}

```
---------------------- MODULE QuvEndToEndRefinement ----------------------
EXTENDS Naturals, Sequences, FiniteSets, TLC

(***************************************************************************)
(* QUV-M17Q-005 transition-level composition witness.                      *)
(*                                                                         *)
(* Finite explicit-state model of the QUV end-to-end path over an          *)
(* abstraction of the production state machine: per-member durable         *)
(* conflict stores keyed WITHOUT the predecessor, two-phase authenticated   *)
(* record/anchor durability with crash recovery, expected-predecessor       *)
(* admission with typed refusal, timed executor operations with a rooted    *)
(* reply cutoff, own-head advancement, process-local continuations with     *)
(* absolute expiry and a height fence, T10 claim-before-call stable-key    *)
(* externalization, and successor-root handoff through the successor's own *)
(* live operation.  Executors are the correct relying members themselves:  *)
```

```
(* the head-state design advances a frontier only through that member's    *)
(* own live operation.  Byzantine members are silent.  Nonce freshness is  *)
(* abstracted as exact request-context plus start-time binding.  This is   *)
(* NOT a mechanized refinement of the Rust implementation; see README.      *)
(***************************************************************************)

CONSTANTS Correct, Domains, MaxSlot, Delta, Lifetime, MaxTime, MaxHeight,
          MaxGen, AuthorityMode, HandoffEnabled,
          PredecessorInKey, LateReplyAdmitted, SkipExecutorQuv,
          ClaimAfterExpiry, CallBeforeClaim, UnauthenticatedRecovery,
          ResourceIdSplitsKey, ActivateFromBytes

ASSUME Correct # {} /\ Domains # {} /\ MaxSlot >= 1 /\ Delta >= 1 /\ Lifetime >= 0
ASSUME AuthorityMode \in {"owned", "unowned"}

Candidates == {"X", "Y"}
OldRoot == "r0"
Successor == "r1"
Handoff == "handoff"
Init0 == "init"
Stable == "*"
None == "none"

Roots == IF HandoffEnabled THEN {OldRoot, Successor} ELSE {OldRoot}
AllDoms == IF HandoffEnabled THEN Domains \cup {Handoff} ELSE Domains
Values == IF HandoffEnabled THEN Candidates \cup {Successor} ELSE Candidates
Preds == Candidates \cup {Init0}
Rids == {"ra", "rb"}
Slots == 1..MaxSlot
KeyPreds == IF PredecessorInKey THEN Preds ELSE {"any"}
Keys == [root : Roots, dom : AllDoms, slot : Slots, pred : KeyPreds]
KeyOf(root, dom, slot, pred) ==
    [root |-> root, dom |-> dom, slot |-> slot,
     pred |-> IF PredecessorInKey THEN pred ELSE "any"]
EmptyEntry == [cands |-> {}, first |-> None]
EmptyStore == [k \in Keys |-> EmptyEntry]
Entries == [cands : SUBSET Values, first : Values \cup {None}]
SomeDom == CHOOSE d \in Domains : TRUE
SomeMember == CHOOSE c \in Correct : TRUE
SomeKey == CHOOSE k \in Keys : TRUE

\* Replies are uniform records: kind \in {"none", "refused", "snapshot"}.
NoReply == [kind |-> "none", cands |-> {}, first |-> None]
Refused == [kind |-> "refused", cands |-> {}, first |-> None]
Snapshot(entry) == [kind |-> "snapshot", cands |-> entry.cands, first |-> entry.first]
Replies == [kind : {"none", "refused", "snapshot"}, cands : SUBSET Values,
            first : Values \cup {None}]

Requests == [root : Roots, dom : AllDoms, slot : Slots, value : Values,
             pred : Preds, start : 0..MaxTime]
NoRequest == [root |-> OldRoot, dom |-> SomeDom, slot |-> 0, value |-> "X",
              pred |-> Init0, start |-> 0]
\* Sentinels are same-shaped records; slot = 0 / gen = 0 means absent.
Idle == [root |-> OldRoot, dom |-> SomeDom, slot |-> 0, value |-> "X",
         pred |-> Init0, start |-> 0, rid |-> "ra",
         replies |-> [c \in Correct |-> NoReply]]
NoGrant == [root |-> OldRoot, dom |-> SomeDom, slot |-> 0, value |-> "X",
            pred |-> Init0, rid |-> "ra", expires |-> 0, height |-> 0]
NoStaged == [gen |-> 0, auth |-> FALSE, content |-> EmptyStore,
             exec |-> SomeMember, key |-> SomeKey, req |-> NoRequest]
NoInflight == [rid |-> "ra", dom |-> SomeDom, slot |-> 0, claimed |-> FALSE]
StableKeys == [dom : Domains, slot : Slots]
```

```
ClaimKeys == [rid : Rids \cup {Stable}, dom : Domains, slot : Slots]

VARIABLES
  now, height,
  \* member durable + volatile state
  anchor, anchorGen, staged, authGen, head, activeRoot, gate, gateLive,
  boundary, bytes, crashed,
  \* executor state
  op, grant, accepted, claims, inflight, calls, register, lateClaim

vars == <<now, height, anchor, anchorGen, staged, authGen, head,
          activeRoot, gate, gateLive, boundary, bytes, crashed,
          op, grant, accepted, claims, inflight, calls, register, lateClaim>>
memberVars == <<anchor, anchorGen, staged, authGen, head, activeRoot, gate,
                gateLive, boundary, bytes, crashed>>
storeVars == <<anchor, anchorGen, staged, authGen>>
execVars == <<op, grant, accepted, claims, inflight, calls, register, lateClaim>>
clockVars == <<now, height>>

Active(e) == op[e].slot # 0
HasGrant(e) == grant[e].slot # 0
HasInflight(e) == inflight[e].slot # 0
IsStaged(c) == staged[c].gen # 0

\* Admitted manifest chain (fixed, documented reduction): every admitted slot
\* binds candidate X; slot 1 binds the initial coordinate and slot s binds the
\* admitted slot s-1 candidate as its online-authorization predecessor.  Y is
\* the competing, never-admitted candidate.
Admitted(dom, s) == "X"
AdmittedPred(dom, s) == IF s = 1 THEN Init0 ELSE Admitted(dom, s - 1)

Init ==
    /\ now = 0 /\ height = 0
    /\ anchor = [c \in Correct |-> EmptyStore]
    /\ anchorGen = [c \in Correct |-> 0]
    /\ staged = [c \in Correct |-> NoStaged]
    /\ authGen = [c \in Correct |-> 0]
    /\ head = [c \in Correct |-> [d \in Domains |-> <<>>]]
    /\ activeRoot = [c \in Correct |-> OldRoot]
    /\ gate = [c \in Correct |-> FALSE]
    /\ gateLive = [c \in Correct |-> FALSE]
    /\ boundary = [c \in Correct |-> FALSE]
    /\ bytes = [c \in Correct |-> FALSE]
    /\ crashed = [c \in Correct |-> FALSE]
    /\ op = [c \in Correct |-> Idle]
    /\ grant = [c \in Correct |-> NoGrant]
    /\ accepted = [c \in Correct |-> {}]
    /\ claims = [c \in Correct |-> {}]
    /\ inflight = [c \in Correct |-> NoInflight]
    /\ calls = [c \in Correct |-> [k \in StableKeys |-> 0]]
    /\ register = {}
    /\ lateClaim = [c \in Correct |-> FALSE]

---------------------------------------------------------------------------
\* Model time and execution height.  Tick encodes Q-A3 as an assumption:
\* time cannot leave an operation's inclusive rooted cutoff instant while a
\* correct member has not yet replied to that operation.  The executor never
\* consults Correct: it waits strictly through the cutoff (Decide below).
Deadline(o) == o.start + Delta
AllCorrectReplied(o) == \A c \in Correct : o.replies[c].kind # "none"
Tick ==
    /\ now < MaxTime
```

```
    /\ \A e \in Correct : Active(e) =>
         AllCorrectReplied(op[e]) \/ now < Deadline(op[e])
    /\ now' = now + 1
    /\ UNCHANGED <<height, memberVars, execVars>>
AdvanceHeight ==
    /\ height < MaxHeight /\ height' = height + 1
    /\ UNCHANGED <<now, memberVars, execVars>>

-----------------------------------------------------------------------------
\* Member admission: exact root, next slot or retained old slot, and the
\* head-derived expected predecessor.  PredecessorInKey models the pre-schema
\* store in which the request-supplied predecessor is a key component
\* instead of an admission check.
Expected(c, dom, s) == IF s = 1 THEN Init0 ELSE head[c][dom][s - 1]
Admissible(c, o) ==
    /\ o.root = activeRoot[c]
    /\ IF o.dom = Handoff
       THEN boundary[c] /\ o.slot = 1 /\ o.pred = Init0
       ELSE /\ o.slot <= Len(head[c][o.dom]) + 1
            /\ (PredecessorInKey \/ o.pred = Expected(c, o.dom, o.slot))

\* Documented reductions: an executor tries the expected predecessor or one
\* wrong one; delivery ids vary only where the mutation under test reads them.
WrongPred(p) == IF p = Init0 THEN "X" ELSE Init0
BeginPreds(e, dom, slot) ==
    IF dom = Handoff \/ slot > Len(head[e][dom]) + 1 THEN {Init0}
    ELSE {Expected(e, dom, slot), WrongPred(Expected(e, dom, slot))}
BeginRids == IF ResourceIdSplitsKey \/ CallBeforeClaim THEN Rids ELSE {"ra"}

Request(o) == [root |-> o.root, dom |-> o.dom, slot |-> o.slot,
               value |-> o.value, pred |-> o.pred, start |-> o.start]

\* Executor-side crash losses: process-local continuation, live operation and
\* the in-flight call handle are volatile; claims, calls and register are not.
CrashLosses(c) ==
    /\ op' = [op EXCEPT ![c] = Idle]
    /\ grant' = [grant EXCEPT ![c] = NoGrant]
    /\ inflight' = [inflight EXCEPT ![c] = NoInflight]

\* Record: atomic durable processing of one push at member c.  A refusal
\* writes nothing and replies with a typed refusal.  An admitted push stages
\* generation g+1; the write is either authenticated or torn (crash during
\* write leaves unauthenticated bytes that happen to roll the key back).
Record(c, e) ==
    /\ ~crashed[c] /\ ~IsStaged(c)
    /\ Active(e) /\ op[e].replies[c].kind = "none"
    /\ LET o == op[e] IN
       IF ~Admissible(c, o)
       THEN /\ op' = [op EXCEPT ![e].replies[c] = Refused]
            /\ UNCHANGED <<clockVars, memberVars, grant, accepted, claims,
                           inflight, calls, register, lateClaim>>
       ELSE /\ anchorGen[c] < MaxGen
            /\ LET k == KeyOf(o.root, o.dom, o.slot, o.pred)
                   old == anchor[c][k]
                   new == [cands |-> old.cands \cup {o.value},
                           first |-> IF old.first = None THEN o.value ELSE old.first]
                   good == [gen |-> anchorGen[c] + 1, auth |-> TRUE,
                            content |-> [anchor[c] EXCEPT ![k] = new],
                            exec |-> e, key |-> k, req |-> Request(o)]
                   torn == [gen |-> anchorGen[c] + 1, auth |-> FALSE,
                            content |-> [anchor[c] EXCEPT ![k] = EmptyEntry],
                            exec |-> e, key |-> k, req |-> Request(o)]
```

```
               IN \/ /\ staged' = [staged EXCEPT ![c] = good]
                     /\ authGen' = [authGen EXCEPT ![c] = anchorGen[c] + 1]
                     /\ UNCHANGED <<crashed, op, grant, inflight>>
                  \/ /\ staged' = [staged EXCEPT ![c] = torn]
                     /\ crashed' = [crashed EXCEPT ![c] = TRUE]
                     /\ CrashLosses(c)
                     /\ UNCHANGED authGen
            /\ UNCHANGED <<clockVars, anchor, anchorGen, head, activeRoot, gate,
                           gateLive, boundary, bytes, accepted, claims, calls,
                           register, lateClaim>>

\* Commit: anchor the authenticated record, then expose the nonce-bound reply
\* to the exact operation it answers.  A reply first observed after the
\* rooted cutoff is discarded unless the LateReplyAdmitted mutation is on.
Commit(c) ==
    /\ ~crashed[c] /\ IsStaged(c) /\ staged[c].auth
    /\ LET s == staged[c]
           e == s.exec
           bound == Active(e) /\ Request(op[e]) = s.req
                    /\ op[e].replies[c].kind = "none"
           timely == LateReplyAdmitted \/ now <= Deadline(s.req)
       IN /\ anchor' = [anchor EXCEPT ![c] = s.content]
          /\ anchorGen' = [anchorGen EXCEPT ![c] = s.gen]
          /\ staged' = [staged EXCEPT ![c] = NoStaged]
          /\ op' = IF bound /\ timely
                   THEN [op EXCEPT ![e].replies[c] = Snapshot(s.content[s.key])]
                   ELSE op
    /\ UNCHANGED <<clockVars, authGen, head, activeRoot, gate, gateLive,
                   boundary, bytes, crashed, grant, accepted, claims, inflight,
                   calls, register, lateClaim>>

Crash(c) ==
    /\ ~crashed[c]
    /\ crashed' = [crashed EXCEPT ![c] = TRUE]
    /\ CrashLosses(c)
    /\ UNCHANGED <<clockVars, storeVars, head, activeRoot, gate, gateLive,
                   boundary, bytes, accepted, claims, calls, register, lateClaim>>

\* Recover: complete the single authenticated g+1 window; drop unauthenticated
\* bytes.  Durable head, claims, calls and the installed gate reload as-is.
Recover(c) ==
    /\ crashed[c]
    /\ crashed' = [crashed EXCEPT ![c] = FALSE]
    /\ IF IsStaged(c) /\ (staged[c].auth \/ UnauthenticatedRecovery)
       THEN /\ anchor' = [anchor EXCEPT ![c] = staged[c].content]
            /\ anchorGen' = [anchorGen EXCEPT ![c] = staged[c].gen]
       ELSE UNCHANGED <<anchor, anchorGen>>
    /\ staged' = [staged EXCEPT ![c] = NoStaged]
    /\ UNCHANGED <<clockVars, authGen, head, activeRoot, gate, gateLive,
                   boundary, bytes, execVars>>

ReachBoundary(c) ==
    /\ HandoffEnabled /\ ~boundary[c]
    /\ boundary' = [boundary EXCEPT ![c] = TRUE]
    /\ UNCHANGED <<clockVars, storeVars, head, activeRoot, gate, gateLive,
                   bytes, crashed, execVars>>

\* Handoff transcript bytes reach c from a member that installed the gate.
ReceiveBytes(c) ==
    /\ HandoffEnabled /\ ~bytes[c]
    /\ \E d \in Correct \ {c} : gate[d]
    /\ bytes' = [bytes EXCEPT ![c] = TRUE]
```

```
    /\ UNCHANGED <<clockVars, storeVars, head, activeRoot, gate, gateLive,
                   boundary, crashed, execVars>>

-----------------------------------------------------------------------------
\* Executor operation: fresh context, push to every configured member.
NewOp(root, dom, slot, value, pred, rid) ==
    [root |-> root, dom |-> dom, slot |-> slot, value |-> value, pred |-> pred,
     start |-> now, rid |-> rid, replies |-> [c \in Correct |-> NoReply]]
Begin(e, dom, slot, value, pred, rid) ==
    /\ ~crashed[e] /\ ~Active(e) /\ ~HasGrant(e) /\ ~HasInflight(e)
    /\ now + Delta < MaxTime
    /\ IF dom = Handoff
       THEN value = Successor /\ slot = 1 /\ pred = Init0
            /\ activeRoot[e] = OldRoot /\ ~gate[e]
       ELSE value \in Candidates /\ slot <= Len(head[e][dom]) + 1
    /\ op' = [op EXCEPT ![e] = NewOp(activeRoot[e], dom, slot, value, pred, rid)]
    /\ UNCHANGED <<clockVars, memberVars, grant, accepted, claims, inflight,
                   calls, register, lateClaim>>

\* Decide: after the rooted cutoff, every retained reply must be a valid
\* snapshot (any typed refusal aborts) and the authority rule must hold.
Decide(e) ==
    /\ ~crashed[e] /\ Active(e)
    /\ LET o == op[e]
           replied == {c \in Correct : o.replies[c].kind # "none"}
           valid == {c \in replied : o.replies[c].kind = "snapshot"}
           accept == /\ valid # {} /\ valid = replied
                     /\ \A c \in valid :
                          IF AuthorityMode = "owned"
                          THEN o.replies[c].cands = {o.value}
                          ELSE o.replies[c].first = o.value
           g == [root |-> o.root, dom |-> o.dom, slot |-> o.slot,
                 value |-> o.value, pred |-> o.pred, rid |-> o.rid,
                 expires |-> now + Lifetime, height |-> height]
           a == [root |-> o.root, dom |-> o.dom, slot |-> o.slot,
                 value |-> o.value, pred |-> o.pred]
       IN /\ LateReplyAdmitted \/ now > Deadline(o)
          /\ replied # {}
          /\ grant' = [grant EXCEPT ![e] = IF accept THEN g ELSE NoGrant]
          /\ accepted' = [accepted EXCEPT ![e] = IF accept THEN @ \cup {a} ELSE @]
          /\ op' = [op EXCEPT ![e] = Idle]
    /\ UNCHANGED <<clockVars, memberVars, claims, inflight, calls, register, lateClaim>>

\* Own accepted-history advancement.  A different hash at an occupied offset
\* is the ConflictingAcceptedHistory refusal: no write, the grant is unusable.
Advance(e) ==
    /\ ~crashed[e] /\ HasGrant(e) /\ grant[e].dom # Handoff
    /\ LET g == grant[e] IN
       /\ g.slot = Len(head[e][g.dom]) + 1
       /\ head' = [head EXCEPT ![e][g.dom] = Append(@, g.value)]
    /\ UNCHANGED <<clockVars, storeVars, activeRoot, gate, gateLive, boundary,
                   bytes, crashed, execVars>>
HeadHolds(e, g) ==
    g.slot <= Len(head[e][g.dom]) /\ head[e][g.dom][g.slot] = g.value

\* Claim fence: unexpired continuation, unchanged execution height, durable
\* own head, and exact admitted-manifest binding.
Fresh(g) == now <= g.expires /\ height = g.height
FenceOK(g) == ClaimAfterExpiry \/ Fresh(g)
BindingOK(e, g) ==
    /\ g.dom # Handoff
    /\ HeadHolds(e, g)
```

```
    /\ g.root = activeRoot[e]
    /\ Admitted(g.dom, g.slot) = g.value
    /\ AdmittedPred(g.dom, g.slot) = g.pred
ClaimKey(g) == [rid |-> IF ResourceIdSplitsKey THEN g.rid ELSE Stable,
                dom |-> g.dom, slot |-> g.slot]
SKey(g) == [dom |-> g.dom, slot |-> g.slot]
Bump(n) == IF n < 2 THEN n + 1 ELSE n

Claim(e) ==
    /\ ~CallBeforeClaim
    /\ ~crashed[e] /\ HasGrant(e) /\ ~HasInflight(e)
    /\ LET g == grant[e] IN
       /\ FenceOK(g) /\ BindingOK(e, g)
       /\ ClaimKey(g) \notin claims[e]
       /\ claims' = [claims EXCEPT ![e] = @ \cup {ClaimKey(g)}]
       /\ inflight' = [inflight EXCEPT ![e] =
            [rid |-> g.rid, dom |-> g.dom, slot |-> g.slot, claimed |-> TRUE]]
       /\ lateClaim' = [lateClaim EXCEPT ![e] = @ \/ ~Fresh(g)]
    /\ grant' = [grant EXCEPT ![e] = NoGrant]
    /\ UNCHANGED <<clockVars, memberVars, op, accepted, calls, register>>

\* Register call: put-if-absent on the idempotency key carried by the call.
Call(e) ==
    /\ ~crashed[e] /\ HasInflight(e) /\ inflight[e].claimed
    /\ register' = register \cup {ClaimKey(inflight[e])}
    /\ calls' = [calls EXCEPT ![e][SKey(inflight[e])] = Bump(@)]
    /\ inflight' = [inflight EXCEPT ![e] = NoInflight]
    /\ UNCHANGED <<clockVars, memberVars, op, grant, accepted, claims, lateClaim>>

\* CallBeforeClaim mutation: the call is issued first; the claim is a later,
\* separately crashable step.
CallFirst(e) ==
    /\ CallBeforeClaim
    /\ ~crashed[e] /\ HasGrant(e) /\ ~HasInflight(e)
    /\ LET g == grant[e] IN
       /\ FenceOK(g) /\ BindingOK(e, g)
       /\ ClaimKey(g) \notin claims[e]
       /\ register' = register \cup {ClaimKey(g)}
       /\ calls' = [calls EXCEPT ![e][SKey(g)] = Bump(@)]
       /\ inflight' = [inflight EXCEPT ![e] =
            [rid |-> g.rid, dom |-> g.dom, slot |-> g.slot, claimed |-> FALSE]]
       /\ lateClaim' = [lateClaim EXCEPT ![e] = @ \/ ~Fresh(g)]
    /\ grant' = [grant EXCEPT ![e] = NoGrant]
    /\ UNCHANGED <<clockVars, memberVars, op, accepted, claims>>
ClaimLate(e) ==
    /\ ~crashed[e] /\ HasInflight(e) /\ ~inflight[e].claimed
    /\ claims' = [claims EXCEPT ![e] = @ \cup {ClaimKey(inflight[e])}]
    /\ inflight' = [inflight EXCEPT ![e] = NoInflight]
    /\ UNCHANGED <<clockVars, memberVars, op, grant, accepted, calls, register, lateClaim>>

\* Lookup-only reconciliation: a durable claim without a live call handle
\* never issues a second call; the grant is consumed without externalizing.
Reconcile(e) ==
    /\ ~crashed[e] /\ HasGrant(e) /\ grant[e].dom # Handoff
    /\ ClaimKey(grant[e]) \in claims[e]
    /\ grant' = [grant EXCEPT ![e] = NoGrant]
    /\ UNCHANGED <<clockVars, memberVars, op, accepted, claims, inflight,
                   calls, register, lateClaim>>

\* A process-local continuation may be dropped at any time.
DiscardGrant(e) ==
    /\ HasGrant(e)
```

```
    /\ grant' = [grant EXCEPT ![e] = NoGrant]
    /\ UNCHANGED <<clockVars, memberVars, op, accepted, claims, inflight,
                   calls, register, lateClaim>>

\* SkipExecutorQuv mutation: claim from another executor's cached transcript.
ClaimFromTranscript(e) ==
    /\ SkipExecutorQuv
    /\ ~crashed[e] /\ ~HasInflight(e)
    /\ \E d \in Correct : \E a \in accepted[d] :
         /\ a.dom # Handoff
         /\ LET g == [root |-> a.root, dom |-> a.dom, slot |-> a.slot,
                      value |-> a.value, pred |-> a.pred, rid |-> "ra",
                      expires |-> now, height |-> height]
            IN /\ ClaimKey(g) \notin claims[e]
               /\ claims' = [claims EXCEPT ![e] = @ \cup {ClaimKey(g)}]
               /\ inflight' = [inflight EXCEPT ![e] =
                    [rid |-> "ra", dom |-> g.dom, slot |-> g.slot, claimed |-> TRUE]]
    /\ UNCHANGED <<clockVars, memberVars, op, grant, accepted, calls, register, lateClaim>>

---------------------------------------------------------------------------
\* Handoff: the successor gate is installed only by consuming this member's
\* own live handoff grant; activation requires the installed gate.
InstallGate(e) ==
    /\ ~crashed[e] /\ HasGrant(e) /\ grant[e].dom = Handoff
    /\ ~gate[e] /\ FenceOK(grant[e])
    /\ gate' = [gate EXCEPT ![e] = TRUE]
    /\ gateLive' = [gateLive EXCEPT ![e] = TRUE]
    /\ grant' = [grant EXCEPT ![e] = NoGrant]
    /\ UNCHANGED <<clockVars, storeVars, head, activeRoot, boundary, bytes,
                   crashed, op, accepted, claims, inflight, calls, register, lateClaim>>
SynthesizeGate(e) ==
    /\ ActivateFromBytes
    /\ ~crashed[e] /\ bytes[e] /\ ~gate[e]
    /\ gate' = [gate EXCEPT ![e] = TRUE]
    /\ UNCHANGED <<clockVars, storeVars, head, activeRoot, gateLive, boundary,
                   bytes, crashed, execVars>>
Activate(e) ==
    /\ ~crashed[e] /\ gate[e] /\ activeRoot[e] = OldRoot
    /\ activeRoot' = [activeRoot EXCEPT ![e] = Successor]
    /\ UNCHANGED <<clockVars, storeVars, head, gate, gateLive, boundary,
                   bytes, crashed, execVars>>

---------------------------------------------------------------------------
Next ==
    \/ Tick \/ AdvanceHeight
    \/ \E c \in Correct :
         \/ (\E e \in Correct : Record(c, e))
         \/ Commit(c) \/ Crash(c) \/ Recover(c) \/ ReachBoundary(c) \/ ReceiveBytes(c)
    \/ \E e \in Correct :
         \/ (\E dom \in AllDoms, slot \in Slots, value \in Values :
               \E pred \in BeginPreds(e, dom, slot), rid \in BeginRids :
                 Begin(e, dom, slot, value, pred, rid))
         \/ Decide(e) \/ Advance(e) \/ Claim(e) \/ Call(e) \/ CallFirst(e)
         \/ ClaimLate(e) \/ Reconcile(e) \/ DiscardGrant(e)
         \/ ClaimFromTranscript(e) \/ InstallGate(e) \/ SynthesizeGate(e)
         \/ Activate(e)
Spec == Init /\ [][Next]_vars

---------------------------------------------------------------------------
TypeOK ==
    /\ now \in 0..MaxTime /\ height \in 0..MaxHeight
    /\ anchor \in [Correct -> [Keys -> Entries]]
```

```
    /\ \A c \in Correct : anchorGen[c] \in 0..MaxGen /\ authGen[c] \in 0..MaxGen
                          /\ anchorGen[c] <= authGen[c] + 1
    /\ \A c \in Correct : ~IsStaged(c) \/
         (staged[c].gen = anchorGen[c] + 1 /\ staged[c].auth \in BOOLEAN
          /\ staged[c].content \in [Keys -> Entries] /\ staged[c].exec \in Correct
          /\ staged[c].key \in Keys /\ staged[c].req \in Requests)
    /\ \A c \in Correct, d \in Domains :
         head[c][d] \in Seq(Candidates) /\ Len(head[c][d]) <= MaxSlot
    /\ activeRoot \in [Correct -> Roots]
    /\ gate \in [Correct -> BOOLEAN] /\ gateLive \in [Correct -> BOOLEAN]
    /\ boundary \in [Correct -> BOOLEAN] /\ bytes \in [Correct -> BOOLEAN]
    /\ crashed \in [Correct -> BOOLEAN]
    /\ \A e \in Correct : ~Active(e) \/
         (Request(op[e]) \in Requests /\ op[e].rid \in Rids
          /\ op[e].replies \in [Correct -> Replies])
    /\ \A e \in Correct : ~HasGrant(e) \/
         (grant[e].root \in Roots /\ grant[e].dom \in AllDoms /\ grant[e].slot \in Slots
          /\ grant[e].value \in Values /\ grant[e].pred \in Preds /\ grant[e].rid \in Rids
          /\ grant[e].expires \in 0..(MaxTime + Lifetime) /\ grant[e].height \in 0..MaxHeight)
    /\ \A e \in Correct : claims[e] \subseteq ClaimKeys
    /\ \A e \in Correct : ~HasInflight(e) \/
         (inflight[e].rid \in Rids /\ inflight[e].dom \in Domains
          /\ inflight[e].slot \in Slots /\ inflight[e].claimed \in BOOLEAN)
    /\ calls \in [Correct -> [StableKeys -> 0..2]]
    /\ register \subseteq ClaimKeys
    /\ lateClaim \in [Correct -> BOOLEAN]

\* Q-E1/Q-E2 witness: no two live accepts differ for one (root, domain, slot).
NoConflictingAccepts ==
    \A e1, e2 \in Correct : \A a \in accepted[e1], b \in accepted[e2] :
      (a.root = b.root /\ a.dom = b.dom /\ a.slot = b.slot) => a.value = b.value

\* One conflict store per (root, domain, slot) at every member.
OneCanonicalConflictIdentity ==
    \A c \in Correct : \A k1, k2 \in Keys :
      (k1.root = k2.root /\ k1.dom = k2.dom /\ k1.slot = k2.slot
       /\ anchor[c][k1].cands # {} /\ anchor[c][k2].cands # {}) => k1 = k2

\* Every accept is retained as conflict knowledge at every correct member.
AcceptedKnowledgeRetained ==
    \A e, c \in Correct : \A a \in accepted[e] :
      a.value \in anchor[c][KeyOf(a.root, a.dom, a.slot, a.pred)].cands

\* An executor's own head never contradicts its own accepts
\* (ConflictingAcceptedHistory is a refusal, never a rewrite).
NoConflictingOwnHead ==
    \A e \in Correct : \A a \in accepted[e] :
      (a.dom # Handoff /\ a.slot <= Len(head[e][a.dom])) => head[e][a.dom][a.slot] = a.value

\* Q-EA4: a claim exists only after this executor's own live accept.
NoClaimWithoutOwnLiveAccept ==
    \A e \in Correct : \A k \in claims[e] :
      \E a \in accepted[e] : a.dom = k.dom /\ a.slot = k.slot

\* Every claim was fenced by continuation expiry and execution height.
NoClaimAfterExpiryOrFence == \A e \in Correct : ~lateClaim[e]

\* Q-E3 / T10: one physical mutation per stable (domain, slot) key.
AtMostOneExternalMutationPerStableKey ==
    \A k \in StableKeys :
      Cardinality({r \in register : r.dom = k.dom /\ r.slot = k.slot}) <= 1

```

```
\* T10 NoBlindReplayAfterAmbiguity lifted to the executor: at most one call
\* per executor and stable key, across crashes.
NoBlindReplayAfterCrash == \A e \in Correct, k \in StableKeys : calls[e][k] <= 1

\* Recovery installs only generations that an authenticated write produced.
RecoveryNeverAdvancesBeyondAuthenticatedRecords ==
    \A c \in Correct : anchorGen[c] <= authGen[c]

\* Q-E4 / Q-EA7: successor authority only through an own live-installed gate.
NoSuccessorAuthorityFromBytes ==
    \A c \in Correct : activeRoot[c] = Successor => gate[c] /\ gateLive[c]

\* Reachability probe, NOT a theorem: its required violation exhibits a trace
\* in which one executor accepts, advances, claims and externalizes slot 1
\* and then slot 2 (a sole correct member suffices).
NoExecutedSecondSlot ==
    ~ \E e \in Correct, d \in Domains : calls[e][[dom |-> d, slot |-> 2]] >= 1

GenBound == \A c \in Correct : anchorGen[c] <= MaxGen
\* Correct members are interchangeable model values (documented symmetry).
Symm == Permutations(Correct)
================================================================================
```

References

[1] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan-Gueta, and Ittai Abraham. *HotStuff: BFT Consensus in the Lens of Blockchain.* arXiv:1803.05069. https://arxiv.org/abs/1803.05069

[2] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. *Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus.* arXiv:2105.11827. https://arxiv.org/abs/2105.11827

[3] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Alexander Spiegelman. *Bullshark: DAG BFT Protocols Made Practical.* arXiv:2201.05677. https://arxiv.org/abs/2201.05677

[4] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. *The Honey Badger of BFT Protocols.* Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. https://eprint.iacr.org/2016/199.pdf

[5] Bingyong Guo, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. *Dumbo: Faster Asynchronous BFT Protocols.* Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. https://dblp.org/rec/conf/ccs/GuoL0XZ20

[6] Sourav Das, Sisi Duan, Shengqi Liu, Atsuki Momose, Ling Ren, and Victor Shoup. *Asynchronous Consensus without Trusted Setup or Public-Key Cryptography.* Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security; IACR ePrint 2024/677. https://eprint.iacr.org/2024/677

[7] Victor Shoup. *A Theoretical Take on a Practical Consensus Protocol.* IACR ePrint 2024/696. https://eprint.iacr.org/2024/696

[8] Alistair Stewart and Eleftherios Kokoris-Kogias. *GRANDPA: a Byzantine Finality Gadget.* arXiv:2007.01560. https://arxiv.org/abs/2007.01560

[9] Jelena Dosenovic, Roger Wattenhofer, and others. *Accountable Safety Implies Finality*. arXiv:2308.16902. https://arxiv.org/abs/2308.16902

[10] Chainlink Labs. *Chainlink CCIP's Defense-In-Depth Security and the Risk Management Network*. Chainlink blog, 2023. https://blog.chain.link/ccip-risk-management-network/

[11] John P. Conley. *Proof of Honesty: Coalition-Proof Blockchain Validation without Proof of Work or Stake*. Geeq technical paper, version 2.0, 2019. https://geeq.io/wp-content/uploads/GEEQ-OFFICIAL-TECHNICAL-PAPER.pdf

[12] Sean Bowe, Jack Grigg, and Daira Hopwood. *Halo: Recursive Proof Composition without a Trusted Setup*. IACR ePrint 2019/1021. https://eprint.iacr.org/2019/1021

[13] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. *Proof-Carrying Data from Accumulation Schemes*. IACR ePrint 2020/499. https://eprint.iacr.org/2020/499